



OWASP Top 10:2025 para Desenvolvedores PHP

Vulnerabilidades, Correções e Testes na Prática

Uirá Ribeiro

agosto de 2026

Table of Contents

1. Introdução	1
1.1. Sobre este livro	1
1.2. Como o projeto está organizado	1
1.3. Requisitos para rodar os exemplos	1
1.4. Rodando todos os testes de uma vez	2
1.5. Alternativa: rodando via Docker	2
1.6. Convenção de leitura deste livro	2
2. A01:2025 — Controle de Acesso Quebrado	3
2.1. 01 — Violação do Princípio do Privilégio Mínimo	3
2.1.1. Como testar	5
2.1.2. Checklist de verificação	5
2.2. 02 — Bypass por Manipulação de Parâmetros	6
2.2.1. Como testar	9
2.2.2. Checklist de verificação	9
2.3. 03 — Insecure Direct Object Reference (IDOR)	9
2.3.1. Como testar	12
2.3.2. Checklist de verificação	12
2.4. 04 — Falta de Controle de Acesso em API	12
2.4.1. Como testar	15
2.4.2. Checklist de verificação	15
2.5. 05 — Escalação de Privilégio	15
2.5.1. Como testar	19
2.5.2. Checklist de verificação	19
2.6. 06 — Manipulação de Metadados JWT	19
2.6.1. Como testar	23
2.6.2. Checklist de verificação	23
2.7. 07 — CORS Mal Configurado	23
2.7.1. Como testar	26
2.7.2. Checklist de verificação	26
2.8. 08 — Forced Browsing	26
2.8.1. Como testar	28
2.8.2. Checklist de verificação	29
2.9. 09 — Client-Side Enforcement	29
2.9.1. Como testar	32
2.9.2. Checklist de verificação	32
2.10. Checklist consolidado — Controle de Acesso Quebrado	32
3. A02:2025 — Configuração Incorreta de Segurança	34
3.1. 01 — Credenciais Padrão de Fábrica	34

3.1.1. Como testar	37
3.1.2. Checklist de verificação	37
3.2. 02 — Recursos e Funcionalidades Desnecessárias Expostas	37
3.2.1. Como testar	40
3.2.2. Checklist de verificação	40
3.3. 03 — Mensagens de Erro Excessivas	40
3.3.1. Como testar	42
3.3.2. Checklist de verificação	43
3.4. 04 — Headers de Segurança HTTP Ausentes	43
3.4.1. Como testar	45
3.4.2. Checklist de verificação	45
3.5. 05 — XXE (XML External Entity)	45
3.5.1. Como testar	48
3.5.2. Checklist de verificação	48
3.6. 06 — Listagem de Diretório	48
3.6.1. Como testar	51
3.6.2. Checklist de verificação	51
3.7. Checklist consolidado — Configuração Incorreta de Segurança	51
4. A03:2025 — Falhas na Cadeia de Suprimentos de Software	53
4.1. 01 — Versão Vulnerável Conhecida	53
4.1.1. Como testar	54
4.1.2. Checklist de verificação	54
4.2. 02 — Componente Não Mantido	55
4.2.1. Como testar	55
4.2.2. Checklist de verificação	56
4.3. 03 — Fonte Não Confiável	56
4.3.1. Como testar	57
4.3.2. Checklist de verificação	57
4.4. 04 — Scripts de Instalação Não Confiáveis	57
4.4.1. Como testar	58
4.4.2. Checklist de verificação	59
4.5. 05 — Dependência Sem Versão Fixada	59
4.5.1. Como testar	60
4.5.2. Checklist de verificação	60
4.6. 06 — Função Obsoleta e Perigosa	60
4.6.1. Como testar	63
4.6.2. Checklist de verificação	63
4.7. Checklist consolidado — Falhas na Cadeia de Suprimentos	63
5. A04:2025 — Falhas Criptográficas	64
5.1. 01 — Algoritmos Fracos	64
5.1.1. Como testar	67

5.1.2. Checklist de verificação	68
5.2. 02 — Aleatoriedade Insuficiente	68
5.2.1. Como testar	70
5.2.2. Checklist de verificação	70
5.3. 03 — Gestão de Chaves	70
5.3.1. Como testar	72
5.3.2. Checklist de verificação	72
5.4. 04 — Transporte em Texto Claro	73
5.4.1. Como testar	75
5.4.2. Checklist de verificação	75
5.5. 05 — Hash sem Salt	75
5.5.1. Como testar	78
5.5.2. Checklist de verificação	78
5.6. 06 — IV Problemático	78
5.6.1. Como testar	81
5.6.2. Checklist de verificação	82
5.7. 07 — Certificado Não Validado	82
5.7.1. Como testar	84
5.7.2. Checklist de verificação	84
5.8. 08 — Modo de Criptografia Inadequado	85
5.8.1. Como testar	87
5.8.2. Checklist de verificação	88
5.9. Checklist Consolidado — Falhas Criptográficas	88
6. A05:2025 — Injeção	89
6.1. 01 — SQL Injection	89
6.1.1. Como testar	92
6.1.2. Checklist de verificação	92
6.2. 02 — XSS (Cross-Site Scripting)	92
6.2.1. Como testar	94
6.2.2. Checklist de verificação	95
6.3. 03 — Command Injection	95
6.3.1. Como testar	97
6.3.2. Checklist de verificação	97
6.4. 04 — NoSQL Injection	97
6.4.1. Como testar	100
6.4.2. Checklist de verificação	101
6.5. 05 — LDAP Injection	101
6.5.1. Como testar	103
6.5.2. Checklist de verificação	103
6.6. 06 — Code Injection / EL Injection	104
6.6.1. Como testar	107

6.6.2. Checklist de verificação	107
6.7. 07 — ORM Injection	107
6.7.1. Como testar	110
6.7.2. Checklist de verificação	111
6.8. Checklist Consolidado — Injeção	111
7. A06:2025 — Insecure Design (Design Inseguro)	112
7.1. Sobre esta categoria	112
7.2. 01 — Armazenamento de Credenciais Desprotegido	112
7.2.1. Como testar	115
7.2.2. Checklist de verificação	115
7.3. 02 — Gerenciamento de Privilégio Impróprio	116
7.3.1. Como testar	119
7.3.2. Checklist de verificação	119
7.4. 03 — Upload de Arquivo Perigoso	120
7.4.1. Como testar	123
7.4.2. Checklist de verificação	123
7.5. 04 — Fronteira de Confiança Violada	123
7.5.1. Como testar	126
7.5.2. Checklist de verificação	126
7.6. 05 — Recuperação de Credencial Fraca	127
7.6.1. Como testar	130
7.6.2. Checklist de verificação	130
7.7. 06 — Lógica de Negócio sem Limite	130
7.7.1. Como testar	133
7.7.2. Checklist de verificação	133
7.8. 07 — Falta de Proteção Anti-Bot	134
7.8.1. Como testar	137
7.8.2. Checklist de verificação	137
7.9. Checklist consolidado — A06: Design Inseguro	138
8. A07:2025 — Authentication Failures (Falhas de Autenticação)	139
8.1. Sobre esta categoria	139
8.2. 01 — Ataques Automatizados Sem Limite	139
8.2.1. Como testar	142
8.2.2. Checklist de verificação	143
8.3. 02 — Senhas Fracas Permitidas	143
8.3.1. Como testar	145
8.3.2. Checklist de verificação	146
8.4. 03 — MFA Ausente ou Ineficaz	146
8.4.1. Como testar	149
8.4.2. Checklist de verificação	149
8.5. 04 — Fixação de Sessão	149

8.5.1. Como testar	152
8.5.2. Checklist de verificação	152
8.6. 05 — Expiração de Sessão Insuficiente	152
8.6.1. Como testar	155
8.6.2. Checklist de verificação	155
8.7. 06 — ID de Sessão em Local Inseguro	155
8.7.1. Como testar	158
8.7.2. Checklist de verificação	158
8.8. 07 — Bypass de Autenticação em Caminho Alternativo	158
8.8.1. Como testar	162
8.8.2. Checklist de verificação	162
8.9. 08 — Replay / Captura de Token	162
8.9.1. Como testar	165
8.9.2. Checklist de verificação	165
8.10. Checklist consolidado — A07: Falhas de Autenticação	166
9. A08:2025 — Falhas de Integridade de Software ou Dados	167
9.1. 01 — Desserialização Insegura	167
9.1.1. Como testar	170
9.1.2. Checklist de verificação	170
9.2. 02 — Atualização Sem Assinatura Criptográfica	170
9.2.1. Como testar	173
9.2.2. Checklist de verificação	173
9.3. 03 — Integridade de Biblioteca Externa	173
9.3.1. Como testar	175
9.3.2. Checklist de verificação	175
9.4. 04 — Inclusão de Funcionalidade Não Confiável (SRI)	175
9.4.1. Como testar	178
9.4.2. Checklist de verificação	178
9.5. 05 — Cookie em Decisão de Segurança	178
9.5.1. Como testar	181
9.5.2. Checklist de verificação	181
9.6. 06 — Inclusão de Caminho Não Controlado (Path Traversal)	181
9.6.1. Como testar	183
9.6.2. Checklist de verificação	183
9.7. Checklist consolidado — Falhas de Integridade de Software ou Dados	184
10. A09:2025 — Falhas de Registro e Alerta de Segurança	185
10.1. 01 — Injeção de Log	185
10.1.1. Como testar	187
10.1.2. Checklist de verificação	187
10.2. 02 — Perda de Informação Relevante	187
10.2.1. Como testar	189

10.2.2. Checklist de verificação	190
10.3. 03 — Dados Sensíveis no Log	190
10.3.1. Como testar	192
10.3.2. Checklist de verificação	192
10.4. 04 — Logging Insuficiente	192
10.4.1. Como testar	194
10.4.2. Checklist de verificação	194
10.5. 05 — Falta de Alerta de Anomalia	195
10.5.1. Como testar	197
10.5.2. Checklist de verificação	197
10.6. Checklist consolidado — Falhas de Registro e Alerta de Segurança	198
11. A10:2025 — Tratamento Inadequado de Condições Excepcionais	199
11.1. 01 — Erro com Dados Sensíveis	199
11.1.1. Como testar	202
11.1.2. Checklist de verificação	202
11.2. 02 — Exceção Não Tratada	202
11.2.1. Como testar	204
11.2.2. Checklist de verificação	205
11.3. 03 — Valor de Retorno Não Verificado	205
11.3.1. Como testar	207
11.3.2. Checklist de verificação	207
11.4. 04 — Caso Default Ausente	207
11.4.1. Como testar	210
11.4.2. Checklist de verificação	210
11.5. 05 — Tratamento Genérico de Exceções	210
11.5.1. Como testar	213
11.5.2. Checklist de verificação	213
11.6. 06 — Falha Não-Segura (Fail-Open)	213
11.6.1. Como testar	215
11.6.2. Checklist de verificação	215
11.7. 07 — Transação Não-Atômica Sem Rollback	215
11.7.1. Como testar	218
11.7.2. Checklist de verificação	218
11.8. 08 — Dados Sensíveis em Debug	218
11.8.1. Como testar	221
11.8.2. Checklist de verificação	221
11.9. Checklist consolidado — Tratamento Inadequado de Condições Excepcionais	221
12. Apêndice A: Testando SQL Injection com sqlmap na Prática	222
12.1. Introdução ao sqlmap	222
12.2. Testando 01-sql-injection	222
12.2.1. Sobre as opções sqlmap usadas	224

12.3. Testando 07-orm-injection	224
12.4. Checklist Geral de sqlmap para Qualquer Projeto PHP	225
12.4.1. 0. Antes de Começar	225
12.4.2. 1. Mapear os Pontos de Entrada	225
12.4.3. 2. Teste Rápido de Descoberta (GET Simples)	225
12.4.4. 3. Aumentar a Profundidade do Teste	226
12.4.5. 4. Focar num Parâmetro Específico (Mais Rápido)	226
12.4.6. 5. Testar POST / Formulários	226
12.4.7. 6. Testar Autenticado (Área Logada)	226
12.4.8. 7. Explorar o Site Inteiro Automaticamente	227
12.4.9. 8. Quando Encontrar uma Injeção — Extrair Informação	227
12.4.10. 9. Se o sqlmap Não Encontrar Nada de Primeira	227
12.4.11. 10. Depois de Terminar	227
12.4.12. Opções Mais Usadas — Resumo Rápido	228
12.5. Aviso de Segurança	228
13. Apêndice B: Rodando o Projeto via Docker	229
13.1. Introdução	229
13.2. Subindo o Ambiente	229
13.3. O Índice (index.php)	229
13.4. Parando e Editando	230
13.5. O Dockerfile	230
13.6. Docker Compose	231
13.7. Próximos Passos	231

Chapter 1. Introdução

1.1. Sobre este livro

Este livro documenta, categoria por categoria, o material didático em PHP construído para o curso *OWASP Top 10:2025*. Para cada uma das 10 categorias do [OWASP Top 10:2025](#), você vai encontrar:

- Uma explicação da categoria e por que ela importa;
- Uma lista de falhas específicas cobertas (cada uma mapeada a um ou mais CWEs oficiais);
- Para cada falha: o código **vulnerável** completo, explicado; o código **corrigido** completo, explicado; como rodar o teste automatizado que prova as duas coisas; e um checklist de verificação específico daquele problema;
- Um checklist consolidado ao final de cada capítulo, para usar em code review ou pentest.

Nos apêndices, você encontra um guia prático de uso do `sqlmap` contra os exemplos de SQL Injection, e como rodar o projeto inteiro via Docker.

1.2. Como o projeto está organizado

O código-fonte completo vive na pasta `owasp-php-examples/`, na raiz do projeto (irmã da pasta `livro/` onde este documento está). Cada categoria tem sua própria pasta (`A01-controle-acesso-quebrado/`, `A02-configuracao-incorreta/`, ...), e dentro dela, uma subpasta por falha específica. Cada subpasta contém, na imensa maioria dos casos, três arquivos:

`vulneravel.php`

Implementação com a falha, dentro do namespace `Vulneravel`. Comentário no topo explica o que a torna vulnerável.

`corrigido.php`

Implementação corrigida, dentro do namespace `Corrigido`. Comentário no topo explica a correção aplicada.

`teste.php`

Script standalone (`php teste.php`) que **prova** as duas coisas: que o `vulneravel.php` é de fato explorável, e que o `corrigido.php` bloqueia o mesmo ataque. Termina imprimindo `RESULTADO: N/N verificacoes passaram` e sai com código 0 se tudo passou, 1 caso contrário.

A única exceção é `A03-falhas-cadeia-suprimentos/`, onde 5 das 6 subpastas comparam pares de `composer.vulneravel.json` / `composer.corrigido.json` em vez de código PHP — porque ali o problema é a escolha de dependências no gerenciador de pacotes, não um input malicioso em tempo de execução (isso é detalhado no capítulo correspondente).

1.3. Requisitos para rodar os exemplos

- PHP 8.1 ou superior (o projeto foi desenvolvido e testado com PHP 8.5.8);

- Extensões padrão: `PDO`, `pdo_sqlite`, `openssl` — todas vêm habilitadas na maioria das instalações padrão do PHP;
- Nenhuma dependência externa, sem Composer, sem servidor web ou banco de dados externo necessário para a suíte principal de testes.

1.4. Rodando todos os testes de uma vez

Da raiz do projeto (`owasp-php-examples/`):

```
./run-tests.sh
```

Isso percorre todas as subpastas, roda o `teste.php` de cada uma, e imprime um resumo geral no final (`RESUMO GERAL: N/N pastas passaram`).

1.5. Alternativa: rodando via Docker

Se preferir não instalar PHP localmente:

```
docker compose up -d --build
```

E abra <http://localhost:8080> — há um índice (`index.php`) listando todas as categorias, com link para rodar cada teste diretamente no navegador, ver o código-fonte de cada exemplo com destaque de sintaxe, e um botão para rodar a suíte de testes inteira. Veja o [Apêndice B](#) para mais detalhes.

1.6. Convenção de leitura deste livro

Em cada exemplo, os blocos de código PHP são incluídos diretamente dos arquivos reais do projeto (via `include::` do AsciiDoc) — ou seja, o que você lê aqui é **exatamente** o que está no repositório, sempre em sincronia. O caminho do arquivo aparece no título de cada bloco de código, caso você queira abri-lo diretamente.

Este material tem finalidade exclusivamente educacional. O código `vulneravel.php` de cada exemplo demonstra uma falha real de segurança de forma isolada e controlada — nunca use esse código como referência de implementação, e não rode nenhuma técnica aqui descrita contra sistemas de terceiros sem autorização explícita.

Chapter 2. A01:2025 — Controle de Acesso Quebrado

Controle de Acesso Quebrado é a falha de segurança mais impactante do OWASP Top 10:2025. Ela resulta quando um aplicativo falha em definir e validar quem pode fazer o quê — isto é, falha em implementar controles que garantem que um usuário logado só possa acessar seus próprios dados e executar apenas as ações que sua função autoriza. As causas raiz costumam ser confundida entre *autenticação* (verificar quem você é) e *autorização* (verificar o que você pode fazer), decisões de arquitetura que depositam lógica de segurança no cliente, ou abordagens de *deny-by-default* fragmentadas que negam alguns casos mas esquecem de outros. Um atacante explorando falhas de controle de acesso pode ler dados alheios, modificar ou deletar recursos que não lhes pertencem, escalar-se para privilégios administrativos, ou assumir completamente contas de outros usuários.

Este capítulo cobre nove exemplos práticos de quebras de controle de acesso em PHP, cada um mostrando a falha, como explorá-la, e a correção baseada em princípios de segurança provados:

- **01 — Violação do Privilégio Mínimo:** usar deny-by-exception em vez de allow-list explícita
- **02 — Bypass por Manipulação de Parâmetros:** confiar em valores sensíveis enviados pelo cliente
- **03 — IDOR (Insecure Direct Object Reference):** retornar objetos sem verificar propriedade
- **04 — Falta de Controle em API:** autenticação sem autorização em endpoints que alteram estado
- **05 — Escalação de Privilégio:** aceitar role do cliente em vez de buscá-la do servidor
- **06 — Manipulação de JWT:** decodificar token sem validar assinatura HMAC
- **07 — CORS Mal Configurado:** refletir qualquer origem com credenciais
- **08 — Forced Browsing:** esconder rotas sensíveis sem autenticação/autorização servidor-side
- **09 — Client-Side Enforcement:** acreditar em sinais enviados pelo cliente sobre validação

2.1. 01 — Violação do Princípio do Privilégio Mínimo

CWE-284: Improper Access Control to a Resource by the Administrators

O Princípio do Privilégio Mínimo afirma que cada usuário deve receber apenas as permissões estritamente necessárias para sua função. Uma abordagem de "deny-open" (negar apenas alguns casos, liberar implicitamente todos os outros) é a inversa: assume que tudo é permitido e tenta bloquear alguns casos especiais. Na prática, é fácil esquecer um caso ou introduzir um novo caso que não estava na lista de negação. O resultado é que usuários comuns conseguem executar ações administrativas. A correção é inverter a lógica para "allow-list explícita": você lista exatamente quem *pode* fazer algo, e qualquer outro é bloqueado por padrão.

Código vulnerável (A01-controle-acesso-quebrado/01-violacao-privilegio-minimo/vulneravel.php)

```
<?php
declare(strict_types=1);
```

```

namespace Vulneravel;

/**
 * Violação do Princípio do Privilégio Mínimo (A01:2025 - Broken Access Control)
 *
 * O código usa uma abordagem "deny-open" (nega apenas guest), permitindo que
 * usuários comuns ('user') executem ações administrativas como exclusão de
 * relatórios.
 * Deveria usar "allow-list" explícita (apenas 'admin' pode excluir).
 */

function podeExcluirRelatorio(array $usuario): bool {
    // VULNERÁVEL: nega apenas 'guest', mas permite qualquer outra role
    return $usuario['role'] !== 'guest';
}

function demo(): void {
    echo "=== VULNERÁVEL: Violação do Privilégio Mínimo ===\n";

    $usuarioGuest = ['role' => 'guest'];
    $usuarioComum = ['role' => 'user'];
    $usuarioAdmin = ['role' => 'admin'];

    echo "Guest pode excluir? " . (podeExcluirRelatorio($usuarioGuest) ? 'SIM' :
    'NÃO') . "\n";
    echo "User comum pode excluir? " . (podeExcluirRelatorio($usuarioComum) ? 'SIM' :
    'NÃO') . " (PROBLEMA!)\n";
    echo "Admin pode excluir? " . (podeExcluirRelatorio($usuarioAdmin) ? 'SIM' :
    'NÃO') . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável nega apenas 'guest', deixando todos os outros roles (como 'user') com permissão de excluir relatórios. Se amanhã uma nova role for adicionada, ela automaticamente herda permissão de exclusão — um resultado contra-intuitivo e perigoso.

Código corrigido (A01-controle-acesso-quebrado/01-violacao-privilegio-minimo/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Aplicar o Princípio do Privilégio Mínimo com Allow-List Explícita
 */

```

```

* Usa abordagem "deny-closed" (nega por padrão, apenas 'admin' é liberado).
* Qualquer nova role deve ser explicitamente adicionada à lista de permissões.
*/

function podeExcluirRelatorio(array $usuario): bool {
    // CORRIGIDO: allow-list explícita (apenas admin pode excluir)
    return $usuario['role'] === 'admin';
}

function demo(): void {
    echo "=== CORRIGIDO: Privilégio Mínimo com Allow-List ===\n";

    $usuarioGuest = ['role' => 'guest'];
    $usuarioComum = ['role' => 'user'];
    $usuarioAdmin = ['role' => 'admin'];

    echo "Guest pode excluir? " . (podeExcluirRelatorio($usuarioGuest) ? 'SIM' :
'NÃO') . "\n";
    echo "User comum pode excluir? " . (podeExcluirRelatorio($usuarioComum) ? 'SIM' :
'NÃO') . " (bloqueado!)\n";
    echo "Admin pode excluir? " . (podeExcluirRelatorio($usuarioAdmin) ? 'SIM' :
'NÃO') . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção usa comparação explícita: apenas 'admin' pode excluir. Se alguém sem essa role tentar, é bloqueado. A técnica é chamada "allow-list" ou "whitelist", e escala bem: basta adicionar um novo valor à condição de comparação se uma nova role aprovada emergir. O princípio vale além de roles simples — sempre que você precisar autorizar uma ação, pergunte-se: "Quem está autorizado?" (lista branca), não "Quem está bloqueado?" (lista negra).

2.1.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A01-controle-acesso-quebrado/01-violacao-privilegio-minimo/teste.php`. O teste cria três usuários (guest, user, admin) e verifica que o código vulnerável permite que `user` exclua (o bug), enquanto o código corrigido bloqueia `user` mas continua permitindo `admin` (caso legítimo). A saída termina em `RESULTADO: N/N verificacoes passaram`.

2.1.2. Checklist de verificação

- ☐ Todas as operações sensíveis (delete, update, admin actions) verificam a role/permissão do usuário no servidor? Não só no frontend?
- ☐ A lógica de autorização usa allow-list (apenas roles aprovadas podem fazer X) em vez de deny-list (tudo pode fazer X exceto guests)?

- ❑ Se um novo role/group é adicionado ao sistema, é necessário explicitamente adicionar cada permissão que ele deve ter, ou herda cegamente permissões antigas?
- ❑ O código falha fechado? Isto é, se um valor de role é desconhecido ou ausente, a ação é negada por padrão?

2.2. 02 — Bypass por Manipulação de Parâmetros

CWE-434: Unrestricted Upload of File with Dangerous Type; CWE-1021: Improper Restriction of Rendered UI Layers or Frames

Um aplicativo de e-commerce calcula o total do carrinho baseado no preço que o cliente envia. Um desenvolvedor confiante pode pensar "o preço vem do banco de dados, não preciso verificar", mas esqueceu que o cliente controla os parâmetros HTTP. Manipulando o POST/GET, um atacante reduz os preços para zero ou centavos, efetivamente roubando produtos. Este é um exemplo clássico de "confiar na entrada do cliente em parâmetros críticos": qualquer coisa que afete dinheiro, limites, quotas ou decisões de negócio nunca deve vir diretamente do cliente sem validação servidor-side contra um catálogo confiável.

Código vulnerável ([A01-controle-acesso-quebrado/02-bypass-manipulacao-parametros/vulneravel.php](#))

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Bypass por Manipulação de Parâmetros (A01:2025 - Broken Access Control)
 *
 * O servidor confia no campo "preco" enviado pelo cliente no carrinho.
 * Um atacante pode reduzir o preço de um produto para valores muito baixos
 * ou até zero, realizando compras fraudulentas.
 */

function calcularTotalCompra(array $carrinho): float {
    $total = 0.0;

    // VULNERÁVEL: confia no campo 'preco' vindo do cliente
    foreach ($carrinho as $item) {
        if (isset($item['preco']) && isset($item['qtd'])) {
            $total += $item['preco'] * $item['qtd'];
        }
    }

    return $total;
}

function demo(): void {
    echo "=== VULNERÁVEL: Manipulação de Parâmetros (Preço) ===\n";
}
```

```

$carrinhoLegitimo = [
    ['produto_id' => 1, 'preco' => 100.0, 'qtd' => 2],
    ['produto_id' => 2, 'preco' => 50.0, 'qtd' => 1],
];

$carrinhoFraudado = [
    ['produto_id' => 1, 'preco' => 0.01, 'qtd' => 2], // Atacante mudou para 0.01
    ['produto_id' => 2, 'preco' => 0.01, 'qtd' => 1], // Deveria ser 50.0
];

echo "Carrinho legítimo total: R$ " . number_format(calcularTotalCompra(
    $carrinhoLegitimo), 2, ',', '.') . "\n";
echo "Carrinho fraudado total: R$ " . number_format(calcularTotalCompra(
    $carrinhoFraudado), 2, ',', '.') . " (PROBLEMA!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código loop sobre `$carrinho` e usa diretamente `$item['preco']` para calcular o total. Se um atacante modifica o corpo da requisição para enviar `preco=0.01`, o total cai dramaticamente. O aplicativo nunca valida se aquele preço é o preço real no banco de dados.

Código corrigido (A01-controle-acesso-quebrado/02-bypass-manipulacao-parametros/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Validar Dados Críticos Sempre no Servidor
 *
 * O servidor mantém um catálogo de preços imutável e valida cada item
 * usando apenas o ID do produto e a quantidade enviada pelo cliente.
 * O preço vem sempre do catálogo do servidor, nunca do cliente.
 */

// Catálogo de preços do servidor (imutável)
const CATALOGO = [
    1 => 100.0,
    2 => 50.0,
    3 => 25.5,
];

function calcularTotalCompra(array $carrinho): float {
    $total = 0.0;

    // CORRIGIDO: valida apenas produto_id e qtd; busca preço no catálogo

```

```

foreach ($carrinho as $item) {
    if (!isset($item['produto_id'], $item['qtd'])) {
        continue;
    }

    $idProduto = $item['produto_id'];

    // Verifica se produto existe no catálogo
    if (!isset(CATALOGO[$idProduto])) {
        continue;
    }

    // Usa preço do servidor, NUNCA do cliente
    $precoReal = CATALOGO[$idProduto];
    $qtd = (int) $item['qtd'];

    if ($qtd > 0) {
        $total += $precoReal * $qtd;
    }
}

return $total;
}

function demo(): void {
    echo "=== CORRIGIDO: Preço Validado no Servidor ===\n";

    $carrinhoLegitimo = [
        ['produto_id' => 1, 'qtd' => 2],
        ['produto_id' => 2, 'qtd' => 1],
    ];

    $carrinhoFraudado = [
        ['produto_id' => 1, 'preco' => 0.01, 'qtd' => 2], // Preço ignorado
        ['produto_id' => 2, 'preco' => 0.01, 'qtd' => 1], // Preço ignorado
    ];

    echo "Carrinho legítimo total: R$ " . number_format(calcularTotalCompra(
        $carrinhoLegitimo), 2, ',', '.') . "\n";
    echo "Carrinho fraudado (preço ignorado) total: R$ " . number_format(
        calcularTotalCompra($carrinhoFraudado), 2, ',', '.') . " (bloqueado!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção busca o preço real de um catálogo no servidor, usando apenas o `produto_id` que o cliente envia. O cliente não pode mais enviar um preço; apenas qual produto quer. O preço sempre vem de uma fonte confiável no servidor. Isso é chamado "re-validação servidor-side" ou "confiança mínima

do cliente", e é uma regra de ouro: dados monetários, limites, e decisões de autorização sempre vêm do servidor, nunca do cliente.

2.2.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A01-controle-acesso-quebrado/02-bypass-manipulacao-parametros/teste.php`. O teste monta um carrinho legítimo e um carrinho onde o preço foi manipulado para 0.01. Verifica que o código vulnerável calcula um total falso/baixo (bug), enquanto o corrigido computa o total correto buscando do catálogo (bloqueado e legítimo). A saída termina em **RESULTADO: N/N verificacoes passaram.**

2.2.2. Checklist de verificação

- ☐ Todos os valores que afetam dinheiro (preço, desconto, taxa) são sempre buscados de um catálogo/tabela no servidor, nunca aceitos como parâmetro do cliente?
- ☐ Se um limite de quota, créditos ou saldo é verificado, o valor atual vem do banco de dados do servidor, não de um campo oculto enviado pelo HTML/JS?
- ☐ Operações financeiras (pagamento, reembolso) usam valores server-side confirmados, não repetem cálculos baseados em entrada de cliente?
- ☐ O teste de carga do carrinho verifica se os preços enviados pelo cliente são ignorados e substituídos pelos preços reais?

2.3. 03 — Insecure Direct Object Reference (IDOR)

CWE-639: Authorization Through User-Controlled Key

IDOR ocorre quando um aplicativo usa um identificador (ID) fornecido pelo usuário para buscar um objeto, sem verificar se o usuário logado é realmente o proprietário daquele objeto. Por exemplo, um endpoint retorna `/pedido/123` para qualquer um que solicitar, desde que estejam autenticados — mas nunca verifica se o usuário 1 é de fato o dono do pedido 123. Um atacante pode enumerar IDs (`/pedido/1`, `/pedido/2`, etc.) e acessar pedidos, endereços, dados médicos ou financeiros de outras pessoas. O impacto é elevadíssimo: exfiltração em massa de dados privados sem deixar rastro.

Código vulnerável (A01-controle-acesso-quebrado/03-idor/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Insecure Direct Object Reference (IDOR) - A01:2025 Broken Access Control
 *
 * O servidor retorna um pedido apenas verificando se o ID existe,
 * sem checar se o usuário logado é o dono do pedido.
 * Atacante pode enumerar IDs de pedidos alheios e acessar dados privados.
 */
```

```

function buscarPedido(array $pedidos, int $idPedidoSolicitado, int $idUserLogado):
?array {
    // VULNERÁVEL: retorna o pedido apenas pelo ID, sem verificar propriedade
    foreach ($pedidos as $pedido) {
        if ($pedido['id'] === $idPedidoSolicitado) {
            return $pedido;
        }
    }
    return null;
}

function demo(): void {
    echo "=== VULNERÁVEL: IDOR (sem verificação de propriedade) ===\n";

    $pedidos = [
        ['id' => 1, 'dono_id' => 1, 'valor' => 100.0, 'descricao' => 'Pedido do
usuário 1'],
        ['id' => 2, 'dono_id' => 2, 'valor' => 250.0, 'descricao' => 'Pedido do
usuário 2'],
    ];

    $pedido = buscarPedido($pedidos, 2, 1);
    if ($pedido) {
        echo "Usuário 1 conseguiu acessar pedido de outro: " . json_encode($pedido) .
" (PROBLEMA!)\n";
    } else {
        echo "Acesso negado\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A função `buscarPedido()` procura um pedido pelo ID e o retorna. Nunca compara se o `dono_id` do pedido corresponde ao `idUserLogado`. Logo, qualquer usuário autenticado consegue ler qualquer pedido.

Código corrigido (A01-controle-acesso-quebrado/03-idor/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Verificar Propriedade do Objeto
 *
 * Antes de retornar um recurso, o servidor verifica se o usuário

```

```

* logado é realmente o dono daquele objeto.
*/

function buscarPedido(array $pedidos, int $idPedidoSolicitado, int $idUsuarioLogado):
?array {
    // CORRIGIDO: verifica se o pedido existe E se o usuário é o dono
    foreach ($pedidos as $pedido) {
        if ($pedido['id'] === $idPedidoSolicitado) {
            // Verifica propriedade
            if ($pedido['dono_id'] === $idUsuarioLogado) {
                return $pedido;
            }
            // Pedido existe mas não pertence a este usuário
            return null;
        }
    }
    return null;
}

function demo(): void {
    echo "=== CORRIGIDO: IDOR com verificação de propriedade ===\n";

    $pedidos = [
        ['id' => 1, 'dono_id' => 1, 'valor' => 100.0, 'descricao' => 'Pedido do
usuário 1'],
        ['id' => 2, 'dono_id' => 2, 'valor' => 250.0, 'descricao' => 'Pedido do
usuário 2'],
    ];

    $pedido = buscarPedido($pedidos, 2, 1);
    if ($pedido) {
        echo "Usuário 1 conseguiu acessar pedido de outro: " . json_encode($pedido) .
"\n";
    } else {
        echo "Acesso negado (bloqueado!)\n";
    }

    $pedidoProprio = buscarPedido($pedidos, 1, 1);
    if ($pedidoProprio) {
        echo "Usuário 1 pode acessar seu próprio pedido (caso legítimo)\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção adiciona uma verificação: depois que o pedido é encontrado, compara `pedido['dono_id']` com `idUsuarioLogado`. Se não combinarem, retorna `null` (como se o pedido não existisse, sem revelar que existe mas não é seu). Generalizar: sempre que retornar um objeto após autenticação, verifique

também a propriedade ou permissão daquele objeto. Essa é uma das regras mais críticas: autenticação ≠ autorização.

2.3.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A01-controle-acesso-quebrado/03-idor/teste.php`. O teste cria dois pedidos com donas diferentes, tenta que usuário 1 acesse o pedido de usuário 2. Verifica que o código vulnerável retorna os dados (bug), o corrigido bloqueia (seguro), e usuário 1 ainda pode acessar seu próprio pedido (caso legítimo). A saída termina em **RESULTADO: N/N verificacoes passaram.**

2.3.2. Checklist de verificação

- ☐ Todo endpoint que retorna dados de um usuário/entidade verifica que o usuário logado é realmente o proprietário daquele dado?
- ☐ Se um ID é passado como parâmetro, ele não é usado sozinho — sempre há uma verificação de propriedade (`donor_id`, `user_id`, `team_id`)?
- ☐ Quando acesso é negado, o aplicativo retorna 403 (sem revelar que o recurso existe) em vez de 404 (que pode vaziar informação)?
- ☐ Testes incluem tentativas de acessar recursos de outros usuários com IDs válidos conhecidos, e todas negam acesso?

2.4. 04 — Falta de Controle de Acesso em API

CWE-285: Improper Authorization

Muitos aplicativos verificam autenticação (o usuário está logado?) em todos os endpoints, mas esquecem que diferentes métodos HTTP devem ter diferentes regras. Um endpoint pode permitir GET (leitura) para qualquer usuário autenticado, mas DELETE e PUT (alteração/exclusão) devem exigir role de admin. Se o código só verifica "autenticado == não vazio" e libera DELETE para qualquer um, um usuário comum consegue deletar qualquer recurso. O erro é confundir autenticação com autorização e aplicar as mesmas regras para leitura e escrita.

Código vulnerável (A01-controle-acesso-quebrado/04-falta-controle-api/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Falta de Controle de Acesso em API (A01:2025 - Broken Access Control)
 *
 * O servidor verifica apenas autenticação (se $usuario não é vazio)
 * para todos os métodos HTTP, inclusive DELETE e PUT que alteram estado.
 * Um usuário comum autenticado pode deletar qualquer recurso.
 */
```

```

function tratarRequisicaoApi(string $metodo, array $usuario): array {
    // VULNERÁVEL: aceita DELETE/PUT apenas com autenticação genérica
    if (empty($usuario)) {
        return [
            'status' => 401,
            'mensagem' => 'Não autenticado',
        ];
    }

    // Qualquer método para qualquer usuário autenticado
    if ($metodo === 'DELETE' || $metodo === 'PUT') {
        return [
            'status' => 200,
            'mensagem' => "Operação {$metodo} realizada com sucesso",
        ];
    }

    if ($metodo === 'GET') {
        return [
            'status' => 200,
            'mensagem' => 'Dados retornados com sucesso',
        ];
    }

    return [
        'status' => 400,
        'mensagem' => 'Método não suportado',
    ];
}

function demo(): void {
    echo "=== VULNERÁVEL: Falta de Controle em API ===\n";

    $usuarioComum = ['id' => 1, 'role' => 'user'];

    $respostaDelete = tratarRequisicaoApi('DELETE', $usuarioComum);
    echo "Usuário comum DELETE: " . json_encode($respostaDelete) . " (PROBLEMA!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código verifica apenas se `$usuario` não é vazio. Se true, qualquer método (DELETE, PUT, GET) é permitido. Um usuário comum pode deletar qualquer coisa.

Código corrigido ([A01-controle-acesso-quebrado/04-falta-controle-api/corrigido.php](#))

```

<?php
declare(strict_types=1);

```

```

namespace Corrigido;

/**
 * Correção: Implementar Controle de Acesso por Método HTTP
 *
 * GET (leitura) requer apenas autenticação.
 * POST, PUT, DELETE (escrita) requerem autenticação E role='admin'.
 */

function tratarRequisicaoApi(string $metodo, array $usuario): array {
    // Primeiro: verifica se está autenticado
    if (empty($usuario)) {
        return [
            'status' => 401,
            'mensagem' => 'Não autenticado',
        ];
    }

    // GET: apenas autenticado é suficiente
    if ($metodo === 'GET') {
        return [
            'status' => 200,
            'mensagem' => 'Dados retornados com sucesso',
        ];
    }

    // DELETE, PUT, POST: requerem admin além de autenticado
    if ($metodo === 'DELETE' || $metodo === 'PUT' || $metodo === 'POST') {
        if ($usuario['role'] !== 'admin') {
            return [
                'status' => 403,
                'mensagem' => 'Acesso negado: privilégios insuficientes',
            ];
        }

        return [
            'status' => 200,
            'mensagem' => "Operação {$metodo} realizada com sucesso",
        ];
    }

    return [
        'status' => 400,
        'mensagem' => 'Método não suportado',
    ];
}

function demo(): void {
    echo "=== CORRIGIDO: Controle de Acesso por Método ===\n";
}

```

```

$usuarioComum = ['id' => 1, 'role' => 'user'];
$usuarioAdmin = ['id' => 2, 'role' => 'admin'];

$respostaDelete = tratarRequisicaoApi('DELETE', $usuarioComum);
echo "Usuário comum DELETE: " . json_encode($respostaDelete) . " (bloqueado!)\n";

$respostaDeleteAdmin = tratarRequisicaoApi('DELETE', $usuarioAdmin);
echo "Admin DELETE: " . json_encode($respostaDeleteAdmin) . " (permitido)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção checa a role do usuário, não só a presença. DELETE e PUT exigem `role == 'admin'`. GET continua aberto para `'user'`. Isso é autorização baseada em role (RBAC): diferentes roles têm diferentes permissões. Escala bem quando você tabelar as permissões: "user pode GET", "admin pode DELETE", etc.

2.4.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A01-controle-acesso-quebrado/04-falta-controle-api/teste.php`. Teste um usuário comum tentando DELETE; o código vulnerável retorna 200 OK (bug), o corrigido retorna 403 Forbidden. A saída termina em **RESULTADO:** `N/N verificacoes passaram`.

2.4.2. Checklist de verificação

- ☐ Endpoints que usam DELETE, PUT, PATCH, POST verificam role/permissão do usuário, além de autenticação?
- ☐ O aplicativo não assume que todos os métodos têm as mesmas regras de autorização?
- ☐ GET (leitura) e métodos que alteram estado têm verificações de autorização distintas?
- ☐ Há uma documentação explícita ou tabela descrevendo qual role pode fazer cada método em cada endpoint?

2.5. 05 — Escalação de Privilégio

CWE-638: Use of Externally-Controlled String in Generated SQL Query ('SQL Injection')

Em um formulário de login, um desenvolvedor incauto deixa o cliente enviar não só usuário/senha, mas também um campo `role` (para conveniência, acredita). Se presente no POST, esse valor é usado diretamente na sessão. Um atacante envia `role=admin` e ganha privilégios administrativos sem conhecer a senha do admin. Outro padrão comum: guardar a role em um cookie ou campo oculto e confiar nele em requisições futuras, sem re-validar no banco de dados. A correção é simples: **role sempre vem do banco de dados ou session confiável no servidor, nunca do cliente**.

```

<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Escalção de Privilégio (A01:2025 - Broken Access Control)
 *
 * Ao fazer login, o servidor aceita um campo 'role' enviado pelo cliente
 * em vez de sempre buscar a role correta do banco de dados.
 * Um atacante pode enviar role='admin' e ganhar privilégios elevados.
 */

function autenticar(array $dadosLogin, array $baseDeUsuarios): array {
    $nomeUsuario = $dadosLogin['usuario'] ?? '';

    if (!$nomeUsuario || empty($baseDeUsuarios)) {
        return [
            'autenticado' => false,
            'mensagem' => 'Falha na autenticação',
        ];
    }

    // Verifica se usuário existe na base
    $usuarioEncontrado = null;
    foreach ($baseDeUsuarios as $usr) {
        if ($usr['nome'] === $nomeUsuario) {
            $usuarioEncontrado = $usr;
            break;
        }
    }

    if (!$usuarioEncontrado) {
        return [
            'autenticado' => false,
            'mensagem' => 'Usuário não encontrado',
        ];
    }

    // VULNERÁVEL: usa role vindo do dadosLogin se estiver presente
    $role = $dadosLogin['role'] ?? $usuarioEncontrado['role'];

    return [
        'autenticado' => true,
        'usuario' => $nomeUsuario,
        'role' => $role,
        'mensagem' => 'Login bem-sucedido',
    ];
}

```

```

function demo(): void {
    echo "=== VULNERÁVEL: Escalção de Privilégio ===\n";

    $baseDeUsuarios = [
        ['nome' => 'alice', 'role' => 'user'],
        ['nome' => 'bob', 'role' => 'admin'],
    ];

    $loginFraudado = ['usuario' => 'alice', 'role' => 'admin'];
    $sessao = autenticar($loginFraudado, $baseDeUsuarios);

    echo "Alice enviou role=admin no login: " . json_encode($sessao) . " (PROBLEMA!)
\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A linha chave é `$role = $dadosLogin['role'] ?? $usuarioEncontrado['role'];`. Se `dadosLogin` (que vem do cliente) contém um campo `role`, ele é usado. Caso contrário, usa o `role` do banco. Um atacante pode forçar o cliente a enviar `role=admin`.

Código corrigido (A01-controle-acesso-quebrado/05-escalacao-privilegio/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Sempre Buscar Role do Servidor, Nunca do Cliente
 *
 * Ignora completamente qualquer campo 'role' enviado pelo cliente.
 * A role vem sempre do banco de dados do servidor.
 */

function autenticar(array $dadosLogin, array $baseDeUsuarios): array {
    $nomeUsuario = $dadosLogin['usuario'] ?? '';

    if (!$nomeUsuario || empty($baseDeUsuarios)) {
        return [
            'autenticado' => false,
            'mensagem' => 'Falha na autenticação',
        ];
    }

    // Busca usuário na base de dados
    $usuarioEncontrado = null;

```

```

foreach ($baseDeUsuarios as $usr) {
    if ($usr['nome'] === $nomeUsuario) {
        $usuarioEncontrado = $usr;
        break;
    }
}

if (!$usuarioEncontrado) {
    return [
        'autenticado' => false,
        'mensagem' => 'Usuário não encontrado',
    ];
}

// CORRIGIDO: SEMPRE usa a role do banco de dados, NUNCA do cliente
$role = $usuarioEncontrado['role'];

return [
    'autenticado' => true,
    'usuario' => $nomeUsuario,
    'role' => $role,
    'mensagem' => 'Login bem-sucedido',
];
}

function demo(): void {
    echo "=== CORRIGIDO: Escalação de Privilégio Bloqueada ===\n";

    $baseDeUsuarios = [
        ['nome' => 'alice', 'role' => 'user'],
        ['nome' => 'bob', 'role' => 'admin'],
    ];

    $loginFraudado = ['usuario' => 'alice', 'role' => 'admin'];
    $sessao = autenticar($loginFraudado, $baseDeUsuarios);

    echo "Alice enviou role=admin no login, mas recebeu: " . json_encode($sessao) . "
(bloqueado!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção **ignora completamente** qualquer **role** vindo de **dadosLogin**. O role sempre vem de **usuarioEncontrado['role']**, que veio do banco de dados em um **SELECT** de verdade. O cliente não tem escolha. Regra geral: dados de identidade (role, grupo, nível de permissão) nunca vêm do cliente, sempre do servidor.

2.5.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A01-controle-acesso-quebrado/05-escalacao-privilegio/teste.php`. Teste um usuário comum enviando `role=admin` no login. O código vulnerável o deixa com `role=admin` (bug), o corrigido mantém o role original do banco (seguro). A saída termina em `RESULTADO: N/N verificacoes passaram`.

2.5.2. Checklist de verificação

- ☐ A role/grupo/nível do usuário vem SEMPRE de uma consulta ao banco de dados após autenticação, nunca de campos enviados pelo cliente?
- ☐ Cookies e tokens que carregam informação de role são criados e validados no servidor, não confiam em valores no cliente?
- ☐ Se um usuário modifica um cookie local ou token, as permissões dele continuam aquelas no banco de dados, não as do token modificado?
- ☐ Testes incluem tentar enviar `role=admin` em um login de usuário comum e verificar que é rejeitado?

2.6. 06 — Manipulação de Metadados JWT

CWE-347: Improper Verification of Cryptographic Signature

Um JWT (JSON Web Token) é usado para armazenar claims (dados) sobre um usuário com uma assinatura HMAC. A assinatura garante que o token não foi modificado e vem do servidor. Um erro comum: decodificar o token e confiar nos claims **sem validar a assinatura**. Um atacante pode criar um token forjado com claims falsos (`role=admin`), e o servidor aceita porque nunca verificou a assinatura. Parece óbvio em retrospecto, mas é um erro real em aplicações PHP que copiam código de tutoriais antigos.

Código vulnerável (A01-controle-acesso-quebrado/06-manipulacao-jwt/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Manipulação de Metadados JWT (A01:2025 - Broken Access Control)
 *
 * O servidor decodifica o JWT e retorna os claims SEM validar a assinatura.
 * Um atacante pode forjar um token com claims falsos (role=admin) e o servidor
 * aceita sem verificar se a assinatura é válida.
 */

function criarToken(array $claims, string $chaveSecreta): string {
    // Header: HS256
    $header = ['alg' => 'HS256', 'typ' => 'JWT'];
    $headerEncoded = base64_encode(json_encode($header));
```

```

// Payload
$payloadEncoded = base64_encode(json_encode($claims));

// Signature
$mensagem = "{$headerEncoded}.{$payloadEncoded}";
$assinatura = hash_hmac('sha256', $mensagem, $chaveSecreta, true);
$assinaturaEncoded = base64_encode($assinatura);

return "{$mensagem}.{$assinaturaEncoded}";
}

function validarTokenERetornarClaims(string $token, string $chaveSecreta): ?array {
    $partes = explode('.', $token);

    // VULNERÁVEL: não valida a assinatura, apenas decodifica o payload
    if (count($partes) !== 3) {
        return null;
    }

    $headerEncoded = $partes[0];
    $payloadEncoded = $partes[1];
    // $assinatura = $partes[2]; // Ignorada!

    $payloadJson = base64_decode($payloadEncoded, true);
    if (!$payloadJson) {
        return null;
    }

    $claims = json_decode($payloadJson, true);
    if (!is_array($claims)) {
        return null;
    }

    // VULNERÁVEL: retorna os claims sem verificar se a assinatura é válida
    return $claims;
}

function demo(): void {
    echo "=== VULNERÁVEL: JWT sem Validação de Assinatura ===\n";

    $chave = 'chave-secreta-servidor';

    // Token legítimo
    $tokenLegitimo = criarToken(['usuario' => 'alice', 'role' => 'user'], $chave);
    echo "Token legítimo: " . substr($tokenLegitimo, 0, 40) . "... \n";

    // Token forjado manualmente (sem saber a chave)
    $headerForjado = base64_encode(json_encode(['alg' => 'HS256', 'typ' => 'JWT']));
    $payloadForjado = base64_encode(json_encode(['usuario' => 'alice', 'role' =>
'admin']));

```

```

    $tokenForjado = "{$headerForjado}.{$payloadForjado}
}.assinatura_falsa_qualquer_coisa";

    $claimsForjados = validarTokenERetornarClaims($tokenForjado, $chave);
    echo "Token forjado decodificado: " . json_encode($claimsForjados) . "
(PROBLEMA!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A função `validarTokenERetornarClaims()` divide o token em 3 partes, decodifica o payload (segunda parte) e o retorna. Mas a assinatura (terceira parte) é completamente ignorada — não há validação. Um atacante forja um token novo com claims falsos e o servidor o aceita.

Código corrigido (A01-controle-acesso-quebrado/06-manipulacao-jwt/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Validar Assinatura do JWT
 *
 * Recalcula o HMAC da parte "header.payload" com a chave secreta
 * e compara com a assinatura recebida usando hash_equals() para
 * evitar timing attacks. Se a assinatura não bater, rejeita o token.
 */

function criarToken(array $claims, string $chaveSecreta): string {
    // Header: HS256
    $header = ['alg' => 'HS256', 'typ' => 'JWT'];
    $headerEncoded = base64_encode(json_encode($header));

    // Payload
    $payloadEncoded = base64_encode(json_encode($claims));

    // Signature
    $mensagem = "{$headerEncoded}.{$payloadEncoded}";
    $assinatura = hash_hmac('sha256', $mensagem, $chaveSecreta, true);
    $assinaturaEncoded = base64_encode($assinatura);

    return "{$mensagem}.{$assinaturaEncoded}";
}

function validarTokenERetornarClaims(string $token, string $chaveSecreta): ?array {
    $partes = explode('.', $token);

```

```

    if (count($partes) !== 3) {
        return null;
    }

    $headerEncoded = $partes[0];
    $payloadEncoded = $partes[1];
    $assinaturaRecebida = $partes[2];

    // Decodifica header e payload
    $headerJson = base64_decode($headerEncoded, true);
    $payloadJson = base64_decode($payloadEncoded, true);

    if (!$headerJson || !$payloadJson) {
        return null;
    }

    $header = json_decode($headerJson, true);
    if (!is_array($header) || ($header['alg'] ?? '') !== 'HS256') {
        return null;
    }

    // CORRIGIDO: valida a assinatura recalculando o HMAC
    $mensagem = "{$headerEncoded}.{$payloadEncoded}";
    $assinaturaEsperada = hash_hmac('sha256', $mensagem, $chaveSecreta, true);
    $assinaturaEsperadaEncoded = base64_encode($assinaturaEsperada);

    // Usa hash_equals para evitar timing attack
    if (!hash_equals($assinaturaEsperadaEncoded, $assinaturaRecebida)) {
        return null; // Assinatura inválida
    }

    $claims = json_decode($payloadJson, true);
    if (!is_array($claims)) {
        return null;
    }

    return $claims;
}

function demo(): void {
    echo "=== CORRIGIDO: JWT com Validação de Assinatura ===\n";

    $chave = 'chave-secreta-servidor';

    // Token legítimo
    $tokenLegitimo = criarToken(['usuario' => 'alice', 'role' => 'user'], $chave);
    $claimsLegitimos = validarTokenERetornarClaims($tokenLegitimo, $chave);
    echo "Token legítimo validado: " . json_encode($claimsLegitimos) . "\n";

    // Token forjado manualmente (sem saber a chave)
    $headerForjado = base64_encode(json_encode(['alg' => 'HS256', 'typ' => 'JWT']));

```

```

    $payloadForjado = base64_encode(json_encode(['usuario' => 'alice', 'role' =>
'admin']));
    $tokenForjado = "{$headerForjado}.{$payloadForjado
}.assinatura_falsa_qualquer_coisa";

    $claimsForjados = validarTokenERetornarClaims($tokenForjado, $chave);
    echo "Token forjado (assinatura inválida): " . ($claimsForjados === null ? "NULL
(bloqueado!)" : json_encode($claimsForjados)) . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção recalcula a assinatura usando a mesma chave secreta e a função HMAC original, depois compara com `hash_equals()` (que evita timing attacks). Só se combinarem, o token é considerado válido e seus claims são confiáveis. Regra: qualquer token criptograficamente assinado deve ser verificado **antes** de confiar nos claims. Use bibliotecas JWT maduras ([firebase/php-jwt](#), [spomky-labs/jose](#)) em produção em vez de reimplementar.

2.6.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A01-controle-acesso-quebrado/06-manipulacao-jwt/teste.php`. Teste um token legítimo e um token forjado manualmente com claims falsos. O código vulnerável os aceita ambos (bug), o corrigido aceita só o legítimo (seguro). A saída termina em `RESULTADO: N/N verificacoes passaram`.

2.6.2. Checklist de verificação

- ☐ Todo JWT processado tem sua assinatura validada contra a chave secreta do servidor antes de confiar nos claims?
- ☐ Usa-se `hash_equals()` ou equivalente para evitar timing attacks na comparação de assinatura?
- ☐ Se um algoritmo de assinatura (HS256, RS256) é esperado, o código rejeita tokens com outros algoritmos?
- ☐ Testes incluem forjar um token manualmente com claims falsos e verificar que é rejeitado?

2.7. 07 — CORS Mal Configurado

CWE-346: Origin Validation Error; CWE-1021: Improper Restriction of Rendered UI Layers or Frames

CORS (Cross-Origin Resource Sharing) permite que um navegador de um site A acesse APIs de um site B, se B enviar o header `Access-Control-Allow-Origin: https://a.com`. Um erro comum: refletir dinamicamente a origem que vem da requisição (`Access-Control-Allow-Origin: <valor da requisição>`) sem validação. Se um atacante envia `Origin: https://attacker.com`, o servidor responde `Access-Control-Allow-Origin: https://attacker.com`, e com `Access-Control-Allow-Credentials: true`, o navegador envia cookies/credenciais automaticamente — permitindo CSRF e roubo de sessão.

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * CORS Mal Configurado (A01:2025 - Broken Access Control)
 *
 * O servidor reflete automaticamente a origem do cliente (qualquer uma)
 * no header Access-Control-Allow-Origin E permite credentials (cookies/auth).
 * Um site malicioso pode fazer requisições cross-origin ao servidor e ler
 * dados autenticados do usuário.
 */

function gerarCabecalhosCors(string $origemRequisicao): array {
    // VULNERÁVEL: reflete qualquer origem e permite credentials
    return [
        'Access-Control-Allow-Origin' => $origemRequisicao,
        'Access-Control-Allow-Credentials' => 'true',
        'Access-Control-Allow-Methods' => 'GET, POST, PUT, DELETE',
    ];
}

function demo(): void {
    echo "=== VULNERÁVEL: CORS Mal Configurado ===\n";

    $origemLegitima = 'https://meusite.com';
    $origemMaliciosa = 'https://atacante.com';

    $headersLegitimos = gerarCabecalhosCors($origemLegitima);
    echo "Headers para origem legítima: " . json_encode($headersLegitimos) . "\n";

    $headersMaliciosos = gerarCabecalhosCors($origemMaliciosa);
    echo "Headers para origem maliciosa: " . json_encode($headersMaliciosos) . "
(PROBLEMA!)\n";
}

if (debug_backtrace() === []) {
    demo();
}
```

O código lê o header **Origin** da requisição e o reflete diretamente no response, sem verificação. Um atacante de outro domínio consegue fazer requisições do navegador da vítima e acessar dados privados com credenciais dela.

```
<?php
```

```

declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: CORS Configurado com Allow-List de Domínios Confiáveis
 *
 * O servidor mantém uma lista explícita de domínios confiáveis.
 * Somente requisições dessas origens recebem os headers CORS com credentials.
 * Requisições de origem desconhecida são bloqueadas.
 */

// Allow-list de domínios confiáveis
const DOMINIOS_CONFIADOS = [
    'https://meusite.com',
    'https://app.meusite.com',
    'https://localhost:3000',
];

function gerarCabecalhosCors(string $origemRequisicao): array {
    // CORRIGIDO: valida contra allow-list
    if (!in_array($origemRequisicao, DOMINIOS_CONFIADOS, true)) {
        // Origem não confiável: não inclui headers CORS
        return [];
    }

    // Apenas domínios confiáveis recebem CORS com credentials
    return [
        'Access-Control-Allow-Origin' => $origemRequisicao,
        'Access-Control-Allow-Credentials' => 'true',
        'Access-Control-Allow-Methods' => 'GET, POST, PUT, DELETE',
    ];
}

function demo(): void {
    echo "=== CORRIGIDO: CORS com Allow-List ===\n";

    $origemLegitima = 'https://meusite.com';
    $origemMaliciosa = 'https://atacante.com';

    $headersLegitimos = gerarCabecalhosCors($origemLegitima);
    echo "Headers para origem confiável: " . json_encode($headersLegitimos) . "\n";

    $headersMaliciosos = gerarCabecalhosCors($origemMaliciosa);
    echo "Headers para origem maliciosa: " . json_encode($headersMaliciosos) . "
(bloqueado! vazio)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

```
}
```

A correção define uma allow-list fixa de origens confiáveis. Se a origem da requisição está na lista, ela é refletida; caso contrário, o header CORS não é enviado (requisição é bloqueada pelo navegador). Nunca refita a origem sem validação. Se permitir credenciais, seja muito restritivo sobre quais origens podem acessá-las.

2.7.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A01-controle-acesso-quebrado/07-cors-mal-configurado/teste.php`. Teste uma origem legítima e uma origem atacante. O código vulnerável libera ambas (bug), o corrigido só libera a legítima (seguro). A saída termina em `RESULTADO: N/N verificacoes passaram.`

2.7.2. Checklist de verificação

- ☐ O header `Access-Control-Allow-Origin` nunca reflete o valor de `Origin` da requisição sem validação?
- ☐ Existe uma allow-list explícita de origens permitidas (pode ser carregada de config)?
- ☐ Se `Access-Control-Allow-Credentials` é `true`, o aplicativo é muito restritivo sobre quais origens a recebem?
- ☐ Testes incluem requisições de origens não listadas e verificam que CORS headers não são enviados?

2.8. 08 — Forced Browsing

CWE-922: Insecure Direct Object Reference

Um aplicativo esconde rotas sensíveis (tipo `/admin`) do menu e da UI comum. Um usuário comum não vê um link para `/admin`, então conclui que não pode acessar. Mas se digitar `http://app.com/admin` na barra de endereços, nada a impede — a rota não valida se o usuário é admin. Isso é "segurança por obscuridade": confiar que o usuário não descobrirá a rota. Um atacante força a navegação (daí o nome) experimentando URLs comuns ou usando ferramentas como `dirbuster` ou `ffuf`. A correção é óbvia: a lógica de autorização deve estar **na rota**, não no frontend.

Código vulnerável (`A01-controle-acesso-quebrado/08-forced-browsing/vulneravel.php`)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Forced Browsing (A01:2025 - Broken Access Control)
 *
 * A única "proteção" é não mostrar links de admin no menu para usuários comuns.
 * Mas se alguém acessa a URL diretamente (/admin/painel), o servidor retorna
```

```

* o conteúdo SEM verificar a role. Segurança por obscuridade, não por verificação
real.
*/

function tratarRota(string $rota, ?array $usuario): string {
    // Rota pública
    if ($rota === '/home') {
        return 'Página inicial - conteúdo público';
    }

    // Rota de admin
    if ($rota === '/admin/painel') {
        // VULNERÁVEL: sem verificação de role
        // A única "proteção" é que o link não aparece no menu para usuários comuns
        return 'Painel de Administração - conteúdo altamente sensível';
    }

    return 'Rota não encontrada';
}

function demo(): void {
    echo "=== VULNERÁVEL: Forced Browsing ===\n";

    $usuarioComum = ['id' => 1, 'role' => 'user'];

    echo "Usuário comum acessando /home: " . tratarRota('/home', $usuarioComum) . "\n";
    echo "Usuário comum acessando /admin/painel: " . tratarRota('/admin/painel',
    $usuarioComum) . " (PROBLEMA!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A função `renderarAdmin()` não checa a role. Qualquer um que acesse `/admin` consegue ver a interface, porque a navegação está garantida apenas por "o menu não mostra o link". A rota em si é aberta.

Código corrigido (A01-controle-acesso-quebrado/08-forced-browsing/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Verificar Autorização no Servidor Para Cada Rota
 *
 * A proteção não é apenas no menu (frontend), mas no próprio tratamento

```

```

* da rota no servidor. Antes de retornar conteúdo sensível, verifica
* se o usuário está autenticado E tem a role necessária.
*/

function tratarRota(string $rota, ?array $usuario): string {
    // Rota pública
    if ($rota === '/home') {
        return 'Página inicial - conteúdo público';
    }

    // Rota de admin
    if ($rota === '/admin/painel') {
        // CORRIGIDO: valida autenticação E role antes de retornar conteúdo
        if ($usuario === null || $usuario['role'] !== 'admin') {
            return 'Acesso negado: privilégios insuficientes';
        }
        return 'Painel de Administração - conteúdo altamente sensível';
    }

    return 'Rota não encontrada';
}

function demo(): void {
    echo "=== CORRIGIDO: Forced Browsing Bloqueado ===\n";

    $usuarioComum = ['id' => 1, 'role' => 'user'];
    $usuarioAdmin = ['id' => 2, 'role' => 'admin'];

    echo "Usuário comum acessando /home: " . tratarRota('/home', $usuarioComum) . "\n";
    echo "Usuário comum acessando /admin/painel: " . tratarRota('/admin/painel', $usuarioComum) . " (bloqueado!)\n";
    echo "Admin acessando /admin/painel: " . tratarRota('/admin/painel', $usuarioAdmin) . " (permitido)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção valida no handler da rota: `if ($usuario['role'] !== 'admin') return ['status' => 403, 'corpo' => 'Acesso negado'];`. Agora, mesmo que o usuário acesse `/admin` diretamente, é bloqueado. A validação está server-side, não depende de UI.

2.8.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A01-controle-acesso-quebrado/08-forced-browsing/teste.php`. Teste um usuário comum acessando `/admin` diretamente. O código vulnerável retorna a página (bug), o corrigido retorna 403 (seguro). A saída termina em `RESULTADO: N/N verificacoes passaram`.

2.8.2. Checklist de verificação

- ☐ Toda rota sensível (/admin, /settings, /reports) verifica autorização no handler, não depende de UI/menu para esconder?
- ☐ Se acesso é negado, o aplicativo retorna 403 (Forbidden) em vez de 200 OK com conteúdo reduzido?
- ☐ Mesmo que um link seja removido do menu, a rota continua segura se acessada diretamente?
- ☐ Testes incluem acessar rotas administrativas diretamente como usuário comum e verificar 403?

2.9. 09 — Client-Side Enforcement

CWE-338: Use of Cryptographically Weak Pseudo-Random Number Generator (PRNG)

Um formulário valida no cliente (JavaScript) que o saldo é suficiente antes de deixar o usuário enviar. O campo `saldoValidado` é enviado como hidden field no form. O servidor confia nesse campo e não re-valida. Um atacante modifica o hidden field (ou faz um POST manual) dizendo `saldoValidado=true` mesmo sem saldo, e completa a compra fraudulentamente. Qualquer validação ou decisão crítica feita no cliente pode ser bypassada pelo atacante. A regra é: **nunca confie em sinais enviados pelo cliente sobre validações que já foram feitas.**

Código vulnerável (A01-controle-acesso-quebrado/09-client-side-enforcement/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Client-Side Enforcement (A01:2025 - Broken Access Control)
 *
 * O servidor confia em um campo enviado pelo cliente indicando que
 * a validação foi feita no frontend ("limiteVerificadoPeloClient").
 * Um atacante pode enviar valores maliciosos ignorando a validação do cliente.
 */

function processarTransferencia(array $dadosFormulario): array {
    $valor = $dadosFormulario['valor'] ?? 0.0;
    $limiteVerificadoPeloClient = $dadosFormulario['limiteVerificadoPeloClient'] ??
false;

    // VULNERÁVEL: se o cliente disse que validou, pula a validação no servidor
    if ($limiteVerificadoPeloClient === true) {
        return [
            'sucesso' => true,
            'mensagem' => "Transferência de R$ {$valor} processada com sucesso",
            'valor_processado' => $valor,
        ];
    }
}
```

```

// Só valida no servidor se cliente não validou (raramente acontece)
if ($valor > 1000.0) {
    return [
        'sucesso' => false,
        'mensagem' => 'Limite de transferência excedido',
        'valor_processado' => 0.0,
    ];
}

return [
    'sucesso' => true,
    'mensagem' => "Transferência de R$ {$valor} processada com sucesso",
    'valor_processado' => $valor,
];
}

function demo(): void {
    echo "=== VULNERÁVEL: Client-Side Enforcement ===\n";

    $transferenciaMaliciosa = ['valor' => 999999.0, 'limiteVerificadoPeloClient' =>
true];
    $resultado = processarTransferencia($transferenciaMaliciosa);

    echo "Transferência maliciosa: " . json_encode($resultado) . " (PROBLEMA!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A função aceita um campo `saldoValidado` vindo do cliente. Se é `true`, confia e libera a transação. O servidor nunca re-valida o saldo real.

Código corrigido (A01-controle-acesso-quebrado/09-client-side-enforcement/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Sempre Validar no Servidor, Não Confiar no Cliente
 *
 * O servidor SEMPRE valida regras críticas (limites, saldo) independente
 * do que o cliente disser. A validação do cliente é só para UX, não para segurança.
 */

// Saldo simulado do usuário (seria consultado de um banco real)
const SALDO_USUARIO = 1500.0;
const LIMITE_TRANSFERENCIA = 2000.0;

```

```

function processarTransferencia(array $dadosFormulario): array {
    $valor = (float) ($dadosFormulario['valor'] ?? 0.0);

    // CORRIGIDO: SEMPRE valida no servidor, ignora qualquer flag do cliente
    // Validação 1: valor positivo
    if ($valor <= 0.0) {
        return [
            'sucesso' => false,
            'mensagem' => 'Valor deve ser maior que zero',
            'valor_processado' => 0.0,
        ];
    }

    // Validação 2: dentro do limite de transferência
    if ($valor > LIMITE_TRANSFERENCIA) {
        return [
            'sucesso' => false,
            'mensagem' => "Limite de transferência excedido (máximo R$ " .
LIMITE_TRANSFERENCIA . ")",
            'valor_processado' => 0.0,
        ];
    }

    // Validação 3: saldo disponível
    if ($valor > SALDO_USUARIO) {
        return [
            'sucesso' => false,
            'mensagem' => 'Saldo insuficiente',
            'valor_processado' => 0.0,
        ];
    }

    // Passou em todas as validações do servidor
    return [
        'sucesso' => true,
        'mensagem' => "Transferência de R$ {$valor} processada com sucesso",
        'valor_processado' => $valor,
    ];
}

function demo(): void {
    echo "=== CORRIGIDO: Validação no Servidor ===\n";

    $transferenciaMaliciosa = ['valor' => 999999.0, 'limiteVerificadoPeloClient' =>
true];
    $resultado = processarTransferencia($transferenciaMaliciosa);

    echo "Transferência maliciosa: " . json_encode($resultado) . " (bloqueado!)\n";

    $transferenciaLegitima = ['valor' => 500.0];

```

```

$resultado2 = processarTransferencia($transferenciaLegitima);
echo "Transferência legítima: " . json_encode($resultado2) . " (permitida)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção **ignora completamente** o campo `saldoValidado`. Em vez disso, busca o saldo real do banco de dados e o valida. O cliente não pode forçar uma decisão; o servidor sempre revalida. Essa é a "defesa em profundidade": cliente valida para UX (feedback rápido), mas servidor valida novamente por segurança.

2.9.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A01-controle-acesso-quebrado/09-client-side-enforcement/teste.php`. Teste uma transação legítima com saldo e uma tentativa com `saldoValidado=true` mas saldo insuficiente. O código vulnerável a libera (bug), o corrigido a bloqueia (seguro). A saída termina em **RESULTADO: N/N verificacoes passaram**.

2.9.2. Checklist de verificação

- ☐ Validações críticas (saldo, limite, quota, permissão) são sempre re-validadas no servidor, independente de flags/hidden fields do cliente?
- ☐ O servidor nunca confia em um campo booleano ou contador enviado pelo cliente que afirma "essa validação já foi feita"?
- ☐ Se o cliente envia um saldo ou valor sensível, ele não é usado — em vez disso, o valor é sempre buscado do servidor?
- ☐ Testes incluem forjar um POST com validação "marcada como ok" mas valores inválidos, e verificam que é rejeitado?

2.10. Checklist consolidado — Controle de Acesso Quebrado

Este checklist reunido é para usar como referência rápida em code review e pentest de aplicações PHP:

- ☐ Toda operação sensível (create, read, update, delete, admin action) verifica a role/permissão do usuário autenticado?
- ☐ A lógica de autorização usa allow-list explícita (apenas roles/grupos aprovados podem fazer X)?
- ☐ Dados de identidade (role, grupo, nível) vêm sempre do servidor (banco de dados, sessão), nunca do cliente (POST, cookies, tokens não-validados)?
- ☐ Ao retornar um objeto personalizado (pedido, documento, perfil), o código verifica que o usuário é proprietário/autorizado àquele objeto?
- ☐ Endpoints que alteram estado (DELETE, PUT, POST) têm verificações de autorização **além** de

autenticação?

- ☐ JWTs, se usados, são sempre validados (assinatura HMAC/RS256) antes de confiar nos claims?
- ☐ CORS headers nunca refletem a origem sem validação contra allow-list; credenciais são expostas apenas a origens confiáveis?
- ☐ Rotas administrativas/sensíveis verificam autorização no handler do servidor, não dependem de UI/menu para esconder?
- ☐ Validações críticas (saldo, limite, quota) são re-validadas no servidor, nunca confiando em sinais do cliente?
- ☐ Há testes automatizados tentando acessar/modificar recursos de outros usuários, escalar privilégios, e bypassar validações?

Chapter 3. A02:2025 — Configuração Incorreta de Segurança

Configuração Incorreta de Segurança classifica uma larga gama de erros de setup, deployment e administração que deixam um aplicativo exposto. Diferente de controle de acesso quebrado (que é um erro de lógica), essas falhas são geralmente resultado de ausência de hardening: deixar endpoints de debug ativados em produção, não enviar headers de segurança, aceitar credenciais padrão de fábrica, exibir mensagens de erro que vazam informações sensíveis. A causa-raiz comum é um ciclo de desenvolvimento vs. produção não bem separado, ou uma lista de verificação de segurança (security checklist) que não é validada antes de deploy. O impacto pode ser severo: descoberta da tecnologia usada, enumeração de usuários, exfiltração de arquivos, ou assumir conta de admin com credenciais conhecidas.

Este capítulo cobre seis exemplos de configuração incorreta em PHP, desde credenciais padrão até expor recursos de debug e faltar com headers de segurança:

- **01 — Credenciais Padrão:** aceitar admin/admin123 hardcoded
- **02 — Recursos Desnecessários:** expor rotas de debug (/phpinfo, profiler) em produção
- **03 — Mensagens de Erro Excessivas:** retornar stack traces completos com caminhos e secrets
- **04 — Headers de Segurança Ausentes:** não enviar X-Content-Type-Options, CSP, HSTS, etc.
- **05 — XXE (XML External Entity):** processar XML sem rejeitar DOCTYPE/ENTITY perigosas
- **06 — Listagem de Diretório:** retornar 200 OK listando arquivos quando não há index

3.1. 01 — Credenciais Padrão de Fábrica

CWE-1392: Use of Default Credentials

Aplicações vêm com usuários e senhas pré-configurados "para facilitar" testes ou setup inicial. Um desenvolvedor esquece de remover essas credenciais antes de publicar, ou assume que o administrador vai trocar ao instalar. Na realidade, muitos administradores não sabem que existem, ou deixam para depois e nunca fazem. Um atacante com acesso ao aplicativo tenta credenciais conhecidas (admin/admin, test/test, admin/admin123) — extremamente comuns em software open-source — e ganha acesso administrativo sem quebrar uma única senha. O impacto é crítico: controle total do aplicativo.

Código vulnerável (A02-configuracao-incorreta/01-credenciais-padrao/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Credenciais Padrão de Fábrica - A02:2025 Security Misconfiguration
 *
 * Sistema ainda aceita credenciais de fábrica hardcoded (admin/admin123).
```

```

* Atacante pode fazer login com credenciais conhecidas sem necessidade de quebra de
senha.
*/

function autenticarAdmin(string $usuario, string $senha): bool {
    // VULNERÁVEL: credenciais padrão de fábrica ainda ativas
    return $usuario === 'admin' && $senha === 'admin123';
}

function demo(): void {
    echo "=== VULNERÁVEL: Credenciais padrão de fábrica ===\n";

    if (autenticarAdmin('admin', 'admin123')) {
        echo "Login bem-sucedido com credenciais padrão (PROBLEMA!)\n";
    } else {
        echo "Login negado\n";
    }

    if (!autenticarAdmin('admin', 'senha_forte_personalizada')) {
        echo "Senha customizada é rejeitada (esperado, mas credencial padrão ainda
funciona)\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A função aceita hardcoded `admin/admin123`. Se o administrador não trocar a senha na instalação, qualquer um que conheça essa credencial padrão faz login como admin. Não há proteção contra força bruta nem bloqueio depois de N tentativas.

Código corrigido (A02-configuracao-incorreta/01-credenciais-padrao/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Remover e Bloquear Credenciais Padrão de Fábrica
 *
 * Credenciais padrão são explicitamente bloqueadas, forçando alteração obrigatória
 * na primeira autenticação e permitindo apenas senhas customizadas e seguras.
 */

function senhaEhPadraoDeFabrica(string $senha): bool {
    // Lista de senhas padrão conhecidas que NUNCA devem ser aceitas
    $senhasPadrao = [
        'admin123',

```

```

        '123456',
        'password',
        'admin',
        '12345678',
        'qwerty',
    ];

    return in_array($senha, $senhasPadrao, true);
}

function autenticarAdmin(string $usuario, string $senhaFornecida, string
$hashArmazenado): bool {
    // CORRIGIDO: rejeita credenciais padrão de fábrica MESMO que o hash corresponda
    if (senhaEhPadraoDeFabrica($senhaFornecida)) {
        return false;
    }

    // Verifica se a senha fornecida combina com o hash armazenado
    return password_verify($senhaFornecida, $hashArmazenado);
}

function demo(): void {
    echo "=== CORRIGIDO: Credenciais padrão bloqueadas ===\n";

    // Hash de uma senha customizada segura
    $hashSenhaPersonalizada = password_hash('SenhaForte2024!', PASSWORD_DEFAULT);

    // Tentativa 1: Login com credencial padrão (bloqueado mesmo com hash válido)
    $hashAdmin123 = password_hash('admin123', PASSWORD_DEFAULT);
    if (!autenticarAdmin('admin', 'admin123', $hashAdmin123)) {
        echo "Credencial padrão 'admin123' bloqueada (protegido!)\n";
    }

    // Tentativa 2: Login com senha customizada (bloqueado se não combinar com hash)
    if (!autenticarAdmin('admin', 'senha_errada', $hashSenhaPersonalizada)) {
        echo "Senha incorreta rejeitada (esperado)\n";
    }

    // Tentativa 3: Login com senha customizada correta (aceito)
    if (autenticarAdmin('admin', 'SenhaForte2024!', $hashSenhaPersonalizada)) {
        echo "Senha customizada válida aceita (caso legítimo)\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção remove completamente credenciais hardcoded. O login só funciona contra uma lista de usuários/senhas vindo do banco de dados ou de um arquivo de configuração **fora do repositório**

(via variáveis de ambiente, `.env` não commitado, ou gerenciador de secrets). Adicionalmente, usa `password_verify()` e `password_hash()` para armazenar senhas de forma segura. Credenciais padrão nunca devem estar no código-fonte.

3.1.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A02-configuracao-incorreta/01-credenciais-padrao/teste.php`. O teste tenta fazer login com `admin/admin123`. O código vulnerável permite (bug), o corrigido rejeita (seguro). A saída termina em `RESULTADO: N/N verificacoes passaram`.

3.1.2. Checklist de verificação

- ☐ Nenhuma credencial padrão está hardcoded no código-fonte do aplicativo?
- ☐ Senhas de usuários são sempre armazenadas com `password_hash()` ou algoritmo equivalente, nunca em texto claro?
- ☐ Testes automatizados tentam login com credenciais de fábrica conhecidas (`admin/admin`, `test/test`, etc.) e verificam que falham?
- ☐ Se um arquivo de configuração contém credenciais, ele está em `.gitignore` e nunca é commitado?

3.2. 02 — Recursos e Funcionalidades Desnecessárias Expostas

CWE-693: Protection Mechanism Failure

Um aplicativo expõe endpoints úteis para debugging em desenvolvimento: `/debug/phpinfo`, `/profiler`, `/logs`, `/console`. Um desenvolvedor os deixa ligados "para diagnóstico" e nunca os desativa em produção. Um atacante descobre versão exata do PHP, extensões instaladas, configurações do servidor, caminhos de arquivo, variáveis de ambiente — tudo uma mina de ouro para encontrar outras vulnerabilidades. A correção é óbvia: qualquer recurso de debug só existe em desenvolvimento, não existe (404) em produção.

Código vulnerável (A02-configuracao-incorreta/02-recursos-desnecessarios/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Recursos e Funcionalidades Desnecessárias Expostas - A02:2025 Security
 * Misconfiguration
 *
 * Endpoints de debug (phpinfo, profiler, console) estão ativos em produção.
 * Atacante pode acessar informações sensíveis sobre versão PHP, extensões, caminho do
 * servidor.
```

```

*/

function tratarRota(string $rota, string $ambiente): array {
    // VULNERÁVEL: endpoints de debug sempre ativados, independente do ambiente
    if ($rota === '/debug/phpinfo') {
        return [
            'status' => 200,
            'corpo' => 'PHP Version: 8.5.8 | OS: Linux | Extensions: PDO, XML, JSON,
OpenSSL',
        ];
    }

    if ($rota === '/') {
        return [
            'status' => 200,
            'corpo' => 'Página inicial',
        ];
    }

    return [
        'status' => 404,
        'corpo' => 'Rota não encontrada',
    ];
}

function demo(): void {
    echo "=== VULNERÁVEL: Endpoints de debug expostos em produção ===\n";

    $resposta = tratarRota('/debug/phpinfo', 'producao');
    echo "Status: {$resposta['status']}\n";
    echo "Corpo: {$resposta['corpo']}\n";
    if ($resposta['status'] === 200) {
        echo "Informações sensíveis expostas em produção (PROBLEMA!)\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A rota `/debug/phpinfo` responde com informações sensíveis sempre, independente do ambiente. Um atacante acessa `GET /debug/phpinfo` e descobre tudo sobre o servidor.

Código corrigido ([A02-configuracao-incorreta/02-recursos-desnecessarios/corrigido.php](#))

```

<?php
declare(strict_types=1);

namespace Corrigido;

```

```

/**
 * Correção: Desabilitar Recursos de Debug em Produção
 *
 * Endpoints sensíveis só estão disponíveis em desenvolvimento.
 * Em produção, retornam 404 para evitar exposição de informações sensíveis.
 */

function tratarRota(string $rota, string $ambiente): array {
    // CORRIGIDO: endpoints de debug só estão ativos em desenvolvimento
    if ($rota === '/debug/phpinfo') {
        if ($ambiente === 'desenvolvimento') {
            return [
                'status' => 200,
                'corpo' => 'PHP Version: 8.5.8 | OS: Linux | Extensions: PDO, XML,
JSON, OpenSSL',
            ];
        } else {
            // Em produção, simula como se a rota não existisse
            return [
                'status' => 404,
                'corpo' => 'Rota não encontrada',
            ];
        }
    }

    if ($rota === '/') {
        return [
            'status' => 200,
            'corpo' => 'Página inicial',
        ];
    }

    return [
        'status' => 404,
        'corpo' => 'Rota não encontrada',
    ];
}

function demo(): void {
    echo "=== CORRIGIDO: Debug apenas em desenvolvimento ===\n";

    // Em produção
    $respostaProd = tratarRota('/debug/phpinfo', 'producao');
    echo "Produção - Status: {$respostaProd['status']}\n";
    if ($respostaProd['status'] === 404) {
        echo "Endpoint de debug bloqueado em produção (protegido!)\n";
    }

    // Em desenvolvimento
    $respostaDev = tratarRota('/debug/phpinfo', 'desenvolvimento');
    echo "Desenvolvimento - Status: {$respostaDev['status']}\n";
}

```

```

    if ($respostaDev['status'] === 200) {
        echo "Endpoint de debug ativo em desenvolvimento (útil para diagnóstico)\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção checa a variável de ambiente ou flag `$ambiente`. Se é produção, a rota retorna 404 (como se não existisse). Se é desenvolvimento, retorna as informações (útil para diagnóstico). Generalizar: rotas de debug devem estar dentro de um bloco condicional que verifica o ambiente.

3.2.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A02-configuracao-incorreta/02-recursos-desnecessarios/teste.php`. Teste a rota `/debug/phpinfo` em desenvolvimento (deve retornar 200) e em produção (deve retornar 404). O código vulnerável retorna 200 em ambos (bug), o corrigido retorna diferente conforme ambiente (seguro). A saída termina em **RESULTADO: N/N verificacoes passaram.**

3.2.2. Checklist de verificação

- ☐ Nenhuma rota de debug (`/debug`, `/profiler`, `/logs`, `/console`, `phpinfo`) está acessível em produção?
- ☐ Essas rotas têm uma verificação `if ($ambiente !== 'development') return 404;?`
- ☐ A variável de ambiente que determina desenvolvimento vs. produção vem de config externa, não de um comentário no código?
- ☐ Testes de segurança verificam se endpoints de debug retornam 404 em produção?

3.3. 03 — Mensagens de Erro Excessivas

CWE-209: Information Exposure Through an Error Message

Um formulário falha ao conectar ao banco de dados. O servidor retorna ao usuário: `"Connection failed: mysqli_connect(): (28000/1045): Access denied for user 'root'@'localhost' (using password: YES) in /var/www/html/config/db.php on line 23"`. Isso expõe o nome de usuário, caminho completo do arquivo, número da linha — informações valiosas para um atacante. Em desenvolvimento, stack traces são úteis; em produção, eles são vazamento de informação. A regra é: em produção, mostre ao usuário uma mensagem genérica, mas log os detalhes no servidor para o time de operações.

Código vulnerável (`A02-configuracao-incorreta/03-mensagens-erro-excessivas/vulneravel.php`)

```

<?php
declare(strict_types=1);

namespace Vulneravel;

```

```

/**
 * Mensagens de Erro com Informações Sensíveis - A02:2025 Security Misconfiguration
 *
 * Stack traces completos e mensagens de erro detalhadas são expostas a usuários.
 * Atacante pode descobrir caminhos do servidor, banco de dados, senhas hardcoded no
 * erro.
 */

function tratarErro(\Throwable $erro, string $ambiente): string {
    // VULNERÁVEL: sempre expõe a mensagem completa e stack trace, independente do
    ambiente
    return $erro->getMessage() . "\n" . $erro->getTraceAsString();
}

function demo(): void {
    echo "=== VULNERÁVEL: Stack trace completo exposto ===\n";

    try {
        throw new \RuntimeException('Falha ao conectar em /var/www/config/database.php
com senha=SegredoDB123');
    } catch (\Throwable $e) {
        $mensagem = tratarErro($e, 'producao');
        echo $mensagem . "\n";
        if (str_contains($mensagem, 'SegredoDB123')) {
            echo "\nSenha exposta no erro (PROBLEMA!)\n";
        }
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A função `tratarErro()` sempre retorna `$erro->getMessage() . $erro->getTraceAsString()`, expondo toda a pilha de chamadas e mensagens detalhadas com paths sensíveis.

Código corrigido (A02-configuracao-incorreta/03-mensagens-erro-excessivas/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Mensagens de Erro Genéricas em Produção
 *
 * Em produção, mensagens são genéricas sem detalhes técnicos.
 * Logs detalhados são gravados no servidor para diagnóstico interno apenas.
 */

```

```

function tratarErro(\Throwable $erro, string $ambiente): string {
    if ($ambiente === 'producao') {
        // CORRIGIDO: retorna mensagem genérica que não expõe informações sensíveis
        return 'Ocorreu um erro interno. Tente novamente mais tarde.';
    } else {
        // Em desenvolvimento, mantém o stack trace completo para debugging
        return $erro->getMessage() . "\n" . $erro->getTraceAsString();
    }
}

function demo(): void {
    echo "=== CORRIGIDO: Mensagens de erro seguras em produção ===\n";

    try {
        throw new \RuntimeException('Falha ao conectar em /var/www/config/database.php
com senha=SegredoDB123');
    } catch (\Throwable $e) {
        // Em produção
        $mensagemProd = tratarErro($e, 'producao');
        echo "Produção:\n{$mensagemProd}\n";
        if (!str_contains($mensagemProd, 'SegredoDB123')) {
            echo "Senha não exposta (protegido!)\n";
        }

        // Em desenvolvimento
        $mensagemDev = tratarErro($e, 'desenvolvimento');
        echo "\nDesenvolvimento:\n{$mensagemDev}\n";
        if (str_contains($mensagemDev, 'SegredoDB123')) {
            echo "Detalhes completos disponíveis para debugging\n";
        }
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção checka o ambiente. Em produção, retorna "Ocorreu um erro interno. Tente novamente mais tarde." — genérico, sem detalhes. Em desenvolvimento, retorna o stack trace completo. A chave é separar o que o *usuário* vê do que o *administrador* vê no log do servidor.

3.3.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A02-configuracao-incorreta/03-mensagens-erro-excessivas/teste.php`. Simule um erro em produção e em desenvolvimento. Em produção, a mensagem deve ser genérica (sem senhas); em desenvolvimento, deve conter detalhes. O código vulnerável mostra detalhes em ambos (bug), o corrigido diferencia (seguro). A saída termina em `RESULTADO: N/N verificacoes passaram`.

3.3.2. Checklist de verificação

- ☐ Em produção, mensagens de erro para o usuário são genéricas (não expõem stack trace, caminhos, banco de dados)?
- ☐ Stack traces e mensagens detalhadas são logadas no servidor (em arquivo privado ou gerenciador de logs), não retornadas ao cliente?
- ☐ Exceções não-tratadas têm um handler que as captura e retorna 500 genérico, não o trace completo?
- ☐ Testes incluem forçar erros e verificar que a mensagem é genérica em produção?

3.4. 04 — Headers de Segurança HTTP Ausentes

CWE-693: Protection Mechanism Failure

Um navegador recebe respostas HTTP sem nenhum header de segurança. Sem **X-Frame-Options**, a página pode ser embutida num iframe malicioso (clickjacking). Sem **Content-Security-Policy**, inline scripts são executados (XSS). Sem **X-Content-Type-Options: nosniff**, o navegador pode adivinhar o tipo MIME (MIME sniffing). Sem **Strict-Transport-Security**, uma conexão pode ser rebaixada de HTTPS para HTTP (man-in-the-middle). Esses headers são "quick wins" de segurança: custam zero em lógica de negócio e protegem contra classes inteiras de ataques.

Código vulnerável (A02-configuracao-incorreta/04-headers-seguranca-ausentes/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Headers de Segurança Ausentes - A02:2025 Security Misconfiguration
 *
 * Aplicação não configura headers HTTP essenciais de segurança.
 * Navegador fica vulnerável a clickjacking, MIME sniffing, injeção XSS e man-in-the-middle.
 */

function gerarHeadersSeguranca(): array {
    // VULNERÁVEL: nenhum header de segurança configurado
    return [];
}

function demo(): void {
    echo "=== VULNERÁVEL: Headers de segurança ausentes ===\n";

    $headers = gerarHeadersSeguranca();
    if (empty($headers)) {
        echo "Nenhum header de segurança configurado (PROBLEMA!)\n";
    }
}
```

```

$headersEsperados = [
    'X-Content-Type-Options',
    'X-Frame-Options',
    'Content-Security-Policy',
    'Strict-Transport-Security',
];

foreach ($headersEsperados as $header) {
    if (!isset($headers[$header])) {
        echo "Header {$header} ausente\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A função retorna um array vazio — nenhum header de segurança. O navegador fica vulnerável a clickjacking, MIME sniffing, injeção de conteúdo.

Código corrigido (A02-configuracao-incorreta/04-headers-seguranca-ausentes/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Configurar Headers de Segurança HTTP
 *
 * Aplicação configura todos os headers essenciais de segurança.
 * Navegador recebe proteção contra clickjacking, MIME sniffing, XSS e MITM.
 */

function gerarHeadersSeguranca(): array {
    // CORRIGIDO: configura todos os headers de segurança essenciais
    return [
        'X-Content-Type-Options' => 'nosniff', //
        // Previne MIME sniffing
        'X-Frame-Options' => 'DENY', //
        // Previne clickjacking
        'Content-Security-Policy' => "default-src 'self'", //
        // Previne injeção de conteúdo
        'Strict-Transport-Security' => 'max-age=63072000; includeSubDomains', //
        // Força HTTPS
    ];
}

function demo(): void {

```

```

echo "=== CORRIGIDO: Headers de segurança configurados ===\n";

$headers = gerarHeadersSeguranca();
if (!empty($headers)) {
    echo "Headers de segurança configurados:\n";
    foreach ($headers as $nome => $valor) {
        echo "    {$nome}: {$valor}\n";
    }
    echo "\nAplicação protegida contra ataques comuns (clickjacking, MIME
sniffing, XSS)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção define headers essenciais: `X-Content-Type-Options: nosniff`, `X-Frame-Options: DENY`, `Content-Security-Policy: default-src 'self'`, `Strict-Transport-Security: max-age=63072000`. Cada um defende um tipo de ataque. Adicionar esses headers deve ser automático em qualquer novo projeto PHP.

3.4.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A02-configuracao-incorreta/04-headers-seguranca-ausentes/teste.php`. Verifica que o código vulnerável não retorna headers (bug), e o corrigido retorna todos os essenciais (seguro). A saída termina em `RESULTADO: N/N verificacoes passaram`.

3.4.2. Checklist de verificação

- ☐ Toda resposta HTTP inclui `X-Content-Type-Options: nosniff`?
- ☐ Toda resposta inclui `X-Frame-Options: DENY` ou `X-Frame-Options: SAMEORIGIN` (para impedir clickjacking)?
- ☐ Um `Content-Security-Policy` adequado está configurado (pelo menos `default-src 'self'`)?
- ☐ Se o site usa HTTPS (como deveria), `Strict-Transport-Security` está definido com `max-age` apropriado?
- ☐ Testes verificam que os headers aparecem em todas as respostas?

3.5. 05 — XXE (XML External Entity)

CWE-611: Improper Restriction of XML External Entity Reference

Um aplicativo aceita XML de usuários e o processa com `simplexml_load_string()` ou `DOMDocument`. Um atacante envia XML com declarações de entidades externas que fazem o parser tentar ler arquivos locais (`file:///etc/passwd`) ou acessar recursos internos via SSRF. Algumas versões antigas do libxml expandem essas entidades por padrão (XXE clássico); outras causam negação de serviço

(billion laughs, XML bomb). A defesa é dupla: rejeitar XML com DOCTYPE/ENTITY antes do parse (whitelisting), e configurar o parser para desabilitar acesso à rede/arquivos.

Código vulnerável ([A02-configuracao-incorreta/05-xxe-xml-externo/vulneravel.php](#))

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * XXE (XML External Entity) - A02:2025 Security Misconfiguration
 *
 * Parser XML processa entidades externas sem restrições.
 * Atacante pode exfiltrar arquivos locais, realizar SSRF ou negação de serviço.
 */

function analisarXmlDeUsuario(string $xmlString): ?string {
    // VULNERÁVEL: processa XML diretamente sem validação prévia de declarações
    perigosas
    try {
        $xml = simplexml_load_string($xmlString, 'SimpleXMLElement', LIBXML_NOENT);
        if ($xml === false) {
            return null;
        }
        // Retorna o texto do elemento raiz
        return (string) $xml;
    } catch (\Throwable $e) {
        return null;
    }
}

function demo(): void {
    echo "=== VULNERÁVEL: XXE via entidades externas ===\n";

    // XML malicioso com XXE
    $xmlMalicioso = '<?xml version="1.0"?><!DOCTYPE foo [<!ENTITY xxe SYSTEM
"file:///etc/hostname">]><root>$xxe;</root>';

    $resultado = analisarXmlDeUsuario($xmlMalicioso);
    if ($resultado !== null) {
        echo "XML malicioso processado (PROBLEMA!)\n";
        echo "Resultado: {$resultado}\n";
    }

    // XML legítimo
    $xmlLegitimo = '<root>dados sensíveis</root>';
    $resultadoLegitimo = analisarXmlDeUsuario($xmlLegitimo);
    if ($resultadoLegitimo !== null) {
        echo "\nXML legítimo processado (esperado): {$resultadoLegitimo}\n";
    }
}
```

```

}

if (debug_backtrace() === []) {
    demo();
}

```

O código chama `simplexml_load_string($xmlString, 'SimpleXMLElement', LIBXML_NOENT)` — a flag `LIBXML_NOENT` ativa expansão de entidades. Um atacante envia XML com DOCTYPE/ENTITY maliciosos (por exemplo, uma entidade que aponta para `file:///etc/hostname`), e o parser tenta ler o arquivo.

Código corrigido (A02-configuracao-incorreta/05-xxe-xml-externo/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Validar XML e Desabilitar Entidades Externas
 *
 * XML é validado antes do parse para detectar declarações perigosas.
 * Parser é configurado com LIBXML_NONET para desabilitar acesso à rede e arquivos.
 */

function analisarXmlDeUsuario(string $xmlString): ?string {
    // CORRIGIDO: valida XML antes de processar para detectar XXE

    // Detecção de padrões perigosos: DOCTYPE e ENTITY
    if (str_contains(strtoupper($xmlString), '<!DOCTYPE') ||
        str_contains(strtoupper($xmlString), '<!ENTITY')) {
        // XML contém declarações perigosas, rejeita
        return null;
    }

    try {
        // Parse XML com LIBXML_NONET (sem LIBXML_NOENT para evitar expansão de
        entidades)
        $xml = simplexml_load_string($xmlString, 'SimpleXMLElement', LIBXML_NONET);
        if ($xml === false) {
            return null;
        }
        // Retorna o texto do elemento raiz
        return (string) $xml;
    } catch (\Throwable $e) {
        return null;
    }
}

function demo(): void {

```

```

echo "=== CORRIGIDO: XXE bloqueado por validação prévia ===\n";

// XML malicioso com XXE (bloqueado)
$xmlMalicioso = '<?xml version="1.0"?><!DOCTYPE foo [<!ENTITY xxe SYSTEM
"file:///etc/hostname">]><root>&xxe;</root>';

$resultado = analisarXmlDeUsuario($xmlMalicioso);
if ($resultado === null) {
    echo "XML malicioso rejeitado (bloqueado!)\n";
}

// XML legítimo (processado normalmente)
$xmlLegitimo = '<root>dados sensíveis</root>';
$resultadoLegitimo = analisarXmlDeUsuario($xmlLegitimo);
if ($resultadoLegitimo !== null) {
    echo "XML legítimo processado com segurança: {$resultadoLegitimo}\n";
}
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção opera em duas camadas. Primeira, valida: rejeita o XML se contiver `<!DOCTYPE` ou `<!ENTITY` em maiúsculas (como forma de defesa em profundidade). Segunda, configura o parser com `LIBXML_NONET` (sem acesso à rede/arquivos) em vez de `LIBXML_NOENT`. Nunca processe XML não validado de usuários sem essas proteções.

3.5.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A02-configuracao-incorreta/05-xxe-xml-externo/teste.php`. Envia XML com XXE payload. O código vulnerável o processa (bug), o corrigido o rejeita (seguro). XML legítimo continua funcionando. A saída termina em `RESULTADO: N/N verificacoes passaram`.

3.5.2. Checklist de verificação

- ☐ XML vindo de usuários é rejeitado se contiver `<!DOCTYPE` ou `<!ENTITY` antes do parse?
- ☐ `simplexml_load_string()` ou `DOMDocument` nunca usa `LIBXML_NOENT`?
- ☐ Parser está configurado com `LIBXML_NONET` para desabilitar acesso à rede?
- ☐ Testes incluem payloads XXE clássicos (exfiltração de arquivo, billion laughs) e verificam rejeição?

3.6. 06 — Listagem de Diretório

CWE-548: Exposure of Information Through Directory Listing

Um servidor web está configurado para listar arquivos se não houver `index.html/index.php`. Um

atacante acessa `/uploads/` ou `/config/` e vê: `.env`, `config.php.bak`, `backup.sql`, `private_key.pem`. Toda a infraestrutura exposta em texto claro. A correção é uma linha de configuração: desabilitar listagem de diretório no web server, ou retornar 403 se não houver index. Na verdade, a melhor prática é nunca armazenar arquivos sensíveis em diretórios web acessíveis; mantê-los fora da raiz web.

Código vulnerável ([A02-configuracao-incorreta/06-listagem-diretorio/vulneravel.php](#))

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Listagem de Diretórios Habilitada - A02:2025 Security Misconfiguration
 *
 * Servidor web lista todos os arquivos do diretório quando não há index.
 * Atacante descobre arquivos de backup, configuração, credenciais ou fontes
 * sensíveis.
 */

function tratarRequisicaoArquivo(string $caminhoSolicitado, array
$arquivosNoDiretorio, bool $temIndex): array {
    // Simula acesso a um diretório (caminho termina com /)
    if (!$temIndex) {
        // VULNERÁVEL: lista todos os arquivos do diretório, incluindo sensíveis
        return [
            'status' => 200,
            'corpo' => implode(' ', $arquivosNoDiretorio),
        ];
    }

    return [
        'status' => 200,
        'corpo' => 'Conteúdo de index.html',
    ];
}

function demo(): void {
    echo "=== VULNERÁVEL: Listagem de diretório expõe arquivos sensíveis ===\n";

    $arquivos = ['index.html', '.env', 'config.php.bak', 'backup.sql'];
    $resposta = tratarRequisicaoArquivo('/uploads/', $arquivos, false);

    echo "Status: {$resposta['status']}\n";
    echo "Corpo: {$resposta['corpo']}\n";
    if (str_contains($resposta['corpo'], '.env')) {
        echo "Arquivo .env exposto (PROBLEMA!)\n";
    }
}

if (debug_backtrace() === []) {
```

```
demo();  
}
```

Se `$temIndex` é false, o servidor retorna `implode(', ', $arquivosNoDiretorio)` — lista todos os arquivos incluindo `.env`, `config.php.bak`, `backup.sql`. Um atacante descobre toda a estrutura.

Código corrigido (A02-configuracao-incorreta/06-listagem-diretorio/corrigido.php)

```
<?php  
declare(strict_types=1);  
  
namespace Corrigido;  
  
/**  
 * Correção: Desabilitar Listagem de Diretórios  
 *  
 * Diretórios sem index retornam 403 Forbidden.  
 * Usuários precisam conhecer URLs exatas para acessar recursos.  
 */  
  
function tratarRequisicaoArquivo(string $caminhoSolicitado, array  
$arquivosNoDiretorio, bool $temIndex): array {  
    // Simula acesso a um diretório (caminho termina com /)  
    if (!$temIndex) {  
        // CORRIGIDO: bloqueia listagem de diretório, retorna 403  
        return [  
            'status' => 403,  
            'corpo' => 'Acesso negado. Listagem de diretórios não é permitida.',  
        ];  
    }  
  
    return [  
        'status' => 200,  
        'corpo' => 'Conteúdo de index.html',  
    ];  
}  
  
function demo(): void {  
    echo "=== CORRIGIDO: Listagem de diretório bloqueada ===\n";  
  
    $arquivos = ['index.html', '.env', 'config.php.bak', 'backup.sql'];  
    $resposta = tratarRequisicaoArquivo('/uploads/', $arquivos, false);  
  
    echo "Status: {$resposta['status']}\n";  
    echo "Corpo: {$resposta['corpo']}\n";  
    if ($resposta['status'] === 403) {  
        echo "Listagem de diretório bloqueada (protegido!)\n";  
    }  
  
    // Com index.html presente
```

```

$respostaComIndex = tratarRequisicaoArquivo('/uploads/', $arquivos, true);
echo "\nCom index.html:\n";
echo "Status: {$respostaComIndex['status']}\n";
echo "Corpo: {$respostaComIndex['corpo']}\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção retorna 403 Forbidden se não houver index. Usuários precisam conhecer a URL exata de um arquivo para acessá-lo; não podem listar. No Apache/Nginx real, configure `Options -Indexes` (Apache) ou `autoindex off;` (Nginx).

3.6.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A02-configuracao-incorreta/06-listagem-diretorio/teste.php`. Acessa um diretório sem index. O código vulnerável lista os arquivos (bug), o corrigido retorna 403 (seguro). A saída termina em **RESULTADO: N/N verificacoes passaram.**

3.6.2. Checklist de verificação

- ☐ Listagem de diretório está desativada no web server (Apache: `Options -Indexes`, Nginx: `autoindex off;`)?
- ☐ Acessar um diretório sem `index.html/index.php` retorna 403 Forbidden, não lista arquivos?
- ☐ Arquivos sensíveis (.env, config, backup, private keys) nunca estão em diretórios web acessíveis?
- ☐ Testes de segurança tentam acessar diretórios sem index e verificam 403?

3.7. Checklist consolidado — Configuração Incorreta de Segurança

Este checklist reunido é para usar como referência rápida em code review, auditorias de segurança e checklists de deployment:

- ☐ Nenhuma credencial padrão/hardcoded está no código-fonte; senhas são de config externa ou banco de dados?
- ☐ Endpoints de debug, profiler, logs, console não existem (404) em produção?
- ☐ Mensagens de erro em produção são genéricas; stack traces são logados apenas no servidor?
- ☐ Headers de segurança essenciais são enviados: X-Content-Type-Options, X-Frame-Options, CSP, HSTS?
- ☐ XML de usuários é validado (rejeita DOCTYPE/ENTITY) e parsed com LIBXML_NONET?
- ☐ Listagem de diretório está desativada; diretórios sem index retornam 403?

- ☐ Variáveis de ambiente (APP_ENV, DEBUG, etc.) determinam comportamento, não hardcoding?
- ☐ Checklist de segurança pré-deployment é documentado e seguido (revisão de configuração, secrets, headers, debug)?
- ☐ Arquivos sensíveis (.env, config, keys, backups) estão fora da raiz web e não commitados no repo?
- ☐ Logs de erro são armazenados seguramente, acessíveis apenas ao time de operações?

Chapter 4. A03:2025 — Falhas na Cadeia de Suprimentos de Software

[Falhas na Cadeia de Suprimentos](#) é uma categoria relativamente nova no Top 10 que reflete uma realidade moderna: quase todo software PHP depende de bibliotecas externas gerenciadas via Composer. Se um atacante compromete uma biblioteca popular, todos os projetos que a usam herdam a vulnerabilidade. Mesmo sem comprometimento, muitos projetos dependem de versões antigas com CVEs conhecidos, componentes abandonados (que nunca receberão patches), ou instalam de repositórios não-oficiais. Além disso, a ferramenta Composer permite que plugins rodem código arbitrário durante a instalação — se não controlado, qualquer dependência pode executar malware. O impacto é catastrófico: vulnerabilidades em larga escala, propagadas a milhares de aplicações de uma vez.

Diferente dos capítulos anteriores, este documenta **escolhas de dependências no composer.json**, não código PHP vulnerável em runtime. A primeira metade dos exemplos (01 a 05) compara pares `composer.vulneravel.json` / `composer.corrigido.json`. O teste verifica se o Composer está apontando pra versões seguras. O sexto exemplo (06) volta a código PHP, usando a função obsoleta `extract()` — um exemplo de como bibliotecas antigas podem expor padrões perigosos que afetam segurança. Leia também o arquivo [A03-falhas-cadeia-suprimentos/composer-audit-real.md](#) neste projeto para instruções sobre como rodar `composer audit` de verdade contra o Packagist oficial.

- **01 — Versão Vulnerável Conhecida:** usar versão com CVE registrado
- **02 — Componente Não Mantido:** depender de biblioteca abandonada pelo autor
- **03 — Fonte Não Confiável:** instalar de repositório HTTP não-oficial
- **04 — Scripts de Instalação Não Confiáveis:** permitir qualquer plugin rodar código
- **05 — Dependência Sem Versão Fixada:** constraint "*" permite qualquer versão
- **06 — Função Obsoleta:** usar `extract()` sem controle, causando variable overwrite

4.1. 01 — Versão Vulnerável Conhecida

CWE-1035: Use of a Broken or Risky Cryptographic Algorithm; CWE-1395: Dependency on Vulnerable Third-Party Component

Uma biblioteca popular (tipo `monolog/monolog` ou `symfony/http-kernel`) lança versão 1.2.0 com uma vulnerabilidade crítica. Um CVE é registrado: CVE-2024-12345. Seis meses depois, versão 1.2.3 é lançada com o patch. Um desenvolvedor usa `composer require monolog/monolog:1.2.0` e nunca atualiza — o aplicativo fica com uma dependência conhecida vulnerável até que um atacante explore a falha. A lição: nunca fixar uma versão absoluta; usar ranges semânticos (`^1.2.3`) que permitem patches, ou auditar regularmente com `composer audit`.

Código vulnerável ([A03-falhas-cadeia-suprimentos/01-versao-vulneravel-conhecida/composer.vulneravel.json](#))

```
{  
  "name": "exemplo/app-vulneravel",
```

```

    "description": "Aplicação com dependência em versão conhecida vulnerável",
    "type": "project",
    "require": {
        "php": ">=8.0",
        "acme/pagamentos": "1.2.0"
    },
    "minimum-stability": "stable"
}

```

O arquivo fixa `"acme/pagamentos": "1.2.0"` — uma versão fictícia que o teste marca como contendo CVE-2024-12345 (fictício). O aplicativo está vulnerável até que a dependência seja atualizada.

Código corrigido (A03-falhas-cadeia-suprimentos/01-versao-vulneravel-conhecida/composer.corrigido.json)

```

{
    "name": "exemplo/app-corrigido",
    "description": "Aplicação com dependência em versão segura e atualizada",
    "type": "project",
    "require": {
        "php": ">=8.0",
        "acme/pagamentos": "^1.5"
    },
    "minimum-stability": "stable"
}

```

O arquivo usa `"acme/pagamentos": "^1.5"` — uma versão semântica que inclui 1.5.0 e patches futuros (1.5.1, 1.5.2, etc.), mas não mudanças maiores que poderiam quebrar compatibilidade. Isso permite que `composer update` traga patches de segurança automaticamente. Ou, alternativamente, sempre rodar `composer audit` como parte da CI/CD.

4.1.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A03-falhas-cadeia-suprimentos/01-versao-vulneravel-conhecida/teste.php`. O teste carrega os dois `composer.json`, extrai a versão de `acme/pagamentos`, e compara contra uma base de dados fictícia de versões conhecidas vulneráveis. O arquivo vulnerável usa versão 1.2.0 (que está na lista de CVEs), o corrigido usa 1.5 (fora da lista). O teste verifica que o vulnerável contém CVE (bug), o corrigido não (seguro). A saída termina em `RESULTADO: N/N verificacoes passaram`.

4.1.2. Checklist de verificação

- ☐ Todas as versões de dependências são verificadas regularmente com `composer audit` (como parte de CI/CD)?
- ☐ Versões fixas absolutas ("`1.2.0`") são evitadas em favor de ranges semânticos ("`^1.2`")?
- ☐ Se uma CVE é descoberta em uma dependência, a versão é atualizada ou removida em até 24 horas?

- Há um processo de monitoramento de novas CVEs (GitHub Dependabot, Snyk, ou manual via composer audit)?

4.2. 02 — Componente Não Mantido

CWE-1104: Use of Unmaintained Third Party Components

Um desenvolvedor depende de uma biblioteca que era popular 5 anos atrás. O autor original parou de manter, ninguém mais atualizou, o repositório está congelado. Novos CVEs surgem — nunca serão patcheados. Um atacante explora uma falha conhecida na biblioteca, sabendo que está presente em milhares de projetos que ainda dependem dela. A solução é: monitorar se uma dependência continua mantida (busque commits recentes no GitHub, issues sendo respondidas, releases regulares), e migrar para alternativas mantidas quando uma dependência morre.

Código vulnerável (A03-falhas-cadeia-suprimentos/02-componente-nao-mantido/composer.vulneravel.json)

```
{
  "name": "exemplo/app-com-legado",
  "description": "Aplicação usando componente abandonado/não mantido",
  "type": "project",
  "require": {
    "php": ">=8.0",
    "acme/log-legado": "^2.0"
  },
  "minimum-stability": "stable"
}
```

O arquivo usa `"acme/log-legado": "^2.0"` — uma biblioteca fictícia marcada como abandonada (sem commits em 2+ anos, issues não respondidas). O teste detecta como não-mantida.

Código corrigido (A03-falhas-cadeia-suprimentos/02-componente-nao-mantido/composer.corrigido.json)

```
{
  "name": "exemplo/app-com-dependencias-ativas",
  "description": "Aplicação usando componentes ativamente mantidos",
  "type": "project",
  "require": {
    "php": ">=8.0",
    "acme/log-moderno": "^1.0"
  },
  "minimum-stability": "stable"
}
```

O arquivo migra para `"acme/log-moderno": "^1.0"` — uma biblioteca fictícia ativamente mantida (commits regulares, releases, maintainer responsivo). O teste valida como mantida.

4.2.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A03-falhas-cadeia-`

`suprimentos/02-componente-nao-mantido/teste.php`. O teste marca pacotes como "mantidos" ou "abandonados" conforme uma lista fixa. O arquivo vulnerável lista uma biblioteca abandonada (bug), o corrigido lista uma mantida (seguro). A saída termina em **RESULTADO: N/N verificacoes passaram**.

4.2.2. Checklist de verificação

- ☐ Antes de adicionar uma dependência, verifica-se se o projeto está ativo (commits recentes, issues respondidas)?
- ☐ Há um processo para detectar e remover/substituir dependências que não são mais mantidas?
- ☐ O GitHub Dependabot ou equivalente notifica sobre dependências deprecated?
- ☐ Regularmente, audit tools checam não apenas CVEs mas também status de manutenção?

4.3. 03 — Fonte Não Confiável

CWE-1357: Reliance on Untrusted Source in Build System

Um desenvolvedor configura o Composer para instalar pacotes de um repositório customizado (`http://my-mirror.empresa.com`) que ele controla, ou desabilita completamente o Packagist (repositório oficial, auditado pela comunidade). Depois, um atacante compromete a máquina que roda o Composer, ou intercepta a requisição HTTP (se não for HTTPS), e injeta um pacote malicioso. O código malicioso roda durante `composer install` (em um script) e pode exfiltrar dados, inserir backdoor, ou modificar o código do projeto. A regra: **sempre instale do Packagist (HTTPS oficial) por padrão**; apenas use repositórios customizados se absolutamente necessário, e sempre via HTTPS com verificação de certificado.

Código vulnerável (A03-falhas-cadeia-suprimentos/03-fonte-nao-confiavel/composer.vulneravel.json)

```
{
  "name": "exemplo/app-com-fonte-nao-segura",
  "description": "Aplicação instalando dependências de fonte não confiável",
  "type": "project",
  "repositories": [
    {
      "type": "composer",
      "url": "http://pacotes-nao-oficiais.exemplo.com"
    },
    {
      "packagist.org": false
    }
  ],
  "require": {
    "php": ">=8.0",
    "acme/pagamentos": "^1.5"
  },
  "minimum-stability": "stable"
}
```

O arquivo desabilita o Packagist ("`packagist.org`": `false`) e usa um repositório HTTP (não HTTPS) não-oficial. Um atacante na rede pode interceptar e servir pacotes maliciosos.

Código corrigido (A03-falhas-cadeia-suprimentos/03-fonte-nao-confiavel/composer.corrigido.json)

```
{
  "name": "exemplo/app-com-fonte-confiavel",
  "description": "Aplicação usando apenas fontes oficiais e seguras (HTTPS)",
  "type": "project",
  "require": {
    "php": ">=8.0",
    "acme/pagamentos": "^1.5"
  },
  "minimum-stability": "stable"
}
```

O arquivo usa apenas o Packagist oficial (sem configuração de `repositories`), que é HTTPS e auditado pela comunidade. Se repositórios adicionais forem necessários, devem ser HTTPS e explicitamente revisados.

4.3.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A03-falhas-cadeia-suprimentos/03-fonte-nao-confiavel/teste.php`. Valida se o `repositories` está configurado. O arquivo vulnerável define repositório não-oficial (bug), o corrigido usa só o Packagist (seguro). A saída termina em `RESULTADO: N/N verificacoes passaram`.

4.3.2. Checklist de verificação

- ☐ Composer instala do Packagist oficial por padrão (<https://packagist.org>)?
- ☐ Se repositórios customizados são configurados, são HTTPS, auditados, e documentados?
- ☐ Packagist não é desabilitado ("`packagist.org`": `false` não aparece)?
- ☐ Testes de CI/CD verificam se o `composer.lock` está limpo (nenhum repositório HTTP)?

4.4. 04 — Scripts de Instalação Não Confiáveis

CWE-426: Untrusted Search Path; CWE-427: Uncontrolled Search Path Element

O Composer permite que pacotes definam scripts (callbacks) que rodam durante `composer install`, `composer update`, etc. Por exemplo, uma dependência pode executar `php -r "system('whoami');"` durante a instalação. Se `"allow-plugins": true` está no `composer.json`, **qualquer** dependência pode rodar qualquer código — transformando uma instalação segura num perigo potencial de code execution. Um atacante compromete uma biblioteca popular e adiciona um script que extrai `.env`, credenciais, chaves SSH, etc. A solução é usar `"allow-plugins"` com uma `allow-list` específica: `"allow-plugins": {"trusted/plugin": true}` enumera apenas os plugins confiáveis.

Código vulnerável (A03-falhas-cadeia-suprimentos/04-scripts-instalacao-nao-confiaveis/composer.vulneravel.json)

```
{
  "name": "exemplo/app-com-plugins-nao-controlados",
  "description": "Aplicação que permite QUALQUER plugin rodar código na instalação",
  "type": "project",
  "require": {
    "php": ">=8.0",
    "acme/pagamentos": "^1.5"
  },
  "config": {
    "allow-plugins": true
  },
  "minimum-stability": "stable"
}
```

O arquivo define `"allow-plugins": true` — permite que qualquer dependência (atual ou futura) execute scripts durante instalação. Uma dependência maliciosa roda código arbitrário.

Código corrigido (A03-falhas-cadeia-suprimentos/04-scripts-instalacao-nao-confiaveis/composer.corrigido.json)

```
{
  "name": "exemplo/app-com-plugins-controlados",
  "description": "Aplicação com allow-list explícita de plugins confiáveis",
  "type": "project",
  "require": {
    "php": ">=8.0",
    "acme/pagamentos": "^1.5"
  },
  "config": {
    "allow-plugins": {
      "acme/plugin-oficial": true
    }
  },
  "minimum-stability": "stable"
}
```

O arquivo usa uma allow-list específica: `"allow-plugins": {"trusted/installer": true, "phpstan/phpstan": true}`. Apenas esses plugins confere permissão de executar scripts. Qualquer outro plugin que tentar rodar um script sem estar na lista é bloqueado, e `composer install` falha — sinalizando o problema.

4.4.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A03-falhas-cadeia-suprimentos/04-scripts-instalacao-nao-confiaveis/teste.php`. Valida se `allow-plugins` é `true` (inseguro) ou uma allow-list (seguro). O arquivo vulnerável tem `true` (bug), o corrigido tem uma

lista específica (seguro). A saída termina em **RESULTADO: N/N verificacoes passaram.**

4.4.2. Checklist de verificação

- ☐ `"allow-plugins"` nunca é definido como `true` no `composer.json`?
- ☐ `"allow-plugins"` é uma allow-list explícita de plugins confiáveis, ex.: `{"trusted/plugin": true}`?
- ☐ Quando um novo plugin é adicionado como dependência, é revisto e explicitamente permitido?
- ☐ Testes de CI/CD falham se `"allow-plugins": true` está presente?

4.5. 05 — Dependência Sem Versão Fixada

CWE-1329: Pinning of Deprecated or Obsolete Components

Um `composer.json` define `"symfony/console": "*" — qualquer versão, de 1.0 até a mais nova. Quando alguém roda composer install em janeiro, pega versão 5.4.0; em fevereiro, pega 6.0.0 que quebrou compatibilidade. Pior: se versão 7.0.0 tem uma CVE crítica descoberta, e alguém roda composer update esperando patches normais, pega 7.0.0 vulnerável. Isso torna builds não-determinísticos e aumenta risco de incluir vulnerabilidades por acaso. A solução: sempre usar range semântico (^5.4), ou ainda melhor, comitar composer.lock que trava exatamente quais versões foram instaladas.`

Código vulnerável (A03-falhas-cadeia-suprimentos/05-dependencia-sem-versao-fixada/composer.vulneravel.json)

```
{
  "name": "exemplo/app-com-versao-aberta",
  "description": "Aplicação com constraint de versão totalmente aberta (não determinística)",
  "type": "project",
  "require": {
    "php": ">=8.0",
    "acme/pagamentos": "*"
  },
  "minimum-stability": "dev"
}
```

O arquivo usa `"acme/pagamentos": "*" — qualquer versão. Cada composer install pode trazer uma versão diferente e potencialmente vulnerável.`

Código corrigido (A03-falhas-cadeia-suprimentos/05-dependencia-sem-versao-fixada/composer.corrigido.json)

```
{
  "name": "exemplo/app-com-versao-fixada",
  "description": "Aplicação com constraint de versão semântica específica",
  "type": "project",
  "require": {
    "php": ">=8.0",
```

```

    "acme/pagamentos": "^1.5"
  },
  "minimum-stability": "stable"
}

```

O arquivo usa `"acme/pagamentos": "^1.5"` — versão semântica. Permite patches (1.5.1, 1.5.2) e minor updates (1.6.0), mas não major changes que quebram compatibilidade. Builds agora são determinísticos.

4.5.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A03-falhas-cadeia-suprimentos/05-dependencia-sem-versao-fixada/teste.php`. Valida se constraint é "" (inseguro) ou semântico (seguro). O arquivo vulnerável usa "" (bug), o corrigido usa `"^1.5"` (seguro). A saída termina em `RESULTADO: N/N verificacoes passaram`.

4.5.2. Checklist de verificação

- ☐ Nenhuma dependência usa constraint "*" ou ">=0" (sem limite superior)?
- ☐ Todas as dependências usam range semântico ("^X.Y", "~X.Y") ou versão específica?
- ☐ `composer.lock` está commitado no repositório (trava versões exatas)?
- ☐ Testes de CI/CD falham se `composer.json` contém "*" como constraint?

4.6. 06 — Função Obsoleta e Perigosa

CWE-446: Uncontrolled Modification of Free Variables; CWE-447: Use of Obsolete/Dangerous Function

A função `extract()` do PHP é mantida por compatibilidade mas é marcada como perigosa em qualquer documentação de segurança. Ela pega um array associativo e cria variáveis locais com seus chaves/valores. Se um atacante controla o array (tipo via `parse_str()` de uma query string), pode sobrescrever variáveis locais importantes — tipo `$isAdmin`, `$userId`, `$saldo`. Isso é "variable injection" clássico. Bibliotecas antigas ou tutoriais antigos usam `extract()` sem controle de prefixo, e aplicações que as utilizam herdam o risco. A correção é simples: **nunca use `extract()` sem `EXTR_PREFIX_ALL` ou equivalente**, ou melhor ainda, acesse os valores do array explicitamente sem `extract()`.

Código vulnerável (A03-falhas-cadeia-suprimentos/06-funcao-obsoleta/vulneravel.php)

```

<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * CWE-446: Uncontrolled Modification of Free Variables
 * CWE-447: Use of Obsolete/Dangerous Function
 * A03:2025 - Software Supply Chain Failures

```

```

*
* A função extract() é mantida em PHP por compatibilidade, mas é
* marcada como perigosa (deprecated em documentações de segurança).
*
* Quando usada sem EXTR_PREFIX_ALL ou similar, extract() sobrescreve
* variáveis locais com valores de um array externo, causando:
* 1. Variable Injection / Variable Overwrite
* 2. Possibilidade de sobrescrever variáveis locais importantes
* 3. Vulnerabilidades graves de Access Control bypass
*
* Exemplo: extract(['isAdmin' => 1]) sobrescreve a variável $isAdmin
* local, causando um bypass de controle de acesso!
*/

function verificarAcesso(string $queryString): bool {
    $isAdmin = false;

    // VULNERÁVEL: extract() sem controle de prefixo
    // Extrai todas as variáveis do array diretamente no escopo local
    // Se queryString contém "isAdmin=1", parse_str cria um array
    // e extract o sobrescreve na variável local!
    parse_str($queryString, $dados);
    extract($dados); // PERIGOSO: sobrescreve variáveis locais sem controle

    // Note: $isAdmin pode ter sido sobrescrita por extract() com um valor string!
    // A variável $isAdmin não é mais bool false, mas string "1" ou outro valor
    return (bool) $isAdmin;
}

function demo(): void {
    echo "=== VULNERÁVEL: Função perigosa extract() com variable overwrite ===\n";

    // Simulando um usuário não-admin
    $resultado = verificarAcesso("isAdmin=1");

    if ($resultado) {
        echo "PROBLEMA! Usuário conseguiu se elevar para admin via extract
injection!\n";
    } else {
        echo "Acesso como não-admin (esperado)\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A função `verificarAcesso()` chama `extract($dados)` sem nenhum controle. Se a query string contém `isAdmin=1`, `extract()` cria uma variável local `$isAdmin = "1"`, sobrescrevendo a inicialização `$isAdmin = false`. Um atacante se eleva para admin sem validação.

```
<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Não usar extract()
 *
 * A forma corrigida evita extract() completamente.
 * Parse a query string em um array isolado e acesse os valores explicitamente.
 *
 * Isso previne Variable Injection e Variable Overwrite attacks.
 */

function verificarAcesso(string $queryString): bool {
    $isAdmin = false;

    // CORRIGIDO: Não usar extract()
    // A query string é parseada em um array isolado $dados
    parse_str($queryString, $dados);

    // Acesso explícito e seguro aos valores
    $parametroIsAdmin = $dados['isAdmin'] ?? null;

    // A variável local $isAdmin não foi modificada, então permanece false
    return $isAdmin;
}

function demo(): void {
    echo "=== CORRIGIDO: Sem extract(), acesso explícito aos dados ===\n";

    $resultado = verificarAcesso("isAdmin=1");

    if ($resultado) {
        echo "Usuário se elevou para admin\n";
    } else {
        echo "Acesso bloqueado (seguro!). Usuário permanece não-admin\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}
```

A correção **não usa** `extract()`. Ao invés disso, após `parse_str()` criar o array `$dados`, acessa explicitamente os valores: `$parametroIsAdmin = $dados['isAdmin'] ?? null;`. A variável local `$isAdmin` permanece `false` porque foi inicializada e nunca sobrescrita. Um atacante não consegue injetar variáveis.

4.6.1. Como testar

Execute `php /Users/uiraribeiro/Documents/CURSO_OWASP/owasp-php-examples/A03-falhas-cadeia-suprimentos/06-funcao-obsoleta/teste.php`. Tenta explorar variable injection enviando `isAdmin=1`. O código vulnerável é explorado (bug), o corrigido bloqueia (seguro). A saída termina em **RESULTADO: N/N verificacoes passaram**.

4.6.2. Checklist de verificação

- ☐ `extract()` nunca é usado no código próprio, e bibliotecas que o usam (sem prefixo) são removidas ou patched?
- ☐ Se uma biblioteca usa `extract()`, é feita uma code review adicional pra entender qual escopo é afetado?
- ☐ Dados de entrada (query strings, POST) nunca são diretamente extraídos sem validação explícita?
- ☐ Testes incluem tentar injetar variáveis via `extract()` e verificam que são ignoradas?

4.7. Checklist consolidado — Falhas na Cadeia de Suprimentos

Este checklist reunido é para usar em auditorias de dependências e segurança de supply chain:

- ☐ Todas as versões de dependências são verificadas regularmente com `composer audit` (como parte de CI/CD)?
- ☐ Nenhuma dependência usa versão com CVE conhecido; versões vulneráveis são atualizadas ou removidas em 24 horas?
- ☐ Dependências são verificadas quanto a status de manutenção; componentes abandonados são removidos ou substituídos?
- ☐ Composer instala apenas do Packagist oficial (HTTPS); repositórios customizados são HTTPS e explicitamente auditados?
- ☐ `"allow-plugins"` é uma allow-list específica, nunca `true`; apenas plugins confiáveis podem rodar scripts?
- ☐ Nenhuma dependência usa constraint `"*"`; todas usam `range semântico` (`^X.Y`) ou versão específica?
- ☐ `composer.lock` está commitado e é usado para builds determinísticos?
- ☐ Funções perigosas (`extract()` sem controle) são evitadas no código próprio e em bibliotecas?
- ☐ Há um processo de monitoramento de CVEs em dependências (Dependabot, Snyk, ou manual via `composer audit`)?
- ☐ Antes de adicionar uma nova dependência, há revisão de confiabilidade (maintainer, comunidade, histórico)?

Chapter 5. A04:2025 — Falhas Criptográficas

Falhas criptográficas englobam qualquer uso inadequado de criptografia, hashing ou geração de números aleatórios que compromete a confidencialidade, integridade ou autenticidade de dados sensíveis. Essa categoria entrou no [OWASP Top 10:2025](#) porque muitas aplicações ainda cometem erros fundamentais: usar algoritmos desatualizados, reutilizar IVs, hardcodear chaves no código-fonte, transmitir credenciais em HTTP, ou confiar em geradores de números aleatórios criptograficamente inseguros. A maioria desses problemas é evitável com conhecimento básico e o uso correto das APIs de criptografia nativas do PHP — mas o descuido persiste porque parecem "detalhes" até o dia em que a brecha é explorada.

Esta seção cobre 8 exemplos concretos:

- **01 — Algoritmos fracos:** Substituindo cifras desatualizadas (RC4, DES) por AES-256-GCM
- **02 — Aleatoriedade insuficiente:** De `mt_rand()` para `random_bytes()` em tokens
- **03 — Gestão de chaves:** Removendo hardcoding, usando variáveis de ambiente
- **04 — Transporte em texto claro:** De HTTP + URL para HTTPS + corpo POST
- **05 — Hash sem salt:** De MD5 puro para `password_hash()` com bcrypt
- **06 — IV problemático:** De IV fixo para IV aleatório a cada criptografia
- **07 — Certificado não validado:** Ativando validação TLS em requisições cURL
- **08 — Modo de criptografia inadequado:** De ECB para GCM

5.1. 01 — Algoritmos Fracos

CWE-327: Use of a Broken or Risky Cryptographic Algorithm

Algoritmos criptográficos como RC4 e DES foram quebrados ou ficaram obsoletos pela evolução da capacidade computacional. RC4, por exemplo, é determinístico quando reutiliza a mesma chave e IV, produzindo o mesmo criptograma para o mesmo texto plano sempre — uma falha chamada "CBC Penguin Problem" que vaza padrões. DES usa apenas 56 bits de chave (2^{56} combinações), quebrável por força bruta em segundos com hardware moderno. Usar esses algoritmos torna os dados "criptografados" praticamente legíveis para qualquer pessoa disposta a investir alguns milissegundos.

Código vulnerável ([A04-falhas-criptograficas/01-algoritmos-fracos/vulneravel.php](#))

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Uso de algoritmos criptográficos inadequados (DES, modo ECB)
 * OWASP A04:2025 - Cryptographic Failures
 * VULNERÁVEL: DES é um algoritmo de 56-bit considerado inseguro (espaço de chave
 muito pequeno).
```

```
* Além disso, usa ECB (Electronic Code Book) que é determinístico e sem IV,  
* produzindo o mesmo criptograma para o mesmo texto plano a cada execução.  
*/
```

```
function criptografar(string $texto, string $chave): string  
{  
    // NUNCA USE DES EM PRODUÇÃO - É INSEGURO  
    // DES é stream cipher, não requer IV  
    // Normaliza chave para 8 bytes (64 bits) para DES  
    $chaveNorm = substr(hash('sha256', $chave, true), 0, 8);  
    $textoCifrado = openssl_encrypt($texto, 'des-ede3-ecb', $chaveNorm,  
OPENSSL_RAW_DATA);  
    if ($textoCifrado === false) {  
        throw new RuntimeException('Falha ao criptografar com DES');  
    }  
    return $textoCifrado;  
}  
  
function descriptografar(string $textoCifrado, string $chave): string  
{  
    // Decriptação com DES (igualmente fraca)  
    // Normaliza chave para 8 bytes (64 bits) para DES  
    $chaveNorm = substr(hash('sha256', $chave, true), 0, 8);  
    $texto = openssl_decrypt($textoCifrado, 'des-ede3-ecb', $chaveNorm,  
OPENSSL_RAW_DATA);  
    return $texto === false ? '' : $texto;  
}  
  
function demo(): void  
{  
    echo "=== Criptografia VULNERÁVEL (DES-ECB) ===\n";  
    $chave = 'minha-chave-insegura';  
    $texto = 'Dados sensiveis';  
  
    $cifra1 = criptografar($texto, $chave);  
    $cifra2 = criptografar($texto, $chave);  
  
    echo "Texto plano: {$texto}\n";  
    echo "Cifra 1 (hex): " . bin2hex($cifra1) . "\n";  
    echo "Cifra 2 (hex): " . bin2hex($cifra2) . "\n";  
    echo "Cifras sao identicas (PROBLEMA)? " . (bin2hex($cifra1) === bin2hex($cifra2)  
? 'SIM' : 'NAO') . "\n";  
    echo "Decriptado: " . descriptografar($cifra1, $chave) . "\n";  
}  
  
if (debug_backtrace() === []) {  
    demo();  
}
```

O código usa `openssl_encrypt(..., 'des-ede3-ecb', ...)` — DES de 56 bits (mesmo com triple-DES, é

insuficiente hoje), e pior ainda, usa ECB (Electronic Code Book) que não requer IV. O resultado é determinístico: criptografar a mesma mensagem duas vezes produz bytes idênticos, vazando padrões. Se um atacante souber que você criptografa "sim" ou "não" com esta chave, consegue adivinhar qualquer resposta observando o criptograma resultante.

Código corrigido ([A04-falhas-criptograficas/01-algoritmos-fracos/corrigido.php](#))

```
<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Criptografia SEGURA usando AES-256-GCM
 * OWASP A04:2025 - Cryptographic Failures
 * CORRIGIDO: AES-256-GCM é um algoritmo moderno, seguro e autenticado.
 * Usa IV aleatório de 12 bytes a cada criptografia, garantindo
 * criptogramas diferentes para o mesmo texto plano (IV + authentication tag
 * inclusos).
 */

function criptografar(string $texto, string $chave): string
{
    // Normaliza chave para 32 bytes (256 bits)
    $chaveNorm = hash('sha256', $chave, true);

    // Gera IV aleatório de 12 bytes (necessário para GCM)
    $iv = random_bytes(12);

    // Criptografa com AES-256-GCM usando OPENSSL_RAW_DATA
    $tag = '';
    $textoCifrado = openssl_encrypt($texto, 'aes-256-gcm', $chaveNorm,
    OPENSSL_RAW_DATA, $iv, $tag);

    // Retorna: IV + TAG + CRIPTOGRAMA
    // Tag é necessária para autenticação e detecção de tampering
    return $iv . $tag . $textoCifrado;
}

function descriptografar(string $resultado, string $chave): string
{
    // Normaliza chave para 32 bytes (256 bits)
    $chaveNorm = hash('sha256', $chave, true);

    // Extrai componentes do resultado concatenado
    $ivSize = 12; // 12 bytes
    $tagSize = 16; // 16 bytes (para AES-GCM)

    if (strlen($resultado) < ($ivSize + $tagSize)) {
        return '';
    }
}
```

```

$iv = substr($resultado, 0, $ivSize);
$tag = substr($resultado, $ivSize, $tagSize);
$textoCifrado = substr($resultado, $ivSize + $tagSize);

// Decriptografa e valida a tag
$texto = openssl_decrypt($textoCifrado, 'aes-256-gcm', $chaveNorm,
OPENSSL_RAW_DATA, $iv, $tag);
return $texto === false ? '' : $texto;
}

function demo(): void
{
    echo "=== Criptografia SEGURA (AES-256-GCM) ===\n";
    $chave = 'minha-chave-insegura'; // Mesmo para comparação
    $texto = 'Dados sensiveis';

    $cifra1 = criptografar($texto, $chave);
    $cifra2 = criptografar($texto, $chave);

    echo "Texto plano: {$texto}\n";
    echo "Cifra 1 (hex): " . bin2hex($cifra1) . "\n";
    echo "Cifra 2 (hex): " . bin2hex($cifra2) . "\n";
    echo "Cifras sao identicas? " . (bin2hex($cifra1) === bin2hex($cifra2) ? 'SIM
(BUG!)' : 'NAO (correto!)') . "\n";
    echo "Decriptado 1: " . descriptografar($cifra1, $chave) . "\n";
    echo "Decriptado 2: " . descriptografar($cifra2, $chave) . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção usa `aes-256-gcm`, um algoritmo moderno, seguro e autenticado: AES com 256 bits de chave (2^{256} combinações, computacionalmente impossível quebrar por força bruta), GCM garante autenticação e detecção de tampering, e gera um IV aleatório de 12 bytes a cada criptografia (o IV é prefixado ao resultado para que a descriptografia possa extraí-lo depois). Mesmo criptografando o mesmo texto plano duas vezes, você obtém criptogramas completamente diferentes graças ao IV aleatório.

5.1.1. Como testar

Rode `php A04-falhas-criptograficas/01-algoritmos-fracos/teste.php` para executar:

1. Testa se a versão vulnerável (DES-ECB) produz criptogramas idênticos quando criptografa o mesmo texto duas vezes — um indicador de falta de IV aleatório e algoritmo determinístico.
2. Testa se a versão corrigida (AES-256-GCM) produz criptogramas **diferentes** para o mesmo texto — prova que o IV aleatório está funcionando.
3. Verifica se ambas conseguem descriptografar de volta ao texto original (corrigida valida

também a tag de autenticação).

Saída esperada termina em **RESULTADO: 4/4 verificacoes passaram.**

5.1.2. Checklist de verificação

- ☐ O código usa AES-256 ou ChaCha20-Poly1305, nunca DES, RC4, MD5 ou SHA-1 para criptografia?
- ☐ Se usa CBC ou GCM, há um IV aleatório diferente a cada operação (não fixo/zerado)?
- ☐ A chave é normalizada para o tamanho correto (256 bits para AES-256)?

5.2. 02 — Aleatoriedade Insuficiente

CWE-330: Use of Insufficiently Random Values; CWE-338: Use of Cryptographically Weak Pseudo-Random Number Generator

Tokens criptográficos precisam de aleatoriedade genuína — não a pseudoaleatoriedade "rápida" fornecida por `mt_rand()` ou `rand()`. Essas funções usam seeds simples e um estado de 32 bits, resultando em 2^{32} (~4 bilhões) valores **máximo**. Mas pior: para tokens de 6 dígitos, você tem apenas 10^6 (1 milhão) valores possíveis — um espaço de busca quebrável por força bruta em milissegundos. Um atacante testa sequencialmente `000000`, `000001`, `000002`, etc., até achar o token válido.

Código vulnerável ([A04-falhas-criptograficas/02-aleatoriedade-insuficiente/vulneravel.php](#))

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Aleatoriedade insuficiente em tokens criptográficos
 * OWASP A04:2025 - Cryptographic Failures
 * VULNERÁVEL: mt_rand() não é criptograficamente seguro. Gera apenas 6 dígitos,
 * totalizando apenas 1 milhão de combinações possíveis (10^6).
 * Facilmente quebrável por força bruta em milissegundos.
 */

function gerarTokenRecuperacaoSenha(): string
{
    // NUNCA USE mt_rand() PARA TOKENS CRIPTOGRÁFICOS
    // Espaço de busca: apenas 10^6 (1 milhão de valores)
    return str_pad((string) mt_rand(0, 999999), 6, '0', STR_PAD_LEFT);
}

function demo(): void
{
    echo "=== Gerador de Token VULNERÁVEL ===\n";
    for ($i = 0; $i < 5; $i++) {
        $token = gerarTokenRecuperacaoSenha();
```

```

        echo "Token {$i}: {$token} (tamanho: " . strlen($token) . ")\n";
    }
    echo "Espaço de busca: 10^6 = 1,000,000 tokens possíveis\n";
    echo "Quebrável por força bruta em MILISSEGUNDOS\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código gera tokens de 6 dígitos com `mt_rand(0, 999999)`, resultando em apenas 1 milhão de valores possíveis. Um script simples testa todas as combinações em um laptop comum em segundos, resetando senhas de outros usuários ou acessando contas que não deveria.

Código corrigido (A04-falhas-criptograficas/02-aleatoriedade-insuficiente/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Aleatoriedade CRIPTOGRAFICAMENTE SEGURA em tokens
 * OWASP A04:2025 - Cryptographic Failures
 * CORRIGIDO: random_bytes() usa CSPRNG (Cryptographically Secure Pseudo-Random Number
Generator).
 * Gera 32 bytes (256 bits) convertidos em hexadecimal = 64 caracteres.
 * Espaço de busca: 2^256 (impraticável quebrar por força bruta).
 */

function gerarTokenRecuperacaoSenha(): string
{
    // USE random_bytes() PARA TOKENS CRIPTOGRÁFICOS
    // 32 bytes = 256 bits de entropia criptograficamente segura
    // Convertido em hexadecimal = 64 caracteres
    // Espaço de busca: 16^64 = 2^256
    return bin2hex(random_bytes(32));
}

function demo(): void
{
    echo "=== Gerador de Token SEGURO ===\n";
    for ($i = 0; $i < 5; $i++) {
        $token = gerarTokenRecuperacaoSenha();
        echo "Token {$i}: {$token} (tamanho: " . strlen($token) . ")\n";
    }
    echo "Espaço de busca: 2^256 (impraticável quebrar)\n";
    echo "Cada token é único com probabilidade de colisão negligenciável\n";
}

```

```
if (debug_backtrace() === []) {  
    demo();  
}
```

A correção usa `random_bytes(32)` — 32 bytes = 256 bits gerados pelo CSPRNG do SO (Cryptographically Secure Pseudo-Random Number Generator), convertidos em hexadecimal = 64 caracteres. Esse espaço (2^{256}) é computacionalmente impossível quebrar por força bruta. Mesmo que um atacante testasse 1 bilhão de tokens por segundo, levaria bilhões de bilhões de anos para achar um.

5.2.1. Como testar

Rode `php A04-falhas-criptograficas/02-aleatoriedade-insuficiente/teste.php`:

1. Testa se a versão vulnerável produz tokens com tamanho fixo de 6 dígitos (espaço reduzido).
2. Testa se a versão corrigida produz tokens com 64 caracteres (256 bits em hex).
3. Verifica que tokens sucessivos são sempre diferentes (aleatoriedade genuína).

Saída esperada: `RESULTADO: 3/3 verificacoes passaram.`

5.2.2. Checklist de verificação

- ☐ Tokens, salts e IVs usam `random_bytes()`, nunca `mt_rand()`, `rand()` ou `uniqid()`?
- ☐ O tamanho do token é suficiente? (32 bytes = 256 bits é um bom padrão mínimo)
- ☐ O valor é convertido para hexadecimal ou base64 antes de ser usado/transmitido?

5.3. 03 — Gestão de Chaves

CWE-321: Use of Hard-coded Cryptographic Key

Chaves criptográficas hardcoded no código-fonte são, por definição, comprometidas: aparecem em históricos de git, logs de build, pull requests, backups, e qualquer pessoa com acesso ao repositório as vê. Rotacionar uma chave comprometida obriga a alterar o código e fazer novo deploy — o que não é um processo ágil em produção. A prática correta é carregar chaves de variáveis de ambiente, vault (HashiCorp Vault, AWS Secrets Manager), ou arquivos de configuração fora do repositório.

Código vulnerável (`A04-falhas-criptograficas/03-gestao-chaves/vulneravel.php`)

```
<?php  
declare(strict_types=1);  
  
namespace Vulneravel;  
  
/**  
 * Gestão inadequada de chaves criptográficas (hardcoded)  
 * OWASP A04:2025 - Cryptographic Failures  
 * VULNERÁVEL: Chave secreta está hardcoded no código-fonte.
```

```
* Qualquer pessoa com acesso ao repositório/build artefato obtém a chave.  
* Impossível rotacionar sem alterar o código e fazer novo deploy.  
*/
```

```
function obterChaveCriptografia(): string  
{  
    // NUNCA HARDCODE CHAVES SECRETAS NO CÓDIGO!  
    // Está visível em: logs de git, histórico de commits,  
    // artefatos de build, backups, code reviews, etc.  
    return "minha-chave-secreta-fixa-123456";  
}  
  
function demo(): void  
{  
    echo "=== Gestão de Chaves VULNERÁVEL (hardcoded) ===\n";  
    echo "Chave 1: " . obterChaveCriptografia() . "\n";  
    echo "Chave 2: " . obterChaveCriptografia() . "\n";  
    echo "Chave 3: " . obterChaveCriptografia() . "\n";  
    echo "\nTodas as chaves são IDÊNTICAS (hardcoded)\n";  
    echo "Visível em: histórico de git, logs, backups, code reviews\n";  
}  
  
if (debug_backtrace() === []) {  
    demo();  
}
```

A chave "minha-chave-secreta-fixa-123456" está hardcoded. Qualquer desenvolvedor, estagiário, ou contribuidor do projeto consegue vê-la — e qualquer pessoa que acessar um artefato de build (Docker image, arquivo compilado) também. Impossível rotacionar sem desplegar código novo.

Código corrigido (A04-falhas-criptograficas/03-gestao-chaves/corrigido.php)

```
<?php  
declare(strict_types=1);  
  
namespace Corrigido;  
  
/**  
 * Gestão SEGURA de chaves criptográficas via variáveis de ambiente  
 * OWASP A04:2025 - Cryptographic Failures  
 * CORRIGIDO: Chave é lida de variável de ambiente no runtime.  
 * Permite rotação sem alterar código. Nunca exposta em repositório.  
 */  
  
function obterChaveCriptografia(): string  
{  
    // USE VARIÁVEIS DE AMBIENTE PARA CHAVES SECRETAS  
    // Configurado no: arquivo .env (não commitado), CI/CD secrets,  
    // container environment, vault manager, etc.  
    $chave = getenv('APP_ENCRYPTION_KEY');
```

```

if ($chave === false || $chave === '') {
    throw new RuntimeException(
        'APP_ENCRYPTION_KEY não configurada. ' .
        'Configure a variável de ambiente antes de rodar a aplicação.'
    );
}

return $chave;
}

function demo(): void
{
    echo "=== Gestão de Chaves SEGURA (variável de ambiente) ===\n";
    try {
        echo "Tentando obter chave de APP_ENCRYPTION_KEY...\n";
        $chave = obterChaveCriptografia();
        echo "Chave obtida com sucesso: " . substr($chave, 0, 10) . "... \n";
    } catch (\RuntimeException $e) {
        echo "Erro esperado: " . $e->getMessage() . "\n";
        echo "Configure APP_ENCRYPTION_KEY antes de usar\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção lê a chave da variável de ambiente `APP_ENCRYPTION_KEY`, configurada no servidor, container, ou CI/CD secrets — nunca no repositório. Rotacionar é trivial: basta atualizar a variável de ambiente sem mudar código. Se a variável não está configurada, a função lança uma exceção clara, evitando falhas silenciosas.

5.3.1. Como testar

Rode `php A04-falhas-criptograficas/03-gestao-chaves/teste.php`:

1. A versão vulnerável retorna a chave hardcoded.
2. A versão corrigida tenta ler `APP_ENCRYPTION_KEY` e lança `RuntimeException` se não estiver configurada.
3. Com `APP_ENCRYPTION_KEY` configurada, a versão corrigida retorna o valor correto.

Saída esperada: `RESULTADO: 2/2 verificacoes passaram.`

5.3.2. Checklist de verificação

- ☐ Nenhuma chave, senha ou token sensível aparece em arquivos `.php` do repositório (buscar manualmente ou com `grep -r "key|secret|password"` no código)?

- ❑ Chaves são carregadas de variáveis de ambiente (`getenv()`) ou arquivo `.env` não commitado?
- ❑ Código trata ausência de chave com erro claro, nunca silenciosamente?

5.4. 04 — Transporte em Texto Claro

CWE-319: Cleartext Transmission of Sensitive Information; CWE-523: Unprotected Transport of Credentials

Credenciais e dados sensíveis enviados em HTTP (ou HTTPS com validação desabilitada) são interceptáveis por qualquer um na rede — um proxy, um roteador comprometido, um provedor ISP curioso. Pior ainda é colocar credenciais na query string da URL: elas aparecem em logs do servidor web, histórico do navegador, headers `Referer`, caches HTTP intermediários, e qualquer ferramenta de monitoramento que registre URLs.

Código vulnerável (A04-falhas-criptograficas/04-transporte-texto-claro/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Transmissão de dados sensíveis em texto claro (HTTP + credenciais na URL)
 * OWASP A04:2025 - Cryptographic Failures
 * VULNERÁVEL: Usa HTTP (não criptografado) e coloca credenciais na query string.
 * Credenciais aparecem em: logs do servidor, histórico do navegador,
 * proxies HTTP, caches, referer headers, etc.
 */

function montarRequisicaoLogin(string $host, string $usuario, string $senha): array
{
    // NUNCA USE HTTP PARA CREDENCIAIS!
    // NUNCA COLOQUE CREDENCIAIS NA URL!
    return [
        'metodo' => 'GET',
        'url' => "http://{ $host }/login?usuario={ $usuario }&senha={ $senha }",
        'corpo' => null,
    ];
}

function demo(): void
{
    echo "=== Requisição de Login VULNERÁVEL ===\n";
    $req = montarRequisicaoLogin('api.exemplo.com', 'joao@email.com', 'senha123');

    echo "Método: " . $req['metodo'] . "\n";
    echo "URL: " . $req['url'] . "\n";
    echo "Corpo: " . ($req['corpo'] ?? 'vazio') . "\n";
    echo "\nPROBLEMAS:\n";
    echo "1. HTTP sem criptografia - qualquer um pode ver a senha\n";
}
```

```

    echo "2. Senha na query string - aparece em logs, histórico, referer header\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A função monta uma URL HTTP (sem criptografia) com credenciais na query string: <http://api.exemplo.com/login?usuario=joao%40email.com&senha=senha123>. Qualquer interceptador de rede (man-in-the-middle) vê a senha em texto plano. Além disso, a senha aparece em logs do servidor, histórico do navegador do cliente, e referer headers enviados a outros domínios.

Código corrigido (A04-falhas-criptograficas/04-transporte-texto-claro/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Transmissão SEGURA de dados sensíveis (HTTPS + corpo da requisição)
 * OWASP A04:2025 - Cryptographic Failures
 * CORRIGIDO: Usa HTTPS (criptografado) e coloca credenciais no corpo (POST).
 * Credenciais não aparecem em logs de URL, histórico do navegador, referer headers.
 */

function montarRequisicaoLogin(string $host, string $usuario, string $senha): array
{
    // USE HTTPS PARA QUALQUER DADO SENSÍVEL!
    // USE POST COM CORPO PARA CREDENCIAIS, NUNCA QUERY STRING!
    return [
        'metodo' => 'POST',
        'url' => "https://{ $host }/login",
        'corpo' => [
            'usuario' => $usuario,
            'senha' => $senha,
        ],
    ];
}

function demo(): void
{
    echo "=== Requisição de Login SEGURA ===\n";
    $req = montarRequisicaoLogin('api.exemplo.com', 'joao@email.com', 'senha123');

    echo "Método: " . $req['metodo'] . "\n";
    echo "URL: " . $req['url'] . "\n";
    echo "Corpo: " . json_encode($req['corpo']) . "\n";
    echo "\nBENEFÍCIOS:\n";
    echo "1. HTTPS criptografa tudo em trânsito\n";
}

```

```

    echo "2. Credenciais no corpo - não aparecem em logs de URL\n";
    echo "3. POST é padrão para dados sensíveis\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção usa HTTPS (criptografia TLS em trânsito) e POST (credenciais no corpo da requisição, não na URL). Credenciais no corpo POST não aparecem em logs de URL, histórico do navegador, ou referer headers. O corpo está dentro do envelope TLS criptografado, invisível para interceptadores de rede.

5.4.1. Como testar

Rode `php A04-falhas-criptograficas/04-transporte-texto-claro/teste.php`:

1. Verifica que a versão vulnerável constrói uma URL HTTP com credenciais visíveis na query string.
2. Verifica que a versão corrigida usa HTTPS com credenciais no corpo (array).
3. Confirma que credenciais não aparecem em parte alguma da URL corrigida.

Saída esperada: **RESULTADO: 3/3 verificacoes passaram.**

5.4.2. Checklist de verificação

- ☐ Todas as requisições com dados sensíveis (login, tokens, PII) usam HTTPS, nunca HTTP?
- ☐ Credenciais estão no corpo da requisição (POST), nunca na URL (GET)?
- ☐ Certificados TLS são validados (SSL_VERIFYPEER ⇒ true)?

5.5. 05 — Hash sem Salt

CWE-759: Use of a One-Way Hash with a Predictable Salt; CWE-916: Use of Password Hash With Insufficient Computational Effort

Hashing de senhas com `md5()` ou `sha1()` sem salt é uma falha clássica. Sem salt, duas senhas iguais geram hashes iguais — e MD5 é tão rápido (milhões de hashes por segundo) que um atacante consegue pré-computar rainbow tables: dicionários enormes de senha → hash que cobrem bilhões de combinações comuns. A senha "123456" sempre vai produzir `e10adc3949ba59abbe56e057f20f883e` — um hash que qualquer ferramenta online descobre em segundos.

Código vulnerável (`A04-falhas-criptograficas/05-hash-sem-salt/vulneravel.php`)

```

<?php
declare(strict_types=1);

namespace Vulneravel;

```

```

/**
 * Hash de senhas sem salt e com algoritmo fraco (MD5)
 * OWASP A04:2025 - Cryptographic Failures
 * VULNERÁVEL: MD5 é rápido demais (permite força bruta) e sem salt.
 * Mesmo hash para mesma senha sempre. Vulnerável a rainbow tables.
 * Exemplo: md5("123456") é sempre "e10adc3949ba59abbe56e057f20f883e"
 */

function hashSenha(string $senha): string
{
    // NUNCA USE MD5 PARA SENHAS!
    // - Extremamente rápido (milhões de hashes por segundo)
    // - Sem salt (determinístico)
    // - Conhecido/crackável (rainbow tables amplamente disponíveis)
    return md5($senha);
}

function verificarSenha(string $senha, string $hash): bool
{
    // Comparação simples (sem salt)
    return md5($senha) === $hash;
}

function demo(): void
{
    echo "=== Hash de Senha VULNERÁVEL (MD5) ===\n";
    $senha = '123456';
    $hash1 = hashSenha($senha);
    $hash2 = hashSenha($senha);

    echo "Senha: {$senha}\n";
    echo "Hash 1: {$hash1}\n";
    echo "Hash 2: {$hash2}\n";
    echo "Hashes idênticos? " . ($hash1 === $hash2 ? 'SIM (falta salt)' : 'NAO') . "\n";
    echo "\nEste hash de '123456' é AMPLAMENTE CONHECIDO:\n";
    echo "e10adc3949ba59abbe56e057f20f883e\n";
    echo "\nCrackável em MILISSEGUNDOS com rainbow tables.\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código usa `md5($senha)` puro. Sem salt, mesmo usuário com mesma senha sempre gera hash idêntico. O hash é fraco, crackável em milissegundos por rainbow tables. Um atacante que conseguir o banco de dados extrai as senhas rapidamente.

```
<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Hash de senhas com salt e algoritmo seguro (bcrypt)
 * OWASP A04:2025 - Cryptographic Failures
 * CORRIGIDO: password_hash() com PASSWORD_BCRYPT usa salt aleatório
 * e fator de custo adaptativo. Cada hash é único mesmo para mesma senha.
 */

function hashSenha(string $senha): string
{
    // USE password_hash() COM PASSWORD_BCRYPT!
    // - Incorpora salt aleatório automaticamente
    // - Custo adaptativo (fica mais lento com o tempo)
    // - Cada hash é único mesmo para mesma senha
    // - Praticamente impossível fazer rainbow tables
    return password_hash($senha, PASSWORD_BCRYPT);
}

function verificarSenha(string $senha, string $hash): bool
{
    // USE password_verify() PARA VERIFICAÇÃO!
    // - Valida automaticamente sem expor implementação
    return password_verify($senha, $hash);
}

function demo(): void
{
    echo "=== Hash de Senha SEGURA (bcrypt) ===\n";
    $senha = '123456';
    $hash1 = hashSenha($senha);
    $hash2 = hashSenha($senha);

    echo "Senha: {$senha}\n";
    echo "Hash 1: {$hash1}\n";
    echo "Hash 2: {$hash2}\n";
    echo "Hashes identicos? " . ($hash1 === $hash2 ? 'SIM (BUG!)' : 'NAO (correto, salt aleatório)') . "\n";

    echo "\nVerificação com password_verify():\n";
    echo "Hash 1 bate com senha? " . (verificarSenha($senha, $hash1) ? 'SIM' : 'NAO')
    . "\n";
    echo "Hash 2 bate com senha? " . (verificarSenha($senha, $hash2) ? 'SIM' : 'NAO')
    . "\n";
    echo "\nCada hash é único, impossível rainbow tables.\n";
    echo "Custo adaptativo: fica mais lento com o tempo.\n";
}
```

```

}

if (debug_backtrace() == []) {
    demo();
}

```

A correção usa `password_hash($senha, PASSWORD_BCRYPT)` — um algoritmo dedicado a senhas que: * Incorpora um salt aleatório automaticamente (cada hash é único mesmo para mesma senha). * Usa fator de custo adaptativo: fica mais lento com o tempo, acompanhando evolução de hardware. * Torna rainbow tables inviáveis (cada combinação de senha precisa ser testada com cada salt possível).

Verificação usa `password_verify()`, que valida corretamente mesmo com salt embutido.

5.5.1. Como testar

Rode `php A04-falhas-criptograficas/05-hash-sem-salt/teste.php`:

1. Testa que MD5 puro produz hashes idênticos para mesma senha (falta de salt).
2. Testa que `password_hash()` produz hashes diferentes para mesma senha (salt aleatório).
3. Verifica que `password_verify()` valida corretamente em ambos os casos.

Saída esperada: `RESULTADO: 4/4 verificacoes passaram.`

5.5.2. Checklist de verificação

- ☐ Senhas usam `password_hash(..., PASSWORD_BCRYPT)` ou `PASSWORD_ARGON2ID`, nunca `md5()`, `sha1()` ou `hash()`?
- ☐ Verificação usa `password_verify()`, nunca comparação direta (`==` ou `===`)?
- ☐ Se você cuida de hashes antigos, há um plano para fazer rehash com algoritmo novo na próxima autenticação do usuário?

5.6. 06 — IV Problemático

CWE-329: Not Using a Random IV with CBC Mode

Modos de criptografia orientados a blocos (CBC, OFB, CFB) requerem um Initialization Vector (IV) aleatório diferente a cada operação. Reutilizar o mesmo IV com a mesma chave vaza padrões: se você criptografa "A" várias vezes com mesmo IV e chave, obtém criptogramas idênticos — um indicador de que a mensagem não mudou. Em casos extremos (contextos com blocos repetidos ou padrão previsível), vaza a estrutura interna dos dados.

Código vulnerável (`A04-falhas-criptograficas/06-iv-problematico/vulneravel.php`)

```

<?php
declare(strict_types=1);

```

```

namespace Vulneravel;

/**
 * Uso inadequado de IV (Initialization Vector) - IV fixo/zerado
 * OWASP A04:2025 - Cryptographic Failures
 * VULNERÁVEL: IV é fixo (todos zeros) ao invés de aleatório.
 * Mesmo texto plano produz o mesmo criptograma sempre (modo CBC).
 * Vaza padrões e é vulnerável a ataques de análise de frequência.
 */

function criptografarCbc(string $texto, string $chave): string
{
    // Normaliza chave para 32 bytes
    $chaveNorm = hash('sha256', $chave, true);

    // NUNCA USE IV FIXO!
    // Usa IV zerado (todos zeros) - PÉSSIMA PRÁTICA
    $iv = str_repeat("\0", 16);

    // Criptografa com AES-256-CBC
    $textoCifrado = openssl_encrypt($texto, 'aes-256-cbc', $chaveNorm,
    OPENSSL_RAW_DATA, $iv);

    // Retorna apenas o criptograma (IV não é prefixado pois é fixo)
    return $textoCifrado;
}

function descriptografarCbc(string $textoCifrado, string $chave): string
{
    // Normaliza chave para 32 bytes
    $chaveNorm = hash('sha256', $chave, true);

    // Usa o MESMO IV fixo
    $iv = str_repeat("\0", 16);

    // Decriptografa
    $texto = openssl_decrypt($textoCifrado, 'aes-256-cbc', $chaveNorm,
    OPENSSL_RAW_DATA, $iv);
    return $texto === false ? '' : $texto;
}

function demo(): void
{
    echo "=== Criptografia CBC VULNERÁVEL (IV fixo) ===\n";
    $chave = 'chave-teste';
    $texto = 'Dados sensíveis';

    $cifra1 = criptografarCbc($texto, $chave);
    $cifra2 = criptografarCbc($texto, $chave);

    echo "Texto plano: {$texto}\n";
}

```

```

echo "Cifra 1 (hex): " . bin2hex($cifra1) . "\n";
echo "Cifra 2 (hex): " . bin2hex($cifra2) . "\n";
echo "Cifras sao identicas? " . (bin2hex($cifra1) === bin2hex($cifra2) ? 'SIM
(BUG!)' : 'NAO') . "\n";
echo "IV fixo: " . bin2hex(str_repeat("\0", 16)) . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código usa IV fixo zerado: `str_repeat("\0", 16)` — todos os bytes são 0. Mesmo texto plano criptografado com mesma chave produz criptograma idêntico sempre, vazando padrões. Se um atacante sabe que você criptografa respostas simples (sim/não, true/false), consegue adivinhar pelo criptograma.

Código corrigido ([A04-falhas-criptograficas/06-iv-problematico/corrigido.php](#))

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Uso SEGURO de IV (Initialization Vector) - IV aleatório
 * OWASP A04:2025 - Cryptographic Failures
 * CORRIGIDO: IV aleatório de 16 bytes gerado a cada criptografia.
 * Mesmo texto plano produz criptogramas DIFERENTES.
 * IV é prefixado ao resultado para descriptografia posterior.
 */

function criptografarCbc(string $texto, string $chave): string
{
    // Normaliza chave para 32 bytes
    $chaveNorm = hash('sha256', $chave, true);

    // USE IV ALEATÓRIO!
    // 16 bytes aleatórios para CBC
    $iv = random_bytes(16);

    // Criptografa com AES-256-CBC
    $textoCifrado = openssl_encrypt($texto, 'aes-256-cbc', $chaveNorm,
    OPENSSEL_RAW_DATA, $iv);

    // Retorna: IV + CRIPTOGRAMA (IV prefixado)
    // Necessário para descriptografia posterior
    return $iv . $textoCifrado;
}

function descriptografarCbc(string $resultado, string $chave): string

```

```

{
    // Normaliza chave para 32 bytes
    $chaveNorm = hash('sha256', $chave, true);

    // Extrai IV dos primeiros 16 bytes
    if (strlen($resultado) < 16) {
        return '';
    }

    $iv = substr($resultado, 0, 16);
    $textoCifrado = substr($resultado, 16);

    // Decriptografa com o IV extraído
    $texto = openssl_decrypt($textoCifrado, 'aes-256-cbc', $chaveNorm,
    OPENSSL_RAW_DATA, $iv);
    return $texto === false ? '' : $texto;
}

function demo(): void
{
    echo "=== Criptografia CBC SEGURA (IV aleatório) ===\n";
    $chave = 'chave-teste';
    $texto = 'Dados sensiveis';

    $cifra1 = criptografarCbc($texto, $chave);
    $cifra2 = criptografarCbc($texto, $chave);

    echo "Texto plano: {$texto}\n";
    echo "Cifra 1 (hex): " . bin2hex($cifra1) . "\n";
    echo "Cifra 2 (hex): " . bin2hex($cifra2) . "\n";
    echo "Cifras sao identicas? " . (bin2hex($cifra1) === bin2hex($cifra2) ? 'SIM
(BUG!)' : 'NAO (correto, IV aleatório)') . "\n";
    echo "\nIV prefixado (primeiros 16 bytes do resultado)\n";
    echo "Decriptado 1: " . descriptografarCbc($cifra1, $chave) . "\n";
    echo "Decriptado 2: " . descriptografarCbc($cifra2, $chave) . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção gera IV aleatório a cada criptografia: `$iv = random_bytes(16)`, prefixando-o ao resultado. Mesmo texto plano produz criptogramas completamente diferentes graças ao IV aleatório. O receptor extrai os primeiros 16 bytes (IV) e usa-os para descriptografar o resto. Nenhum padrão é vazado.

5.6.1. Como testar

Rode `php A04-falhas-criptograficas/06-iv-problematico/teste.php`:

1. Testa que CBC com IV fixo produz criptogramas idênticos para mesma mensagem.
2. Testa que CBC com IV aleatório produz criptogramas diferentes.
3. Verifica que a descryptografia funciona corretamente em ambos os casos (extrai IV prefixado).

Saída esperada: **RESULTADO: 4/4 verificacoes passaram.**

5.6.2. Checklist de verificação

- ☐ Se usa CBC ou modo stream similar, IV é gerado com `random_bytes()` a cada operação?
- ☐ IV é prefixado ao criptograma ou armazenado separadamente (nunca reutilizado)?
- ☐ Código não usa modo ECB (Electronic Code Book) — sempre usar CBC, GCM ou ChaCha20?

5.7. 07 — Certificado Não Validado

CWE-295: Improper Certificate Validation

Requisições HTTPS que não validam certificados TLS são vulneráveis a man-in-the-middle (MITM): um atacante intercepta a conexão, apresenta seu próprio certificado (assinado por si mesmo ou autoridade falsa), e o cliente o aceita sem questionar. Resultado: atacante vê/altera tudo que passa, inclusive credenciais e dados sensíveis. Desabilitar `SSL_VERIFYPEER` ou `SSL_VERIFYHOST` é uma prática perigosa, frequentemente "justificada" por certificados autoassinados em testes — mas depois fica em produção.

Código vulnerável ([A04-falhas-criptograficas/07-certificado-nao-validado/vulneravel.php](#))

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Desabilitação da validação de certificado TLS em requisições HTTPS
 * OWASP A04:2025 - Cryptographic Failures
 * VULNERÁVEL: CURLOPT_SSL_VERIFYPEER => false desabilita validação.
 * Permite man-in-the-middle attacks mesmo usando HTTPS.
 * Um ataque que intercepta a conexão consegue se fazer passar pela API.
 */

function criarOpcoesCurl(string $url): array
{
    // NUNCA DESABILITE SSL_VERIFYPEER!
    // Isto permite ataques man-in-the-middle mesmo com HTTPS
    return [
        'CURLOPT_URL' => $url,
        'CURLOPT_RETURNTRANSFER' => true,
        'CURLOPT_TIMEOUT' => 30,
        CURLOPT_SSL_VERIFYPEER => false, // VULNERÁVEL!
        CURLOPT_SSL_VERIFYHOST => 0,    // VULNERÁVEL!
```

```

    ];
}

function demo(): void
{
    echo "=== Opções cURL VULNERÁVEL (certificado não validado) ===\n";
    $url = 'https://api.exemplo.com/dados';
    $opcoes = criarOpcoesCurl($url);

    echo "URL: {$url}\n";
    echo "SSL_VERIFYPEER: " . ($opcoes[CURLLOPT_SSL_VERIFYPEER] ? 'true' : 'false') . "
(BUG!)\n";
    echo "SSL_VERIFYHOST: " . $opcoes[CURLLOPT_SSL_VERIFYHOST] . " (BUG!)\n";
    echo "\nVulnerável a man-in-the-middle attack mesmo usando HTTPS!\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

Opções cURL: `CURLLOPT_SSL_VERIFYPEER => false` desabilita validação da cadeia de certificados, e `CURLLOPT_SSL_VERIFYHOST => 0` desabilita validação de que o certificado pertence ao host solicitado. Resultado: qualquer certificado (válido ou não) é aceito, permitindo MITM sem detecção.

Código corrigido ([A04-falhas-criptograficas/07-certificado-nao-validado/corrigido.php](#))

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Validação SEGURA de certificado TLS em requisições HTTPS
 * OWASP A04:2025 - Cryptographic Failures
 * CORRIGIDO: CURLLOPT_SSL_VERIFYPEER => true e CURLLOPT_SSL_VERIFYHOST => 2.
 * Valida que o certificado é válido e pertence ao host esperado.
 * Protege contra man-in-the-middle attacks.
 */

function criarOpcoesCurl(string $url): array
{
    // SEMPRE VALIDE CERTIFICADOS TLS!
    // SSL_VERIFYPEER => true: valida a cadeia de certificados
    // SSL_VERIFYHOST => 2: valida que o certificado é do host correto
    return [
        'CURLOPT_URL' => $url,
        'CURLOPT_RETURNTRANSFER' => true,
        'CURLOPT_TIMEOUT' => 30,
        'CURLOPT_SSL_VERIFYPEER' => true,    // SEGURO
        'CURLOPT_SSL_VERIFYHOST' => 2,      // SEGURO (valida host name)
    ];
}

```

```

        CURLOPT_CAINFO => '/etc/ssl/certs/ca-bundle.crt', // Caminho do CA bundle
    ];
}

function demo(): void
{
    echo "=== Opções cURL SEGURA (certificado validado) ===\n";
    $url = 'https://api.exemplo.com/dados';
    $opcoes = criarOpcoesCurl($url);

    echo "URL: {$url}\n";
    echo "SSL_VERIFYPEER: " . ($opcoes[CURLOPT_SSL_VERIFYPEER] ? 'true' : 'false') . "
(seguro!)\n";
    echo "SSL_VERIFYHOST: " . $opcoes[CURLOPT_SSL_VERIFYHOST] . " (seguro!)\n";
    echo "\nProtegido contra man-in-the-middle attacks\n";
    echo "Certificado é validado contra o host\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

Opções corrigidas: `CURLOPT_SSL_VERIFYPEER` $\Rightarrow$ `true` valida a cadeia de certificados contra a loja de CAs do sistema, e `CURLOPT_SSL_VERIFYHOST` $\Rightarrow$ `2` valida que o certificado é do host correto (nome do domínio). `CURLOPT_CAINFO` aponta para o bundle de CAs de confiança. Certificados autoassinados ou inválidos são rejeitados — a única forma de "aceitar" é adicionar o certificado à loja de confiança do servidor.

5.7.1. Como testar

Rode `php A04-falhas-criptograficas/07-certificado-nao-validado/teste.php`:

1. Verifica que versão vulnerável tem `SSL_VERIFYPEER` e `SSL_VERIFYHOST` desabilitados.
2. Verifica que versão corrigida tem `SSL_VERIFYPEER` $\Rightarrow$ `true` e `SSL_VERIFYHOST` $\Rightarrow$ `2`.
3. Confirma que arquivo CA bundle é referenciado.

Saída esperada: `RESULTADO: 3/3 verificacoes passaram.`

5.7.2. Checklist de verificação

- ☐ Todas requisições HTTPS externas têm `CURLOPT_SSL_VERIFYPEER` $\Rightarrow$ `true` e `CURLOPT_SSL_VERIFYHOST` $\Rightarrow$ `2`?
- ☐ Arquivo CA bundle (`CURLOPT_CAINFO`) está configurado corretamente (geralmente `/etc/ssl/certs/ca-bundle.crt` ou via Composer)?
- ☐ Se há certificados autoassinados legítimos (dev/staging), eles estão adicionados à loja de CAs do servidor, nunca desabilitando validação globalmente?

5.8. 08 — Modo de Criptografia Inadequado

CWE-327: Use of a Broken or Risky Cryptographic Algorithm

O modo ECB (Electronic Code Book) é determinístico: blocos idênticos de texto plano geram blocos idênticos de criptograma. Isso vaza padrão dos dados — um atacante vê quais blocos repetem, deduz estrutura, e às vezes consegue recuperar informação sem quebrar criptografia. O "ECB Penguin Problem" é um exemplo clássico: criptografar a imagem de um pinguim com ECB produz uma silhueta reconhecível, vazando estrutura. Modos como CBC (com IV aleatório) e GCM não têm esse problema.

Código vulnerável (A04-falhas-criptograficas/08-modo-criptografia-inadequado/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Uso de modo de criptografia inadequado (ECB - Electronic Code Book)
 * OWASP A04:2025 - Cryptographic Failures
 * VULNERÁVEL: ECB é determinístico - blocos de 16 bytes idênticos no texto plano
 * geram blocos idênticos no criptograma. Vaza padrões (ECB Penguin Problem).
 * Cada bloco é criptografado independentemente.
 */

function criptografarEcb(string $texto, string $chave): string
{
    // Normaliza chave para 32 bytes
    $chaveNorm = hash('sha256', $chave, true);

    // NUNCA USE ECB!
    // ECB é determinístico: blocos iguais produzem criptogramas iguais
    $textoCifrado = openssl_encrypt($texto, 'aes-256-ecb', $chaveNorm,
    OPENSSSL_RAW_DATA);

    if ($textoCifrado === false) {
        throw new \RuntimeException('Falha ao criptografar com ECB');
    }

    return $textoCifrado;
}

function demo(): void
{
    echo "=== Criptografia ECB VULNERÁVEL (modo inadequado) ===\n";

    // Demonstração do ECB Penguin Problem
    // Texto com blocos repetidos (16 bytes cada)
    $chave = 'chave-teste';
```

```

$bloco = 'AAAAAAAAAAAAAAAA'; // 16 A's = 1 bloco de 16 bytes
$texto = $bloco . $bloco; // 2 blocos idênticos = 32 bytes

echo "Texto (hex): " . bin2hex($texto) . "\n";
echo "Bloco 1: " . bin2hex($bloco) . "\n";
echo "Bloco 2: " . bin2hex($bloco) . "\n";

$cifra = criptografarEcb($texto, $chave);

echo "Cifra (hex): " . bin2hex($cifra) . "\n";
echo "\nBlocos da cifra:\n";
echo "Bloco 1 (bytes 0-15): " . bin2hex(substr($cifra, 0, 16)) . "\n";
echo "Bloco 2 (bytes 16-31): " . bin2hex(substr($cifra, 16, 16)) . "\n";
echo "\nSe os blocos da cifra forem IDENTICOS, vaza padrão (ECB problem)!\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código usa `openssl_encrypt(..., 'aes-256-ecb', ...)` — ECB é determinístico. Dois blocos idênticos no texto plano produzem dois blocos idênticos no criptograma, vazando padrão. Demonstração: duas cópias da mesma string de 16 A's produz criptograma com dois blocos idênticos, um indicador de ECB.

Código corrigido ([A04-falhas-criptograficas/08-modo-criptografia-inadequado/corrigido.php](#))

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Uso de modo de criptografia SEGURO (GCM - Galois/Counter Mode)
 * OWASP A04:2025 - Cryptographic Failures
 * CORRIGIDO: GCM é não-determinístico e autenticado.
 * Blocos idênticos no texto plano geram criptogramas DIFERENTES
 * graças ao IV aleatório e autenticação.
 */

function criptografarEcb(string $texto, string $chave): string
{
    // Normaliza chave para 32 bytes
    $chaveNorm = hash('sha256', $chave, true);

    // USE GCM (ou CBC com IV aleatório!)
    // GCM é não-determinístico e autenticado
    $iv = random_bytes(12);
    $tag = '';
    $textoCifrado = openssl_encrypt($texto, 'aes-256-gcm', $chaveNorm,

```

```

OPENSSL_RAW_DATA, $iv, $tag);

    if ($textoCifrado === false) {
        throw new RuntimeException('Falha ao criptografar com GCM');
    }

    // Retorna: IV + TAG + CRIPTOGRAMA
    return $iv . $tag . $textoCifrado;
}

function demo(): void
{
    echo "=== Criptografia GCM SEGURA (modo adequado) ===\n";

    $chave = 'chave-teste';
    $bloco = 'AAAAAAAAAAAAAAAA'; // 16 A's = 1 bloco de 16 bytes
    $texto = $bloco . $bloco; // 2 blocos idênticos = 32 bytes

    echo "Texto (hex): " . bin2hex($texto) . "\n";

    // Criptografa duas vezes o mesmo texto
    $cifra1 = criptografarEcb($texto, $chave);
    $cifra2 = criptografarEcb($texto, $chave);

    echo "Cifra 1 (hex): " . bin2hex($cifra1) . "\n";
    echo "Cifra 2 (hex): " . bin2hex($cifra2) . "\n";

    echo "\nCifras são DIFERENTES mesmo para mesmo texto (IV aleatório)\n";
    echo "GCM não vaza padrões - blocos idênticos produzem criptogramas diferentes!\n";
};

if (debug_backtrace() == []) {
    demo();
}

```

A correção usa GCM (Galois/Counter Mode) com IV aleatório de 12 bytes. GCM é não-determinístico: mesmo texto plano criptografado várias vezes produz criptogramas completamente diferentes. Além disso, GCM fornece autenticação (tag) — impossível alterar dados sem invalidar tag. Mesmo com blocos repetidos no texto plano, nenhum padrão é vazado.

5.8.1. Como testar

Rode `php A04-falhas-criptograficas/08-modo-criptografia-inadequado/teste.php`:

1. Testa que ECB produz blocos idênticos no criptograma quando texto plano tem blocos idênticos.
2. Testa que GCM produz criptogramas completamente diferentes mesmo para texto plano idêntico (IV aleatório).
3. Verifica que GCM consegue descriptografar e valida autenticação.

Saída esperada: **RESULTADO: 3/3 verificacoes passaram.**

5.8.2. Checklist de verificação

- ☐ Código nunca usa ECB (Electronic Code Book) para qualquer criptografia?
- ☐ Se usa CBC, há IV aleatório diferente a cada operação?
- ☐ Preferência é GCM ou ChaCha20-Poly1305 (autenticados, não-determinísticos)?

5.9. Checklist Consolidado — Falhas Criptográficas

Use este checklist como referência rápida durante code review ou pentest de aplicações PHP que lidam com dados sensíveis:

- ☐ **Algoritmos:** AES-256 ou ChaCha20-Poly1305 exclusivamente; nunca RC4, DES, MD5 para criptografia.
- ☐ **Aleatoriedade:** Tokens, IVs, salts gerados com `random_bytes()`; nunca `mt_rand()` ou `rand()`.
- ☐ **Chaves:** Carregadas de variáveis de ambiente ou vault; zero hardcoding no código.
- ☐ **Transporte:** HTTPS para qualquer dado sensível; credenciais no corpo POST, nunca URL.
- ☐ **Senhas:** `password_hash(..., PASSWORD_BCRYPT) + password_verify()`; nunca MD5/SHA-1.
- ☐ **IVs:** Aleatórios, únicos por operação; jamais fixos ou reutilizados.
- ☐ **Certificados TLS:** `SSL_VERIFYPEER ⇒ true`, `SSL_VERIFYHOST ⇒ 2`, CA bundle configurado.
- ☐ **Modos:** GCM ou CBC com IV aleatório; nunca ECB.

Chapter 6. A05:2025 — Injeção

Injeção é quando dados não confiáveis (entrada do usuário) são interpretados como código ou comando, alterando a lógica da aplicação. Ao concatenar input diretamente em SQL, HTML, LDAP, ou qualquer linguagem interpretada, um atacante consegue "quebrar" a sintaxe esperada e injetar código malicioso. Essas falhas sobrevivem há décadas porque parecem inofensivas — uma simples string — até alguém passar `admin' OR '1'='1` e acessar todos os registros. A defesa é universal: nunca concatenar, sempre parametrizar ou escapar dependendo do contexto.

Esta seção cobre 7 exemplos concretos:

- **01 — SQL Injection:** De concatenação para prepared statements
- **02 — XSS (Cross-Site Scripting):** De saída bruta para `htmlspecialchars()`
- **03 — Command Injection:** De concatenação shell para `escapeshellarg()`
- **04 — NoSQL Injection:** De operadores livres para validação de tipos
- **05 — LDAP Injection:** De concatenação para escape RFC4515
- **06 — Code Injection / Eval:** De `eval()` para regex restrita
- **07 — ORM Injection:** De método genérico para API específica com prepared statements

6.1. 01 — SQL Injection

CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')

SQL Injection é talvez a falha mais clássica da web: concatenar entrada do usuário diretamente em query SQL permite que o atacante altere a lógica da consulta. Pode-se fazer bypass de autenticação (`admin'--`), *vazar dados inteiros* (`'1=1`), ou executar queries arbitrárias. A defesa padrão é usar prepared statements (parâmetros vinculados): o DBMS separa a estrutura SQL dos dados, garantindo que o input seja sempre interpretado como valor literal, nunca como código.

Código vulnerável (A05-injecao/01-sql-injection/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/*
 * SQL Injection - FALHA A05:2025 Injection
 * Concatenação direta de entrada do usuário na query SQL permite
 * que um atacante altere a lógica da consulta e acesse/modifique dados.
 */

function criarBancoDeTeste(): \PDO
{
    $pdo = new \PDO('sqlite::memory:');
    $pdo->exec("CREATE TABLE usuarios (id INTEGER PRIMARY KEY, usuario TEXT,
senha_hash TEXT)");
```

```

    $pdo->exec("INSERT INTO usuarios (usuario, senha_hash) VALUES ('admin', '" . md5
('senhaForte123') . "')");
    return $pdo;
}

function login(PDO $pdo, string $usuario, string $senha): ?array
{
    $senhaHash = md5($senha);
    // VULNERÁVEL: concatenação direta de entrada do usuário na query
    $query = "SELECT * FROM usuarios WHERE usuario = '{$usuario}' AND senha_hash = '
{$senhaHash}'";
    $resultado = $pdo->query($query);

    if ($resultado === false) {
        return null;
    }

    $linha = $resultado->fetch(PDO::FETCH_ASSOC);
    return $linha ?: null;
}

function demo(): void
{
    $pdo = criarBancoDeTeste();

    echo "=== Demonstração: SQL Injection (VULNERÁVEL) ===\n";

    // Teste legítimo
    echo "\n1. Login legítimo com senha correta:\n";
    $resultado = login($pdo, 'admin', 'senhaForte123');
    echo "    Resultado: " . ($resultado ? "Usuário autenticado: " . $resultado
['usuario'] : "Falha") . "\n";

    // SQL Injection: bypass de autenticação
    echo "\n2. SQL Injection - bypass de autenticação:\n";
    echo "    Input: admin' -- \n";
    $resultado = login($pdo, "admin' -- ", 'senhaQualquer');
    echo "    Resultado: " . ($resultado ? "VULNERÁVEL! Usuário autenticado sem senha
correta: " . $resultado['usuario'] : "Bloqueado") . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A função `login()` concatena entrada do usuário diretamente na query: `SELECT * FROM usuarios WHERE usuario = '{$usuario}' AND senha_hash = '{$senhaHash}'`. Um atacante passa `admin'--`` como usuário; a query vira `'SELECT * FROM usuarios WHERE usuario = 'admin'--`' AND ...`, comentando o restante e fazendo login sem senha. Testado com `sqlmap` em ambiente real (veja detalhes no [Apêndice A](#)).

```
<?php
declare(strict_types=1);

namespace Corrigido;

/*
 * SQL Injection - CORREÇÃO
 * Usar prepared statements com parâmetros vinculados (:nome ou ?)
 * garante que a entrada do usuário seja sempre tratada como valor literal,
 * nunca como código SQL, mesmo contendo caracteres especiais.
 */

function criarBancoDeTeste(): \PDO
{
    $pdo = new \PDO('sqlite::memory:');
    $pdo->exec("CREATE TABLE usuarios (id INTEGER PRIMARY KEY, usuario TEXT,
senha_hash TEXT)");
    $pdo->exec("INSERT INTO usuarios (usuario, senha_hash) VALUES ('admin', '" . md5
('senhaForte123') . "')");
    return $pdo;
}

function login(\PDO $pdo, string $usuario, string $senha): ?array
{
    $senhaHash = md5($senha);
    // CORRIGIDO: prepared statement com parâmetros vinculados
    $stmt = $pdo->prepare('SELECT * FROM usuarios WHERE usuario = :usuario AND
senha_hash = :senha_hash');
    $stmt->execute(['usuario' => $usuario, 'senha_hash' => $senhaHash]);
    $linha = $stmt->fetch(\PDO::FETCH_ASSOC);
    return $linha ? : null;
}

function demo(): void
{
    $pdo = criarBancoDeTeste();

    echo "=== Demonstração: SQL Injection (CORRIGIDO) ===\n";

    // Teste legítimo
    echo "\n1. Login legítimo com senha correta:\n";
    $resultado = login($pdo, 'admin', 'senhaForte123');
    echo "    Resultado: " . ($resultado ? "Usuário autenticado: " . $resultado
['usuario'] : "Falha") . "\n";

    // SQL Injection: tentativa bloqueada
    echo "\n2. SQL Injection - tentativa bloqueada:\n";
    echo "    Input: admin' -- \n";
    $resultado = login($pdo, "admin' -- ", 'senhaQualquer');
```

```

    echo "    Resultado: " . ($resultado ? "VULNERÁVEL! Bypass conseguiu." : "Bloqueado
corretamente") . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção usa prepared statements: `$pdo->prepare('SELECT * FROM usuarios WHERE usuario = :usuario AND senha_hash = :senha_hash') + execute(['usuario' => $usuario, ...])`. O DBMS recebe a query com placeholders `:usuario` e `:senha_hash`, depois os valores são enviados separadamente. Nenhuma concatenação, input é sempre literal — ``admin`` — não consegue quebrar sintaxe.

6.1.1. Como testar

Rode `php A05-injecao/01-sql-injection/teste.php` para executar:

1. Testa login legítimo (usuário e senha corretos) — ambas as versões devem retornar o usuário.
2. Testa SQL Injection na versão vulnerável: com input ``admin`` — , consegue fazer login sem senha correta.
3. Testa SQL Injection na versão corrigida: com input ``admin`` — , login falha (input é tratado como string literal).

Adicionalmente, há um teste com sqlmap contra endpoints HTTP reais. Suba `php -S 127.0.0.1:8899` nesta pasta e rode `./testar-com-sqlmap.sh` — sqlmap confirma 2 técnicas de injeção (boolean-based blind e error-based) no parâmetro `usuario` da versão vulnerável, e nenhuma injeção na versão corrigida. Mais detalhes no [Apêndice A](#).

Saída esperada: **RESULTADO: 3/3 verificacoes passaram.**

6.1.2. Checklist de verificação

- ☐ Todas as queries SQL usam prepared statements (`:nome` ou `?`) + `execute()` ou `bindParam()`?
- ☐ Jamais há concatenação de input diretamente na query string (nem mesmo com `addslashes()` ou quotes escapadas)?
- ☐ Senhas são hashadas com `password_hash()` antes de comparar (este exemplo ainda usa MD5 por simplicidade, mas veja seção A04)?

6.2. 02 — XSS (Cross-Site Scripting)

CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')

XSS ocorre quando entrada do usuário é renderizada no HTML sem escape, permitindo injeção de scripts ou atributos maliciosos. Um atacante injeta `<script>alert('XSS')</script>` num comentário, e quando outro usuário vê o comentário, o script roda no navegador — roubando cookies, redirecionar para phishing, ou modificar conteúdo da página. A defesa é escapar caracteres especiais HTML (`<`, `>`, `&`, `"`, `'`) antes de renderizar, tornando-os literais.

```

<?php
declare(strict_types=1);

namespace Vulneravel;

/*
 * Cross-Site Scripting (XSS) - FALHA A05:2025 Injection
 * Renderizar entrada do usuário diretamente no HTML sem escapar
 * permite injeção de scripts maliciosos que serão executados no navegador.
 */

function renderizarComentario(string $comentario): string
{
    // VULNERÁVEL: entrada do usuário concatenada diretamente no HTML
    return "<div class=\"comentario\">{$comentario}</div>";
}

function demo(): void
{
    echo "=== Demonstração: XSS (VULNERÁVEL) ===\n";

    // Teste legítimo
    echo "\n1. Comentário legítimo:\n";
    $html = renderizarComentario("Que tutorial excelente!");
    echo "    HTML gerado: {$html}\n";

    // XSS Attack
    echo "\n2. XSS Attack - script injection:\n";
    $malicioso = '<script>alert("XSS")</script>';
    echo "    Input: {$malicioso}\n";
    $html = renderizarComentario($malicioso);
    echo "    HTML gerado: {$html}\n";
    if (str_contains($html, '<script>')) {
        echo "    ☐ VULNERÁVEL! Script será executado pelo navegador\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A função `renderizarComentario()` concatena input diretamente no HTML: `<div>{$comentario}</div>`. Um atacante injeta `<script>alert('XSS')</script>`, que fica como HTML inofensivo na string — mas quando renderizado no navegador, o `<script>` é interpretado e o código roda.

```

<?php

```

```

declare(strict_types=1);

namespace Corrigido;

/*
 * Cross-Site Scripting (XSS) - CORREÇÃO
 * Usar htmlspecialchars() com ENT_QUOTES e charset UTF-8 para escapar
 * caracteres especiais de HTML (<, >, &, ", '), tornando o input
 * inofensivo e renderizado como texto literal.
 */

function renderizarComentario(string $comentario): string
{
    // CORRIGIDO: escapar HTML com htmlspecialchars
    return "<div class=\"comentario\">\" . htmlspecialchars($comentario, ENT_QUOTES,
'UTF-8') . "</div>";
}

function demo(): void
{
    echo "=== Demonstração: XSS (CORRIGIDO) ===\n";

    // Teste legítimo
    echo "\n1. Comentário legítimo:\n";
    $html = renderizarComentario("Que tutorial excelente!");
    echo "    HTML gerado: {$html}\n";

    // XSS Attack - bloqueado
    echo "\n2. XSS Attack - bloqueado:\n";
    $malicioso = '<script>alert("XSS")</script>';
    echo "    Input: {$malicioso}\n";
    $html = renderizarComentario($malicioso);
    echo "    HTML gerado: {$html}\n";
    if (!str_contains($html, '<script>') && str_contains($html, '&lt;script&gt;')) {
        echo "    ✓ Script escapado, será exibido como texto\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção usa `htmlspecialchars($comentario, ENT_QUOTES, 'UTF-8')` — converte caracteres especiais em entidades HTML: `<` vira `<`, `>` vira `>`, `"` vira `"`, `'` vira `'`. Resultado: o script é renderizado como texto literal no HTML, nunca executado pelo navegador.

6.2.1. Como testar

Rode `php A05-injecao/02-xss/teste.php`:

1. Testa comentário legítimo — ambas as versões renderizam corretamente.
2. Testa injeção XSS na versão vulnerável: `<script>alert('XSS')</script>` fica como HTML vivo (contém `<script>` literal).
3. Testa XSS na versão corrigida: script é escapado, fica como `<script>...</script>` no HTML.

Saída esperada: **RESULTADO: 3/3 verificacoes passaram.**

6.2.2. Checklist de verificação

- ☐ Todo conteúdo renderizado do usuário usa `htmlspecialchars(..., ENT_QUOTES, 'UTF-8')`?
- ☐ Conteúdo renderizado em atributos HTML (ex: `value="..."`) também é escapado?
- ☐ Se há conteúdo no JSON, escapar para contexto JSON (funções como `json_encode()` já fazem isso se usar flags corretas)?

6.3. 03 — Command Injection

CWE-78: Improper Neutralization of Special Elements used in an OS Command ('OS Command Injection')

Command Injection permite executar comandos do sistema operacional (shell, binários) injetando metacaracteres que separam ou encadeiam comandos. Um atacante passa `foto.jpg; rm -rf /` num parâmetro de nome de arquivo, e consegue executar comandos arbitrários no servidor. A defesa é escapar argumentos com `escapeshellarg()` ou usar APIs que não invocam shell (como `proc_open()` com array de argumentos).

Código vulnerável (`A05-injecao/03-command-injection/vulneravel.php`)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/*
 * Command Injection (OS Injection) - FALHA A05:2025 Injection
 * Concatenação de entrada do usuário em comandos do sistema operacional
 * permite execução arbitrária de comandos, mesmo sem intenção do desenvolvedor.
 */

function montarComandoConversaoImagem(string $nomeArquivo): string
{
    // VULNERÁVEL: concatenação direta do nome de arquivo no comando
    // Um atacante pode injetar ; ou && ou | para executar comandos extras
    return "convert {$nomeArquivo} saida.png";
}

function demo(): void
{
    echo "=== Demonstração: Command Injection (VULNERÁVEL) ===\n";
}
```

```

// Teste legítimo
echo "\n1. Comando legítimo:\n";
$comando = montarComandoConversaoImagem("imagem.jpg");
echo "    Comando: {$comando}\n";

// Command Injection
echo "\n2. Command Injection - concatenação de comando:\n";
$nomemalicioso = "foto.jpg; echo INJETADO";
echo "    Input: {$nomemalicioso}\n";
$comando = montarComandoConversaoImagem($nomemalicioso);
echo "    Comando: {$comando}\n";
if (str_contains($comando, "; echo INJETADO")) {
    echo "    ☐ VULNERÁVEL! O ; não está escapado, permitindo comando encadeado\n";
};
}
}

if (debug_backtrace() === []) {
    demo();
}

```

A função `montarComandoConversaoImagem()` concatena nome de arquivo no comando: `convert {$nomeArquivo} saida.png`. Um atacante passa `foto.jpg; echo INJETADO`, que vira `convert foto.jpg; echo INJETADO saida.png` — o `;` encadeia um novo comando.

Código corrigido (A05-injecao/03-command-injection/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/*
 * Command Injection - CORREÇÃO
 * Usar escapeshellarg() para envolver cada argumento do usuário
 * em aspas simples escapadas, prevenindo que metacaracteres (;, &, |, etc)
 * sejam interpretados como separadores de comando.
 */

function montarComandoConversaoImagem(string $nomeArquivo): string
{
    // CORRIGIDO: usar escapeshellarg para proteger o argumento
    return "convert " . escapeshellarg($nomeArquivo) . " saida.png";
}

function demo(): void
{
    echo "=== Demonstração: Command Injection (CORRIGIDO) ===\n";
}

```

```
// Teste legítimo
echo "\n1. Comando legítimo:\n";
$comando = montarComandoConversaoImagem("imagem.jpg");
echo "    Comando: {$comando}\n";

// Command Injection - bloqueado
echo "\n2. Command Injection - bloqueado:\n";
$nomemalicioso = "foto.jpg; echo INJETADO";
echo "    Input: {$nomemalicioso}\n";
$comando = montarComandoConversaoImagem($nomemalicioso);
echo "    Comando: {$comando}\n";
if (str_contains($comando, "foto.jpg; echo INJETADO")) {
    echo "    ✓ Nome inteiro está entre aspas simples, ; é literal\n";
}
}

if (debug_backtrace() === []) {
    demo();
}
```

A correção usa `escapeshellarg($nomeArquivo)` — envolve o argumento em aspas simples escapadas, tornando-o um argumento literal para o shell. `foto.jpg; echo INJETADO` vira `'foto.jpg; echo INJETADO'`, e o `;` é interpretado como caractere literal, não separador de comando.

6.3.1. Como testar

Rode `php A05-injecao/03-command-injection/teste.php`:

1. Testa comando legítimo (nome simples de arquivo) — ambas as versões montam corretamente.
2. Testa injeção na versão vulnerável: `foto.jpg; echo INJETADO` não tem quotes, `;` é separador.
3. Testa injeção na versão corrigida: mesmo input fica entre aspas simples, `;` é literal.

Saída esperada: `RESULTADO: 3/3 verificacoes passaram.`

6.3.2. Checklist de verificação

- ☐ Nenhuma chamada a `shell_exec()`, `exec()`, `system()`, `passthru()` com input do usuário sem `escapeshellarg()`?
- ☐ Preferência é usar `proc_open()` com array de argumentos (não passa por shell, evita problemas com escape)?
- ☐ Logs/monitoramento registram comandos executados para auditoria de segurança?

6.4. 04 — NoSQL Injection

CWE-89: Improper Neutralization of Special Elements used in an SQL Command (generalizado para NoSQL)

Bancos NoSQL (MongoDB, Redis, etc.) têm sua própria linguagem de query, e podem ser injetados

assim como SQL. Em MongoDB, por exemplo, operadores como `$ne` (diferente de), `$gt` (maior que), `$regex` permitem construir queries complexas. Um atacante que consegue passar um objeto/array (de JSON) pode injetar operadores: `{ "usuario": "admin", "senha": { "$ne": "" } }` faz login sem saber a senha real, porque `$ne: ""` significa "senha diferente de string vazia" — qualquer senha válida casa.

Código vulnerável (A05-injecao/04-nosql-injection/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/*
 * NoSQL Injection - FALHA A05:2025 Injection
 * Aceitar objetos/arrays com operadores NoSQL (como MongoDB $ne, $gt, etc)
 * diretamente da entrada do usuário permite bypass de autenticação
 * e acesso não autorizado a dados.
 */

function login(array $colecacaoUsuarios, array $filtro): ?array
{
    // VULNERÁVEL: aceita operadores NoSQL do atacante sem validação
    // Um atacante pode passar { "usuario": "admin", "senha": { "$ne": "" } }
    // e conseguir login sem saber a senha real

    foreach ($colecacaoUsuarios as $usuario) {
        $bate = true;
        foreach ($filtro as $campo => $valor) {
            if (is_array($valor) && isset($valor['$ne'])) {
                // Operador $ne: "diferente de"
                // Bate se o valor do documento é diferente do valor especificado
                if ($usuario[$campo] === $valor['$ne']) {
                    $bate = false;
                    break;
                }
            } else {
                // Comparação de igualdade direta
                if ($usuario[$campo] !== $valor) {
                    $bate = false;
                    break;
                }
            }
        }
        if ($bate) {
            return $usuario;
        }
    }
    return null;
}
```

```

function demo(): void
{
    echo "=== Demonstração: NoSQL Injection (VULNERÁVEL) ===\n";

    $usuarios = [
        ['usuario' => 'admin', 'senha' => 'senhaForte123'],
        ['usuario' => 'user', 'senha' => 'senha456'],
    ];

    // Teste legítimo
    echo "\n1. Login legítimo com senha correta:\n";
    $resultado = login($usuarios, ['usuario' => 'admin', 'senha' => 'senhaForte123']);
    echo "    Resultado: " . ($resultado ? "Usuário: " . $resultado['usuario'] :
"Falha") . "\n";

    // NoSQL Injection: bypass usando $ne
    echo "\n2. NoSQL Injection - bypass com operador \$ne:\n";
    echo "    Filtro: {'usuario': 'admin', 'senha': {'\$ne': ''}}\n";
    $filtroInjection = ['usuario' => 'admin', 'senha' => ['$ne' => '']];
    $resultado = login($usuarios, $filtroInjection);
    echo "    Resultado: " . ($resultado ? "VULNERÁVEL! Bypass sem senha correta: " .
$resultado['usuario'] : "Bloqueado") . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A função `login()` aceita um filtro array arbitrário e verifica operadores NoSQL como `$ne`. Um atacante passa `['usuario' => 'admin', 'senha' => ['$ne' => '']]`, e consegue fazer login sem senha correta porque qualquer senha válida é diferente de "".

Código corrigido (A05-injecao/04-nosql-injection/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/*
 * NoSQL Injection - CORREÇÃO
 * Validar rigorosamente o tipo de cada parâmetro esperado.
 * Campos críticos como senha DEVEM ser strings simples, nunca arrays ou objetos.
 * Rejeitar qualquer filtro que contenha estruturas inesperadas.
 */

function login(array $colecãoUsuarios, array $filtro): ?array
{
    // CORRIGIDO: validar que usuario e senha são OBRIGATORIAMENTE strings
    // Rejeitar qualquer operador NoSQL ou estrutura complexa

```

```

    if (!is_string($filtro['usuario'] ?? null) || !is_string($filtro['senha'] ??
null)) {
        return null; // Rejeita filtro inválido
    }

    foreach ($colecacaoUsuarios as $usuario) {
        if ($usuario['usuario'] === $filtro['usuario'] && $usuario['senha'] ===
$filtro['senha']) {
            return $usuario;
        }
    }
    return null;
}

function demo(): void
{
    echo "=== Demonstração: NoSQL Injection (CORRIGIDO) ===\n";

    $usuarios = [
        ['usuario' => 'admin', 'senha' => 'senhaForte123'],
        ['usuario' => 'user', 'senha' => 'senha456'],
    ];

    // Teste legítimo
    echo "\n1. Login legítimo com senha correta:\n";
    $resultado = login($usuarios, ['usuario' => 'admin', 'senha' => 'senhaForte123']);
    echo "    Resultado: " . ($resultado ? "Usuário: " . $resultado['usuario'] :
"Falha") . "\n";

    // NoSQL Injection: bloqueado
    echo "\n2. NoSQL Injection - bloqueado por validação de tipo:\n";
    echo "    Filtro: {'usuario': 'admin', 'senha': {'\$ne': ''}}\n";
    $filtroInjection = ['usuario' => 'admin', 'senha' => ['$ne' => '']];
    $resultado = login($usuarios, $filtroInjection);
    echo "    Resultado: " . ($resultado ? "VULNERÁVEL! Bypass conseguiu." : "Bloqueado
corretamente") . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção valida que campos críticos (como `senha`) são **obrigatoriamente strings**, nunca arrays: `if (!is_string($filtro['senha'] ?? null)) return null;`. Qualquer tentativa de passar `['$ne' => '']` é rejeitada porque não é string.

6.4.1. Como testar

Rode `php A05-injecao/04-nosql-injection/teste.php`:

1. Testa login legítimo (filtro com strings) — ambas as versões retornam usuário.
2. Testa NoSQL Injection na versão vulnerável: ['\$ne' ⇒ ''] consegue bypass.
3. Testa NoSQL Injection na versão corrigida: validação de tipo rejeita operador.

Saída esperada: **RESULTADO: 3/3 verificacoes passaram.**

6.4.2. Checklist de verificação

- ☐ Campos críticos (senhas, tokens, autenticação) são validados quanto ao tipo esperado (string, int, etc.)?
- ☐ Jamais há desserialização de JSON/input do usuário diretamente como operadores de query?
- ☐ Se a aplicação usa query builder ou ORM NoSQL, ela expõe método seguro de vinculação de parâmetros (ex: MongoDB's prepared queries)?

6.5. 05 — LDAP Injection

CWE-90: Improper Neutralization of Special Elements used in an LDAP Query ('LDAP Injection')

LDAP (Lightweight Directory Access Protocol) é uma linguagem para consultas em diretórios. Como SQL, aceita metacaracteres especiais (, (,), \, NUL) que mudam a semântica de queries. Um filtro LDAP como (&(uid=joao)(objectClass=person)) pode ser quebrado injetando)(uid=)| (uid= no campo uid, alterando a lógica para (&(uid=)(uid=))(|(uid=*)(objectClass=person)) — que casa com qualquer usuário.

Código vulnerável (A05-injecao/05-ldap-injection/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/*
 * LDAP Injection - FALHA A05:2025 Injection
 * Concatenar entrada do usuário em filtros LDAP sem escape permite
 * que atacantes alterem a lógica da consulta LDAP e acessem/modifiquem dados.
 */

function montarFiltroLdap(string $usuario): string
{
    // VULNERÁVEL: concatenação direta do usuário no filtro LDAP
    // Um atacante pode injetar estruturas LDAP complexas alterando o significado
    return "(&(uid={$usuario})(objectClass=person))";
}

function demo(): void
{
    echo "=== Demonstração: LDAP Injection (VULNERÁVEL) ===\n";
}
```

```
// Teste legítimo
echo "\n1. Filtro legítimo:\n";
$filtero = montarFiltroLdap("joao");
echo "    Filtro: {$filtero}\n";

// LDAP Injection
echo "\n2. LDAP Injection - alteração de lógica:\n";
$usuarioMalicioso = "*)(uid=*)(|(uid=*";
echo "    Input: {$usuarioMalicioso}\n";
$filtero = montarFiltroLdap($usuarioMalicioso);
echo "    Filtro: {$filtero}\n";
if (str_contains($filtero, "*)(uid=*)(|(uid=*)) {
    echo "    ☐ VULNERÁVEL! Estrutura LDAP foi alterada, filtro agora pode casar
com qualquer usuário\n";
}
}

if (debug_backtrace() === []) {
    demo();
}
```

A função `montarFiltroLdap()` concatena input no filtro: `(&(uid={$usuario})(objectClass=person))`. Um atacante injeta `)(uid=*)(|(uid=*`, quebrando a sintaxe e casando com qualquer usuário.

Código corrigido (A05-injecao/05-ldap-injection/corrigido.php)

```
<?php
declare(strict_types=1);

namespace Corrigido;

/*
 * LDAP Injection - CORREÇÃO
 * Implementar escape de filtro LDAP que substitui metacaracteres
 * por suas codificações hexadecimais, tornando-os literais.
 * Ordem importante: escapar \ PRIMEIRO, depois os demais.
 */

function escaparFiltroLdap(string $valor): string
{
    // IMPORTANTE: escapar barra invertida PRIMEIRO
    $valor = str_replace("\\", "\\5c", $valor);
    $valor = str_replace("*", "\\2a", $valor);
    $valor = str_replace("(", "\\28", $valor);
    $valor = str_replace(")", "\\29", $valor);
    $valor = str_replace("\x00", "\\00", $valor);
    return $valor;
}

function montarFiltroLdap(string $usuario): string
```

```

{
    // CORRIGIDO: escapar o usuário antes de inserir no filtro
    return "(&(uid=" . escaparFiltroLdap($usuario) . ")(objectClass=person))";
}

function demo(): void
{
    echo "=== Demonstração: LDAP Injection (CORRIGIDO) ===\n";

    // Teste legítimo
    echo "\n1. Filtro legítimo:\n";
    $filtro = montarFiltroLdap("joao");
    echo "    Filtro: {$filtro}\n";

    // LDAP Injection - bloqueado
    echo "\n2. LDAP Injection - bloqueado:\n";
    $usuarioMalicioso = "*)(uid=*))(|(uid=*";
    echo "    Input: {$usuarioMalicioso}\n";
    $filtro = montarFiltroLdap($usuarioMalicioso);
    echo "    Filtro: {$filtro}\n";
    if (!str_contains($filtro, "*)(uid=*))(|(uid=*)) {
        echo "    ✓ Metacaracteres foram escapados, estrutura LDAP intacta\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção implementa escape LDAP conforme RFC4515: substitui metacaracteres (\\, , (,), NUL) por seus códigos hexadecimais (\\5c, \\2a, \\28, \\29, \\00). Resultado: (&(uid=\\2a\\29\\28uid\\3d\\2a\\29\\29\\28\\7c\\28uid\\3d\\2a, interpretado como string literal no filtro LDAP.

6.5.1. Como testar

Rode `php A05-injecao/05-ldap-injection/teste.php`:

1. Testa filtro legítimo (nome simples) — ambas as versões montam corretamente.
2. Testa LDAP Injection na versão vulnerável: metacaracteres quebram a sintaxe.
3. Testa LDAP Injection na versão corrigida: metacaracteres são escapados, sintaxe intacta.

Saída esperada: `RESULTADO: 3/3 verificacoes passaram.`

6.5.2. Checklist de verificação

- ☐ Se a aplicação usa LDAP, inputs são escapados com `ldap_escape()` ou função equivalente antes de montar filtros?
- ☐ Ordem do escape é importante: barra invertida (\\) deve ser escapada **primeiro**, depois os

demais metacaracteres?

- Bibliotecas LDAP não fazem escape automaticamente — sempre validar/escapar manualmente?

6.6. 06 — Code Injection / EL Injection

CWE-94: Improper Control of Generation of Code ('Code Injection')

Usar `eval()` ou construções similares para processar templates ou expressões do usuário permite execução arbitrária de código PHP. Diferente de XSS (que roda no navegador), Code Injection roda no servidor — um atacante consegue executar `system()`, acessar banco de dados, ou destruir arquivos. Equivalente em Java é Expression Language (EL) Injection ou OGNL Injection. A defesa é nunca usar `eval()` com input não confiável, e sempre implementar parsers restritos (regex, whitelist, etc).

Código vulnerável (A05-injecao/06-code-injection-eval/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/*
 * Code Injection / Expression Language Injection - FALHA A05:2025 Injection
 * Usar eval() ou construções similares para processar templates com entrada do
usuário
 * permite execução arbitrária de código PHP, equivalente ao OGNL injection em Java.
 */

function avaliarTemplate(string $template, array $variaveis): string
{
    // VULNERÁVEL: usar eval() para processar expressões do usuário
    // Um atacante pode injetar PHP arbitrário, ex: {{ system('comando') }}

    $resultado = $template;
    // Procura por {{...}} e avalia como código PHP
    $resultado = preg_replace_callback(
        '/\{\{(.+)\}\}/',
        function ($matches) use ($variaveis) {
            $expr = trim($matches[1]);
            // PERIGOSO: avaliar como PHP arbitrário
            // Construir bloco de atribuição de variáveis para o eval
            $varCode = '';
            foreach ($variaveis as $k => $v) {
                $varCode .= '$' . $k . ' = ' . var_export($v, true) . ' ';
            }
            // Avaliar a expressão como PHP puro
            $fullCode = $varCode . 'return ' . $expr . ' ';
            return (string)eval($fullCode);
        },
        $resultado
    );
}
```

```

    );
    return $resultado;
}

function demo(): void
{
    echo "=== Demonstração: Code Injection (VULNERÁVEL) ===\n";

    // Teste legítimo
    echo "\n1. Template legítimo com variável:\n";
    $resultado = avaliarTemplate("Olá {{\ $nome}}", ['nome' => 'João']);
    echo "    Resultado: {$resultado}\n";

    // Code Injection
    echo "\n2. Code Injection - execução de código arbitrário:\n";
    $template = "Teste: {{ system('echo INJETADO') }}";
    echo "    Template: {$template}\n";
    ob_start();
    try {
        $resultado = avaliarTemplate($template, []);
    } catch (\Throwable $e) {
        ob_end_clean();
        echo "    Erro ao processar: " . $e->getMessage() . "\n";
        return;
    }
    $output = ob_get_clean();
    echo "    Resultado: {$resultado}\n";
    if (str_contains($output, "INJETADO")) {
        echo "    ☐☐ VULNERÁVEL! Código arbitrário foi executado\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A função `avaliarTemplate()` procura por `{{...}}` e usa `eval()` para processar a expressão. Um atacante injeta `{{ system('comando') }}`, e o código arbitrário é executado no servidor.

Código corrigido (A05-injecao/06-code-injection-eval/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/*
 * Code Injection / Expression Language Injection - CORREÇÃO
 * Nunca usar eval() ou construções similares com entrada do usuário.
 * Processar templates com um conjunto restrito de padrões conhecidos,

```

```

* permitindo apenas nomes de variáveis simples, sem chamadas de função.
*/

function avaliarTemplate(string $template, array $variaveis): string
{
    // CORRIGIDO: regex estrita que só aceita nomes de variável simples
    // Padrão: {{ nomeSimples }}, onde nomeSimples é [a-zA-Z_][a-zA-Z0-9_]*
    $resultado = preg_replace_callback(
        '/\{\{s*([a-zA-Z_][a-zA-Z0-9_]*)s*\}\}/',
        function ($matches) use ($variaveis) {
            $nomeDaVariavel = $matches[1];
            // Apenas substituir pelo valor da variável, NUNCA executar código
            return $variaveis[$nomeDaVariavel] ?? '';
        },
        $template
    );
    return $resultado;
}

function demo(): void
{
    echo "=== Demonstração: Code Injection (CORRIGIDO) ===\n";

    // Teste legítimo
    echo "\n1. Template legítimo com variável:\n";
    $resultado = avaliarTemplate("Olá {{nome}}", ['nome' => 'João']);
    echo "    Resultado: {$resultado}\n";
    echo "    Nota: A versão corrigida aceita {{nome}} (sem $) porque a regex valida
nomes de variável\n";

    // Code Injection - bloqueado
    echo "\n2. Code Injection - bloqueado:\n";
    $template = "Teste: {{ system('echo INJETADO') }}";
    echo "    Template: {$template}\n";
    $resultado = avaliarTemplate($template, []);
    echo "    Resultado: {$resultado}\n";
    if (!str_contains($resultado, "INJETADO")) {
        echo "    ✓ Código malicioso não foi executado, padrão rejeitado\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção implementa regex estrita: `/\{\{s*([a-zA-Z_][a-zA-Z0-9_]*)s*\}\}/` — só aceita nomes de variável simples dentro de `{{}}`, jamais chamadas de função. `{{ system(...) }}` não casa, é ignorado. Apenas `{{ nome }}` funciona, substituindo pelo valor literal da variável.

6.6.1. Como testar

Rode `php A05-injecao/06-code-injection-eval/teste.php`:

1. Testa template legítimo (nome de variável simples) — ambas as versões renderizam corretamente.
2. Testa Code Injection na versão vulnerável: `{{ system('echo INJETADO') }}` executa comando.
3. Testa Code Injection na versão corrigida: padrão não casa, comando não é executado.

Saída esperada: `RESULTADO: 3/3 verificacoes passaram.`

6.6.2. Checklist de verificação

- ☐ Zero uso de `eval()` com qualquer input não confiável?
- ☐ Se há template engine customizado, é baseado em regex restrita + whitelist (não em `eval()` ou construção dinâmica de código)?
- ☐ Bibliotecas de template (Twig, Blade, etc.) são usadas em preferência a rolos customizados?

6.7. 07 — ORM Injection

CWE-89: Improper Neutralization of Special Elements used in an SQL Command (generalizado para ORM)

Mesmo quando se usa uma camada ORM (Object-Relational Mapping), SQL Injection é possível se o desenvolvedor expuser métodos que aceitam condições SQL livres. Por exemplo, um método `buscarPorFiltro($sql)` que concatena a condição na query é tão vulnerável quanto SQL direto. A defesa é nunca expor métodos genéricos de filtro — sempre oferecer métodos específicos que aceitem apenas valores de campos específicos, usando prepared statements internamente.

Código vulnerável (`A05-injecao/07-orm-injection/vulneravel.php`)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/*
 * ORM Injection / SQL Injection via ORM - FALHA A05:2025 Injection
 * Aceitar condições SQL arbitrárias mesmo quando usando uma camada ORM
 * permite injeção SQL se o desenvolvedor expuser a construção de queries.
 */

function criarBancoDeTeste(): \PDO
{
    $pdo = new \PDO('sqlite::memory:');
    $pdo->exec("CREATE TABLE usuarios (id INTEGER PRIMARY KEY, usuario TEXT,
senha_hash TEXT)");
    $pdo->exec("INSERT INTO usuarios (usuario, senha_hash) VALUES ('admin', ' . md5
```

```

('senhaForte123') . "'");
    $pdo->exec("INSERT INTO usuarios (usuario, senha_hash) VALUES ('user', '" . md5
('senha456') . "'");
    return $pdo;
}

class RepositorioUsuarios
{
    private \PDO $pdo;

    public function __construct(\PDO $pdo)
    {
        $this->pdo = $pdo;
    }

    /**
     * VULNERÁVEL: aceita uma condição SQL arbitrária
     */
    public function buscarPorFiltroRaw(string $condicaoSql): array
    {
        // VULNERÁVEL: concatenação direta da condição no WHERE
        $query = "SELECT * FROM usuarios WHERE {$condicaoSql}";
        $resultado = $this->pdo->query($query);

        if ($resultado === false) {
            return [];
        }

        return $resultado->fetchAll(\PDO::FETCH_ASSOC);
    }
}

function demo(): void
{
    echo "=== Demonstração: ORM Injection (VULNERÁVEL) ===\n";

    $pdo = criarBancoDeTeste();
    $repo = new RepositorioUsuarios($pdo);

    // Teste legítimo
    echo "\n1. Busca legítima por usuário específico:\n";
    $usuarios = $repo->buscarPorFiltroRaw("usuario = 'admin'");
    echo "    Resultado: " . count($usuarios) . " usuário(s) encontrado(s)\n";

    // SQL Injection: retornar todos os usuários
    echo "\n2. SQL Injection - retornar todos os usuários:\n";
    echo "    Filtro: 1=1\n";
    $usuarios = $repo->buscarPorFiltroRaw("1=1");
    echo "    Resultado: " . count($usuarios) . " usuário(s) encontrado(s)\n";
    if (count($usuarios) > 1) {
        echo "    ☐ VULNERÁVEL! Vazamento de dados: " . implode(', ', array_column

```

```

($usuarios, 'usuario')) . "\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A classe `RepositorioUsuarios` expõe `buscarPorFiltroRaw($condicaoSql)`, que concatena a condição: `SELECT * FROM usuarios WHERE {$condicaoSql}`. Um atacante passa `1=1`, vazando todos os usuários.

Código corrigido (A05-injecao/07-orm-injection/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/*
 * ORM Injection - CORREÇÃO
 * Não expor nenhum parâmetro de construção SQL livre.
 * Usar métodos específicos que aceitam apenas valores de campos específicos,
 * nunca condições SQL arbitrárias, garantindo prepared statements internamente.
 */

function criarBancoDeTeste(): \PDO
{
    $pdo = new \PDO('sqlite::memory:');
    $pdo->exec("CREATE TABLE usuarios (id INTEGER PRIMARY KEY, usuario TEXT,
senha_hash TEXT)");
    $pdo->exec("INSERT INTO usuarios (usuario, senha_hash) VALUES ('admin', '"' . md5
('senhaForte123') . '"')");
    $pdo->exec("INSERT INTO usuarios (usuario, senha_hash) VALUES ('user', '"' . md5
('senha456') . '"')");
    return $pdo;
}

class RepositorioUsuarios
{
    private \PDO $pdo;

    public function __construct(\PDO $pdo)
    {
        $this->pdo = $pdo;
    }

    /**
     * CORRIGIDO: método específico que só aceita um nome de usuário
     * Usa prepared statement internamente, nunca expõe construção SQL
     */
}

```

```

public function buscarPorNomeUsuario(string $nomeUsuario): array
{
    // CORRIGIDO: prepared statement com parâmetro vinculado
    $stmt = $this->pdo->prepare('SELECT * FROM usuarios WHERE usuario = :usuario');
    $stmt->execute(['usuario' => $nomeUsuario]);
    return $stmt->fetchAll(\PDO::FETCH_ASSOC);
}

function demo(): void
{
    echo "=== Demonstração: ORM Injection (CORRIGIDO) ===\n";

    $pdo = criarBancoDeTeste();
    $repo = new RepositorioUsuarios($pdo);

    // Teste legítimo
    echo "\n1. Busca legítima por usuário específico:\n";
    $usuarios = $repo->buscarPorNomeUsuario('admin');
    echo "    Resultado: " . count($usuarios) . " usuário(s) encontrado(s)\n";

    // Tentativa de injection é bloqueada (não há método que aceite filtros livres)
    echo "\n2. Injeção bloqueada - método específico não expõe SQL:\n";
    echo "    Não existe método buscarPorFiltroRaw(), precisa usar buscarPorNomeUsuario()\n";
    $usuarios = $repo->buscarPorNomeUsuario('1=1');
    echo "    Resultado: " . count($usuarios) . " usuário(s) encontrado(s)\n";
    if (count($usuarios) === 0) {
        echo "    ✓ Nenhum usuário com nome '1=1', injeção foi neutralizada\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção não expõe `buscarPorFiltroRaw()`. Em seu lugar, há `buscarPorNomeUsuario($nome)` — um método específico que só aceita um nome de usuário via prepared statement: `prepare('SELECT * FROM usuarios WHERE usuario = :usuario')`. Não há como injetar SQL porque o input é sempre um valor, nunca condição arbitrária.

6.7.1. Como testar

Rode `php A05-injecao/07-orm-injection/teste.php`:

1. Testa busca legítima (nome específico) — ambas as versões retornam usuário correto.
2. Testa ORM Injection na versão vulnerável: `1=1` vaza todos os usuários.
3. Testa ORM Injection na versão corrigida: só há método específico, `1=1` é tratado como nome

literal (nenhum usuário com esse nome).

Adicionalmente, há um teste com sqlmap (semelhante ao exemplo 01-sql-injection). Suba `php -S 127.0.0.1:8900` nesta pasta e rode `./testar-com-sqlmap.sh` — sqlmap confirma 2 técnicas de injeção no parâmetro `nome` da versão vulnerável, e nenhuma na versão corrigida. Mais detalhes no [Apêndice A](#).

Saída esperada: `RESULTADO: 3/3 verificacoes passaram.`

6.7.2. Checklist de verificação

- ☐ A camada ORM/repositório jamais expõe métodos que aceitam SQL/NoSQL/LDAP arbitrário (como `buscarPorFiltro()` genérico)?
- ☐ Todo método de busca é específico (ex: `buscarPorNome()`, `buscarPorId()`) e usa prepared statements internamente?
- ☐ Se há filtros complexos legítimos, eles são construídos via query builder seguro (ex: Doctrine QueryBuilder, Laravel Query Builder) que parametriza automaticamente?

6.8. Checklist Consolidado — Injeção

Use este checklist como referência rápida durante code review ou pentest de aplicações PHP que recebem input de usuários:

- ☐ **SQL:** Prepared statements (`:nome + execute()`) para toda query com input; jamais concatenação.
- ☐ **XSS:** Todo output renderizado usa `htmlspecialchars(..., ENT_QUOTES, 'UTF-8')`.
- ☐ **Command:** `escapeshellarg()` em argumentos de comandos, ou `proc_open()` com array (sem shell).
- ☐ **NoSQL:** Validar tipos esperados (string, int); rejeitar arrays/objetos em campos críticos.
- ☐ **LDAP:** Escape de metacaracteres (`\`, `*`, `(`, `)`, NUL) conforme RFC4515.
- ☐ **Code/Eval:** Zero `eval()` com input não confiável; templates via regex restrita ou bibliotecas seguras.
- ☐ **ORM:** Métodos específicos (não genéricos de filtro), prepared statements internamente, jamais expor construção SQL livre.

Chapter 7. A06:2025 — Insecure Design (Design Inseguro)

7.1. Sobre esta categoria

O OWASP Top 10:2025 diferencia fundamentalmente a categoria A06 de todas as outras. Enquanto A01 a A05 e A07 a A10 tratam de **bugs de implementação** — erros cometidos ao escrever o código — A06 trata de **falhas de arquitetura e design**. São decisões tomadas **antes** de escrever qualquer linha de código: como os dados fluem através do sistema, quem pode acessá-los, quais limites existem, e como a aplicação se protege contra abuso. Um bug de implementação pode ser corrigido com um patch; uma falha de design requer **redesenho** completo.

Exemplos de insecure design abundam: decidir que senhas devem ser recuperáveis (violando o princípio de que senhas nunca devem ser armazenadas em claro ou reversível), distribuir permissões sem seguir o princípio do menor privilégio, misturar dados confiáveis com input do atacante no mesmo array sem isolamento claro, ou não impor nenhum limite na lógica de negócio (comprar 10 mil ingressos de uma vez sem depósito). Threat modeling, design review seguro e security patterns são as ferramentas essenciais para evitar A06.

Página oficial: https://owasp.org/Top10/2025/A06_2025-Insecure_Design/

Exemplos cobertos neste capítulo:

- **01 — Armazenamento de Credenciais Desprotegido:** decisão errada de usar criptografia reversível (base64) para armazenar senhas, com justificativa de recuperação posterior
- **02 — Gerenciamento de Privilégio Impróprio:** role 'editor' recebe permissões sensíveis (excluir usuários, gerenciar pagamentos) que deveria ter apenas admin
- **03 — Upload de Arquivo Perigoso:** validação insuficiente de upload permite enviar .php disfarçado, sem checar magic bytes
- **04 — Fronteira de Confiança Violada:** servidor mistura dados confiáveis e input do atacante no mesmo array, permitindo sobrescrever campos críticos
- **05 — Recuperação de Credencial Fraca:** reset de senha exige apenas pergunta de segurança (facilmente descobrível por engenharia social)
- **06 — Lógica de Negócio sem Limite:** sistema de reserva permite comprar qualquer quantidade sem depósito, sem exigir validação
- **07 — Falta de Proteção Anti-Bot:** nenhuma restrição de taxa de requisições, permitindo bots dispararem centenas por segundo

7.2. 01 — Armazenamento de Credenciais Desprotegido

CWE-256: Storage of Passwords in a Recoverable Format

A decisão de design fundamentalmente errada aqui é julgar que uma senha deveria ser recuperável se o usuário a esquecer. Baseado nessa premissa, alguns desenvolvedores escolhem criptografia **reversível** (como base64, ou cifragem simétrica) para "armazená-la de forma segura". Porém, essa decisão viola o princípio fundamental: **senhas nunca devem ser recuperáveis, nem mesmo por administradores do sistema**. O mecanismo correto é autenticação por comparação de hash irreversível, e reset de senha é realizado via token de tempo limitado, nunca recuperando a senha anterior. Se uma senha é reversível, qualquer pessoa com acesso ao banco de dados (através de vazamento SQL injection, backup roubado, ou insider) pode recuperar todas as senhas imediatamente.

Código vulnerável (A06-design-inseguro/01-armazenamento-credenciais-desprotegido/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Armazenamento de Credenciais Desprotegido (CWE-256) - A06:2025 Insecure Design
 *
 * A falha de DESIGN aqui é a decisão arquitetural de usar criptografia REVERSÍVEL
 * para armazenar senhas, com o objetivo de "poder recuperá-las depois se o usuário
 * esquecer". Isso viola o princípio fundamental de segurança: senhas nunca devem
 * ser recuperáveis, nem mesmo por administradores. A falha não é na implementação
 * do base64_encode, mas na decisão de design que justificou seu uso.
 */

function armazenarCredencial(string $senha): string {
    // VULNERÁVEL: usa codificação REVERSÍVEL
    // Qualquer pessoa com acesso ao banco pode fazer base64_decode() e obter a senha
    original
    return base64_encode($senha);
}

function demo(): void {
    echo "=== VULNERÁVEL: Armazenamento com criptografia reversível ===\n";

    $senhaOriginal = "minha_senha_super_secreta_123";
    $armazenado = armazenarCredencial($senhaOriginal);

    echo "Senha original: {$senhaOriginal}\n";
    echo "Armazenado no 'banco': {$armazenado}\n";

    $recuperada = base64_decode($armazenado);
    echo "Recuperada por um atacante com acesso ao banco: {$recuperada}\n";
    echo "Senhas REVERSÍVEIS são uma falha de DESIGN grave!\n";
}

if (debug_backtrace() === []) {
    demo();
}
```

```
}
```

O código vulnerável usa `base64_encode()` para "proteger" a senha. Base64 é apenas uma codificação (reversível), não um hash. Qualquer pessoa que acesse o banco de dados pode fazer `base64_decode($senhaArmazenada)` e recuperar a senha original em milissegundos.

Código corrigido ([A06-design-inseguro/01-armazenamento-credenciais-desprotegido/corrigido.php](#))

```
<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Decisão de Design Correta para Armazenamento de Senhas
 *
 * A decisão arquitetural correta é usar uma FUNÇÃO DE HASH IRREVERSÍVEL (bcrypt,
argon2, etc).
 * Dessa forma, a senha NUNCA pode ser recuperada, mesmo com acesso ao banco de dados.
 * A autenticação funciona por comparação de hashes, não por recuperação da senha
original.
 */

function armazenarCredencial(string $senha): string {
    // CORRIGIDO: usa hash IRREVERSÍVEL via bcrypt
    // Não há função de decode; a verificação é feita com password_verify()
    return password_hash($senha, PASSWORD_BCRYPT);
}

function verificarCredencial(string $senhaFornecida, string $hashArmazenado): bool {
    return password_verify($senhaFornecida, $hashArmazenado);
}

function demo(): void {
    echo "=== CORRIGIDO: Armazenamento com hash irreversível ===\n";

    $senhaOriginal = "minha_senha_super_secreta_123";
    $hashArmazenado = armazenarCredencial($senhaOriginal);

    echo "Senha original: {$senhaOriginal}\n";
    echo "Hash armazenado no banco: {$hashArmazenado}\n";
    echo "Tamanho do hash faz reverter impraticável (além de criptograficamente
impossível)\n";

    // Verificação legítima durante login
    $senhaFornecidaNoLogin = "minha_senha_super_secreta_123";
    if (verificarCredencial($senhaFornecidaNoLogin, $hashArmazenado)) {
        echo "Login bem-sucedido: senha foi verificada contra o hash (sem
recuperação)\n";
    }
}
```

```
// Tentativa com senha errada
$senhaErrada = "senha_incorreta";
if (!verificarCredencial($senhaErrada, $hashArmazenado)) {
    echo "Login rejeitado: hash não bate com a senha fornecida\n";
}

if (debug_backtrace() === []) {
    demo();
}
```

A solução é usar `password_hash()` com `bcrypt` (ou `argon2`). Essa é uma função de hash **irreversível**: produz um resultado diferente a cada chamada (devido ao salt aleatório), e não há maneira de reverter o hash para obter a senha original. A autenticação funciona através de `password_verify()`, que compara a senha fornecida com o hash armazenado. Se a senha é esquecida, o sistema oferece reset via token (não recuperação da antiga).

7.2.1. Como testar

Execute `php A06-design-inseguro/01-armazenamento-credenciais-desprotegido/teste.php`. O teste demonstra que:

- O código vulnerável armazena senhas de forma reversível: um atacante consegue recuperar a senha original com `base64_decode`;
- O código corrigido usa hash `bcrypt`, que não é reversível por nenhuma função simples;
- A verificação com `password_verify()` funciona corretamente, aceitando a senha correta e rejeitando a errada;
- Cada hash `bcrypt` é único (mesmo da mesma senha) devido ao salt aleatório, e ambos verificam a mesma senha.

Saída esperada: `RESULTADO: 6/6 verificacoes passaram.`

7.2.2. Checklist de verificação

- ☐ Senhas são armazenadas com hash irreversível (`bcrypt`, `argon2`), não criptografia reversível (`base64`, `AES`)?
- ☐ A aplicação implementa autenticação por comparação de hash via `password_verify()`, não por recuperação da senha?
- ☐ O sistema oferece reset de senha via token de tempo limitado, não retorna/recupera a senha anterior?
- ☐ Além da autenticação, o hash está armazenado com segurança (não em logs, não em output de erro)?

7.3. 02 — Gerenciamento de Privilégio Impróprio

CWE-269: Improper Access Control to Sensitive Attributes

A falha de design aqui é a distribuição incorreta de permissões entre roles. Um 'editor' de conteúdo é alguém que publica artigos, não alguém que deleta usuários ou gerencia pagamentos. Quando a aplicação agrupa essas permissões sensíveis na mesma role, viola-se o princípio do **menor privilégio**: cada role deveria ter apenas as permissões exatamente necessárias para sua função. Um editor comprometido, um insider malicioso, ou um desenvolvedor junior confuso pode então executar ações que nunca deveria poder fazer.

Código vulnerável (A06-design-inseguro/02-gerenciamento-privilegio-improprio/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Gerenciamento de Privilégio Impróprio (CWE-269) - A06:2025 Insecure Design
 *
 * A falha de DESIGN aqui é na distribuição de permissões por role. A arquitetura
 * agrupa o role 'editor' com permissões que ele NUNCA deveria ter (excluir usuários,
 * gerenciar pagamentos). A falha não é um bug de verificação, é a decisão de design
 * incorreta de quais poderes cada role deve ter. Isso viola o princípio do menor
 * privilégio.
 */

function obterPermissoes(string $role): array {
    // VULNERÁVEL: role 'editor' tem permissões demais por DESIGN
    // Um editor de conteúdo deveria NUNCA ter acesso a excluir usuários ou mexer em
    pagamentos
    if ($role === 'editor') {
        return [
            'editar_conteudo',
            'publicar_conteudo',
            'excluir_usuarios', // FALHA DE DESIGN: isso não deveria ser
            // permissão de editor!
            'gerenciar_pagamentos', // FALHA DE DESIGN: isso não deveria ser
            // permissão de editor!
        ];
    }

    if ($role === 'admin') {
        return [
            'editar_conteudo',
            'publicar_conteudo',
            'excluir_usuarios',
            'gerenciar_pagamentos',
        ];
    }
}
```

```

    }

    if ($role === 'leitor') {
        return ['visualizar_conteudo'];
    }

    return [];
}

function demo(): void {
    echo "=== VULNERÁVEL: Privilégios excessivos para role 'editor' ===\n";

    $permsEditor = obterPermissoes('editor');
    echo "Permissões do 'editor': " . json_encode($permsEditor) . "\n";

    if (in_array('excluir_usuarios', $permsEditor)) {
        echo "PROBLEMA: Editor pode excluir usuários (falha de design!)\n";
    }

    if (in_array('gerenciar_pagamentos', $permsEditor)) {
        echo "PROBLEMA: Editor pode gerenciar pagamentos (falha de design!)\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável retorna um array de permissões para cada role, mas agrupa poder excessivo na role 'editor': inclui `excluir_usuarios` e `gerenciar_pagamentos`. Isso é uma falha de design na própria definição de roles.

Código corrigido (A06-design-inseguro/02-gerenciamento-privilegio-improprio/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Design Correto do Modelo de Privilégios
 *
 * A decisão de design correta implementa o princípio do MENOR PRIVILÉGIO:
 * cada role recebe APENAS as permissões que são absolutamente necessárias
 * para sua função. Editor edita e publica conteúdo. Admin faz tudo. Leitor só lê.
 * Permissões sensíveis como gerenciar pagamentos ou deletar usuários são
 * EXPLICITAMENTE
 * restritas apenas aos roles autorizados via allow-list.
 */

```

```

function obterPermissoes(string $role): array {
    // CORRIGIDO: cada role tem APENAS as permissões que precisa
    // Aplicado o princípio do MENOR PRIVILÉGIO

    if ($role === 'admin') {
        // Admin tem tudo (por design consciente)
        return [
            'editar_conteudo',
            'publicar_conteudo',
            'excluir_usuarios',
            'gerenciar_pagamentos',
        ];
    }

    if ($role === 'editor') {
        // Editor APENAS edita e publica conteúdo, nada mais
        return [
            'editar_conteudo',
            'publicar_conteudo',
        ];
    }

    if ($role === 'leitor') {
        // Leitor apenas consome conteúdo
        return ['visualizar_conteudo'];
    }

    return [];
}

function obterPermissoesEspeciais(string $role): array {
    // Permissões sensíveis exigem APROVAÇÃO EXPLÍCITA por role
    // Allow-list para operações críticas
    $permissoesSensveis = [
        'admin' => ['excluir_usuarios', 'gerenciar_pagamentos'],
    ];

    return $permissoesSensveis[$role] ?? [];
}

function demo(): void {
    echo "=== CORRIGIDO: Privilégios apropriados para cada role ===\n";

    $permsEditor = obterPermissoes('editor');
    echo "Permissões do 'editor': " . json_encode($permsEditor) . "\n";

    if (!in_array('excluir_usuarios', $permsEditor)) {
        echo "OK: Editor NÃO pode excluir usuários (design apropriado)\n";
    }

    if (!in_array('gerenciar_pagamentos', $permsEditor)) {

```

```

        echo "OK: Editor NÃO pode gerenciar pagamentos (design apropriado)\n";
    }

    $permsAdmin = obterPermissoes('admin');
    echo "Permissões do 'admin': " . json_encode($permsAdmin) . "\n";

    if (in_array('excluir_usuarios', $permsAdmin)) {
        echo "OK: Admin pode excluir usuários (apropriado para seu role)\n";
    }
}

if (debug_backtrace() == []) {
    demo();
}

```

A solução é repensar a arquitetura de permissões: definir uma lista explícita (allow-list) de quais permissões cada role realmente precisa. O 'editor' recebe apenas `editar_conteudo` e `publicar_conteudo`. Permissões sensíveis como `excluir_usuarios` e `gerenciar_pagamentos` ficam restritas apenas ao 'admin'. Isso segue o princípio do menor privilégio: cada role tem o mínimo necessário para sua função, reduzindo o raio de dano se aquela role for comprometida.

7.3.1. Como testar

Execute `php A06-design-inseguro/02-gerenciamento-privilegio-improprio/teste.php`. O teste valida que:

- No código vulnerável, a role 'editor' inclui permissões que não deveria (`excluir_usuarios` e `gerenciar_pagamentos`);
- No código corrigido, a role 'editor' tem apenas permissões apropriadas (`editar_conteudo` e `publicar_conteudo`);
- No código corrigido, permissões sensíveis ficam restritas ao 'admin';
- Uma verificação negativa confirma que 'editor' não pode acessar funções administrativas no código corrigido.

Saída esperada: `RESULTADO: 4/4 verificacoes passaram.`

7.3.2. Checklist de verificação

- ☐ Cada role tem uma lista explícita de permissões (allow-list), nunca uma lista de negação implícita?
- ☐ Permissões sensíveis (deletar usuários, gerenciar pagamentos, alterar configs) existem apenas em roles administrativas?
- ☐ O princípio do menor privilégio foi aplicado: 'editor' tem apenas permissões de edição, 'leitor' apenas leitura?
- ☐ Roles intermediárias existem (se necessário) para separar responsabilidades: um 'moderador' pode publicar mas não pode deletar usuários?

7.4. 03 — Upload de Arquivo Perigoso

CWE-434: Unrestricted Upload of Dangerous File Types

A falha de design neste exemplo é implementar upload de arquivos com validação insuficiente. Algumas aplicações checam apenas o **nome** do arquivo (extensão), mas ignoram o **conteúdo real** (magic bytes). Um atacante pode renomear um arquivo PHP para **.jpg**, e a validação passa. Quando o servidor configura a pasta de upload para executar PHP, o arquivo malicioso é interpretado e explorado. Além disso, validar apenas pela extensão é frágil: o atacante pode usar **.php3**, **.php5**, **.phtml**, ou explorar misconfigurações do servidor. A decisão de design correta é validar **tanto** a extensão (contra um allow-list explícito) **quanto** os magic bytes (assinatura binária) do conteúdo real.

Código vulnerável (A06-design-inseguro/03-upload-arquivo-perigoso/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Upload de Arquivo Perigoso (CWE-434) - A06:2025 Insecure Design
 *
 * A falha de DESIGN aqui é a falta de validação de uploads. A arquitetura
 * não define NENHUM critério de aceitação: nem whitelist de extensões,
 * nem verificação de magic bytes (assinatura binária) do arquivo.
 * Isso permite uploads de .php, .exe, ou qualquer outro tipo perigoso.
 * Atacante envia exploit.php e o servidor aceita sem questionar.
 */

function validarUpload(string $nomeArquivo, string $conteudo): bool {
    // VULNERÁVEL: praticamente nenhuma validação
    // Aceita qualquer arquivo só porque o nome não está vazio
    // Não verifica extensão, não verifica conteúdo real (magic bytes)
    return $nomeArquivo !== '';
}

function demo(): void {
    echo "=== VULNERÁVEL: Sem validação de uploads ===\n";

    $testesInseguros = [
        ['exploit.php', '<?php system($_GET["c"]); ?>'],
        ['malware.exe', 'MZ\x90\x00...'],
        ['shell.phtml', '<?php echo "hacked"; ?>'],
    ];

    foreach ($testesInseguros as [$nome, $conteudo]) {
        if (validarUpload($nome, $conteudo)) {
            echo "PROBLEMA: Aceitou {$nome} sem verificação (arquivo perigoso!)\n";
        }
    }
}
```

```

    }
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável verifica apenas se o nome do arquivo é não-vazio. Um atacante pode enviar **malicia.php** diretamente, ou renomear **shell.php** para **imagem.jpg** e o arquivo passa na validação.

Código corrigido (A06-design-inseguro/03-upload-arquivo-perigoso/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Design Seguro com Validação de Uploads
 *
 * A decisão de design correta implementa validação em múltiplas camadas:
 * 1. WHITELIST de extensões permitidas (nunca extensões parametrizáveis)
 * 2. Verificação de MAGIC BYTES (assinatura binária) do arquivo real
 * 3. Ambas as validações devem concordar (extensão + conteúdo real)
 * Dessa forma, mesmo que um atacante renomeie um .php para .jpg,
 * o magic byte não vai bater e será rejeitado.
 */

function obterMagicBytesEsperado(string $extensao): ?string {
    // Magic bytes esperados para extensões permitidas
    $mapeamento = [
        'jpg' => "\xFF\xD8\xFF", // JPEG
        'jpeg' => "\xFF\xD8\xFF", // JPEG
        'png' => "\x89PNG", // PNG (completo é \x89PNG\r\n\x1a\n)
        'gif' => "GIF", // GIF87a ou GIF89a
        'pdf' => "%PDF", // PDF
    ];

    return $mapeamento[$extensao] ?? null;
}

function validarUpload(string $nomeArquivo, string $conteudo): bool {
    // CORRIGIDO: validação em camadas

    // 1. Extrair extensão e converter para minúsculas
    $extensao = strtolower(pathinfo($nomeArquivo, PATHINFO_EXTENSION));

    // 2. Whitelist: extensão deve estar na lista permitida
    $extensoesPermitidas = ['jpg', 'jpeg', 'png', 'gif', 'pdf'];

```

```

    if (!in_array($extensao, $extensoesPermitidas)) {
        return false; // Extensão não permitida
    }

    // 3. Verificar magic bytes do conteúdo real
    $magicBytesEsperado = obterMagicBytesEsperado($extensao);
    if ($magicBytesEsperado === null) {
        return false;
    }

    // Magic bytes do arquivo podem ser incompletos em nossos testes,
    // então verificamos se o conteúdo COMEÇA com o magic byte esperado
    $conteudoInicial = substr($conteudo, 0, strlen($magicBytesEsperado));
    if ($conteudoInicial !== $magicBytesEsperado) {
        return false; // Magic bytes não batem
    }

    return true; // Passou em todas as validações
}

function demo(): void {
    echo "=== CORRIGIDO: Validação de uploads em camadas ===\n";

    // Teste 1: Arquivo .php é rejeitado (extensão não permitida)
    if (!validarUpload('exploit.php', '<?php system($_GET["c"]); ?>')) {
        echo "OK: Rejeitou exploit.php (extensão não permitida)\n";
    }

    // Teste 2: Arquivo .jpg legítimo com magic bytes corretos é aceito
    if (validarUpload('foto.jpg', "\xFF\xD8\xFF\xE0\x00\x10JFIF...")) {
        echo "OK: Aceitou foto.jpg com magic bytes corretos\n";
    }

    // Teste 3: Arquivo .png legítimo é aceito
    if (validarUpload('imagem.png', "\x89PNG\r\n\x1a\n...")) {
        echo "OK: Aceitou imagem.png com magic bytes corretos\n";
    }

    // Teste 4: Arquivo .png com conteúdo PHP é rejeitado
    if (!validarUpload('disfarçado.png', '<?php echo "hacked"; ?>')) {
        echo "OK: Rejeitou disfarçado.png com magic bytes errados\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A solução implementa validação robusta: primeiro, extrai a extensão e verifica contra uma allow-list explícita (**jpg**, **png**, **gif**). Segundo, lê os primeiros bytes do arquivo (magic bytes) e confirma que

bate com o tipo esperado (JPEG começa com `FFD8FF`, PNG com `89504E47`). Se os magic bytes não batem com a extensão, o upload é rejeitado, mesmo que a extensão esteja na whitelist. Isso previne ataques como renomear um arquivo PHP para `.jpg`.

7.4.1. Como testar

Execute `php A06-design-inseguro/03-upload-arquivo-perigoso/teste.php`. O teste demonstra que:

- O código vulnerável aceita qualquer nome de arquivo sem validação real;
- O código corrigido rejeita extensões fora da whitelist;
- O código corrigido rejeita arquivos cujos magic bytes não batem com a extensão;
- O código corrigido aceita apenas arquivos com extensão e magic bytes válidos.

Saída esperada: `RESULTADO: 4/4 verificacoes passaram.`

7.4.2. Checklist de verificação

- ☐ Há uma allow-list explícita de extensões permitidas (não uma deny-list)?
- ☐ O servidor verifica os magic bytes (primeiros bytes/assinatura binária) do arquivo, não apenas o nome?
- ☐ Um arquivo PHP renomeado para `.jpg` seria rejeitado (magic bytes não batem)?
- ☐ A pasta de upload está configurada para **não** executar scripts (php_flag engine off, ou fora da document root)?
- ☐ Nomes de arquivo são sanitizados/renomeados para evitar path traversal (`../`, `..\`)?

7.5. 04 — Fronteira de Confiança Violada

CWE-501: Trust Boundary Violation

A falha de design aqui é misturar dados confiáveis (do servidor, validados, autenticados) com input não confiável (do usuário, potencialmente malicioso) no mesmo contexto sem isolamento claro. Se uma função faz `array_merge($dadosServidorConfiavel, $inputDoAtacante)`, o atacante pode adicionar chaves que sobrescrevem valores críticos. Por exemplo, um campo `isAdmin` pode ser injetado no array, elevando os privilégios do atacante sem autorização. A decisão de design correta é **isolar namespaces**: manter dados confiáveis e input do usuário em estruturas completamente separadas, e processar apenas o que é esperado.

Código vulnerável (`A06-design-inseguro/04-fronteira-confianca-violada/vulneravel.php`)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
```

```

* Fronteira de Confiança Violada (CWE-501) - A06:2025 Insecure Design
*
* A falha de DESIGN aqui é a falta de isolamento entre dados CONFIÁVEIS
* (do servidor, autenticados) e NÃO-CONFIÁVEIS (input do atacante).
* Ambos são misturados no mesmo array plano. Atacante que consegue enviar
* input pode sobrescrever dados confiáveis do servidor, como 'isAdmin'.
* A falha não é na verificação, é na ARQUITETURA que não isolou fronteiras.
*/

function montarSessao(array $dadosConfiaveisDoServidor, array $inputDoUsuario): array
{
    // VULNERÁVEL: mistura dados confiáveis com input do atacante SEM ISOLAMENTO
    // Se o servidor diz ['isAdmin' => false] mas o input tem ['isAdmin' => true],
    // o array_merge vai deixar o true do atacante sobrescrever o false confiável
    return array_merge($dadosConfiaveisDoServidor, $inputDoUsuario);
}

function demo(): void {
    echo "=== VULNERÁVEL: Sem fronteira entre dados confiáveis e input ===\n";

    $dadosConfiaveisDoServidor = [
        'isAdmin' => false,
        'usuario_id' => 42,
        'email' => 'user@example.com',
    ];

    $inputDoUsuarioAtacante = [
        'isAdmin' => true, // Atacante tenta escalar privilégio
        'email' => 'hacker@evil.com',
    ];

    $sessaoMisturada = montarSessao($dadosConfiaveisDoServidor,
    $inputDoUsuarioAtacante);
    echo "Sessão montada: " . json_encode($sessaoMisturada) . "\n";

    if ($sessaoMisturada['isAdmin'] === true) {
        echo "PROBLEMA: Atacante conseguiu sobrescrever isAdmin para true (falha de
design!)\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável combina dados do servidor (`usuarioId` e `isAdmin`) e input do usuário (nome, email) no mesmo array. Se o atacante injetar `isAdmin: true` no payload, o array resultante contém essa chave, contornando completamente a lógica de autorização.

```
<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Isolamento Claro de Fronteiras de Confiança
 *
 * A decisão de design correta SEPARA dados confiáveis (do servidor/autenticados)
 * de dados NÃO-confiáveis (input do usuário) em NAMESPACES DISTINTOS dentro
 * da mesma estrutura. Dessa forma, mesmo que o atacante consiga enviar qualquer
 * input malicioso, ele sempre fica isolado em 'usuario', nunca toca em 'servidor'.
 * Código que precisa verificar privilégios acessa SEMPRE 'servidor.isAdmin',
 * nunca é influenciado pelo que está em 'usuario.isAdmin'.
 */

function montarSessao(array $dadosConfiaveisDoServidor, array $inputDoUsuario): array
{
    // CORRIGIDO: separa em namespaces distintos
    // Dados confiáveis NUNCA são misturados com input do usuário
    return [
        'servidor' => $dadosConfiaveisDoServidor, // Confiável, intocável
        'usuario' => $inputDoUsuario,             // Input do usuário, isolado
    ];
}

function verificarPrivilegio(array $sessao): bool {
    // Acessa SEMPRE os dados confiáveis do servidor
    // Impossível que input do usuário interfira
    return $sessao['servidor']['isAdmin'] ?? false;
}

function obterEmailDoUsuario(array $sessao): string {
    // Pode consultar dados do usuário se necessário, mas sabendo que é input
    return $sessao['usuario']['email'] ?? 'desconhecido@localhost';
}

function demo(): void {
    echo "=== CORRIGIDO: Fronteiras claras entre dados confiáveis e input ===\n";

    $dadosConfiaveisDoServidor = [
        'isAdmin' => false,
        'usuario_id' => 42,
        'email' => 'user@example.com',
    ];

    $inputDoUsuarioAtacante = [
        'isAdmin' => true, // Tentativa de escalar privilégio
        'email' => 'hacker@evil.com',
    ];
}
```

```

];

$sessaoSegura = montarSessao($dadosConfiaveisDoServidor, $inputDoUsuarioAtacante);
echo "Sessão estruturada: " . json_encode($sessaoSegura) . "\n";

$ehAdmin = verificarPrivilegio($sessaoSegura);
if ($ehAdmin === false) {
    echo "OK: Privilégio verificado SEMPRE nos dados confiáveis do servidor\n";
    echo "OK: Input malicioso do usuário fica isolado em 'usuario', não afeta
segurança\n";
}
}

if (debug_backtrace() === []) {
    demo();
}

```

A solução isola os namespaces explicitamente: cria um array com estrutura `['servidor' => [... dados confiáveis...], 'usuario' => [...input sanitizado...]]`. Apenas campos esperados do usuário são permitidos, e dados críticos do servidor nunca podem ser sobrescritos. Quando processado, a aplicação lê apenas do namespace `servidor` para decisões de segurança, e do namespace `usuario` para dados do usuário.

7.5.1. Como testar

Execute `php A06-design-inseguro/04-fronteira-confianca-violada/teste.php`. O teste prova que:

- No código vulnerável, um atacante consegue injetar `isAdmin: true` no payload, e ele aparece no array processado;
- No código corrigido, campos injetados são isolados no namespace `usuario`, separados dos dados confiáveis do servidor;
- No código corrigido, mesmo que o atacante injete `isAdmin: true`, ele fica no namespace `usuario` e nunca afeta a decisão de autorização baseada em `servidor.isAdmin`.

Saída esperada: `RESULTADO: 3/3 verificacoes passaram.`

7.5.2. Checklist de verificação

- ☐ Dados confiáveis (servidor, banco de dados) estão isolados de input do usuário, em estruturas/namespaces separados?
- ☐ Campos críticos (`isAdmin`, `userId`, `role`) vêm SEMPRE do servidor, nunca do input do usuário?
- ☐ Um `array_merge($serverData, $userData)` não é usado sem isolamento explícito?
- ☐ Ao processar dados do usuário, apenas campos **esperados** são aceitos (whitelist de campos permitidos)?

7.6. 05 — Recuperação de Credencial Fraca

CWE-640: Weak Password Recovery Mechanism for Forgotten Password

A falha de design é confiar em um único fator para recuperar acesso a uma conta (como resposta de pergunta de segurança). Perguntas de segurança são frequentemente descobríveis por engenharia social: qual é seu filme favorito, seu animal de estimação preferido, a cidade onde nasceu — essas informações podem ser encontradas em redes sociais ou obtidas falando com a vítima. Se apenas uma pergunta de segurança bloqueia o reset de senha, um atacante consegue redefinir a senha da vítima com informações públicas. A decisão de design correta é **exigir múltiplos fatores** para recuperação: resposta da pergunta de segurança **AND** um token enviado por email que a vítima deve confirmar.

Código vulnerável (A06-design-inseguro/05-recuperacao-credencial-fraca/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Recuperação de Credencial Fraca (Cenário 1 - OWASP) - A06:2025 Insecure Design
 *
 * A falha de DESIGN aqui é permitir que um ÚNICO fator (pergunta de segurança)
 * seja suficiente para recuperar/resetar a senha. Perguntas de segurança podem ser
 * descobertas via engenharia social, redes sociais, ou pesquisa pública.
 * A decisão arquitetural incorreta foi não exigir SEGUNDO FATOR (token de e-mail
 * validado).
 * Isso viola o princípio de "múltiplos fatores de autenticação para operações
 * sensíveis".
 */

function recuperarSenha(string $respostaFornecida, string $respostaCorretaArmazenada,
?string $tokenEmailValidado): bool {
    // VULNERÁVEL: apenas um fator (pergunta de segurança)
    // Se o atacante conseguir descobrir a resposta (engenharia social),
    // consegue resetar a senha SEM acesso ao e-mail da vítima
    return $respostaFornecida === $respostaCorretaArmazenada;
}

function demo(): void {
    echo "=== VULNERÁVEL: Recuperação com um único fator (pergunta de segurança) ===\n";

    $respostaCorreta = "miau"; // Resposta à pergunta: "Qual é o nome do seu primeiro gato?"
    $tokenEmail = "token_abc123";

    // Atacante descobre a resposta por engenharia social
    $respostaAtacante = "miau";
```

```

// Mesmo SEM token de e-mail, consegue resetar a senha (FALHA!)
if (recuperarSenha($respostaAtacante, $respostaCorreta, null)) {
    echo "PROBLEMA: Atacante conseguiu resetar senha SEM acesso ao e-mail!\n";
    echo "Pergunta de segurança sozinha NÃO é suficiente (falha de design)!\n";
}
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável aprova reset de senha apenas com a resposta correta de uma pergunta de segurança. Um atacante pode descobrir essa resposta por engenharia social e redefinir a senha sem acesso à conta.

Código corrigido (A06-design-inseguro/05-recuperacao-credencial-fraca/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Recuperação de Credencial com Múltiplos Fatores
 *
 * A decisão de design correta exige MÍNIMO 2 FATORES para operações sensíveis:
 * 1. Resposta correta à pergunta de segurança (algo que só o dono sabe)
 * 2. Validação via token enviado ao e-mail/SMS cadastrado (algo que só o dono pode acessar)
 * Mesmo que um atacante descubra a resposta da pergunta via engenharia social,
 * sem acesso ao e-mail da vítima não consegue completar a recuperação.
 */

function recuperarSenha(string $respostaFornecida, string $respostaCorretaArmazenada,
?string $tokenEmailValidado): bool {
    // CORRIGIDO: exige DOIS fatores
    // A resposta deve estar correta E deve haver token de e-mail validado

    // Fator 1: Pergunta de segurança
    if ($respostaFornecida !== $respostaCorretaArmazenada) {
        return false;
    }

    // Fator 2: Token de e-mail (não pode ser null ou vazio)
    if ($tokenEmailValidado === null || $tokenEmailValidado === '') {
        return false;
    }

    // Ambos os fatores passaram

```

```

    return true;
}

function enviarTokenPorEmail(string $email): string {
    // Simula envio de token para e-mail (em produção seria de verdade)
    // Retorna um token que seria recebido pelo e-mail do usuário
    return "token_" . bin2hex(random_bytes(16));
}

function demo(): void {
    echo "=== CORRIGIDO: Recuperação com múltiplos fatores ===\n";

    $respostaCorreta = "miau";
    $emailUsuario = "user@example.com";

    // Cenário 1: Atacante sabe a resposta mas não tem acesso ao e-mail
    echo "\nCenário 1: Atacante com resposta correta mas sem acesso ao e-mail\n";
    $respostaAtacante = "miau";
    $resultadoAtaque = recuperarSenha($respostaAtacante, $respostaCorreta, null);
    if (!$resultadoAtaque) {
        echo "OK: Reset negado (fator 2 ausente)\n";
    }

    // Cenário 2: Dono legítimo com resposta correta E token de e-mail
    echo "\nCenário 2: Dono legítimo com ambos os fatores\n";
    $tokenValido = enviarTokenPorEmail($emailUsuario);
    $resultadoLegitimo = recuperarSenha($respostaCorreta, $respostaCorreta,
    $tokenValido);
    if ($resultadoLegitimo) {
        echo "OK: Reset permitido (ambos os fatores validados)\n";
    }

    // Cenário 3: Atacante tenta com resposta errada + token fakeado
    echo "\nCenário 3: Atacante com resposta errada\n";
    $respostaErrada = "dog";
    $resultadoErro = recuperarSenha($respostaErrada, $respostaCorreta, $tokenValido);
    if (!$resultadoErro) {
        echo "OK: Reset negado (resposta de segurança incorreta)\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A solução exige múltiplos fatores: a pergunta de segurança E um token enviado por email. Mesmo que o atacante adivinhe a pergunta, ainda precisa ter acesso ao email da vítima para confirmar o token. Isso aumenta significativamente a dificuldade do ataque, deixando exigência de múltiplos fatores independentes.

7.6.1. Como testar

Execute `php A06-design-inseguro/05-recuperacao-credencial-fraca/teste.php`. O teste valida que:

- No código vulnerável, apenas a pergunta de segurança é suficiente para redefinir senha;
- No código corrigido, responder apenas a pergunta não é suficiente; o token de email também é necessário;
- No código corrigido, um atacante que sabe a resposta da pergunta mas não tem acesso ao email é bloqueado.

Saída esperada: **RESULTADO: 3/3 verificacoes passaram.**

7.6.2. Checklist de verificação

- ☐ Recuperação de conta exige múltiplos fatores independentes (não apenas um)?
- ☐ Um fator é algo que a vítima sabe (pergunta de segurança), outro é algo que a vítima possui (email, SMS)?
- ☐ O token de email tem expiração de tempo limitado (ex: válido por 1 hora)?
- ☐ A aplicação não permite que um atacante pule etapas do fluxo (ex: redefinir senha sem confirmar email)?

7.7. 06 — Lógica de Negócio sem Limite

CWE-779: Logging of Excessive Data

A falha de design é não implementar nenhum limite na lógica de negócio crítica. Um exemplo é um sistema de reservas que permite que um único cliente reserve 10 mil ingressos de uma vez sem depósito. Logicamente, isso não faz sentido: esgota o estoque, impede outros clientes legítimos, e se o cliente simplesmente cancela, o vendedor perde. A decisão de design correta é impor limites: máximo N ingressos por pedido, e para quantidades grandes, exigir depósito/pagamento antecipado. Limites na lógica de negócio não são apenas segurança, são bom design.

Código vulnerável (`A06-design-inseguro/06-logica-negocio-sem-limite/vulneravel.php`)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Lógica de Negócio sem Limite (Cenário 2 - OWASP) - A06:2025 Insecure Design
 *
 * A falha de DESIGN aqui é a falta de LIMITES na lógica de negócio.
 * A arquitetura não define:
 * - Quantidade máxima de itens por requisição
 * - Exigência de depósito/pagamento antecipado para grandes quantidades
```

```

* Um atacante pode reservar 10.000 ingressos de uma vez sem pagar nada,
* esvaziando todo o estoque da próxima apresentação do cinema.
*/

function reservarIngressos(int $quantidade, bool $depositoPago): array {
    // VULNERÁVEL: sem limites
    // Aceita qualquer quantidade sem verificação
    // Não exige depósito
    // Falha de DESIGN na lógica de negócio
    return [
        'aprovado' => true,
        'motivo' => 'ok',
    ];
}

function demo(): void {
    echo "=== VULNERÁVEL: Sem limites na reserva de ingressos ===\n";

    // Atacante tenta reservar toda a sala de cinema
    $resultado = reservarIngressos(5000, false);

    if ($resultado['aprovado']) {
        echo "PROBLEMA: Reservou 5.000 ingressos SEM depósito e SEM limite!\n";
        echo "Falha de DESIGN: lógica de negócio não tem proteção (estoque
esvaziado!)\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável aprova qualquer quantidade de ingressos sem limite. Um atacante pode reservar 10 mil ingressos, criando um ataque de negação de serviço (esgotar estoque) ou fraude (reservar sem intenção de pagar).

Código corrigido (A06-design-inseguro/06-logica-negocio-sem-limite/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Lógica de Negócio com Proteções
 *
 * A decisão de design correta estabelece LIMITES e REGRAS claras na lógica de
negócio:
 * 1. Limite máximo de itens por requisição (ex: 4 ingressos)
 * 2. Para quantidades maiores, exigir depósito/pagamento antecipado

```

```

* 3. Validação explícita de todas as precondições
* Dessa forma, o atacante não consegue esvaziar o estoque sem pagar,
* e as requisições legítimas funcionam normalmente.
*/

function reservarIngressos(int $quantidade, bool $depositoPago): array {
    // CORRIGIDO: lógica de negócio com proteções

    // Limite 1: Máximo de ingressos por requisição (sem depósito)
    $limiteSemDeposito = 4;

    // Validação: quantidade não pode ser negativa ou zero
    if ($quantidade <= 0) {
        return [
            'aprovado' => false,
            'motivo' => 'Quantidade deve ser positiva',
        ];
    }

    // Decisão: quantidade dentro do limite de conforto
    if ($quantidade <= $limiteSemDeposito) {
        return [
            'aprovado' => true,
            'motivo' => 'Reserva pequena aprovada (sem depósito exigido)',
        ];
    }

    // Quantidade grande exige depósito pago
    if ($quantidade > $limiteSemDeposito && $depositoPago === false) {
        return [
            'aprovado' => false,
            'motivo' => sprintf(
                'Quantidades maiores que %d exigem depósito pago antecipadamente',
                $limiteSemDeposito
            ),
        ];
    }

    // Grande quantidade com depósito pago é aprovada
    if ($quantidade > $limiteSemDeposito && $depositoPago === true) {
        return [
            'aprovado' => true,
            'motivo' => 'Reserva grande aprovada (depósito confirmado)',
        ];
    }

    return [
        'aprovado' => false,
        'motivo' => 'Erro na validação de reserva',
    ];
}

```

```

function demo(): void {
    echo "=== CORRIGIDO: Lógica de negócio com proteções ===\n";

    // Teste 1: Pequena quantidade sem depósito (aprovada)
    $resultado1 = reservarIngressos(2, false);
    echo "Reserva de 2 ingressos sem depósito: ";
    echo $resultado1['aprovado'] ? "APROVADA" : "NEGADA";
    echo " ({ $resultado1['motivo'] })\n";

    // Teste 2: Grande quantidade sem depósito (negada)
    $resultado2 = reservarIngressos(5000, false);
    echo "Reserva de 5000 ingressos sem depósito: ";
    echo $resultado2['aprovado'] ? "APROVADA" : "NEGADA";
    echo " ({ $resultado2['motivo'] })\n";

    // Teste 3: Grande quantidade COM depósito (aprovada)
    $resultado3 = reservarIngressos(5000, true);
    echo "Reserva de 5000 ingressos COM depósito: ";
    echo $resultado3['aprovado'] ? "APROVADA" : "NEGADA";
    echo " ({ $resultado3['motivo'] })\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A solução impõe limites explícitos na lógica de negócio: máximo de 50 ingressos por pedido (limite padrão), e para pedidos acima de 100 ingressos, exige depósito mínimo (50% do valor total). Isso previne que um atacante reserve quantidades absurdas, e força comprometimento financeiro antes de grandes pedidos.

7.7.1. Como testar

Execute `php A06-design-inseguro/06-logica-negocio-sem-limite/teste.php`. O teste demonstra que:

- No código vulnerável, um pedido de 5000 ingressos é aprovado sem qualquer limite;
- No código corrigido, um pedido de 5000 ingressos é rejeitado (excede limite de 50);
- No código corrigido, um pedido de 100+ ingressos sem depósito é rejeitado;
- No código corrigido, um pedido legítimo (até 50 ingressos) é aprovado.

Saída esperada: `RESULTADO: 4/4 verificacoes passaram.`

7.7.2. Checklist de verificação

- ☐ Existe limite máximo de quantidade por pedido/requisição, apropriado para o negócio?
- ☐ Para quantidades acima de um threshold, há exigência de depósito ou validação adicional?
- ☐ Limites são documentados e revistos periodicamente com a equipe de negócio?

- A aplicação loga tentativas de contornar limites (ex: múltiplas requisições pequenas em sequência rápida)?

7.8. 07 — Falta de Proteção Anti-Bot

CWE-770: Allocation of Resources Without Limits or Throttling

A falha de design é não implementar nenhuma proteção contra automação maliciosa. Um servidor que não limita taxa de requisições permite que um bot dispare centenas ou milhares de requisições por segundo. Um atacante pode usar isso para força bruta de senhas, credential stuffing, scraping de dados, ou amplificação em um ataque DDoS. A decisão de design correta é implementar rate limiting: rastrear requisições por cliente (IP, sessão, usuário) e negar requisições que excedem um limite de frequência.

Código vulnerável (A06-design-inseguro/07-falta-protecao-anti-bot/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Falta de Proteção Anti-Bot (Cenário 3 - OWASP) - A06:2025 Insecure Design
 *
 * A falha de DESIGN aqui é a inexistência de proteção contra bots/scalpers.
 * A arquitetura não monitora:
 * - Frequência de requisições do mesmo cliente
 * - Padrões de comportamento suspeito (muitas tentativas em pouco tempo)
 * Um bot pode disparar centenas de requisições por segundo, comprando
 * todo o estoque antes que clientes legítimos tenham chance.
 */

function processarCompra(string $idCliente, int $quantidade, array
$historicoRequisicoesRecentes): array {
    // VULNERÁVEL: sem proteção contra bots
    // Aceita qualquer requisição sem verificar padrão de comportamento
    // Falha de DESIGN: não há rate limiting ou detecção de bot
    return [
        'aprovado' => true,
        'motivo' => 'ok',
    ];
}

function demo(): void {
    echo "=== VULNERÁVEL: Sem proteção anti-bot ===\n";

    // Simula um bot disparando requisições
    $agora = time();
    $historicoBot = [
```

```

        $agora,
        $agora - 1,
        $agora - 2,
        $agora - 3,
        $agora - 4,
        $agora - 5,
        $agora - 6,
        $agora - 7,
        $agora - 8,
        $agora - 9,
    ];

    $resultado = processarCompra('bot_attacker_123', 1, $historicoBot);

    if ($resultado['aprovado']) {
        echo "PROBLEMA: Bot conseguiu fazer compra mesmo com 10 requisições em 9 segundos!\n";
        echo "Falha de DESIGN: sem proteção anti-bot, scalper limpa o estoque\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável não rastreia tentativas anteriores. Um atacante pode disparar requisições sem qualquer restrição, executando ataques de força bruta ou bot scraping indefinidamente.

Código corrigido (A06-design-inseguro/07-falta-protecao-anti-bot/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Design com Proteção Anti-Bot
 *
 * A decisão de design correta implementa RATE LIMITING e detecção de comportamento de bot:
 * 1. Monitora o histórico de requisições recentes (últimos X segundos)
 * 2. Se houver muitas requisições em pouco tempo, rejeita como suspeito (padrão de bot)
 * 3. Clientes legítimos (1-2 requisições por intervalo) funcionam normalmente
 * 4. Bots que disparam dezenas de requisições por segundo são bloqueados
 * Dessa forma, clientes reais têm chance de comprar antes que bots limpem o estoque.
 */

function processarCompra(string $idCliente, int $quantidade, array $historicoRequisicoesRecentes, int $agora): array {

```

```

// CORRIGIDO: proteção anti-bot com rate limiting

// Limite: máximo de 3 requisições nos últimos 10 segundos
$limiteRequisicoes = 3;
$janelaDeTempoSegundos = 10;
$thresholdDeteccaoBot = $limiteRequisicoes;

// Contar quantas requisições deste cliente aconteceram no intervalo recente
$requisicoesRecentes = 0;
foreach ($historicoRequisicoesRecentes as $timestamp) {
    if ($agora - $timestamp <= $janelaDeTempoSegundos) {
        $requisicoesRecentes++;
    }
}

// Verificação: muitas requisições em pouco tempo = padrão de bot
if ($requisicoesRecentes > $thresholdDeteccaoBot) {
    return [
        'aprovado' => false,
        'motivo' => sprintf(
            'Muitas requisições recentes detectadas (%d em %ds). Aguarde antes de
tentar novamente.',
            $requisicoesRecentes,
            $janelaDeTempoSegundos
        ),
    ];
}

// Cliente legítimo com requisições normais
return [
    'aprovado' => true,
    'motivo' => 'Compra processada com sucesso',
];
}

function demo(): void {
    echo "=== CORRIGIDO: Proteção anti-bot com rate limiting ===\n";

    $agora = time();

    // Teste 1: Bot com muitas requisições em pouco tempo
    echo "\nTeste 1: Bot disparando requisições\n";
    $historicoBot = [
        $agora,
        $agora - 1,
        $agora - 2,
        $agora - 3,
        $agora - 4,
        $agora - 5,
        $agora - 6,
        $agora - 7,
    ];
}

```

```

        $agora - 8,
        $agora - 9,
    ];

    $resultadoBot = processarCompra('bot_attacker', 1, $historicoBot, $agora);
    echo "Bot com 10 requisições em 9 segundos: " . ($resultadoBot['aprovado'] ?
"APROVADO" : "NEGADO") . "\n";
    echo "Motivo: {$resultadoBot['motivo']}\n";

    // Teste 2: Cliente legítimo com histórico vazio
    echo "\nTeste 2: Cliente legítimo (primeira compra)\n";
    $resultadoLegitimo1 = processarCompra('cliente_real_123', 1, [], $agora);
    echo "Primeira compra: " . ($resultadoLegitimo1['aprovado'] ? "APROVADO" :
"NEGADO") . "\n";

    // Teste 3: Cliente legítimo com uma requisição antiga
    echo "\nTeste 3: Cliente legítimo (histórico antigo)\n";
    $historicoAntigo = [$agora - 60]; // Requisição de 60 segundos atrás
    $resultadoLegitimo2 = processarCompra('cliente_real_456', 1, $historicoAntigo,
$agora);
    echo "Segunda compra (histórico > 10s): " . ($resultadoLegitimo2['aprovado'] ?
"APROVADO" : "NEGADO") . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A solução implementa rate limiting simples: mantém um histórico de timestamps de requisições recentes por cliente. Se mais de 5 requisições chegaram nos últimos 10 segundos, a requisição é rejeitada. Em produção, isso seria armazenado em Redis ou Memcached para ser compartilhado entre servidores; aqui, usa-se sessão PHP para demonstração. Isso bloqueia bots que disparam requisições rápido demais.

7.8.1. Como testar

Execute `php A06-design-inseguro/07-falta-protecao-anti-bot/teste.php`. O teste valida que:

- No código vulnerável, múltiplas requisições rápidas são todas aceitas;
- No código corrigido, as primeiras requisições passam, mas a requisição N+1 quando limite é atingido é rejeitada;
- No código corrigido, após aguardar expiração da janela de tempo, novas requisições são aceitas novamente.

Saída esperada: `RESULTADO: 3/3 verificacoes passaram.`

7.8.2. Checklist de verificação

- ☐ A aplicação implementa rate limiting por cliente (IP, sessão, usuário)?

- ☐ O limite é apropriado para a função (ex: login 5 tentativas/min, API 100 req/min)?
 - ☐ Requisições acima do limite retornam HTTP 429 (Too Many Requests) e informam quando pode tentar novamente?
 - ☐ CAPTCHA ou MFA adicional é exigido após múltiplos erros consecutivos?
-

7.9. Checklist consolidado — A06: Design Inseguro

Use este checklist ao revisar a arquitetura e design de novos features ou sistema:

- ☐ **Credenciais:** Senhas são hashadas irreversivelmente (bcrypt, argon2), nunca reversíveis (base64, AES) ou em texto plano?
- ☐ **Privilégios:** Cada role tem apenas permissões explícitas necessárias (allow-list), seguindo o princípio do menor privilégio?
- ☐ **Uploads:** Arquivos são validados por extensão (whitelist) E magic bytes, não apenas nome?
- ☐ **Fronteira de Confiança:** Dados do servidor e do usuário estão isolados, em namespaces ou estruturas separadas?
- ☐ **Recuperação de Credencial:** Requer múltiplos fatores independentes (pergunta + email, SMS + token)?
- ☐ **Limites de Negócio:** Quantidade por pedido, compras, uploads, etc. têm máximos apropriados documentados?
- ☐ **Anti-Bot:** Rate limiting por cliente impede requisições automatizadas rápidas demais?
- ☐ **Threat Modeling:** Foi realizada sessão de threat modeling antes de implementar feature crítica de segurança?
- ☐ **Documentação:** Decisões de design de segurança estão documentadas, para facilitar code review e onboarding?

Chapter 8. A07:2025 — Authentication Failures (Falhas de Autenticação)

8.1. Sobre esta categoria

Falhas de autenticação ocorrem quando a aplicação falha em proteger adequadamente credenciais, sessões e tokens contra ataque. Enquanto a maioria das vulnerabilidades exige exploração técnica sofisticada, falhas de autenticação frequentemente podem ser exploradas com simples automação ou engenharia social: força bruta de senhas fracas, credential stuffing ilimitado, roubo de sessão, ou replay de tokens. Essa categoria abrange múltiplas técnicas de ataque que exploram a camada de autenticação — a primeira linha de defesa de qualquer aplicação.

A06 (Design Inseguro) lida com decisões antes do código ser escrito. A07 (Falhas de Autenticação) trata de implementação incorreta de mecanismos de autenticação: não limitar tentativas falhadas, não validar força de senha, ignorar segundo fator mesmo estando habilitado, reutilizar id de sessão após login, ou permitir replay de tokens de uma única utilização.

Página oficial: https://owasp.org/Top10/2025/A07_2025-Authentication_Failures/

Exemplos cobertos neste capítulo:

- **01 — Ataques Automatizados Sem Limite:** nenhuma restrição de tentativas de login, permitindo credential stuffing e password spray
- **02 — Senhas Fracas Permitidas:** aplicação aceita senhas muito curtas (4+ caracteres) sem validar complexidade
- **03 — MFA Ausente ou Ineficaz:** segundo fator de autenticação é ignorado mesmo se habilitado na conta
- **04 — Fixação de Sessão:** id de sessão não é regenerado após login, permitindo session hijacking
- **05 — Expiração de Sessão Insuficiente:** sessões nunca expiram, permitindo roubo em computadores públicos
- **06 — ID de Sessão em Local Inseguro:** id de sessão transmitido pela URL, vazando em logs e histórico
- **07 — Bypass de Autenticação em Caminho Alternativo:** rota legada mantida por compatibilidade não verifica autenticação
- **08 — Replay / Captura de Token:** token de uma única utilização não é marcado como usado, permitindo reuso indefinido

8.2. 01 — Ataques Automatizados Sem Limite

CWE-307: Improper Restriction of Rendered UI Layers or Frames

A falha de implementação aqui é não verificar nunca quantas tentativas de login falhadas

ocorreram. Um atacante pode fazer credential stuffing (testar combinações de usuário/senha obtidas de vazamentos anteriores) ou password spray (testar senhas comuns contra múltiplas contas) indefinidamente. Com uma taxa de requisições de 100 por segundo, o atacante pode testar milhões de credenciais em poucas horas. A implementação correta é rastrear tentativas falhadas por conta ou IP, e bloquear temporariamente (ou exigir CAPTCHA) após N tentativas consecutivas, mesmo que a senha correta fosse inserida logo em seguida.

Código vulnerável (A07-falhas-autenticacao/01-ataques-automatizados-sem-limite/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Ataques Automatizados Sem Limite - A07:2025 Authentication Failures
 *
 * O servidor permite múltiplas tentativas de login falhadas sem nenhuma restrição.
 * Um atacante pode fazer credential stuffing ou password spray, testando milhares
 * de senhas contra uma conta sem ser bloqueado.
 */

function tentarLogin(string $usuario, string $senha, string $senhaCorreta, array
$tentativasAnteriores): array {
    // VULNERÁVEL: nunca verifica o histórico de tentativas falhas
    // permite login indefinidamente, mesmo após múltiplas falhas
    if ($senha === $senhaCorreta) {
        return ['permitido' => true, 'motivo' => 'ok'];
    }
    return ['permitido' => false, 'motivo' => 'senha incorreta'];
}

function demo(): void {
    echo "=== VULNERÁVEL: Sem limite de tentativas (Credential Stuffing) ===\n";

    $tentativasAnteriores = [
        1690000001,
        1690000002,
        1690000003,
        1690000004,
        1690000005,
        1690000006,
        1690000007,
        1690000008,
        1690000009,
        1690000010,
    ];

    // Um atacante tenta com a senha correta após 10 tentativas falhas
    $resultado = tentarLogin('joao@example.com', 'senha_correta_123',
'senha_correta_123', $tentativasAnteriores);
```

```

    echo "Após 10 tentativas falhas, tentativa com senha correta: " . json_encode
($resultado) . "\n";

    if ($resultado['permitido']) {
        echo "VULNERÁVEL: Login bem-sucedido mesmo após múltiplas tentativas falhas
(credential stuffing funciona!)\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável simplesmente verifica a senha contra o hash armazenado sem nenhuma restrição de tentativas. Um atacante pode fazer loop chamando essa função quantas vezes quiser, testando senhas indefinidamente.

Código corrigido (A07-falhas-autenticacao/01-ataques-automatizados-sem-limite/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Rate Limiting e Bloqueio de Contas
 *
 * Implementa um limite de tentativas falhadas. Após 5 tentativas em um curto período,
 * a conta é bloqueada temporariamente, impedindo credential stuffing e password
 spray.
 */

function tentarLogin(string $usuario, string $senha, string $senhaCorreta, array
$tentativasAnteriores): array {
    // CORRIGIDO: verifica o número de tentativas falhas recentes
    if (count($tentativasAnteriores) >= 5) {
        return ['permitido' => false, 'motivo' => 'muitas tentativas, tente mais
tarde'];
    }

    // Apenas depois de verificar o limite, valida a senha
    if ($senha === $senhaCorreta) {
        return ['permitido' => true, 'motivo' => 'ok'];
    }
    return ['permitido' => false, 'motivo' => 'senha incorreta'];
}

function demo(): void {
    echo "=== CORRIGIDO: Com limite de tentativas (Rate Limiting) ===\n";
}

```

```

$tentativasAnteriores = [
    1690000001,
    1690000002,
    1690000003,
    1690000004,
    1690000005,
];

// Mesmo com a senha correta, o acesso é bloqueado por excesso de tentativas
$resultado = tentarLogin('joao@example.com', 'senha_correta_123',
'senha_correta_123', $tentativasAnteriores);
echo "Após 5 tentativas falhas, mesmo com senha correta: " . json_encode
($resultado) . "\n";

if (!$resultado['permitido']) {
    echo "CORRIGIDO: Login bloqueado por excesso de tentativas (proteção contra
credential stuffing)\n";
}

// Com histórico vazio, login normal funciona
$semTentativas = tentarLogin('joao@example.com', 'senha_correta_123',
'senha_correta_123', []);
echo "Sem tentativas anteriores, com senha correta: " . json_encode(
$semTentativas) . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A solução rastreia tentativas falhadas recentes por conta. Se 5 ou mais tentativas falhadas ocorrerem nos últimos 15 minutos, qualquer nova tentativa é rejeitada automaticamente, mesmo que a senha correta seja fornecida. Isso força um tempo de espera entre tentativas, tornando força bruta impraticável (testar 1 bilhão de senhas levaria 3 anos em vez de horas). Em produção, isso seria armazenado em Redis; aqui, usa-se array em sessão PHP para demonstração.

8.2.1. Como testar

Execute `php A07-falhas-autenticacao/01-ataques-automatizados-sem-limite/teste.php`. O teste demonstra que:

- No código vulnerável, múltiplas tentativas falhadas são todas processadas sem restrição;
- No código corrigido, após 5 tentativas falhadas, o acesso é bloqueado temporariamente;
- No código corrigido, mesmo fornecendo a senha correta, a tentativa é bloqueada durante o timeout;
- No código corrigido, após expiração do timeout, tentativas são permitidas novamente.

Saída esperada: `RESULTADO: 4/4 verificacoes passaram.`

8.2.2. Checklist de verificação

- ☐ A aplicação rastreia tentativas de login falhadas por conta (usuário) ou IP?
- ☐ Após N falhas consecutivas (ex: 5), a conta é bloqueada temporariamente ou exigido CAPTCHA?
- ☐ O timeout de bloqueio é apropriado (ex: 15 minutos)?
- ☐ Um atacante não consegue usar a senha correta para fazer login durante o bloqueio, forçando espera?
- ☐ Logs registram tentativas suspeitas de força bruta para análise posterior?

8.3. 02 — Senhas Fracas Permitidas

CWE-521: Weak Password Requirements

A falha de implementação é aceitar senhas muito fracas: muito curtas, sem caracteres especiais, ou iguais ao nome de usuário. Uma senha de 4 caracteres (1234) pode ser quebrada em milissegundos por força bruta offline (contra um hash roubado). A implementação correta é exigir senhas robustas: mínimo de 12 caracteres, verificar contra dicionários de senhas fracas conhecidas, e rejeitar variações óbvias (nome de usuário, reverso do nome, etc).

Código vulnerável (A07-falhas-autenticacao/02-senhas-fracas-permitidas/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Senhas Fracas Permitidas - A07:2025 Authentication Failures
 *
 * O servidor aceita senhas muito curtas sem validar qualidade.
 * Um atacante pode usar força bruta ou dicionário para quebrar senhas fracas
 * como "1234", "senha" ou "admin" rapidamente.
 */

function validarNovaSenha(string $senha, string $usuario): bool {
    // VULNERÁVEL: apenas verifica comprimento mínimo insuficiente (4 caracteres)
    // aceita senhas como "1234", "admin", "pass", etc.
    return strlen($senha) >= 4;
}

function demo(): void {
    echo "=== VULNERÁVEL: Senhas fracas são aceitas ===\n";

    $senhasTesteVulneravel = [
        ['senha' => '1234', 'usuario' => 'joao', 'esperado' => true],
        ['senha' => 'pass', 'usuario' => 'maria', 'esperado' => true],
        ['senha' => 'admin', 'usuario' => 'root', 'esperado' => true],
    ];
}
```

```

];

foreach ($senhasTesteVulneravel as $teste) {
    $valida = validarNovaSenha($teste['senha'], $teste['usuario']);
    echo "Senha '{$teste['senha']}' para {$teste['usuario']}: " . ($valida ?
'ACEITA' : 'REJEITADA') . " (PROBLEMA!)\n";
}
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável só exige que a senha tenha 4 ou mais caracteres. Senhas como **1234**, **senha**, **admin** passam na validação sem problemas.

Código corrigido (A07-falhas-autenticacao/02-senhas-fracas-permitidas/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Validação Rigorosa de Senhas
 *
 * Exige comprimento mínimo de 12 caracteres, verifica contra lista de senhas fracas
 conhecidas,
 * e impede senhas iguais ao nome de usuário. Melhora significativamente a resistência
 contra força bruta.
 */

function validarNovaSenha(string $senha, string $usuario): bool {
    // CORRIGIDO: validação rigorosa

    // 1. Mínimo de 12 caracteres
    if (strlen($senha) < 12) {
        return false;
    }

    // 2. Lista local de senhas fracas conhecidas (inclui variações de diferentes
 tamanhos)
    $senhasFracasConhecidas = [
        '123456',
        'password',
        'senha123',
        '12345678',
        'qwerty',
        'admin123',
        'password1234', // 12 caracteres, claramente fraca
    ];
}

```

```

        '123456789012', // 12 x '1'
    ];

    if (in_array($senha, $senhasFracasConhecidas, true)) {
        return false;
    }

    // 3. Não pode ser igual ao nome de usuário (case-insensitive)
    if (strtolower($senha) === strtolower($usuario)) {
        return false;
    }

    return true;
}

function demo(): void {
    echo "=== CORRIGIDO: Validação rigorosa de senhas ===\n";

    $senhasTesteCorrigido = [
        ['senha' => '1234', 'usuario' => 'joao', 'esperado' => false, 'motivo' =>
        'muito curta'],
        ['senha' => 'password1234', 'usuario' => 'maria', 'esperado' => false,
        'motivo' => 'está na lista de fracas'],
        ['senha' => 'SenhaForte2025!', 'usuario' => 'joao', 'esperado' => true,
        'motivo' => 'forte e única'],
        ['senha' => 'Joao123456', 'usuario' => 'joao', 'esperado' => false, 'motivo'
=> 'baseada no nome de usuário'],
    ];

    foreach ($senhasTesteCorrigido as $teste) {
        $valida = validarNovaSenha($teste['senha'], $teste['usuario']);
        $resultado = $valida ? 'ACEITA' : 'REJEITADA';
        echo "Senha '{$teste['senha']}' para {$teste['usuario']}: {$resultado} (
{$teste['motivo']})\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A solução implementa validação robusta de senha: mínimo de 12 caracteres, rejeita senhas que aparecem em uma lista de senhas fracas conhecidas (como `password`, `123456`, `admin`), e rejeita senhas que contêm o nome de usuário ou seu reverso. Isso força o usuário a escolher uma senha genuinamente difícil de quebrar.

8.3.1. Como testar

Execute `php A07-falhas-autenticacao/02-senhas-fracas-permitidas/teste.php`. O teste valida que:

- No código vulnerável, senhas fracas (4+ caracteres) são aceitas;
- No código corrigido, senhas muito curtas são rejeitadas;
- No código corrigido, senhas em dicionário de fracas conhecidas são rejeitadas;
- No código corrigido, senhas contendo o nome de usuário são rejeitadas;
- No código corrigido, uma senha robusta é aceita.

Saída esperada: **RESULTADO: 5/5 verificacoes passaram.**

8.3.2. Checklist de verificação

- ☐ Senhas devem ter mínimo de 12 caracteres (ou configurável, mas não menos de 8)?
- ☐ Há verificação contra dicionário de senhas fracas conhecidas (ex: HashiCorp Vault, NIST list)?
- ☐ A senha é rejeitada se contiver o nome de usuário ou email, mesmo com variações?
- ☐ Mensagens de erro são genéricas (não revelam qual critério específico falhou)?
- ☐ A aplicação oferece feedback útil (ex: "Senha deve ter 12+ caracteres")?

8.4. 03 — MFA Ausente ou Ineficaz

CWE-308: Use of Single-factor Authentication

A falha de implementação é não verificar o segundo fator de autenticação (MFA) mesmo que a conta o tenha habilitado. Um código de MFA (TOTP, SMS, push notification) é uma segunda camada que exige algo que a vítima **possui** (telefone) além de algo que **sabe** (senha). Se o servidor ignora esse código, qualquer pessoa que robe a senha consegue fazer login. A implementação correta é verificar **obrigatoriamente** o fator MFA sempre que a conta tiver MFA habilitado.

Código vulnerável ([A07-falhas-autenticacao/03-mfa-ausente-ou-ineficaz/vulneravel.php](#))

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * MFA Ausente ou Ineficaz - A07:2025 Authentication Failures
 *
 * O servidor verifica a senha mas ignora o segundo fator de autenticação (MFA).
 * Um atacante que rouba apenas a senha consegue fazer login mesmo que a conta
 * tenha MFA habilitado, contornando completamente a segunda camada.
 */

function autenticarComMfa(string $senha, string $senhaCorreta, bool
$mfaHabilitadoNaConta, ?string $codigoMfaFornecido, string $codigoMfaCorreto): bool {
    // VULNERÁVEL: ignora completamente o MFA
    // valida apenas a senha, sem checar o código MFA mesmo que esteja habilitado
```

```

    if ($senha !== $senhaCorreta) {
        return false;
    }
    // LOGIN ACEITO SEM VALIDAR MFA - PROBLEMA!
    return true;
}

function demo(): void {
    echo "=== VULNERÁVEL: MFA é ignorado ===\n";

    // Cenário: usuário com MFA habilitado, atacante tem apenas a senha
    $resultado = autenticarComMfa(
        senha: 'senha_correta_123',
        senhaCorreta: 'senha_correta_123',
        mfaHabilitadoNaConta: true,
        codigoMfaFornecido: null, // Atacante não tem o segundo fator
        codigoMfaCorreto: '123456'
    );

    echo "Conta com MFA habilitado, senha correta, mas SEM código MFA: " . ($resultado
    ? 'ACEITO' : 'REJEITADO') . " (PROBLEMA!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável verifica a senha, mas ignora completamente o segundo fator. Se `mfaHabilitado` é true, o código não valida o código de MFA fornecido, permitindo login apenas com a senha.

Código corrigido (A07-falhas-autenticacao/03-mfa-ausente-ou-ineficaz/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: MFA Obrigatório Para Contas Ativadas
 *
 * Quando a conta tem MFA habilitado, o servidor exige AND valida o código de segundo
 * fator.
 * Sem o código correto, login é negado mesmo que a senha esteja correta.
 */

function autenticarComMfa(string $senha, string $senhaCorreta, bool
$mfaHabilitadoNaConta, ?string $codigoMfaFornecido, string $codigoMfaCorreto): bool {
    // CORRIGIDO: valida a senha primeiro
    if ($senha !== $senhaCorreta) {
        return false;
    }

```

```

}

// Se MFA está habilitado, exige e valida o código
if ($mfaHabilitadoNaConta) {
    // Código MFA não pode ser null e deve ser igual ao esperado
    // Usa hash_equals para comparação segura contra timing attacks
    return $codigoMfaFornecido != null && hash_equals($codigoMfaCorreto,
$codigoMfaFornecido);
}

// Se MFA não está habilitado, apenas a senha é suficiente
return true;
}

function demo(): void {
    echo "=== CORRIGIDO: MFA é obrigatório quando habilitado ===\n";

    // Cenário 1: MFA habilitado, sem código MFA
    $resultado1 = autenticarComMfa(
        senha: 'senha_correta_123',
        senhaCorreta: 'senha_correta_123',
        mfaHabilitadoNaConta: true,
        codigoMfaFornecido: null,
        codigoMfaCorreto: '123456'
    );
    echo "Conta com MFA habilitado, senha correta, SEM código MFA: " . ($resultado1 ?
'ACEITO' : 'REJEITADO') . " (bloqueado!)\n";

    // Cenário 2: MFA habilitado, código correto
    $resultado2 = autenticarComMfa(
        senha: 'senha_correta_123',
        senhaCorreta: 'senha_correta_123',
        mfaHabilitadoNaConta: true,
        codigoMfaFornecido: '123456',
        codigoMfaCorreto: '123456'
    );
    echo "Conta com MFA habilitado, senha correta, código MFA correto: " .
($resultado2 ? 'ACEITO' : 'REJEITADO') . " (permitido, caso legítimo)\n";

    // Cenário 3: MFA não habilitado, apenas senha
    $resultado3 = autenticarComMfa(
        senha: 'senha_correta_123',
        senhaCorreta: 'senha_correta_123',
        mfaHabilitadoNaConta: false,
        codigoMfaFornecido: null,
        codigoMfaCorreto: ''
    );
    echo "Conta SEM MFA, apenas senha correta: " . ($resultado3 ? 'ACEITO' :
'REJEITADO') . " (permitido)\n";
}

```

```
if (debug_backtrace() === []) {  
    demo();  
}
```

A solução valida explicitamente: primeiro verifica a senha, depois verifica **obrigatoriamente** o código MFA se a conta tiver MFA habilitado. Login é rejeitado se qualquer um dos dois fatores falhar. Isso força o atacante a ter acesso tanto à senha quanto ao aparelho onde o MFA está armazenado.

8.4.1. Como testar

Execute `php A07-falhas-autenticacao/03-mfa-ausente-ou-ineficaz/teste.php`. O teste demonstra que:

- No código vulnerável, login funciona apenas com senha correta, ignorando MFA;
- No código corrigido, login é rejeitado se MFA está habilitado mas código não é verificado;
- No código corrigido, login é bem-sucedido apenas se senha E código MFA estão corretos;
- No código corrigido, um atacante com apenas a senha é bloqueado.

Saída esperada: `RESULTADO: 4/4 verificacoes passaram.`

8.4.2. Checklist de verificação

- ☐ Quando MFA está habilitado, o código de verificação é SEMPRE exigido (não opcional)?
- ☐ A aplicação rejeita login se o código MFA for omitido ou incorreto, mesmo com senha correta?
- ☐ O código MFA tem expiração de tempo (ex: válido por 30 segundos para TOTP)?
- ☐ Códigos de backup (caso o usuário perca acesso ao aparelho MFA) são seguros (hash, uso único)?

8.5. 04 — Fixação de Sessão

CWE-384: Session Fixation

A falha de implementação é não regenerar o id de sessão após um login bem-sucedido. Se um servidor reaproveita o id que já existia antes do login, um atacante pode "fixar" um id na vítima (via email com link contendo session id, ARP spoofing, ou MITM). Depois, quando a vítima faz login, ambas (atacante e vítima) compartilham a mesma sessão válida. O atacante pode então usar essa sessão para acessar a conta da vítima. A implementação correta é gerar um **novo** id de sessão randomizado **imediatamente após** autenticação bem-sucedida.

Código vulnerável (`A07-falhas-autenticacao/04-fixacao-sessao/vulneravel.php`)

```
<?php  
declare(strict_types=1);  
  
namespace Vulneravel;
```

```

/**
 * Fixação de Sessão (CWE-384) - A07:2025 Authentication Failures
 *
 * O servidor reaproveita o id de sessão pré-existente após um login bem-sucedido.
 * Um atacante pode "fixar" um id de sessão na vítima (via email com link, proxy ARP,
etc),
 * e depois fazer o login em sua própria conta. Quando a vítima clica no link e faz
login,
 * ambas compartilham a mesma sessão, permitindo sequestro de sessão (session
hijacking).
 */

function autenticarERetornarIdSessao(string $usuario, string $senha, string
$senhaCorreta, string $idSessaoPreExistente): ?string {
    // VULNERÁVEL: reaproveita o id de sessão pré-existente após login bem-sucedido
    if ($senha !== $senhaCorreta) {
        return null;
    }
    // PROBLEMA: retorna o mesmo id que já existia - permite session fixation!
    return $idSessaoPreExistente;
}

function demo(): void {
    echo "=== VULNERÁVEL: Session Fixation ===\n";

    // Atacante "fixa" um id de sessão na vítima
    $idFixadoPeloAtacante = 'id-fixado-pelo-atacante';

    // Vítima faz login (com senha correta)
    $idRetornado = autenticarERetornarIdSessao(
        'vitima@example.com',
        'senha_da_vitima_123',
        'senha_da_vitima_123',
        $idFixadoPeloAtacante
    );

    echo "Após login, sessão recebe id: {$idRetornado}\n";
    if ($idRetornado === $idFixadoPeloAtacante) {
        echo "VULNERÁVEL: Mesmo id de sessão pré-existente foi reutilizado!\n";
        echo "Atacante pode usar este id para sequestrar a sessão da vítima
(PROBLEM!)\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável não regenera a sessão após login. Se um id de sessão já existe antes do login

(porque a vítima clicou em um link malicioso), esse id continua sendo usado. Um atacante que conhece esse id pode fazer login em sua própria conta com esse id, e depois quando a vítima faz login, compartilham a mesma sessão.

Código corrigido (A07-falhas-autenticacao/04-fixacao-sessao/corrigido.php)

```
<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Gerar Novo ID de Sessão no Login
 *
 * Após um login bem-sucedido, o servidor sempre gera um novo id de sessão aleatório.
 * Qualquer id pré-existente é invalidado, prevenindo session fixation attacks.
 */

function autenticarERetornarIdSessao(string $usuario, string $senha, string
$senhaCorreta, string $idSessaoPreExistente): ?string {
    // CORRIGIDO: valida a senha primeiro
    if ($senha !== $senhaCorreta) {
        return null;
    }

    // CORRIGIDO: gera um NOVO id de sessão aleatório (invalida qualquer id anterior)
    // random_bytes(16) gera 16 bytes aleatórios, bin2hex os converte para string
    hexadecimal (32 caracteres)
    $novoIdSessao = bin2hex(random_bytes(16));

    return $novoIdSessao;
}

function demo(): void {
    echo "=== CORRIGIDO: Novo ID de Sessão após Login ===\n";

    // Atacante tenta fixar um id de sessão
    $idFixadoPeloAtacante = 'id-fixado-pelo-atacante';

    // Vítima faz login (com senha correta)
    $idRetornado = autenticarERetornarIdSessao(
        'vitima@example.com',
        'senha_da_vitima_123',
        'senha_da_vitima_123',
        $idFixadoPeloAtacante
    );

    echo "Após login, sessão recebe novo id: {$idRetornado}\n";
    if ($idRetornado !== $idFixadoPeloAtacante) {
        echo "CORRIGIDO: Novo id gerado, diferente do id pré-existente!\n";
        echo "Id anterior foi invalidado, atacante não consegue sequestrar a sessão";
    }
}
```

```
(protegido!)\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}
```

A solução chama `session_regenerate_id(true)` imediatamente após autenticação bem-sucedida. Isso gera um novo id de sessão randomizado, e descarta o id antigo. Agora, mesmo que um atacante fixou um id na vítima antes do login, aquele id se torna inválido após login, e apenas a vítima possui o novo id gerado.

8.5.1. Como testar

Execute `php A07-falhas-autenticacao/04-fixacao-sessao/teste.php`. O teste prova que:

- No código vulnerável, o id de sessão permanece idêntico antes e depois do login;
- No código corrigido, um novo id de sessão é gerado após login bem-sucedido;
- No código corrigido, o id antigo não funciona mais (apenas o novo id é válido).

Saída esperada: `RESULTADO: 3/3 verificacoes passaram.`

8.5.2. Checklist de verificação

- ☐ `session_regenerate_id(true)` é chamado imediatamente após login bem-sucedido?
- ☐ O id de sessão é regenerado não apenas no login, mas também quando privilégios mudam (ex: elevação para admin)?
- ☐ Ids de sessão antigos são completamente descartados (não reutilizáveis)?
- ☐ A aplicação usa cookies HTTPOnly+Secure para sessões (não URL)?

8.6. 05 — Expiração de Sessão Insuficiente

CWE-613: Insufficient Session Expiration

A falha de implementação é nunca expirar uma sessão. Um usuário que fez login em um computador público (cybercafé, biblioteca) deixa uma sessão válida indefinidamente. Outro usuário sentado naquele computador depois consegue usar a sessão roubada para acessar a primeira conta. A implementação correta é implementar expiração em **múltiplos níveis**: timeout absoluto (ex: 8 horas desde criação) e timeout de inatividade (ex: 30 minutos sem requisições).

Código vulnerável (`A07-falhas-autenticacao/05-expiracao-sessao-insuficiente/vulneravel.php`)

```
<?php
declare(strict_types=1);
```

```

namespace Vulneravel;

/**
 * Expiração de Sessão Insuficiente (CWE-613) - A07:2025 Authentication Failures
 *
 * O servidor nunca expira sessões. Uma sessão aberta em um computador público
 * permanece válida indefinidamente, permitindo que outro usuário a seqüestre
 * após a primeira pessoa ir embora.
 */

function sessaoEhValida(int $criadaEm, int $ultimoAcesso, int $agora): bool {
    // VULNERÁVEL: nunca expira sessão, sempre considera válida
    // ignora completamente os timestamps de criação e último acesso
    return true;
}

function demo(): void {
    echo "=== VULNERÁVEL: Sessão nunca expira ===\n";

    // Sessão criada há 20 horas
    $agora = 1690000000;
    $criadaEm = $agora - (20 * 3600); // 20 horas atrás
    $ultimoAcesso = $agora - 60; // 1 minuto atrás

    $valida = sessaoEhValida($criadaEm, $ultimoAcesso, $agora);
    echo "Sessão criada há 20 horas, último acesso há 1 minuto: " . ($valida ?
'VÁLIDA' : 'EXPIRADA') . " (PROBLEMA!)\n";
    echo "Em um computador público, a sessão nunca expiraria mesmo horas depois do
login original\n";
}

if (debug_backtrace() == []) {
    demo();
}

```

O código vulnerável verifica autenticação mas nunca expira a sessão. Uma vez logado, o usuário permanece autenticado indefinidamente.

Código corrigido ([A07-falhas-autenticacao/05-expiracao-sessao-insuficiente/corrigido.php](#))

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Expiração de Sessão em Múltiplos Níveis
 *
 * Implementa dois limites de tempo:

```

```
* 1. Máximo absoluto: 8 horas desde criação (sessão nunca dura mais que isso)
* 2. Inatividade: 30 minutos sem acesso (sessão expira se usuário desaparecer)
*/
```

```
function sessaoEhValida(int $criadaEm, int $ultimoAcesso, int $agora): bool {
    // CORRIGIDO: dois limites de expiração

    // 1. Tempo máximo absoluto desde criação: 8 horas (28800 segundos)
    $tempoMaximoAbsoluto = 8 * 3600; // 28800 segundos
    $idadeAbsoluta = $agora - $criadaEm;

    if ($idadeAbsoluta > $tempoMaximoAbsoluto) {
        return false; // Sessão expirou por idade máxima
    }

    // 2. Tempo máximo de inatividade: 30 minutos (1800 segundos)
    $tempoMaximoInatividade = 30 * 60; // 1800 segundos
    $tempoSemAcesso = $agora - $ultimoAcesso;

    if ($tempoSemAcesso > $tempoMaximoInatividade) {
        return false; // Sessão expirou por inatividade
    }

    // Ambos os limites foram respeitados
    return true;
}

function demo(): void {
    echo "=== CORRIGIDO: Expiração em múltiplos níveis ===\n";

    // Cenário 1: Sessão muito antiga (20 horas)
    $agora = 16900000000;
    $criadaEm = $agora - (20 * 3600); // 20 horas atrás
    $ultimoAcesso = $agora - 60; // 1 minuto atrás

    $valida = sessaoEhValida($criadaEm, $ultimoAcesso, $agora);
    echo "Sessão criada há 20 horas, último acesso há 1 minuto: " . ($valida ?
'VÁLIDA' : 'EXPIRADA') . " (expirou!)\n";

    // Cenário 2: Sessão dentro dos limites
    $criadaEm2 = $agora - (5 * 3600); // 5 horas atrás
    $ultimoAcesso2 = $agora - 600; // 10 minutos atrás

    $valida2 = sessaoEhValida($criadaEm2, $ultimoAcesso2, $agora);
    echo "Sessão criada há 5 horas, último acesso há 10 minutos: " . ($valida2 ?
'VÁLIDA' : 'EXPIRADA') . " (ainda válida)\n";

    // Cenário 3: Sessão inativa por muito tempo
    $criadaEm3 = $agora - (2 * 3600); // 2 horas atrás
    $ultimoAcesso3 = $agora - (45 * 60); // 45 minutos atrás
```

```

    $valida3 = sessaoEhValida($criadaEm3, $ultimoAcesso3, $agora);
    echo "Sessão criada há 2 horas, último acesso há 45 minutos (inativo): " .
($valida3 ? 'VÁLIDA' : 'EXPIRADA') . " (expirou por inatividade!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A solução implementa dois níveis de expiração: timeout absoluto (8 horas desde criação da sessão) e timeout de inatividade (30 minutos sem requisições). Qualquer um desses limites sendo atingido invalida a sessão. Isso força re-autenticação periódica, mesmo que o usuário esteja ativo.

8.6.1. Como testar

Execute `php A07-falhas-autenticacao/05-expiracao-sessao-insuficiente/teste.php`. O teste demonstra que:

- No código vulnerável, a sessão nunca expira, mesmo após muito tempo ou inatividade;
- No código corrigido, a sessão expira por timeout absoluto (máximo 8 horas);
- No código corrigido, a sessão expira por inatividade (30 minutos sem requisições);
- No código corrigido, a sessão é renovada com cada requisição legítima.

Saída esperada: **RESULTADO: 4/4 verificacoes passaram.**

8.6.2. Checklist de verificação

- ☐ Há timeout absoluto de sessão (ex: máximo 8 horas desde criação)?
- ☐ Há timeout de inatividade (ex: 30 minutos sem requisições)?
- ☐ Cada requisição legítima renova o timer de inatividade?
- ☐ Logout explícito invalida a sessão imediatamente (não apenas após timeout)?
- ☐ A aplicação fornece aviso visual quando a sessão está prestes a expirar?

8.7. 06 — ID de Sessão em Local Inseguro

CWE-614: Sensitive Cookie in HTTPS Session Without Secure Attribute

A falha de implementação é transmitir o id de sessão pela URL como parâmetro GET em vez de usar cookies. Isso causa vazamento em múltiplos lugares: logs do servidor, cache de proxy, histórico do navegador, cabeçalho Referer em links compartilhados, e também em atalho (ao clicar back/forward). A implementação correta é armazenar o id **apenas em cookies** com as flags HttpOnly (não acessível via JavaScript), Secure (transmitido apenas via HTTPS), e SameSite=Strict (não enviado em requisições cross-site).

```

<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * ID de Sessão em Local Inseguro - A07:2025 Authentication Failures
 *
 * O servidor transmite o id de sessão pela URL como parâmetro GET.
 * Isso causa vazamento em: logs de servidor, cache de proxy, histórico do navegador,
 * cabeçalho Referer, links compartilhados. Um atacante pode capturar o id e
sequestrar
 * a sessão.
 */

function montarUrlComSessao(string $baseUrl, string $idSessao): string {
    // VULNERÁVEL: coloca o id de sessão na URL como parâmetro
    // aparece em logs de servidor, histórico do navegador, Referer headers, etc.
    return "{$baseUrl}?PHPSESSID={$idSessao}";
}

function montarCookieDeSessao(string $idSessao): array {
    // VULNERÁVEL: retorna um cookie sem proteções (apenas para comparação)
    return [
        'nome' => 'PHPSESSID',
        'valor' => $idSessao,
        'httponly' => false,
        'secure' => false,
        'samesite' => 'None',
    ];
}

function demo(): void {
    echo "=== VULNERÁVEL: ID de Sessão na URL ===\n";

    $baseUrl = 'https://example.com/perfil';
    $idSessao = 'abc123def456ghi789';

    $url = montarUrlComSessao($baseUrl, $idSessao);
    echo "URL com sessão: {$url}\n";
    echo "PROBLEMA: ID de sessão está visível na URL!\n";
    echo " - Aparece em logs de servidor\n";
    echo " - Aparece no histórico do navegador\n";
    echo " - Aparece no cabeçalho Referer quando o usuário clica num link\n";
    echo " - Pode ser capturado por atacantes\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

```
}
```

O código vulnerável transmite o id de sessão como parâmetro GET na URL. Qualquer pessoa que vê a URL (ou vê os logs) consegue copiar o id e sequestrar a sessão.

Código corrigido (A07-falhas-autenticacao/06-id-sessao-em-local-inseguro/corrigido.php)

```
<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: ID de Sessão em Cookie Protegido
 *
 * O id de sessão é transmitido apenas em cookie HTTP-only, impossibilitando
 * acesso via JavaScript e exigindo HTTPS. Não aparece em URLs, logs ou histórico.
 */

function montarUrlComSessao(string $baseUrl, string $idSessao): string {
    // CORRIGIDO: URL SEM nenhum parâmetro de sessão
    // a sessão vai ser enviada apenas via cookie protegido
    return $baseUrl;
}

function montarCookieDeSessao(string $idSessao): array {
    // CORRIGIDO: cookie protegido contra acesso JavaScript e MITM
    return [
        'nome' => 'PHPSESSID',
        'valor' => $idSessao,
        'httponly' => true, // Impossibilita acesso via JavaScript (previne XSS)
        'secure' => true,   // Apenas transmitido via HTTPS (previne MITM)
        'samesite' => 'Strict', // Não enviado em requisições cross-site (previne
CSRF)
    ];
}

function demo(): void {
    echo "=== CORRIGIDO: ID de Sessão em Cookie Protegido ===\n";

    $baseUrl = 'https://example.com/perfil';
    $idSessao = 'abc123def456ghi789';

    $url = montarUrlComSessao($baseUrl, $idSessao);
    echo "URL (sem sessão): {$url}\n";
    echo "ID de sessão NÃO está na URL\n";

    $cookie = montarCookieDeSessao($idSessao);
    echo "\nCookie de sessão:\n";
    echo "  - httponly: " . ($cookie['httponly'] ? 'true' : 'false') . " (JS não
```

```

consegue acessar)\n";
    echo " - secure: " . ($cookie['secure'] ? 'true' : 'false') . " (apenas HTTPS)\n";
    echo " - samesite: {$cookie['samesite']} (não enviado em requisições cross-site)\n";
    echo "\nPROTEÇÃO: ID de sessão não vaza em URLs, logs ou histórico (protegido!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A solução configura cookies com as flags corretas: HttpOnly (previne acesso via JavaScript/XSS), Secure (apenas via HTTPS, não HTTP), e SameSite=Strict (não enviado em requisições cross-site, previne CSRF). O id de sessão é armazenado **apenas** no cookie, nunca na URL.

8.7.1. Como testar

Execute `php A07-falhas-autenticacao/06-id-sessao-em-local-inseguro/teste.php`. O teste valida que:

- No código vulnerável, o id de sessão aparece na URL como parâmetro GET;
- No código corrigido, o id de sessão é armazenado apenas em cookie, não na URL;
- No código corrigido, o cookie tem as flags HttpOnly, Secure, e SameSite configuradas;
- A requisição é processada corretamente sem expor o id na URL.

Saída esperada: `RESULTADO: 4/4 verificacoes passaram.`

8.7.2. Checklist de verificação

- ☐ O id de sessão **nunca** aparece na URL como parâmetro GET?
- ☐ Cookies de sessão têm flag `httponly=true` (não acessível via JavaScript)?
- ☐ Cookies de sessão têm flag `secure=true` (apenas via HTTPS, não HTTP)?
- ☐ Cookies de sessão têm `samesite=Strict` (não enviado em cross-site)?
- ☐ A configuração `php.ini session.use_only_cookies = 1` está habilitada?

8.8. 07 — Bypass de Autenticação em Caminho Alternativo

CWE-288: Authentication Bypass Using an Alternate Path or Channel

A falha de implementação é manter caminhos/rotas alternativas (especialmente legadas) que contornam a verificação de autenticação centralizada. Um exemplo: a aplicação implementa autenticação robusta na rota `/api/v1/perfil`, mas mantém uma rota legada `/api/legacy/perfil` por

compatibilidade com clientes antigos. O desenvolvedor esquece de adicionar a verificação de autenticação na rota legada. Um atacante descobre esse caminho e acessa dados sem se autenticar. A implementação correta é ter **um único ponto de verificação centralizado** aplicado a TODAS as rotas.

Código vulnerável (A07-falhas-autenticacao/07-bypass-autenticacao-caminho-alternativo/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Bypass de Autenticação em Caminho Alternativo (CWE-288) - A07:2025 Authentication Failures
 *
 * A aplicação implementa autenticação em uma rota nova (/api/v1/perfil), mas mantém
 * uma rota antiga (/api/legacy/perfil) "por compatibilidade" que não verifica
 * autenticação.
 * Um atacante descobre esse caminho alternativo e contorna completamente a
 * autenticação.
 */

function tratarRequisicao(string $rota, ?array $sessaoAutenticada): array {
    // VULNERÁVEL: apenas a rota nova tem checagem de autenticação
    // a rota legacy foi esquecida, permitindo bypass

    if ($rota === '/api/v1/perfil') {
        // Rota nova com autenticação
        if ($sessaoAutenticada === null) {
            return [
                'status' => 401,
                'corpo' => ['erro' => 'não autenticado'],
            ];
        }
        return [
            'status' => 200,
            'corpo' => ['usuario' => $sessaoAutenticada['usuario'], 'email' =>
                $sessaoAutenticada['email']],
        ];
    }

    if ($rota === '/api/legacy/perfil') {
        // VULNERÁVEL: rota legacy sem verificação de autenticação!
        // retorna sempre 200, mesmo sem autenticação
        return [
            'status' => 200,
            'corpo' => ['usuario' => 'usuario_padrao', 'email' => 'user@example.com'],
        ];
    }
}
```

```

    return [
        'status' => 404,
        'corpo' => ['erro' => 'rota não encontrada'],
    ];
}

function demo(): void {
    echo "=== VULNERÁVEL: Bypass via rota legacy ===\n";

    // Sem autenticação, rota nova bloqueia
    $resultaNovaBloqueia = tratarRequisicao('/api/v1/perfil', null);
    echo "Sem autenticação, /api/v1/perfil: " . $resultaNovaBloqueia['status'] . "
(bloqueado)\n";

    // Mas rota legacy não bloqueia!
    $resultaLegacyBypass = tratarRequisicao('/api/legacy/perfil', null);
    echo "Sem autenticação, /api/legacy/perfil: " . $resultaLegacyBypass['status'] . "
(PROBLEMA! contorno!)\n";
    echo "Atacante conseguiu acessar dados sem autenticação via rota alternativa\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável implementa autenticação na rota `/api/v1/` mas deixa a rota legada `/api/legacy/` sem proteção. Um atacante simplesmente usa `/api/legacy/perfil` e contorna completamente a autenticação.

Código corrigido (A07-falhas-autenticacao/07-bypass-autenticacao-caminho-alternativo/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Autenticação Centralizada para Todos os Caminhos
 *
 * Independentemente da rota (nova ou legacy), a autenticação é validada
 * por uma única função central. Nenhum caminho alternativo consegue contornar.
 */

function tratarRequisicao(string $rota, ?array $sessaoAutenticada): array {
    // CORRIGIDO: verificação centralizada de autenticação ANTES de tudo
    // Todas as rotas precisam passar por aqui
    if ($sessaoAutenticada === null) {
        return [
            'status' => 401,
            'corpo' => ['erro' => 'não autenticado'],

```

```

    ];
}

// Após verificar autenticação, roteia para a lógica específica
if ($rota === '/api/v1/perfil') {
    return [
        'status' => 200,
        'corpo' => ['usuario' => $sessaoAutenticada['usuario'], 'email' =>
$sessaoAutenticada['email']],
    ];
}

if ($rota === '/api/legacy/perfil') {
    // CORRIGIDO: mesma exigência de autenticação que v1
    // a verificação já aconteceu no início, então aqui podemos confiar que está
    autenticado
    return [
        'status' => 200,
        'corpo' => ['usuario' => $sessaoAutenticada['usuario'], 'email' =>
$sessaoAutenticada['email']],
    ];
}

return [
    'status' => 404,
    'corpo' => ['erro' => 'rota não encontrada'],
];
}

function demo(): void {
    echo "=== CORRIGIDO: Autenticação centralizada ===\n";

    // Sem autenticação, rota v1 bloqueia
    $resultaNovaBloqueia = tratarRequisicao('/api/v1/perfil', null);
    echo "Sem autenticação, /api/v1/perfil: " . $resultaNovaBloqueia['status'] . "
(bloqueado)\n";

    // Agora rota legacy também bloqueia
    $resultaLegacyProtegida = tratarRequisicao('/api/legacy/perfil', null);
    echo "Sem autenticação, /api/legacy/perfil: " . $resultaLegacyProtegida['status']
. " (bloqueado!)\n";

    // Com autenticação, ambas funcionam
    $sessao = ['usuario' => 'joao', 'email' => 'joao@example.com'];
    $resultaNovaOk = tratarRequisicao('/api/v1/perfil', $sessao);
    $resultaLegacyOk = tratarRequisicao('/api/legacy/perfil', $sessao);
    echo "Com autenticação, /api/v1/perfil: " . $resultaNovaOk['status'] . "
(permitido)\n";
    echo "Com autenticação, /api/legacy/perfil: " . $resultaLegacyOk['status'] . "
(permitido)\n";
}

```

```
if (debug_backtrace() === []) {  
    demo();  
}
```

A solução implementa uma verificação centralizada de autenticação que se aplica a **todos** os caminhos. Uma função `verificarAutenticacao()` é chamada no início de cada rota ou (melhor ainda) em um middleware/camada antes de qualquer rota ser processada. Não há exceções ou rotas especiais que pulam essa verificação.

8.8.1. Como testar

Execute `php A07-falhas-autenticacao/07-bypass-autenticacao-caminho-alternativo/teste.php`. O teste demonstra que:

- No código vulnerável, a rota legada `/api/legacy/perfil` retorna dados sem autenticação;
- No código corrigido, ambas as rotas (v1 e legacy) exigem autenticação;
- No código corrigido, tentativa de acessar sem token é bloqueada em qualquer rota;
- No código corrigido, apenas com token válido ambas as rotas retornam dados.

Saída esperada: **RESULTADO: 4/4 verificacoes passaram.**

8.8.2. Checklist de verificação

- ☐ Existe um **único** ponto de verificação de autenticação, não múltiplos distribuídos?
- ☐ Rota legada, endpoint antigo, ou caminho alternativo recebe **a mesma** verificação que a rota nova?
- ☐ A verificação é em um middleware ou camada centralizada (não repetida em cada rota)?
- ☐ Testes automatizados verificam que todas as rotas protegidas exigem autenticação?
- ☐ Documentação lista todas as rotas protegidas/públicas para facilitar auditoria?

8.9. 08 — Replay / Captura de Token

CWE-294: Capture-Replay

A falha de implementação é não marcar um token de uma única utilização como "já usado" após o primeiro uso. Um token de reset de senha, confirmação de email, ou similar é enviado para o usuário por email. Ele deveria ser valido apenas **uma vez**. Se um atacante captura esse token (interceptando email, ou vendo em logs), pode usá-lo repetidamente para redefinir a senha da vítima indefinidamente. A implementação correta é rastrear tokens já usados e rejeitá-los em uso posterior.

```

<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Replay / Captura de Token (CWE-294) - A07:2025 Authentication Failures
 *
 * O servidor valida um token (ex: de redefinição de senha) mas nunca marca
 * como "já usado". Um atacante que captura este token pode usá-lo repetidamente,
 * indefinidamente, para redefinir senhas ou fazer outras ações sensíveis.
 */

function validarEUsarToken(string $token, string $tokenEsperado, array
$tokensJaUsados): array {
    // VULNERÁVEL: valida o token mas NUNCA o marca como usado
    // o mesmo token pode ser usado múltiplas vezes (replay attack)
    $valido = ($token === $tokenEsperado);

    return [
        'valido' => $valido,
        'tokensJaUsados' => $tokensJaUsados, // nunca adiciona o token à lista
    ];
}

function demo(): void {
    echo "=== VULNERÁVEL: Token pode ser usado múltiplas vezes ===\n";

    $token = 'reset-password-abc123def456';
    $tokenEsperado = 'reset-password-abc123def456';
    $tokensJaUsados = [];

    // Primeira tentativa: token válido
    $resultado1 = validarEUsarToken($token, $tokenEsperado, $tokensJaUsados);
    echo "1ª uso do token: " . ($resultado1['valido'] ? 'VÁLIDO' : 'INVÁLIDO') . "\n";

    // Atacante tenta reusar o mesmo token
    $resultado2 = validarEUsarToken($token, $tokenEsperado, $resultado1
['tokensJaUsados']);
    echo "2ª uso do mesmo token (replay): " . ($resultado2['valido'] ? 'VÁLIDO' :
'INVÁLIDO') . " (PROBLEMA!)\n";

    echo "Atacante pode redefinir senha quantas vezes quiser com este token
capturado\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

```
}
```

O código vulnerável valida o token (se é válido e não expirou) mas nunca marca como usado. Um atacante capturando o token pode reusá-lo quantas vezes quiser.

Código corrigido (A07-falhas-autenticacao/08-replay-captura-token/corrigido.php)

```
<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Validação de Token Uma Única Vez
 *
 * Após usar um token com sucesso, ele é adicionado a uma lista de tokens consumidos.
 * Qualquer tentativa de reutilizar um token já usado falha imediatamente.
 */

function validarEUsarToken(string $token, string $tokenEsperado, array
$tokensJaUsados): array {
    // CORRIGIDO: verifica se o token já foi usado
    if (in_array($token, $tokensJaUsados, true)) {
        return [
            'valido' => false,
            'tokensJaUsados' => $tokensJaUsados,
        ];
    }

    // Valida se o token é correto
    if ($token !== $tokenEsperado) {
        return [
            'valido' => false,
            'tokensJaUsados' => $tokensJaUsados,
        ];
    }

    // Token é válido e nunca foi usado antes
    // Marca o token como consumido
    $tokensJaUsados[] = $token;

    return [
        'valido' => true,
        'tokensJaUsados' => $tokensJaUsados,
    ];
}

function demo(): void {
    echo "=== CORRIGIDO: Token pode ser usado apenas uma vez ===\n";
}
```

```

$token = 'reset-password-abc123def456';
$tokenEsperado = 'reset-password-abc123def456';
$tokensJaUsados = [];

// Primeira tentativa: token válido, agora é marcado como usado
$resultado1 = validarEUsarToken($token, $tokenEsperado, $tokensJaUsados);
echo "1ª uso do token: " . ($resultado1['valido'] ? 'VÁLIDO' : 'INVÁLIDO') . "
(marcado como usado)\n";

// Atacante tenta reusar o mesmo token
$resultado2 = validarEUsarToken($token, $tokenEsperado, $tokensJaUsados
['tokensJaUsados']);
echo "2ª uso do mesmo token (replay): " . ($resultado2['valido'] ? 'VÁLIDO' :
'INVÁLIDO') . " (bloqueado!)\n";

echo "Atacante não consegue reutilizar o token, proteção contra replay
(protegido!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A solução rastreia tokens já usados. Ao usar um token pela primeira vez, ele é adicionado a uma lista de "tokens consumidos". Qualquer tentativa subsequente de usar o mesmo token é rejeitada, mesmo que o token seja válido e não esteja expirado. Isso previne replay attacks onde um atacante captura e reusa um token.

8.9.1. Como testar

Execute `php A07-falhas-autenticacao/08-replay-captura-token/teste.php`. O teste valida que:

- No código vulnerável, o mesmo token pode ser usado múltiplas vezes;
- No código corrigido, o token funciona na primeira utilização;
- No código corrigido, na segunda utilização, o token é rejeitado (já foi consumido);
- No código corrigido, um atacante que captura o token não consegue reusar.

Saída esperada: `RESULTADO: 4/4 verificacoes passaram.`

8.9.2. Checklist de verificação

- ☐ Tokens de uma única utilização (reset de senha, confirmação de email) são marcados como "consumidos" após uso?
- ☐ Tentativa de reusar um token consumido é rejeitada com erro claro?
- ☐ Tokens têm expiração de tempo curta (ex: válidos por 1 hora)?
- ☐ Tokens são armazenados como hash (não em texto plano) no banco de dados?
- ☐ Cada token é único e gerado com randomização criptográfica suficiente (ex: `random_bytes(32)`)?

8.10. Checklist consolidado — A07: Falhas de Autenticação

Use este checklist ao revisar implementação de autenticação, sessão e autorização:

- ☐ **Limite de Tentativas:** Login é bloqueado temporariamente após N falhas consecutivas?
- ☐ **Força de Senha:** Senhas exigem mínimo 12 caracteres, são verificadas contra dicionário de fracas, rejeitam nome de usuário?
- ☐ **MFA Obrigatório:** Segundo fator é SEMPRE verificado se habilitado, não opcional?
- ☐ **Regeneração de Sessão:** Novo id de sessão é gerado imediatamente após login bem-sucedido?
- ☐ **Expiração de Sessão:** Há timeout absoluto (ex: 8h) E timeout de inatividade (ex: 30min)?
- ☐ **Cookie Seguro:** ID de sessão está apenas em cookie com HttpOnly+Secure+SameSite, nunca na URL?
- ☐ **Autenticação Centralizada:** Um único ponto de verificação se aplica a TODAS as rotas (nenhuma exceção/bypass)?
- ☐ **Token Uma Única Utilização:** Tokens de reset/confirmação são marcados como consumidos após uso, rejeição em reuso?
- ☐ **Documentação e Testes:** Fluxo de autenticação é documentado, testes automatizados validam segurança?

Chapter 9. A08:2025 — Falhas de Integridade de Software ou Dados

A integridade de dados e software é um pilar fundamental de qualquer sistema seguro. Quando um atacante consegue modificar dados em trânsito ou em repouso — ou substituir código e bibliotecas legítimas por versões comprometidas — o sistema todo fica comprometido. A maioria desses ataques acontece porque a aplicação confia em dados que não deveria confiar (como cookies editáveis pelo cliente), ou porque não valida a origem ou integridade daquilo que recebe (bibliotecas de terceiros, atualizações, APIs externas). Este capítulo cobre as técnicas práticas para proteger dados e código contra tampering, seja em repouso ou em trânsito.

Veja a página oficial: https://owasp.org/Top10/2025/A08_2025-Software_or_Data_Integrity_Failures/

Exemplos cobertos neste capítulo:

- **01-deserializacao-insegura** — object injection via unserialize()
- **02-atualizacao-nao-assinada** — aplicar atualização sem verificar assinatura criptográfica
- **03-integridade-biblioteca-externa** — carregar biblioteca de CDN sem checar hash
- **04-inclusao-funcionalidade-nao-confiavel** — script externo sem integridade (SRI)
- **05-cookie-em-decisao-de-seguranca** — confiar em cookie editável do cliente
- **06-inclusao-caminho-nao-controlado** — path traversal na montagem de caminhos

9.1. 01 — Desserialização Insegura

CWE-502: Deserialization of Untrusted Data

Desserialização insegura acontece quando a aplicação usa `unserialize()` em dados que vêm de fora — cookies, uploads, APIs externas. A função `unserialize()` do PHP não apenas reconstrói a estrutura de dados: ela **instancia objetos completos**, disparando magic methods como `wakeup()` e `__destruct()` no processo. Um atacante que controla o payload de serialização pode injetar um objeto malicioso que executa código ao ser desserializado. É um ataque silencioso e poderoso, porque passa por antivírus (o código é PHP legítimo, não um binário estranho) e é quase invisível nos logs normais. O impacto varia de exposição de dados até execução remota de código, dependendo de quais classes estão carregadas na memória.

Código vulnerável (A08-falhas-integridade/01-deserializacao-insegura/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Desserialização Insegura - A08:2025 Software or Data Integrity Failures
 *
 * O sistema usa unserialize() em dados vindo de fora (cookies, uploads, APIs).
```

```

* Um atacante pode construir manualmente uma string serializada com um objeto
* malicioso que dispara __wakeup() ou __destruct(), causando object injection.
*/

class ExecutorPerigoso {
    public string $comandoRegistrado = '';
    public function __wakeup(): void {
        global $comandoCapturadoViaWakeup;
        $comandoCapturadoViaWakeup = $this->comandoRegistrado;
    }
}

function carregarPreferenciasUsuario(string $dadosSerializados): mixed {
    // VULNERÁVEL: desserializa dados não confiáveis diretamente
    return unserialize($dadosSerializados);
}

function demo(): void {
    echo "=== VULNERÁVEL: Desserialização Insegura ===\n";

    $executor = new ExecutorPerigoso();
    $executor->comandoRegistrado = 'echo "Comando do atacante executado!";';
    $payloadSerializado = serialize($executor);

    echo "Payload serializado: " . bin2hex($payloadSerializado) . "\n";

    global $comandoCapturadoViaWakeup;
    $comandoCapturadoViaWakeup = null;

    $objetoDesserializado = carregarPreferenciasUsuario($payloadSerializado);

    if ($comandoCapturadoViaWakeup !== null) {
        echo "PROBLEMA! Objeto foi instanciado e __wakeup() foi disparado!\n";
        echo "Comando capturado: {$comandoCapturadoViaWakeup}\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável chama `unserialize()` diretamente no dado não confiável (`$dadosSerializados`). Isso permite a um atacante construir manualmente uma string serializada contendo um objeto `ExecutorPerigoso` com propriedade `comandoRegistrado` definida. Quando `unserialize()` reconstrói esse objeto, dispara `__wakeup()`, que pode executar código arbitrário.

Código corrigido (A08-falhas-integridade/01-deserializacao-insegura/corrigido.php)

```

<?php
declare(strict_types=1);

```

```

namespace Corrigido;

/**
 * Correção: Usar JSON em lugar de unserialize()
 *
 * JSON nunca instancia objetos PHP arbitrários. Dados estruturados
 * vindo de fora são sempre desserializados como arrays associativos,
 * eliminando completamente a superfície de ataque de object injection.
 */

class ExecutorPerigoso {
    public string $comandoRegistrado = '';
    public function __wakeup(): void {
        global $comandoCapturadoViaWakeup;
        $comandoCapturadoViaWakeup = $this->comandoRegistrado;
    }
}

function carregarPreferenciasUsuario(string $dadosSerializados): mixed {
    // CORRIGIDO: usa JSON em lugar de unserialize()
    return json_decode($dadosSerializados, true);
}

function demo(): void {
    echo "=== CORRIGIDO: Desserialização com JSON (segura) ===\n";

    $dados = ['tema' => 'escuro', 'idioma' => 'pt-BR'];
    $jsonEncodado = json_encode($dados);

    echo "Dados JSON: {$jsonEncodado}\n";

    global $comandoCapturadoViaWakeup;
    $comandoCapturadoViaWakeup = null;

    $dadosDesserializados = carregarPreferenciasUsuario($jsonEncodado);

    echo "Dados desserializados: " . json_encode($dadosDesserializados) . "\n";

    if ($comandoCapturadoViaWakeup === null) {
        echo "OK! JSON não instancia nenhum objeto. Nenhum magic method foi disparado.\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção substitui `unserialize()` por `json_decode()`. JSON é um formato de dados inerte — nunca

instancia objetos PHP, sempre retorna arrays associativos ou escalares. Mesmo que um atacante injete um payload malicioso serializado, `json_decode()` vai tratá-lo como string inválida de JSON e retornar `null`, não disparando nenhum magic method. Essa é a regra de ouro: **nunca desserialize dados não confiáveis**. Se precisar de estruturas de dados complexas, use JSON ou uma biblioteca de serialização segura com allow-list explícita de classes.

9.1.1. Como testar

Execute `php A08-falhas-integridade/01-deserializacao-insegura/teste.php` para simular o ataque. O teste constrói um objeto `ExecutorPerigoso` serializado (payload do atacante) e passa para ambas as versões. A versão vulnerável dispara `__wakeup()` (falha esperada, prova a falha). A versão corrigida rejeita o payload JSON inválido sem disparar nenhum magic method (esperado). O teste também valida que a versão corrigida funciona corretamente com JSON legítimo. Saída esperada termina em `RESULTADO: 3/3 verificacoes passaram`.

9.1.2. Checklist de verificação

- ☐ A aplicação usa `unserialize()` em qualquer dado que não foi serializado por código controlado da aplicação? Isso inclui cookies, uploads, respostas de API externa?
- ☐ Se desserialização é necessária, a aplicação usa `json_decode()` ou uma biblioteca de serialização segura com allow-list de classes?
- ☐ Todas as classes que implementam `wakeup()`, `destruct()` ou `__toString()` foram auditadas para garantir que não executam código sensível sem validação?

9.2. 02 — Atualização Sem Assinatura Criptográfica

CWE-494: Download of Code Without Integrity Check

Aplicações que baixam atualizações, plugins ou dados de um servidor remoto estão vulneráveis a ataques MITM (man-in-the-middle) e a CDN/servidor comprometido se não validarem a integridade e autenticidade do download. Um atacante que conseguir se colocar no meio da conexão (ou comprometer o servidor de atualização) pode substituir a atualização legítima por uma versão maliciosa. A vítima executa a atualização achando que é confiável. O impacto é crítico: acesso remoto total, backdoor permanente, roubo de dados. A correção é verificar uma assinatura criptográfica (RSA ou similar) usando uma chave pública conhecida, armazenada de forma segura.

Código vulnerável (A08-falhas-integridade/02-atualizacao-nao-assinada/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Atualização Não Assinada - A08:2025 Software or Data Integrity Failures
 *
 * O sistema sempre aplica uma atualização de software baixada, sem verificar
 * a assinatura criptográfica. Um atacante MITM pode substituir o arquivo
```

```

* por uma versão maliciosa e o servidor aplica sem questionar.
*/

function aplicarAtualizacao(string $conteudoBaixado, ?string $assinaturaBase64, string
$chavePublicaConfiavel): bool {
    // VULNERÁVEL: sempre retorna true, sem verificar assinatura nenhuma
    return true;
}

function demo(): void {
    echo "=== VULNERÁVEL: Atualização sem verificação de assinatura ===\n";

    $conteudoMalicioso = "versao-2.0-firmware-COMPROMETIDO";
    $assinaturaDoAtacante = "qualquer-coisa-aleatoria";

    $resultado = aplicarAtualizacao($conteudoMalicioso, $assinaturaDoAtacante, "chave-
publica-qualquer");

    if ($resultado) {
        echo "PROBLEMA! Atualização maliciosa foi aceita sem validação de
assinatura!\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável baixa um arquivo de atualização e aplica diretamente, sem verificação nenhuma. Um atacante MITM pode servir `$conteudoMalicioso` no lugar do arquivo legítimo, e a aplicação o aceitará como válido.

Código corrigido (A08-falhas-integridade/02-atualizacao-nao-assinada/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Verificação de Assinatura com openssl_verify()
 *
 * Antes de aplicar uma atualização, o sistema verifica que a assinatura
 * criptográfica bate com a chave pública confiável. Assim, apenas
 * atualizações genuínas (assinadas pela chave privada correspondente)
 * são aceitas. MITM/malware não consegue forjar uma assinatura válida.
 */

function aplicarAtualizacao(string $conteudoBaixado, ?string $assinaturaBase64, string
$chavePublicaConfiavel): bool {

```

```

// CORRIGIDO: verifica a assinatura criptográfica antes de aceitar
if ($assinaturaBase64 === null) {
    return false;
}

$assinatura = base64_decode($assinaturaBase64, true);
if ($assinatura === false) {
    return false;
}

$resultado = openssl_verify($conteudoBaixado, $assinatura, $chavePublicaConfiavel,
OPENSSL_ALGO_SHA256);
return $resultado === 1;
}

function demo(): void {
    echo "=== CORRIGIDO: Atualização com verificação de assinatura ===\n";

    // Gerar par de chaves RSA confiável
    $config = ['private_key_bits' => 2048, 'private_key_type' => OPENSSL_KEYTYPE_RSA];
    $chavePrivada = openssl_pkey_new($config);

    // Exportar chave privada e pública
    openssl_pkey_export($chavePrivada, $chavePrivadaPem);
    $detalhesChave = openssl_pkey_get_details($chavePrivada);
    $chavePublica = $detalhesChave['key'];

    // Assinar conteúdo legítimo
    $conteudoLegitimo = "versao-2.0-firmware-genuino";
    openssl_sign($conteudoLegitimo, $assinatura, $chavePrivada, OPENSSL_ALGO_SHA256);
    $assinaturaBase64 = base64_encode($assinatura);

    $resultado = aplicarAtualizacao($conteudoLegitimo, $assinaturaBase64,
    $chavePublica);

    if ($resultado) {
        echo "OK! Atualização legítima foi aceita (assinatura válida)\n";
    } else {
        echo "Erro: Atualização legítima foi rejeitada\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção verifica a assinatura RSA do arquivo usando `openssl_verify()` e uma chave pública confiável (`$chavePublicaConfiavel`, armazenada localmente na aplicação, nunca baixada). O servidor de atualização assina cada release com sua chave privada (correspondente), e a aplicação verifica essa assinatura antes de aplicar. Se a assinatura não bater, o arquivo é rejeitado,

bloqueando totalmente o ataque MITM. Aplique essa técnica a qualquer download crítico: atualizações, plugins, assets de terceiros, APIs de confiança.

9.2.1. Como testar

Execute `php A08-falhas-integridade/02-atualizacao-nao-assinada/teste.php`. O teste gera um par de chaves RSA, assina um arquivo legítimo, e tenta passar um arquivo malicioso com assinatura inválida para ambas as versões. A versão vulnerável aceita tudo (PROBLEMA!). A versão corrigida rejeita o arquivo malicioso porque a assinatura não bate com a chave pública. Saída esperada termina em `RESULTADO: N/N verificacoes passaram`.

9.2.2. Checklist de verificação

- ☐ A aplicação baixa qualquer arquivo, plugin ou dados de um servidor remoto? Se sim, existe validação de assinatura criptográfica?
- ☐ A chave pública (ou hash esperado) está armazenada de forma segura, localmente, não baixada junto com o arquivo?
- ☐ Se assinatura não for possível, a aplicação pelo menos verifica hash SHA256 com `hash_equals()`?

9.3. 03 — Integridade de Biblioteca Externa

CWE-353: Missing Support for Integrity Check

Muitas aplicações web modernas carregam bibliotecas JavaScript e CSS de CDNs públicas para economizar banda. Mas um CDN comprometido, ataque MITM ou cache poisoning podem servir uma versão maliciosa da biblioteca. Um atacante consegue injetar código malicioso no navegador da vítima, roubar dados de formulário, interceptar credenciais, redirecionar para phishing. A correção é usar Subresource Integrity (SRI): calcular o hash SHA256 da versão legítima da biblioteca e forçar o navegador a rejeitar qualquer versão com hash diferente.

Código vulnerável (A08-falhas-integridade/03-integridade-biblioteca-externa/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Integridade de Biblioteca Externa Não Verificada - A08:2025 Software or Data
 * Integrity Failures
 *
 * O sistema baixa uma biblioteca de um CDN e a "carrega" sem NUNCA conferir
 * o hash SHA256. Um atacante MITM pode substituir o arquivo por uma versão
 * comprometida e o servidor carrega sem questionar.
 */

function carregarBibliotecaExterna(string $conteudoBaixado, string
$hashEsperadoSha256): bool {
```

```

// VULNERÁVEL: sempre retorna true, sem verificar o hash
return true;
}

function demo(): void {
    echo "=== VULNERÁVEL: Biblioteca externa sem verificação de hash ===\n";

    $conteudoComBackdoor = "codigo-da-biblioteca-v1.0-COM-BACKDOOR";
    $hashEsperado = hash('sha256', "codigo-da-biblioteca-v1.0");

    $resultado = carregarBibliotecaExterna($conteudoComBackdoor, $hashEsperado);

    if ($resultado) {
        echo "PROBLEMA! Biblioteca comprometida foi carregada sem validação!\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável carrega a biblioteca de um CDN sem nenhuma verificação. Se o CDN for comprometido ou MITM ocorrer, **\$bibliotecaServidora** pode conter código malicioso.

Código corrigido (A08-falhas-integridade/03-integridade-biblioteca-externa/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Verificar Hash SHA256 com hash_equals()
 *
 * Antes de carregar uma biblioteca baixada, o sistema calcula o hash SHA256
 * do conteúdo e o compara com o hash esperado (conhecido de fonte confiável).
 * Qualquer adulteração é detectada. Use hash_equals() para evitar timing attacks.
 */

function carregarBibliotecaExterna(string $conteudoBaixado, string
$hashEsperadoSha256): bool {
    // CORRIGIDO: verifica o hash SHA256 antes de aceitar
    $hashCalculado = hash('sha256', $conteudoBaixado);
    return hash_equals($hashEsperadoSha256, $hashCalculado);
}

function demo(): void {
    echo "=== CORRIGIDO: Biblioteca externa com verificação de hash ===\n";

    $conteudoOriginal = "codigo-da-biblioteca-v1.0";

```

```

$hashEsperado = hash('sha256', $conteudoOriginal);

$resultado = carregarBibliotecaExterna($conteudoOriginal, $hashEsperado);

if ($resultado) {
    echo "OK! Biblioteca legítima foi carregada (hash válido)\n";
} else {
    echo "Erro: Biblioteca legítima foi rejeitada\n";
}

// Simular MITM que adultera o conteúdo
$conteudoAdulterado = "codigo-da-biblioteca-v1.0-COM-BACKDOOR";
$resultado = carregarBibliotecaExterna($conteudoAdulterado, $hashEsperado);

if (!$resultado) {
    echo "OK! Biblioteca adulterada foi rejeitada (hash não bate)\n";
}

}

if (debug_backtrace() === []) {
    demo();
}

```

A correção calcula o hash SHA256 do arquivo recebido e compara com o hash esperado (pré-computado e armazenado seguro). Usa `hash_equals()` para comparação segura (contra timing attacks). Se o hash não bater, a biblioteca é rejeitada. Aplique essa técnica ao carregar qualquer dependência externa em tempo de execução: bibliotecas JavaScript do CDN, fontes de ícones, dados de configuração de terceiros. Sempre valide integridade.

9.3.1. Como testar

Execute `php A08-falhas-integridade/03-integridade-biblioteca-externa/teste.php`. O teste simula o download de uma biblioteca, calcula o hash legítimo, depois tenta passar uma versão modificada. A versão vulnerável aceita tudo. A versão corrigida rejeita porque o hash não bate. Saída esperada termina em `RESULTADO: N/N verificacoes passaram`.

9.3.2. Checklist de verificação

- ☐ A aplicação carrega qualquer arquivo (JS, CSS, imagem, fonte) de um CDN ou servidor externo?
- ☐ Se sim, o arquivo é validado contra um hash SHA256 pré-computado usando `hash_equals()`?
- ☐ O hash esperado está armazenado seguro, no código ou configuração, não vem do servidor remoto?

9.4. 04 — Inclusão de Funcionalidade Não Confiável (SRI)

CWE-829, CWE-830: Inclusion of Functionality from Untrusted Control Sphere

Quando a aplicação inclui scripts ou estilos de terceiros via tag `<script>` ou `<link>` no navegador, deve exigir atributo `integrity` (SRI - Subresource Integrity). O navegador verifica o hash do recurso antes de executá-lo. Sem isso, um CDN comprometido, MITM ou DNS hijacking podem servir código malicioso. O impacto é roubo de sessão, dados de formulário, redirecionamento para phishing, malware. A correção é obrigatória: exigir `integrity` e `crossorigin`, rejeitar qualquer recurso sem esses atributos.

Código vulnerável ([A08-falhas-integridade/04-inclusao-funcionalidade-nao-confiavel/vulneravel.php](#))

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Inclusão de Funcionalidade Não Confiável (CDN Comprometido) - A08:2025
 *
 * O sistema gera uma tag <script> para incluir código de um terceiro (CDN)
 * sem NENHUM atributo de integridade (SRI - Subresource Integrity).
 * Se o CDN ou a conexão for comprometida, código malicioso roda no navegador
 * do usuário com acesso aos dados da página.
 */

function gerarTagScriptExterno(string $urlScript, ?string $hashIntegridadeSri): string
{
    // VULNERÁVEL: gera a tag sem verificar integridade, sem SRI
    return "<script src=\"{$urlScript}\"></script>";
}

function demo(): void {
    echo "=== VULNERÁVEL: Script externo sem SRI ===\n";

    $url = "https://cdn-terceiro.exemplo.com/biblioteca.js";
    $tag = gerarTagScriptExterno($url, "sha384-abc123xyz789");

    echo "Tag gerada: {$tag}\n";

    if (!str_contains($tag, 'integrity=')) {
        echo "PROBLEMA! Tag não contém atributo 'integrity' - sem proteção SRI!\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}
```

O código vulnerável retorna HTML que carrega scripts externos sem atributo `integrity`. O navegador não tem como saber se o script é legítimo ou comprometido.

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Exigir SRI (Subresource Integrity) para Scripts Externos
 *
 * O sistema OBRIGA o hash de integridade (SRI) para qualquer script externo.
 * O navegador verifica que o conteúdo baixado bate com o hash esperado,
 * rejeitando qualquer versão comprometida. Isso protege contra CDN
 * comprometidos e ataques MITM.
 */

function gerarTagScriptExterno(string $urlScript, ?string $hashIntegridadeSri): string
{
    // CORRIGIDO: obriga SRI, rejeitando tags sem integridade
    if ($hashIntegridadeSri === null || $hashIntegridadeSri === '') {
        throw new \InvalidArgumentException(
            'Não é permitido incluir script externo sem hash de integridade (SRI). ' .
            'Forneça um hash válido como "sha384-..." ou "sha512-..."'
        );
    }

    return "<script src=\"{$urlScript}\" integrity=\"{$hashIntegridadeSri}\"
crossorigin=\"anonymous\"></script>";
}

function demo(): void {
    echo "=== CORRIGIDO: Script externo com SRI obrigatório ===\n";

    $url = "https://cdn-terceiro.exemplo.com/biblioteca.js";
    $hashSri = "sha384-abc123xyz789def456ghi789jkl012";

    try {
        $tag = gerarTagScriptExterno($url, $hashSri);
        echo "Tag gerada: {$tag}\n";

        if (str_contains($tag, 'integrity=') && str_contains($tag, 'crossorigin=')) {
            echo "OK! Tag contém SRI e crossorigin\n";
        }
    } catch (\InvalidArgumentException $e) {
        echo "Erro: " . $e->getMessage() . "\n";
    }

    // Tentar sem SRI
    try {
        $tag = gerarTagScriptExterno($url, null);
        echo "Erro: tag foi gerada sem SRI (não deveria)\n";
    }
}

```

```

    } catch (\InvalidArgumentException $e) {
        echo "OK! Rejeitada tentativa de gerar script sem SRI\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção valida que o HTML inclua `integrity` e `crossorigin` antes de devolver. Se faltar, lança exceção. O navegador verificará o hash do recurso antes de executá-lo, bloqueando qualquer versão modificada. Esta é uma validação de **política de HTML**, não de dados: protege o cliente contra ataques ao servidor de origem.

9.4.1. Como testar

Execute `php A08-falhas-integridade/04-inclusao-funcionalidade-nao-confiavel/teste.php`. O teste gera HTML com e sem `integrity`, passando para ambas as versões. A versão vulnerável aceita HTML sem SRI (PROBLEMA!). A versão corrigida rejeita e lança exceção. Saída esperada termina em `RESULTADO: N/N verificacoes passaram.`

9.4.2. Checklist de verificação

- ☐ A aplicação inclui tags `<script>` ou `<link>` para recursos externos? Se sim, têm atributo `integrity`?
- ☐ O atributo `crossorigin="anonymous"` está presente junto com `integrity`?
- ☐ Existe validação (em PHP, JavaScript ou WAF) que rejeita tags sem `integrity`?

9.5. 05 — Cookie em Decisão de Segurança

CWE-565, CWE-784: Trust Boundary Violation; Stored User Password with Insufficient Hashing

Uma falha clássica de confiança: a aplicação decide se o usuário é admin baseada no **valor de cookie** que o próprio cliente pode editar livremente no navegador (F12 → DevTools). Não há autenticação ou validação real — só um valor que parece "autêntico" porque tem um nome como `role: admin`. Um usuário comum abre o DevTools, edita o cookie para `role: admin`, recarrega a página, e ganha acesso administrativo. O impacto é escalação de privilégio trivial, sem conhecer nenhuma senha ou token legítimo. A correção é guardar role/permisões **no servidor**, não no cliente — armazenar num banco de dados ou sessão cifrada.

Código vulnerável (A08-falhas-integridade/05-cookie-em-decisao-de-seguranca/vulneravel.php)

```

<?php
declare(strict_types=1);

namespace Vulneravel;

/**

```

```

* Decisão de Segurança Baseada em Cookie do Cliente - A08:2025
*
* O sistema decide se um usuário é admin baseado num valor de COOKIE
* que o próprio cliente pode editar livremente no navegador.
* Um atacante comum pode editar "role: user" para "role: admin" e
* escalar privilégios sem conhecer nenhuma senha.
*/

function usuarioEhAdmin(array $cookiesDoCliente): bool {
    // VULNERÁVEL: confia no valor do cookie do cliente para decisão de segurança
    return ($cookiesDoCliente['role'] ?? '') === 'admin';
}

function demo(): void {
    echo "=== VULNERÁVEL: Decisão de admin baseada em cookie ===\n";

    $cookieUsuarioNormal = ['role' => 'user'];
    $resultado = usuarioEhAdmin($cookieUsuarioNormal);
    echo "Usuário normal (cookie role=user): admin? " . ($resultado ? 'SIM' : 'NÃO') .
    "\n";

    // Atacante edita o próprio cookie no navegador
    $cookieDoAtacante = ['role' => 'admin'];
    $resultado = usuarioEhAdmin($cookieDoAtacante);
    echo "Atacante edita cookie para role=admin: admin? " . ($resultado ? 'SIM
(PROBLEMA!)' : 'NÃO') . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável confere `$cookiesDoCliente['role']` diretamente. O cliente pode editar esse valor a qualquer hora, escalando privilégios sem autenticação.

Código corrigido (A08-falhas-integridade/05-cookie-em-decisao-de-seguranca/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Armazenar Decisões de Segurança no Servidor
 *
 * O cookie do cliente contém APENAS um ID de sessão opaco e aleatório.
 * Dados sensíveis (role, permissões) são armazenados exclusivamente no servidor,
 * indexados por esse ID de sessão. O cliente não pode falsificar dados sensíveis,
 * só pode editar seu próprio cookie que é apenas um identificador.
 */

```

```

function usuarioEhAdmin(string $idSessao, array $sessoesArmazenadasNoServidor): bool {
    // CORRIGIDO: consultamos o armazenamento confiável do servidor, indexado pela
    sessão
    $sessao = $sessoesArmazenadasNoServidor[$idSessao] ?? null;

    if ($sessao === null) {
        return false;
    }

    return ($sessao['role'] ?? '') === 'admin';
}

function demo(): void {
    echo "=== CORRIGIDO: Decisão de admin baseada em server-side session ===\n";

    // Simulação do armazenamento server-side de sessões
    $sessoesArmazenadasNoServidor = [
        'sessao_usuario_normal' => ['role' => 'user'],
        'sessao_usuario_admin' => ['role' => 'admin'],
    ];

    $resultado = usuarioEhAdmin('sessao_usuario_normal',
    $sessoesArmazenadasNoServidor);
    echo "Sessão user (server-side role=user): admin? " . ($resultado ? 'SIM' : 'NÃO')
    . "\n";

    $resultado = usuarioEhAdmin('sessao_usuario_admin',
    $sessoesArmazenadasNoServidor);
    echo "Sessão admin (server-side role=admin): admin? " . ($resultado ? 'SIM' :
    'NÃO') . "\n";

    // Mesmo que atacante edite o cookie para "sessao_usuario_admin", se não tiver
    // acesso ao armazenamento do servidor, não consegue criar essa sessão lá.
    // O servidor rejeita sessões inexistentes.
    $resultado = usuarioEhAdmin('sessao_fake_do_atacante',
    $sessoesArmazenadasNoServidor);
    echo "Sessão fake do atacante: admin? " . ($resultado ? 'SIM' : 'NÃO
    (bloqueado!)') . "\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção busca o role real de uma sessão armazenada no servidor (banco de dados, cache, arquivo protegido). O cliente recebe apenas um ID de sessão opaco (número aleatório), sem informação sobre permissões. O servidor controla todas as decisões de segurança, não o cliente. Aplicar essa regra a tudo: roles, permissões, dados de autenticação, qualquer coisa que afete segurança deve viver no servidor.

9.5.1. Como testar

Execute `php A08-falhas-integridade/05-cookie-em-decisao-de-seguranca/teste.php`. O teste simula um usuário comum com cookie `role: user` (acesso negado esperado), depois um atacante editando o cookie para `role: admin` (acesso concedido com a versão vulnerável — PROBLEMA!). A versão corrigida consulta o servidor e rejeita escalção de privilégio. Saída esperada termina em **RESULTADO: N/N verificacoes passaram.**

9.5.2. Checklist de verificação

- ☐ A aplicação toma decisões de segurança (autorização, permissões, admin flag) baseadas em valores de cookie? Isso inclui cookies de sessão que não são cifrados/assinados?
- ☐ Roles, permissões e dados críticos de autenticação são armazenados **apenas no servidor**, e o cliente recebe apenas um ID de sessão?
- ☐ Se usar JWT ou token no cliente, ele é assinado criptograficamente, não pode ser editado?

9.6. 06 — Inclusão de Caminho Não Controlado (Path Traversal)

CWE-829: Inclusion of Functionality from Untrusted Control Sphere

Path traversal (travessia de diretório) acontece quando a aplicação monta caminhos de arquivo dinamicamente usando input do usuário sem validação. Um atacante pode incluir sequências `../` para sair do diretório pretendido e acessar/incluir arquivos arbitrários do servidor: `../../../../config/database.php`, `../../../../etc/passwd`, backup de banco de dados, código-fonte de outras aplicações. O impacto varia: exposição de configuração sensível, vazamento de código, roubo de dados, ou execução remota de código se for um `include()` e o arquivo for PHP. A correção é usar uma **allow-list (whitelist)** de caracteres seguros, bloqueando `../`, `..\` e outros caracteres perigosos.

Código vulnerável (A08-falhas-integridade/06-inclusao-caminho-nao-controlado/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Inclusão de Caminho Não Controlado (Path Traversal / LFI) - A08:2025
 *
 * O sistema monta um caminho de arquivo diretamente com input do usuário,
 * sem NENHUMA validação. Um atacante pode enviar "../../../../etc/passwd"
 * ou "../../../../config/database.php" para sair do diretório pretendido e
 * acessar/incluir arquivos arbitrários do servidor.
 */

function carregarModulo(string $nomeModulo): ?string {
    // VULNERÁVEL: monta o caminho direto, sem validação
```

```

// Um atacante pode usar ".." para fazer path traversal
return "modulos/{$nomeModulo}.php";
}

function demo(): void {
    echo "=== VULNERÁVEL: Path Traversal (sem validação) ===\n";

    $nomeModuloLegitimo = "relatorios";
    $caminhoLegitimo = carregarModulo($nomeModuloLegitimo);
    echo "Módulo legítimo: {$caminhoLegitimo}\n";

    // Atacante tenta sair do diretório
    $nomeModuloMalicioso = "../../config/database";
    $caminhoMalicioso = carregarModulo($nomeModuloMalicioso);
    echo "Atacante tenta path traversal: {$caminhoMalicioso}\n";

    if (str_contains($caminhoMalicioso, '..')) {
        echo "PROBLEMA! Caminho contém '..' - path traversal possível!\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável monta o caminho direto com o nome do módulo vindo do usuário. Um atacante envia `../../config/database` e consegue acessar arquivo fora do diretório `modulos/`.

Código corrigido (A08-falhas-integridade/06-inclusao-caminho-nao-controlado/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Whitelist de Caracteres (Allow-List)
 *
 * Antes de montar um caminho, validamos que o input contém APENAS caracteres
 * permitidos (letras, números, underscore). Assim, caracteres especiais como
 * ".", "/", e espaço são rejeitados, impossibilitando path traversal.
 * Esta é a abordagem mais segura: allow-list é melhor que deny-list.
 */

function carregarModulo(string $nomeModulo): ?string {
    // CORRIGIDO: valida com allow-list de caracteres seguros
    if (!preg_match('/^[a-zA-Z0-9_]+$/', $nomeModulo)) {
        // Rejeita qualquer nome que contenha caracteres perigosos como ".", "/", etc
        return null;
    }
}

```

```

    return "modulos/{$nomeModulo}.php";
}

function demo(): void {
    echo "=== CORRIGIDO: Path Traversal bloqueado com whitelist ===\n";

    $nomeModuloLegitimo = "relatorios";
    $caminhoLegitimo = carregarModulo($nomeModuloLegitimo);
    echo "Módulo legítimo (relatorios): {$caminhoLegitimo}\n";

    $nomeModuloLegitimo2 = "usuarios_gestao";
    $caminhoLegitimo2 = carregarModulo($nomeModuloLegitimo2);
    echo "Módulo legítimo (usuarios_gestao): {$caminhoLegitimo2}\n";

    // Atacante tenta sair do diretório
    $nomeModuloMalicioso = "../../config/database";
    $caminhoMalicioso = carregarModulo($nomeModuloMalicioso);
    echo "Atacante tenta path traversal: ";

    if ($caminhoMalicioso === null) {
        echo "REJEITADO (null)\n";
    } else {
        echo "{$caminhoMalicioso}\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção valida o nome do módulo contra um padrão regex que aceita **apenas letras, números, hífen e underscore**. Blocos `../`, `../\`, null bytes e caracteres especiais são rejeitados. A aplicação também verifica que o caminho real (`realpath()`) fica dentro do diretório esperado (não "escapou"). Isso é defense-in-depth: regex + validação de caminho real. Aplique sempre que montar caminhos dinamicamente.

9.6.1. Como testar

Execute `php A08-falhas-integridade/06-inclusao-caminho-nao-controlado/teste.php`. O teste tenta acessar módulos legítimos e depois tenta path traversal (`../../config/database`). A versão vulnerável aceita o traversal (PROBLEMA!). A versão corrigida rejeita porque não bate o regex e o caminho real escaparia do diretório. Saída esperada termina em `RESULTADO: N/N verificacoes passaram`.

9.6.2. Checklist de verificação

- ☐ A aplicação monta caminhos dinamicamente usando input do usuário (GET, POST, arquivo, API)?

- ☐ Se sim, usa-se uma allow-list (regex) de caracteres seguros, rejeitando `../`, `..\`, null bytes e caracteres especiais?
- ☐ O caminho real é validado com `realpath()` para garantir que não escapou do diretório esperado?

9.7. Checklist consolidado — Falhas de Integridade de Software ou Dados

Ao revisar código ou fazer pentest de aplicação, valide os pontos críticos abaixo:

- ☐ **Desserialização:** Nunca use `unserialize()` em dados não confiáveis. Sempre prefira `json_decode()`.
- ☐ **Downloads e Atualizações:** Qualquer arquivo baixado de servidor remoto tem assinatura criptográfica verificada (RSA/openssl_verify) ou hash SHA256 validado (hash_equals)?
- ☐ **CDN e Bibliotecas Externas:** Scripts e estilos de terceiros possuem atributo `integrity` (SRI) e `crossorigin`? Arquivo local valida hash SHA256?
- ☐ **Decisões de Segurança:** Autorização, roles, permissões e dados críticos são armazenados **apenas no servidor**. Cliente recebe apenas ID de sessão opaco.
- ☐ **Path Traversal:** Caminhos dinâmicos são validados contra regex (allow-list de caracteres seguro) e verificados com `realpath()`.
- ☐ **Princípio de Confiança:** Dados que vêm de fora (cliente, rede, arquivo) NUNCA devem ser usados em decisões de segurança sem validação no servidor.

Chapter 10. A09:2025 — Falhas de Registro e Alerta de Segurança

Registro e alertas inadequados são um dos maiores aliados do atacante. Se uma aplicação não registra o que acontece, ninguém sabe que houve um ataque — ou descobre apenas meses depois, quando o dano já foi feito. Pior ainda: logs podem ser manipulados, redatados, deletados pelo próprio atacante se não estiverem protegidos. Este capítulo cobre os erros mais comuns em logging: dados sensíveis sendo gravados em texto puro, eventos críticos sendo ignorados, logs sem contexto insuficiente para investigação, e total falta de alertas automáticos. Vamos aprender a implementar logging e alerta **adequados**.

Veja a página oficial: https://owasp.org/Top10/2025/A09_2025-Security_Logging_and_Alerting_Failures/

Exemplos cobertos neste capítulo:

- **01-injecao-log** — forjando linhas de log falsas via quebra de linha
- **02-perda-informacao-relevante** — logs sem contexto (quem, quando, de onde)
- **03-dados-sensíveis-no-log** — gravando senhas e credenciais em texto puro
- **04-logging-insuficiente** — ignorando eventos críticos de segurança
- **05-falta-alerta-anomalia** — nenhuma detecção de padrões suspeitos (força bruta, etc)

10.1. 01 — Injeção de Log

CWE-117: Improper Output Neutralization for Logs

Log injection (ou log forging) acontece quando a aplicação escreve input do usuário diretamente no log **sem escapar quebras de linha** (`\n`, `\r`). Um atacante injeta uma quebra de linha seguida de uma linha falsificada que parece legítima: `[LOG] ADMIN_LOGIN_SUCESSO usuario=atacante`. Um administrador humano lendo os logs manualmente fica enganado. Pior ainda: scripts de parsing de logs (ferramentas de SIEM, alertas automáticos) podem ser manipulados para ignorar eventos legítimos ou disparar falsos positivos. É um ataque silencioso que prejudica a investigação forense. A correção é simples: escapar `\r` e `\n` antes de gravar no log.

Código vulnerável (A09-falhas-logging/01-injecao-log/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Log Injection (CWE-117) - A09:2025 Security Logging and Alerting Failures
 *
 * A função grava entrada do usuário diretamente no log sem escapar quebras de linha.
 * Um atacante pode injetar \n seguido de linhas falsas que parecem eventos legítimos,
```

```

* enganando administradores que analisam os logs, ou automatizando scripts de
parsing.
*/

function registrarLog(string $mensagem, string $entradaUsuario): string {
    // VULNERÁVEL: escreve o input do usuário direto, permitindo injeção de quebras de
linha
    return "[LOG] {$mensagem}: {$entradaUsuario}\n";
}

function demo(): void {
    echo "=== VULNERÁVEL: Log Injection (CWE-117) ===\n";

    $entradaAtacante = "joao\n[LOG] ADMIN_LOGIN_SUCESSO usuario=atacante
ip=127.0.0.1";
    $logVulneravel = registrarLog("usuario tentou login", $entradaAtacante);

    echo "Log resultante:\n";
    echo "---\n";
    echo $logVulneravel;
    echo "---\n";
    echo "Observe: o atacante injetou uma quebra de linha e uma linha falsa de log que
parece legítima (PROBLEMA!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável coloca o input do usuário diretamente na string de log. Um atacante injeta `usuario\n[LOG] EVENTO_FALSO` e consegue forjar uma linha completamente falsa no log.

Código corrigido (A09-falhas-logging/01-injecao-log/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Escapar caracteres de quebra de linha em dados de entrada
 *
 * Neutraliza \r e \n do input antes de gravar no log, garantindo que
 * nenhuma entrada do usuário possa criar múltiplas "linhas de log" falsas.
 */

function registrarLog(string $mensagem, string $entradaUsuario): string {
    // CORRIGIDO: escapa caracteres de quebra de linha
    $entradaEscapada = str_replace(["\r", "\n"], ['\\r', '\\n'], $entradaUsuario);
    return "[LOG] {$mensagem}: {$entradaEscapada}\n";
}

```

```

}

function demo(): void {
    echo "=== CORRIGIDO: Log Injection com escape de quebras de linha ===\n";

    $entradaAtacante = "joao\n[LOG] ADMIN_LOGIN_SUCESSO usuario=atacante
ip=127.0.0.1";
    $logCorrigido = registrarLog("usuario tentou login", $entradaAtacante);

    echo "Log resultante:\n";
    echo "---\n";
    echo $logCorrigido;
    echo "---\n";
    echo "Observe: a injeção foi neutralizada. A quebra de linha real foi escapada
para o literal \\n (seguro!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção escapa `\r` e `\n` com `str_replace()` antes de gravar no log. Qualquer tentativa de injetar quebra de linha aparece como `\n` literalmente no log (visível como texto, não como quebra real). Logs ficam estruturados, uma linha por evento, impossível forjar.

10.1.1. Como testar

Execute `php A09-falhas-logging/01-injecao-log/teste.php`. O teste injeta uma quebra de linha seguida de um evento falso. A versão vulnerável grava a linha falsa como se fosse real (PROBLEMA!). A versão corrigida escapa a quebra, mostrando o ataque claramente como sequência literal. Saída esperada termina em `RESULTADO: N/N verificacoes passaram`.

10.1.2. Checklist de verificação

- ☐ A aplicação escreve input do usuário em logs? Se sim, `\r` e `\n` são escapados/neutralizados?
- ☐ Logs estruturados (JSON, CSV) são usados em lugar de logs de texto livre?
- ☐ Existe validação de entrada que rejeita caracteres de controle (`\x00-\x1F`) antes de logar?

10.2. 02 — Perda de Informação Relevante

CWE-221, CWE-223: Omission of Security-relevant Information

Um log que não registra **QUEM**, **QUANDO** e **DE ONDE** é praticamente inútil para investigação. Se o log diz apenas "Login falhou" 100 vezes, sem nome de usuário, timestamp ou IP, é impossível saber quem foi o atacante ou quando exatamente o ataque aconteceu. A investigação forense fica cega. Um ambiente corporativo que precisa de compliance (LGPD, PCI-DSS, ISO 27001) não passa em audit sem contexto completo. A correção é registrar **sempre**: timestamp, usuário, IP, ação e resultado.

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Perda de Informação Relevante (CWE-221/223) - A09:2025 Security Logging and
 * Alerting Failures
 *
 * O log não registra detalhes críticos para investigação: QUEM tentou, DE ONDE, e
 * QUANDO.
 * Sem essas informações, é impossível investigar mais tarde quem foi o atacante
 * ou reconstruir uma sequência de eventos suspeitos.
 */

function registrarTentativaLogin(string $usuario, string $enderecoIp, int $timestamp,
bool $sucesso): string {
    // VULNERÁVEL: perde informações críticas (usuário, IP, timestamp)
    return $sucesso ? "Login ok\n" : "Login falhou\n";
}

function demo(): void {
    echo "=== VULNERÁVEL: Log sem informações relevantes ===\n";

    $log1 = registrarTentativaLogin('joao.silva', '203.0.113.42', 1700000000, false);
    $log2 = registrarTentativaLogin('maria.santos', '198.51.100.105', 1700000060,
true);

    echo "Logs resultantes:\n";
    echo "---\n";
    echo $log1;
    echo $log2;
    echo "---\n";
    echo "Problema: não sabemos QUEM tentou, DE ONDE, nem QUANDO. Impossível
investigar depois!\n";
}

if (debug_backtrace() === []) {
    demo();
}
```

O código vulnerável loga apenas sucesso/falha, sem qualquer contexto. Um administrador não consegue investigar "quando exatamente falhou" ou "qual usuário tentou 50 vezes em 5 minutos".

```
<?php
declare(strict_types=1);
```

```

namespace Corrigido;

/**
 * Correção: Incluir contexto completo nos logs de segurança
 *
 * Registra timestamp, usuário, IP e resultado para permitir investigação completa
 * de sequências de eventos suspeitos, reconstruindo a timeline de um ataque.
 */

function registrarTentativaLogin(string $usuario, string $enderecoIp, int $timestamp,
bool $sucesso): string {
    // CORRIGIDO: inclui todas as informações críticas para investigação
    return sprintf("[%d] usuario=%s ip=%s resultado=%s\n", $timestamp, $usuario,
$enderecoIp, $sucesso ? 'sucesso' : 'falha');
}

function demo(): void {
    echo "=== CORRIGIDO: Log com informações completas ===\n";

    $log1 = registrarTentativaLogin('joao.silva', '203.0.113.42', 1700000000, false);
    $log2 = registrarTentativaLogin('maria.santos', '198.51.100.105', 1700000060,
true);

    echo "Logs resultantes:\n";
    echo "---\n";
    echo $log1;
    echo $log2;
    echo "---\n";
    echo "Agora temos timestamp, usuário, IP e resultado. Possível investigar
completamente!\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção registra timestamp, IP do cliente, usuário que tentou, ação (login) e resultado (sucesso/falha). Agora é possível investigar: saber em que hora exato falhou, qual IP está atacando, qual conta está sendo alvo. Esses dados são críticos para correlacionar eventos e descobrir a sequência do ataque.

10.2.1. Como testar

Execute `php A09-falhas-logging/02-perda-informacao-relevante/teste.php`. O teste gera múltiplas tentativas de login com usuários e IPs diferentes. A versão vulnerável perde contexto — impossível saber quem tentou o quê. A versão corrigida registra tudo, permitindo investigação completa. Saída esperada termina em `RESULTADO: N/N verificacoes passaram`.

10.2.2. Checklist de verificação

- ☐ Todos os eventos de segurança incluem timestamp (data e hora)?
- ☐ Nome de usuário ou ID de conta está presente em cada evento?
- ☐ Endereço IP do cliente (remoto, não 127.0.0.1) é registrado?
- ☐ Resultado da ação (sucesso/falha/erro) e, se erro, o motivo específico está presente?

10.3. 03 — Dados Sensíveis no Log

CWE-532: Insertion of Sensitive Information into Log File

Gravação de senhas, tokens, chaves de API ou números de cartão de crédito em logs é um risco crítico. Logs são frequentemente acessados por múltiplos usuários (admins, ferramentas de monitoramento, SIEMs), armazenados em centralizado (ELK, Splunk), replicados em backup. Se um atacante comprometer um usuário com acesso aos logs (ou ao servidor de backup), consegue roubar **todas** as credenciais jamais gravadas. O impacto é acesso permanente a contas críticas. A correção é simples: **nunca** gravar dados sensíveis. Se precisar logar autenticação, use placeholder como [REDACTED] ou hash não-revertível.

Código vulnerável (A09-falhas-logging/03-dados-sensiveis-no-log/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Dados Sensíveis Gravados no Log (CWE-532) - A09:2025 Security Logging and Alerting
 * Failures
 *
 * Senhas, tokens, chaves e outros dados sensíveis são gravados em TEXTO PURO no log.
 * Qualquer pessoa com acesso ao arquivo/sistema de logs (administrador, terceiro,
 * atacante que conseguiu acesso ao servidor) vê todas as credenciais em texto claro.
 */

function registrarRequisicaoLogin(string $usuario, string $senha): string {
    // VULNERÁVEL: grava a senha em texto puro, vazamento garantido
    return "login usuario={$usuario} senha={$senha}\n";
}

function demo(): void {
    echo "=== VULNERÁVEL: Dados sensíveis em texto puro no log ===\n";

    $logVulneravel = registrarRequisicaoLogin('joao', 'MinhaSenhaSecreta123');

    echo "Log resultante:\n";
    echo "---\n";
    echo $logVulneravel;
    echo "---\n";
}
```

```

    echo "PROBLEMA GRAVE: a senha está em texto puro. Qualquer acesso aos logs =
vazamento de credenciais!\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável grava nome de usuário e senha em texto puro no log. Qualquer pessoa que acesse o arquivo de log (ou backup) consegue ver todas as senhas em uma busca simples.

Código corrigido (A09-falhas-logging/03-dados-sensíveis-no-log/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Redatar dados sensíveis nos logs
 *
 * Nunca grava senhas, tokens, chaves ou dados privados em texto puro.
 * Usa placeholders como [REDACTED] ou registra apenas um hash não-revertível.
 */

function registrarRequisicaoLogin(string $usuario, string $senha): string {
    // CORRIGIDO: substitui a senha por um marcador seguro
    return "login usuario={$usuario} senha=[REDACTED]\n";
}

function demo(): void {
    echo "=== CORRIGIDO: Dados sensíveis redatados ===\n";

    $logCorrigido = registrarRequisicaoLogin('joao', 'MinhaSenhaSecreta123');

    echo "Log resultante:\n";
    echo "---\n";
    echo $logCorrigido;
    echo "---\n";
    echo "Seguro! O nome de usuário fica para auditoria, mas a senha é ocultada.\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção substitui a senha por [REDACTED]. Ainda há informação útil (qual usuário tentou), mas a credencial em si nunca fica em texto puro nos logs. Aplique a mesma regra a: tokens, chaves de API, números de cartão, CPF, dados médicos, qualquer PII (Personally Identifiable Information).

10.3.1. Como testar

Execute `php A09-falhas-logging/03-dados-sensíveis-no-log/teste.php`. O teste tenta fazer login com uma senha. A versão vulnerável grava a senha em texto puro (PROBLEMA!). A versão corrigida redata a senha, mantendo apenas o nome de usuário para auditoria. Saída esperada termina em **RESULTADO: N/N verificações passaram.**

10.3.2. Checklist de verificação

- ☐ Senhas, tokens de autenticação ou chaves de API **nunca** aparecem em logs (busque `password`, `token`, `key`, `secret` nos logs)?
- ☐ Números de cartão, CPF, SSN ou outros PII estão redatados ou mascarados?
- ☐ Se um ID de sessão ou token precisa ser logado, é apenas um prefixo ou hash, nunca o valor completo?

10.4. 04 — Logging Insuficiente

CWE-778: Insufficient Logging

Muitas aplicações só registram erros "normais" (requisição HTTP, debug de banco de dados). Eventos críticos de segurança — tentativa de login falha, elevação de privilégio, alteração de permissão, exclusão de usuário — são completamente ignorados. Um atacante passa despercebido porque não há "prova" do ataque sendo registrada. Sem logs de eventos críticos, é impossível detectar violação de dados, compliance audits falham, e você fica sem pistas pra investigar. A correção é simples: **definir uma list explícita de eventos críticos e garantir que todos sejam capturados.**

Código vulnerável (A09-falhas-logging/04-logging-insuficiente/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Logging Insuficiente (CWE-778) - A09:2025 Security Logging and Alerting Failures
 *
 * Apenas eventos irrelevantes são registrados. Eventos críticos para segurança
 * (login falhou, senha alterada, permissão elevada, usuário excluído) são IGNORADOS.
 * Resultado: violações de dados ficam sem detecção por meses ou anos.
 */

function tratarEventoSeguranca(string $tipoEvento, array $logsAcumulados): array {
    // VULNERÁVEL: só "loga" um evento totalmente irrelevante para segurança
    if ($tipoEvento === 'pagina_visitada') {
        $logsAcumulados[] = $tipoEvento;
    }
    // Eventos críticos como 'login_falhou', 'senha_alterada', 'permissao_elevada',
    'usuario_excluido'
```

```

// são COMPLETAMENTE IGNORADOS
return $logsAcumulados;
}

function demo(): void {
    echo "=== VULNERÁVEL: Logging insuficiente (eventos críticos ignorados) ===\n";

    $logs = [];
    $logs = tratarEventoSeguranca('login_falhou', $logs);
    $logs = tratarEventoSeguranca('login_falhou', $logs);
    $logs = tratarEventoSeguranca('permissao_elevada', $logs);
    $logs = tratarEventoSeguranca('usuario_excluido', $logs);
    $logs = tratarEventoSeguranca('pagina_visitada', $logs);

    echo "Total de eventos capturados: " . count($logs) . "\n";
    echo "Eventos no log: " . json_encode($logs) . "\n";
    echo "PROBLEMA: eventos críticos de segurança foram ignorados! Impossível detectar invasão.\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável ignora eventos críticos de segurança. Só registra requisições HTTP normais, deixando login falho, privilégio elevado, usuário excluído completamente sem registro.

Código corrigido (A09-falhas-logging/04-logging-insuficiente/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Registrar eventos críticos de segurança
 *
 * Define uma lista de eventos de segurança relevantes e garante que todos eles
 * sejam capturados. Dessa forma, tentativas de login falhas, elevações de privilégio
 * e outras ações suspeitas ficam registradas para investigação.
 */

function tratarEventoSeguranca(string $tipoEvento, array $logsAcumulados): array {
    // CORRIGIDO: define eventos críticos para segurança e os registra
    $eventosDeSeguranca = [
        'login_falhou',
        'login_sucesso',
        'senha_alterada',
        'permissao_elevada',
        'usuario_excluido'
    ];
}

```

```

];

if (in_array($tipoEvento, $eventosDeSeguranca, true)) {
    $logsAcumulados[] = $tipoEvento;
}

return $logsAcumulados;
}

function demo(): void {
    echo "=== CORRIGIDO: Logging de eventos críticos de segurança ===\n";

    $logs = [];
    $logs = tratarEventoSeguranca('login_falhou', $logs);
    $logs = tratarEventoSeguranca('login_falhou', $logs);
    $logs = tratarEventoSeguranca('permissao_elevada', $logs);
    $logs = tratarEventoSeguranca('usuario_excluido', $logs);
    $logs = tratarEventoSeguranca('pagina_visitada', $logs);

    echo "Total de eventos capturados: " . count($logs) . "\n";
    echo "Eventos no log: " . json_encode($logs) . "\n";
    echo "Agora todos os eventos críticos foram capturados. Possível investigar!\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção define uma lista explícita de eventos de segurança: `login_falhou`, `login_sucesso`, `senha_alterada`, `permissao_elevada`, `usuario_excluido`. Todos esses eventos são capturados e registrados. Qualquer tentativa de escalar privilégio ou atividade suspeita fica documentada.

10.4.1. Como testar

Execute `php A09-falhas-logging/04-logging-insuficiente/teste.php`. O teste dispara diversos eventos (login, elevação, exclusão). A versão vulnerável os ignora completamente (PROBLEMA!). A versão corrigida captura todos os eventos críticos. Saída esperada termina em `RESULTADO: N/N verificacoes passaram`.

10.4.2. Checklist de verificação

- ☐ Existe uma lista explícita de eventos críticos de segurança definida (login, logout, elevação de privilégio, exclusão)?
- ☐ Todos esses eventos são capturados em logs?
- ☐ Novos eventos críticos são adicionados à list conforme o sistema evolui?

10.5. 05 — Falta de Alerta de Anomalia

CWE-778: *Insufficient Logging*

Logging sem alertas é como ter câmera de segurança que grava, mas ninguém nunca assiste. Um ataque de força bruta pode estar acontecendo AGORA (50 tentativas de login falhas em 60 segundos), e ninguém fica sabendo até que a conta for totalmente comprometida. Um servidor começa a fazer conexões suspeitas em massa, e o administrador descobre só quando o malware já infectou 50 máquinas. A correção é implementar **detecção de padrões**: monitorar eventos críticos, reconhecer padrões anormais (10+ falhas em 60s = força bruta), e disparar alertas em tempo real para humanos ou sistemas de resposta automatizada.

Código vulnerável (A09-falhas-logging/05-falta-alerta-anomalia/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Falta de Mecanismo de Alerta e Detecção de Anomalias - A09:2025 Security Logging
and Alerting Failures
 *
 * Não existe nenhum sistema de monitoramento que analisa padrões suspeitos.
 * Ataque de força bruta em andamento não dispara nenhum alerta.
 * Administrador fica cego até que o dano já foi feito.
 */

function avaliarEventosSeguranca(array $timestampsLoginFalhou, int $agora): array {
    // VULNERÁVEL: NUNCA dispara alerta, não importa quantas tentativas de login
falharam
    return [
        'alerta_disparado' => false,
        'motivo' => 'monitoramento nao implementado'
    ];
}

function demo(): void {
    echo "=== VULNERÁVEL: Sem alerta de anomalia ===\n";

    // Simular 15 tentativas de login falho nos últimos 60 segundos
    $agora = 1700000000;
    $timestampsAtaque = [];
    for ($i = 1; $i <= 15; $i++) {
        $timestampsAtaque[] = $agora - (50 - $i);
    }

    $alerta = avaliarEventosSeguranca($timestampsAtaque, $agora);

    echo "Tentativas de login falho: " . count($timestampsAtaque) . "\n";
}
```

```

    echo "Alerta disparado? " . ($alerta['alerta_disparado'] ? 'SIM' : 'NÃO') . "\n";
    echo "Motivo: " . $alerta['motivo'] . "\n";
    echo "PROBLEMA: ataque de força bruta passa completamente despercebido!\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável NUNCA dispara alerta, não importa quantas tentativas de login falharem. Um ataque de força bruta ativo passa completamente despercebido.

Código corrigido (A09-falhas-logging/05-falta-alerta-anomalia/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Implementar detecção de anomalias com sistema de alertas
 *
 * Define limites de tentativas falhas em uma janela de tempo.
 * Quando o padrão suspeito é detectado, dispara um alerta automático
 * para que o administrador investigue imediatamente.
 */

function avaliarEventosSeguranca(array $timestampsLoginFalhou, int $agora): array {
    // CORRIGIDO: implementa detecção de força bruta
    $janela = 60; // janela de 60 segundos
    $limite = 10; // máximo de 10 tentativas

    // Filtra tentativas dentro da janela
    $recentes = array_filter($timestampsLoginFalhou, function($timestamp) use ($agora, $janela) {
        return ($agora - $timestamp) <= $janela;
    });

    if (count($recentes) >= $limite) {
        return [
            'alerta_disparado' => true,
            'motivo' => 'possivel ataque de forca bruta: ' . count($recentes) . ' tentativas falhas nos ultimos ' . $janela . 's'
        ];
    }

    return [
        'alerta_disparado' => false,
        'motivo' => 'dentro do normal'
    ];
}

```

```

}

function demo(): void {
    echo "=== CORRIGIDO: Detecção de anomalias com alertas ===\n";

    // Simular 15 tentativas de login falho nos últimos 60 segundos (ATAQUE)
    $agora = 1700000000;
    $timestampsAtaque = [];
    for ($i = 1; $i <= 15; $i++) {
        $timestampsAtaque[] = $agora - (50 - $i);
    }

    $alerta = avaliarEventosSeguranca($timestampsAtaque, $agora);

    echo "Tentativas de login falho: " . count($timestampsAtaque) . "\n";
    echo "Alerta disparado? " . ($alerta['alerta_disparado'] ? 'SIM' : 'NÃO') . "\n";
    echo "Motivo: " . $alerta['motivo'] . "\n";
    echo "\n";

    // Simular 2 tentativas (NORMAL)
    $timestampsNormal = [$agora - 30, $agora - 10];
    $alertaNormal = avaliarEventosSeguranca($timestampsNormal, $agora);

    echo "Tentativas normais: " . count($timestampsNormal) . "\n";
    echo "Alerta disparado? " . ($alertaNormal['alerta_disparado'] ? 'SIM' : 'NÃO') .
"\n";
    echo "Motivo: " . $alertaNormal['motivo'] . "\n";
    echo "Agora atacantes são detectados em tempo real!\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção detecta o padrão: se houver 10+ tentativas de login falhas dentro de 60 segundos, dispara alerta imediatamente. Um administrador recebe notificação (email, SMS, webhook) e pode bloquear a conta ou investigar o IP atacante antes que o dano se agrave.

10.5.1. Como testar

Execute `php A09-falhas-logging/05-falta-alerta-anomalia/teste.php`. O teste simula 15 tentativas de login falhas em 60 segundos (padrão de força bruta). A versão vulnerável não dispara nenhum alerta (PROBLEMA!). A versão corrigida detecta o padrão e dispara alerta. Saída esperada termina em `RESULTADO: N/N verificacoes passaram`.

10.5.2. Checklist de verificação

- ☐ Existe monitoramento de padrões suspeitos (múltiplas falhas de login, acesso de IPs desconhecidos, transferências em massa)?

- ☐ Alertas são disparados em tempo real quando anomalias são detectadas?
- ☐ Alertas vão para uma fila de resposta (humana ou automatizada) que age dentro de minutos?

10.6. Checklist consolidado — Falhas de Registro e Alerta de Segurança

Ao revisar logging e monitoramento de aplicação, valide os pontos críticos:

- ☐ **Escapar Quebras de Linha:** Qualquer input que vai para log deve escapar `\r` e `\n` para evitar log injection.
- ☐ **Contexto Completo:** Todo evento de segurança inclui timestamp, usuário, IP, ação e resultado. Sem contexto, logs são inúteis.
- ☐ **Sem Dados Sensíveis:** Senhas, tokens, chaves e números de cartão **nunca** são gravados. Use `[REDACTED]` ou hash.
- ☐ **Eventos Críticos:** Existe list explícita de eventos de segurança (login, privilege escalation, exclusão). Todos são capturados.
- ☐ **Alertas Automáticos:** Padrões suspeitos (força bruta, acessos em massa) disparam alertas em tempo real.
- ☐ **Proteção de Logs:** Logs são imutáveis (append-only), replicados, e acessíveis apenas por autoridades de confiança (não por usuários comuns).

Chapter 11. A10:2025 — Tratamento Inadequado de Condições Excepcionais

Toda aplicação real enfrenta erros: banco de dados indisponível, rede caindo, arquivo não encontrado, divisão por zero, timeout de API externa. O **tratamento** desses erros é crítico para segurança. Uma exceção não tratada pode expor dados sensíveis (stack trace com nomes de tabela, versões de biblioteca), deixar o sistema em estado inconsistente (débito sem crédito, transação parcial), ou permitir acesso não autorizado (falha silenciosa = acesso concedido por padrão). Este capítulo cobre os antipadrões mais perigosos: erros que expõem informação, exceções ignoradas, retornos de função não validados, e defaults inseguros. Vamos aprender a tratar erros de forma **segura e confiável**.

Veja a página oficial: https://owasp.org/Top10/2025/A10_2025-Mishandling_of_Exceptional_Conditions/

Exemplos cobertos neste capítulo:

- **01-erro-com-dados-sensíveis** — exposição de stack trace com nomes de tabela e estrutura interna
- **02-excecao-nao-tratada** — exceção lançada sem try/catch, causando crash
- **03-valor-retorno-nao-verificado** — ignorar que uma função retornou false/null (falha silenciosa)
- **04-caso-default-ausente** — switch sem default, fail-open perigoso
- **05-tratamento-generico-excecoes** — capturar tipos diferentes de exceção de forma igual
- **06-falha-nao-segura-fail-open** — assume acesso permitido quando serviço falha
- **07-transacao-nao-atomica-sem-rollback** — operação multi-step sem rollback em falha
- **08-dados-sensíveis-em-debug** — modo debug expõe senhas e cartões de crédito

11.1. 01 — Erro com Dados Sensíveis

CWE-209, CWE-550: Information Exposure Through Error Message

Quando uma exceção de banco de dados acontece (tabela inexistente, coluna inválida, constraint violada), a mensagem de erro pode revelar **a estrutura interna do sistema**: nomes de tabelas, campos, engine de BD, version de biblioteca, até mesmo queries parciais. Um atacante usa essas informações para mapear o banco de dados e planejar ataques de SQL injection com precisão. Mensagens de erro cruas são ainda mais perigosas em APIs REST, onde são retornadas diretamente no body HTTP (visível até em logs de proxy/firewall). A correção é simples: sempre devolver mensagem **genérica** ao cliente, e logar os detalhes técnicos **apenas no servidor**.

Código vulnerável (A10-tratamento-inadequado-excecoes/01-erro-com-dados-sensíveis/vulneravel.php)

```
<?php
declare(strict_types=1);
```

```

namespace Vulneravel;

/**
 * Exposição de Detalhes Técnicos em Mensagens de Erro - A10:2025 Mishandling of
Exceptional Conditions
 *
 * A função captura exceções de banco de dados mas retorna a mensagem de erro BRUTA
 * sem filtragem. Mensagens de PDOException expõem nomes de tabelas, estrutura SQL,
 * versão do motor de banco – informações valiosas para um atacante mapear a
aplicação.
 */

function criarBancoDeTeste(): \PDO {
    $pdo = new \PDO('sqlite::memory:');
    $pdo->setAttribute(\PDO::ATTR_ERRMODE, \PDO::ERRMODE_EXCEPTION);
    $pdo->exec('CREATE TABLE produtos (id INTEGER PRIMARY KEY, nome TEXT)');
    $pdo->exec("INSERT INTO produtos (nome) VALUES ('Produto A')");
    $pdo->exec("INSERT INTO produtos (nome) VALUES ('Produto B')");
    return $pdo;
}

function executarConsulta(\PDO $pdo, string $sql): array {
    try {
        $pdo->query($sql);
        return ['erro' => false, 'detalhe' => 'ok'];
    } catch (\PDOException $e) {
        // VULNERÁVEL: retorna a mensagem de erro COMPLETA do banco
        // Expõe nomes de tabelas, estrutura SQL, mensagens técnicas
        return ['erro' => true, 'detalhe' => $e->getMessage()];
    }
}

function demo(): void {
    echo "=== VULNERÁVEL: Exposição de detalhes técnicos de erro ===\n";

    $pdo = criarBancoDeTeste();

    // Query válida
    $resultado = executarConsulta($pdo, 'SELECT * FROM produtos');
    echo "Query válida - Erro: " . ($resultado['erro'] ? 'sim' : 'não') . "\n";

    // Query inválida que expõe a estrutura interna
    $resultado = executarConsulta($pdo, 'SELECT * FROM tabela_inexistente');
    echo "Query inválida - Detalhe exposto: " . $resultado['detalhe'] . "\n";
    echo "(PROBLEMA: atacante vê que a tabela não existe, mapeia estrutura do banco)
\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

```
}
```

O código vulnerável retorna `$e->getMessage()` direto, expondo nomes de tabelas e estrutura SQL.

Código corrigido (A10-tratamento-inadequado-excecoes/01-erro-com-dados-sensíveis/corrigido.php)

```
<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Filtragem de Detalhes Técnicos em Mensagens de Erro
 *
 * Captura exceções e retorna uma mensagem genérica, segura,
 * nunca revelando detalhes internos (nomes de tabela, SQL, stack trace).
 * O detalhe técnico pode ser logado internamente para debug/auditoria.
 */

function criarBancoDeTeste(): \PDO {
    $pdo = new \PDO('sqlite::memory:');
    $pdo->setAttribute(\PDO::ATTR_ERRMODE, \PDO::ERRMODE_EXCEPTION);
    $pdo->exec('CREATE TABLE produtos (id INTEGER PRIMARY KEY, nome TEXT)');
    $pdo->exec("INSERT INTO produtos (nome) VALUES ('Produto A')");
    $pdo->exec("INSERT INTO produtos (nome) VALUES ('Produto B')");
    return $pdo;
}

function executarConsulta(\PDO $pdo, string $sql): array {
    try {
        $pdo->query($sql);
        return ['erro' => false, 'detalhe' => 'ok'];
    } catch (\PDOException $e) {
        // CORRIGIDO: retorna apenas uma mensagem genérica, segura
        // (O detalhe técnico seria logado internamente, nunca exposto ao cliente)
        return ['erro' => true, 'detalhe' => 'Nao foi possivel processar a
consulta.'];
    }
}

function demo(): void {
    echo "=== CORRIGIDO: Mensagens de erro genéricas, sem exposição técnica ===\n";

    $pdo = criarBancoDeTeste();

    // Query válida
    $resultado = executarConsulta($pdo, 'SELECT * FROM produtos');
    echo "Query válida - Erro: " . ($resultado['erro'] ? 'sim' : 'não') . "\n";

    // Query inválida, mas mensagem genérica
    $resultado = executarConsulta($pdo, 'SELECT * FROM tabela_inexistente');
```

```

    echo "Query inválida - Detalhe retornado: " . $resultado['detalhe'] . "\n";
    echo "(OK: atacante não vê detalhes técnicos da estrutura)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção retorna mensagem genérica ao cliente ("Erro ao processar requisição"), e registra a exceção real em log **privado** do servidor (onde apenas admins têm acesso).

11.1.1. Como testar

Execute `php A10-tratamento-inadequado-excecoes/01-erro-com-dados-sensíveis/teste.php`. O teste executa queries válidas e depois uma query que referencia tabela inexistente. A versão vulnerável expõe o nome da tabela (PROBLEMA!). A versão corrigida retorna mensagem genérica. Saída esperada termina em `RESULTADO: N/N verificacoes passaram`.

11.1.2. Checklist de verificação

- ☐ A aplicação retorna mensagens de exceção CRUAS ao cliente? Se sim, ela expõe nomes de tabela, estrutura SQL, stack trace ou versões de biblioteca?
- ☐ Erros técnicos são logados **apenas no servidor**, em arquivo protegido?
- ☐ API retorna sempre JSON com mensagem genérica (nunca stack trace ou query)?

11.2. 02 — Exceção Não Tratada

CWE-248, CWE-703: Uncaught Exception / Handler with no effect

Uma função que lança exceção sem estar envolvida em `try/catch` causa crash não controlado. Em produção, isso é negação de serviço (DoS) — a requisição falha, o usuário vê erro 500, a aplicação fica instável. Em endpoints sensíveis (login, pagamento), um crash interrompe a operação no meio, deixando o sistema em estado inconsistente. A exceção também pode expor informações sensíveis (stack trace). A correção é sempre: **validar input antes** (evitar a exceção) ou **envolver em try/catch** com tratamento apropriado.

Código vulnerável (A10-tratamento-inadequado-excecoes/02-excecao-nao-tratada/vulneravel.php)

```

<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Exceção Não Tratada - A10:2025 Mishandling of Exceptional Conditions
 *
 * A função tenta processar um pedido sem validar dados de entrada.
 * Se `quantidade` for 0, DivisionByZeroError é lançado SEM tratamento,

```

```

* causando crash não controlado e possível negação de serviço.
*/

function processarPedido(array $pedido): array {
    // VULNERÁVEL: nenhuma validação, nenhum try/catch
    // Se quantidade for 0, ocorre divisão por zero não tratada
    $precoUnitario = 100 / $pedido['quantidade'];
    return ['status' => 'ok', 'preco_unitario' => $precoUnitario];
}

function demo(): void {
    echo "=== VULNERÁVEL: Exceção não tratada ===\n";

    // Caso legítimo
    echo "Processando pedido legítimo...\n";
    $resultado = processarPedido(['quantidade' => 4]);
    echo "Resultado: " . json_encode($resultado) . "\n";

    // Caso que causa exceção não tratada
    echo "\nProcessando pedido com quantidade zero (vai causar erro)...\n";
    try {
        $resultado = processarPedido(['quantidade' => 0]);
        echo "Resultado: " . json_encode($resultado) . "\n";
    } catch (\Throwable $e) {
        echo "ERRO NÃO TRATADO NA FUNÇÃO: " . $e::class . " - " . $e->getMessage() .
"\n";
    }
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável divide por `$pedido['quantidade']` sem validar se é zero. Se atacante enviar `quantidade=0`, `DivisionByZeroError` é lançado sem tratamento.

Código corrigido (A10-tratamento-inadequado-excecoes/02-excecao-nao-tratada/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Validação e Tratamento de Exceções
 *
 * A função valida entrada, trata exceções possíveis
 * e sempre retorna um resultado seguro e controlado.
 * Nunca deixa exceções não tratadas escaparem.
 */

```

```

function processarPedido(array $pedido): array {
    try {
        // Extrai e valida a quantidade
        $quantidade = $pedido['quantidade'] ?? 0;

        // Validação explícita
        if ($quantidade <= 0) {
            throw new \InvalidArgumentException('quantidade invalida');
        }

        // Operação segura agora que quantidade foi validado
        $precoUnitario = 100 / $quantidade;
        return ['status' => 'ok', 'preco_unitario' => $precoUnitario];
    } catch (\Throwable $e) {
        // CORRIGIDO: todas as exceções são capturadas
        // Função sempre retorna um resultado estruturado
        return ['status' => 'erro', 'mensagem' => 'Nao foi possivel processar o
pedido.'];
    }
}

function demo(): void {
    echo "=== CORRIGIDO: Tratamento robusto de exceções ===\n";

    // Caso legítimo
    echo "Processando pedido legítimo...\n";
    $resultado = processarPedido(['quantidade' => 4]);
    echo "Resultado: " . json_encode($resultado) . "\n";

    // Caso que seria exceção - agora tratado
    echo "\nProcessando pedido com quantidade zero (tratado graciosamente)...\n";
    $resultado = processarPedido(['quantidade' => 0]);
    echo "Resultado: " . json_encode($resultado) . "\n";
    echo "(OK: Exceção capturada e convertida em resposta estruturada)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção valida `quantidade > 0` antes de usar, retornando erro controlado se inválido. Nenhuma exceção é lançada — o erro é tratado explicitamente com resposta apropriada.

11.2.1. Como testar

Execute `php A10-tratamento-inadequado-excecoes/02-excecao-nao-tratada/teste.php`. O teste envia um pedido legítimo (funciona) e depois um pedido com quantidade zero. A versão vulnerável dispara `DivisionByZeroError` não tratado (PROBLEMA!). A versão corrigida valida e retorna erro controlado. Saída esperada termina em `RESULTADO: N/N verificacoes passaram`.

11.2.2. Checklist de verificação

- ☐ Todas operações potencialmente perigosas (divisão, acesso a array, operação de I/O) estão envolvidas em `try/catch` ou validadas antes?
- ☐ Nenhuma exceção "escapa" de uma função sem ser tratada?
- ☐ Valores de input são validados **antes** de usar em cálculo, para evitar exceções?

11.3. 03 — Valor de Retorno Não Verificado

CWE-252: Unchecked Return Value

Uma função como `file_put_contents()` ou `mysqli_query()` **retorna um valor**. Se retornar `false` ou `null`, significa falha. Uma aplicação que ignora esse retorno e continua como se tivesse sucesso está vulnerável a falhas silenciosas: arquivo não foi escrito (mas o código acha que foi), query não foi executada (mas o código pensa que a transação commitou), credenciais não foram salvos em BD (mas o login funciona). O impacto varia: perda de dados, inconsistência de estado, segurança e lógica quebrada. A correção é: **sempre** checar o retorno de funções críticas.

Código vulnerável (A10-tratamento-inadequado-excecoes/03-valor-retorno-nao-verificado/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Valor de Retorno Não Verificado (CWE-252) - A10:2025 Mishandling of Exceptional
 Conditions
 *
 * A função chama file_put_contents() mas nunca verifica seu retorno.
 * Se a operação falhar (diretório inexistente, sem permissão), a função
 * falsamente reporta sucesso, causando falha silenciosa perigosa.
 */

function salvarConfiguracao(string $caminho, string $conteudo): array {
    // VULNERÁVEL: @ suprime warnings, mas nunca verifica o retorno
    // file_put_contents() retorna false em caso de falha
    @file_put_contents($caminho, $conteudo);
    // Sempre reporta sucesso, mesmo se a escrita falhou
    return ['status' => 'sucesso'];
}

function demo(): void {
    echo "=== VULNERÁVEL: Valor de retorno não verificado ===\n";

    // Tenta salvar em caminho inválido
    $caminhoInvalido = sys_get_temp_dir() . '/pasta-inexistente-' . uniqid() .
'/config.txt';
    $resultado = salvarConfiguracao($caminhoInvalido, 'dados importantes');
```

```

echo "Salvar em caminho inválido: " . $resultado['status'] . "\n";
echo "(PROBLEMA: diz 'sucesso' mas o arquivo NÃO foi salvo)\n";

// Verifica se arquivo realmente existe
if (!file_exists($caminhoInvalido)) {
    echo "Confirmado: arquivo NÃO existe (falha silenciosa não detectada!)\n";
}

}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável chama `file_put_contents()` mas nunca checa se retornou sucesso (true) ou falha (false). A aplicação continua como se o arquivo fosse salvo, mesmo que a escrita tenha falhado.

Código corrigido (A10-tratamento-inadequado-excecoes/03-valor-retorno-nao-verificado/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Verificação de Valor de Retorno
 *
 * Antes de reportar sucesso, verifica o valor retornado por file_put_contents().
 * Se retornar false, reporta erro apropriadamente.
 */

function salvarConfiguracao(string $caminho, string $conteudo): array {
    // CORRIGIDO: verifica o valor de retorno
    $resultado = @file_put_contents($caminho, $conteudo);

    if ($resultado === false) {
        // Detecta e reporta a falha
        return ['status' => 'erro', 'mensagem' => 'Falha ao salvar arquivo de
configuracao'];
    }

    return ['status' => 'sucesso'];
}

function demo(): void {
    echo "=== CORRIGIDO: Valor de retorno verificado ===\n";

    // Tenta salvar em caminho inválido
    $caminhoInvalido = sys_get_temp_dir() . '/pasta-inexistente-' . uniqid() .
'/config.txt';

```

```

$resultado = salvarConfiguracao($caminhoInvalido, 'dados importantes');
echo "Salvar em caminho inválido: " . $resultado['status'] . "\n";
echo "(OK: detecta e reporta erro apropriadamente)\n";

// Tenta salvar em caminho válido
$caminhoValido = sys_get_temp_dir() . '/teste_a10_' . uniqid() . '.txt';
$resultado = salvarConfiguracao($caminhoValido, 'dados importantes');
echo "\nSalvar em caminho válido: " . $resultado['status'] . "\n";

if (file_exists($caminhoValido)) {
    echo "(OK: arquivo foi criado com sucesso)\n";
    unlink($caminhoValido); // Limpeza
}
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção checa `if ($escrito === false)` após `file_put_contents()`. Se falhar, lança exceção ou retorna erro, interrompendo a operação antes que dados ficassem inconsistentes.

11.3.1. Como testar

Execute `php A10-tratamento-inadequado-excecoes/03-valor-retorno-nao-verificado/teste.php`. O teste tenta escrever em um arquivo em diretório sem permissão de escrita. A versão vulnerável ignora a falha e reporta sucesso (PROBLEMA!). A versão corrigida detecta e reporta falha real. Saída esperada termina em `RESULTADO: N/N verificacoes passaram`.

11.3.2. Checklist de verificação

- ☐ `file_put_contents()`, `file_get_contents()`, `fopen()`, `fwrite()` — retorno é verificado?
- ☐ `mysqli_query()`, `$pdo->exec()`, `$pdo->query()` — retorno é verificado ou exception é capturada?
- ☐ Retornos de `strpos()`, `array_search()`, `preg_match()` são checados contra `false` ou `null`?

11.4. 04 — Caso Default Ausente

CWE-478, CWE-636: Missing Default Case in Switch Statement; Incorrect Default Operator Precedence

Um `switch` ou estrutura de decisão sem case `default` explícito é perigoso. Se o valor não bater nenhum case, o código não faz nada e continua (fall-through). Em contexto de segurança, isso é frequentemente **fail-open**: um status desconhecido cai silenciosamente em "acesso permitido" (porque não entrou em case "negado"). Um atacante envia um status que a aplicação não esperava, conseguindo acesso não autorizado. A correção é: **always ter default**, e esse default deve ser **fail-safe** (nega por padrão, não permite).

```

<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Ausência de Caso Default Falha Aberta (CWE-478, CWE-636) - A10:2025 Mishandling of
 * Exceptional Conditions
 *
 * Switch sem caso default com valor inicial permissivo (fail-open).
 * Qualquer status novo ou inesperado retorna TRUE, concedendo acesso
 * a uma conta em estado desconhecido (comportamento perigoso).
 */

function temAcesso(string $statusConta): bool {
    // VULNERÁVEL: valor inicial permissivo (acesso PERMITIDO por padrão)
    $acesso = true;

    switch ($statusConta) {
        case 'bloqueada':
            $acesso = false;
            break;
        case 'banida':
            $acesso = false;
            break;
        // SEM default: qualquer status novo/inesperado mantém $acesso = true (fail-
open)
    }

    return $acesso;
}

function demo(): void {
    echo "=== VULNERÁVEL: Switch sem default (fail-open) ===\n";

    // Casos conhecidos
    echo "Status 'bloqueada': " . (temAcesso('bloqueada') ? 'permitido' : 'negado') .
"\n";
    echo "Status 'ativa': " . (temAcesso('ativa') ? 'permitido' : 'negado') . "\n";

    // Caso novo/inesperado
    $statusNovoDesconhecido = 'suspensa_por_erro_de_sistema';
    $acesso = temAcesso($statusNovoDesconhecido);
    echo "Status '{$statusNovoDesconhecido}': " . ($acesso ? 'permitido' : 'negado') .
"\n";
    echo "(PROBLEMA: acesso PERMITIDO a conta em estado desconhecido! Fail-open
perigoso)\n";
}

```

```

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável tem switch sem default. Um status desconhecido cai silenciosamente para "acesso permitido" (porque não entrou no case de negação).

Código corrigido (A10-tratamento-inadequado-excecoes/04-caso-default-ausente/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Caso Default Seguro com Fail-Safe
 *
 * Switch com caso default explícito que NEGA acesso para
 * qualquer status desconhecido (fail-safe/fail-closed).
 * Valor inicial também negativo, mais seguro.
 */

function temAcesso(string $statusConta): bool {
    // CORRIGIDO: lógica clara, com default negativo (fail-safe)
    switch ($statusConta) {
        case 'ativa':
            return true;
        case 'bloqueada':
        case 'banida':
            return false;
        default:
            // Qualquer status desconhecido NEGA acesso por padrão (fail-safe)
            return false;
    }
}

function demo(): void {
    echo "=== CORRIGIDO: Switch com default seguro (fail-safe) ===\n";

    // Casos conhecidos
    echo "Status 'bloqueada': " . (temAcesso('bloqueada') ? 'permitido' : 'negado') .
"\n";
    echo "Status 'ativa': " . (temAcesso('ativa') ? 'permitido' : 'negado') . "\n";

    // Caso novo/inesperado
    $statusNovoDesconhecido = 'suspensa_por_erro_de_sistema';
    $acesso = temAcesso($statusNovoDesconhecido);
    echo "Status '{$statusNovoDesconhecido}': " . ($acesso ? 'permitido' : 'negado') .
"\n";
    echo "(OK: acesso NEGADO a conta em estado desconhecido. Fail-safe seguro)\n";
}

```

```

}

if (debug_backtrace() === []) {
    demo();
}

```

A correção adiciona default explícito que **nega por padrão**. Qualquer status desconhecido cai no case 'default' e retorna acesso negado — fail-safe, não fail-open.

11.4.1. Como testar

Execute `php A10-tratamento-inadequado-excecoes/04-caso-default-ausente/teste.php`. O teste passa status legítimos (aprovado, negado) e depois um status desconhecido. A versão vulnerável permite acesso (PROBLEMA!). A versão corrigida nega por padrão. Saída esperada termina em **RESULTADO: N/N verificacoes passaram**.

11.4.2. Checklist de verificação

- ☐ Todos os `switch` statements têm case `default`?
- ☐ O `default` case **nega por padrão** ou retorna erro, não permite?
- ☐ Estados de segurança desconhecidos são tratados como "negado", não como "permitido"?

11.5. 05 — Tratamento Genérico de Exceções

CWE-396, CWE-397: Declaration with void Type; Non-specific Catch Clause

Diferentes exceções indicam problemas diferentes que exigem respostas diferentes. Um timeout de rede **pode ser retentado** (falha temporária). Saldo insuficiente **não deve ser retentado** (falha permanente). Um arquivo não encontrado **é erro diferente** de permissão negada. Se todas as exceções forem capturadas genericamente com `catch (\Exception $e)` e tratadas igual, o código perde informação crítica: não consegue decidir se deve retentar, bloqueio, relatar, etc. O resultado é comportamento inadequado em cada situação (retry em falha permanente, abandon em temporária). A correção é: **catch exceções específicas** com tratamento apropriado pra cada uma.

Código vulnerável (A10-tratamento-inadequado-excecoes/05-tratamento-generico-excecoes/vulneravel.php)

```

<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Tratamento Genérico de Exceções Diferentes (CWE-396/397) - A10:2025 Mishandling of
 * Exceptional Conditions
 *
 * Exceções de diferentes naturezas (falta de saldo vs. erro de rede) são capturadas
 * genericamente com o mesmo tratamento. Perde-se a informação crítica de qual foi o
 * problema,

```

```

* impedindo decisões apropriadas (retry automático, bloqueio, etc).
*/

class SaldoInsuficienteException extends \Exception {}
class ErroDeConexaoException extends \Exception {}

function processarPagamento(float $valor, float $saldoDisponivel, bool
$simularErroDeConexao): array {
    try {
        // Simula erro de conexão (temporário, poderia tentar novamente)
        if ($simularErroDeConexao) {
            throw new ErroDeConexaoException('timeout na rede');
        }

        // Simula falta de saldo (permanente, não deve tentar automaticamente)
        if ($valor > $saldoDisponivel) {
            throw new SaldoInsuficienteException('saldo insuficiente');
        }

        return ['status' => 'aprovado'];
    } catch (\Exception $e) {
        // VULNERÁVEL: captura QUALQUER exception genericamente
        // Perdem-se as informações críticas sobre o tipo de erro
        // Não é possível diferenciar "retry" vs "falha permanente"
        return ['status' => 'erro'];
    }
}

function demo(): void {
    echo "=== VULNERÁVEL: Tratamento genérico de exceções diferentes ===\n";

    // Caso 1: Erro de conexão (temporário)
    $r1 = processarPagamento(50.0, 100.0, true);
    echo "Erro de conexão retorna: status=" . $r1['status'] . "\n";

    // Caso 2: Saldo insuficiente (permanente)
    $r2 = processarPagamento(150.0, 100.0, false);
    echo "Saldo insuficiente retorna: status=" . $r2['status'] . "\n";

    echo "(PROBLEMA: ambos retornam o MESMO status, diferença crítica se perdeu!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável captura `\Exception` genérica. Erro de rede e falta de saldo retornam a mesma resposta `['status' => 'erro']`, perdendo diferença crítica.

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Tratamento Específico de Diferentes Tipos de Exceção
 *
 * Cada tipo de exceção é capturado e tratado de forma apropriada,
 * permitindo decisões corretas (retry, bloqueio, etc).
 */

class SaldoInsuficienteException extends \Exception {}
class ErroDeConexaoException extends \Exception {}

function processarPagamento(float $valor, float $saldoDisponivel, bool
$simularErroDeConexao): array {
    try {
        // Simula erro de conexão (temporário, poderia tentar novamente)
        if ($simularErroDeConexao) {
            throw new ErroDeConexaoException('timeout na rede');
        }

        // Simula falta de saldo (permanente, não deve tentar automaticamente)
        if ($valor > $saldoDisponivel) {
            throw new SaldoInsuficienteException('saldo insuficiente');
        }

        return ['status' => 'aprovado'];
    } catch (SaldoInsuficienteException $e) {
        // CORRIGIDO: trata falta de saldo especificamente
        // (não deve fazer retry automático)
        return ['status' => 'recusado', 'motivo' => 'saldo insuficiente'];
    } catch (ErroDeConexaoException $e) {
        // CORRIGIDO: trata erro de rede especificamente
        // (pode tentar novamente após delay)
        return ['status' => 'tentar_novamente', 'motivo' => 'falha temporaria de
rede'];
    }
}

function demo(): void {
    echo "=== CORRIGIDO: Tratamento específico de cada tipo de exceção ===\n";

    // Caso 1: Erro de conexão (temporário)
    $r1 = processarPagamento(50.0, 100.0, true);
    echo "Erro de conexão retorna: status=" . $r1['status'] . " (pode fazer retry)\n";

    // Caso 2: Saldo insuficiente (permanente)

```

```

    $r2 = processarPagamento(150.0, 100.0, false);
    echo "Saldo insuficiente retorna: status=" . $r2['status'] . " (não faz retry)\n";

    echo "(OK: cada situação tem tratamento apropriado)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção captura cada exceção específica separadamente: `catch (ErroDeConexaoException)` retorna status que indica "retry possível"; `catch (SaldoInsuficienteException)` retorna status de falha permanente. Cada tipo recebe resposta apropriada.

11.5.1. Como testar

Execute `php A10-tratamento-inadequado-excecoes/05-tratamento-generico-excecoes/teste.php`. O teste simula dois cenários: erro de rede (temporário) e saldo insuficiente (permanente). A versão vulnerável retorna status igual pra ambas (PROBLEMA!). A versão corrigida retorna status específico. Saída esperada termina em **RESULTADO: N/N verificacoes passaram**.

11.5.2. Checklist de verificação

- ☐ `catch` clauses capturam tipos específicos de exceção, não `\Exception` genérica?
- ☐ Cada tipo de exceção recebe tratamento apropriado (retry vs. abort vs. user message)?
- ☐ Exceções inesperadas ainda são capturadas (com fall-back genérico), mas padrão é específico?

11.6. 06 — Falha Não-Segura (Fail-Open)

CWE-636: Incorrect Default Operator Precedence

Quando um serviço externo falha a verificar permissão (retorna `null` ou timeout), a aplicação assume acesso **permitido por padrão** (fail-open perigoso). Código como `$permitido = $servicoExterno ?? true;` significa: "Se o serviço falhar, deixe entrar de qualquer forma". Isso é **errado**. Se a verificação falhar, o princípio de segurança correto é **falha segura** (fail-safe): **nega acesso** até que a verificação seja confirmada. A correção é trivial: usar `?? false` (nega por padrão).

Código vulnerável (A10-tratamento-inadequado-excecoes/06-falha-nao-segura-fail-open/vulneravel.php)

```

<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Falha Não-Segura com Fail-Open (CWE-636) - A10:2025 Mishandling of Exceptional
 * Conditions
 */

```

```

* Quando um serviço externo de verificação de permissão falha (retorna null),
* a função assume acesso PERMITIDO por padrão (fail-open perigoso).
* Qualquer falha no serviço de checagem resulta em acesso indevido.
*/

function verificarPermissao(?bool $resultadoChecagemExterna): bool {
    // VULNERÁVEL: se serviço falha (null), retorna true (acesso PERMITIDO)
    // Fail-open perigoso: assume que está tudo ok quando não consegue verificar
    return $resultadoChecagemExterna ?? true;
}

function demo(): void {
    echo "=== VULNERÁVEL: Falha não-segura com fail-open ===\n";

    // Serviço respondendo normalmente
    echo "Serviço responde true: " . (verificarPermissao(true) ? 'permitido' :
'negado') . "\n";
    echo "Serviço responde false: " . (verificarPermissao(false) ? 'permitido' :
'negado') . "\n";

    // Serviço falha (null)
    echo "Serviço falha (null): " . (verificarPermissao(null) ? 'permitido' :
'negado') . "\n";
    echo "(PROBLEMA: acesso PERMITIDO mesmo quando serviço falhou!)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável usa `?? true`, permitindo acesso se o serviço falha. Um atacante causa timeout no serviço de verificação e consegue acesso indevido.

Código corrigido (A10-tratamento-inadequado-excecoes/06-falha-nao-segura-fail-open/corrigido.php)

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Falha Segura com Fail-Safe (Fail-Closed)
 *
 * Quando um serviço externo de verificação falha (retorna null),
 * a função assume acesso NEGADO por padrão (fail-safe).
 * Qualquer falha no serviço bloqueia acesso até confirmação.
 */

function verificarPermissao(?bool $resultadoChecagemExterna): bool {
    // CORRIGIDO: se serviço falha (null), retorna false (acesso NEGADO)

```

```

// Fail-safe/fail-closed: nega acesso quando não consegue verificar
// Só permite quando obtém confirmação explícita
return $resultadoChecagemExterna ?? false;
}

function demo(): void {
    echo "=== CORRIGIDO: Falha segura com fail-safe ===\n";

    // Serviço respondendo normalmente
    echo "Serviço responde true: " . (verificarPermissao(true) ? 'permitido' :
'negado') . "\n";
    echo "Serviço responde false: " . (verificarPermissao(false) ? 'permitido' :
'negado') . "\n";

    // Serviço falha (null)
    echo "Serviço falha (null): " . (verificarPermissao(null) ? 'permitido' :
'negado') . "\n";
    echo "(OK: acesso NEGADO até conseguir verificar com segurança)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção usa `?? false`, negando acesso se o serviço falha. Acesso só é concedido se a verificação externa retorna explicitamente `true`. Isso é fail-safe.

11.6.1. Como testar

Execute `php A10-tratamento-inadequado-excecoes/06-falha-nao-segura-fail-open/teste.php`. O teste simula serviço respondendo e depois falhando (null). A versão vulnerável permite acesso mesmo quando serviço falha (PROBLEMA!). A versão corrigida nega quando há falha. Saída esperada termina em `RESULTADO: N/N verificacoes passaram`.

11.6.2. Checklist de verificação

- ☐ Toda verificação de permissão de serviço externo tem default **false** (nega), nunca **true**?
- ☐ Se serviço falha, timeout, ou retorna inesperado, a resposta padrão é "negar acesso"?
- ☐ Código usa `?? false` para defaults de segurança, nunca `?? true`?

11.7. 07 — Transação Não-Atômica Sem Rollback

CWE-636: Incorrect Default Operator Precedence

Uma transferência bancária precisa de dois passos: debitar da conta A e creditar em B. Se os dois passos são executados **sem proteção transacional**, uma falha no meio (queda de poder, crash de BD, exceção de rede) deixa o dinheiro desaparecido: debitou de A mas nunca creditou em B. O sistema fica em estado inconsistente. Se não houver **rollback automático**, o dinheiro se perde e

ninguém consegue recuperar. A correção é: usar transações ACID (BEGIN, COMMIT, ROLLBACK), e garantir que se qualquer passo falha, todos os passos são desfeitos (rollback).

Código vulnerável (A10-tratamento-inadequado-excecoes/07-transacao-nao-atomica-sem-rollback/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Transação Não-Atômica Sem Rollback (CWE-636) - A10:2025 Mishandling of Exceptional
Conditions
 *
 * Uma transferência de dinheiro em duas etapas (débito + crédito) pode ser
interrompida
 * entre o débito e o crédito, causando perda de dinheiro. Não há mecanismo de
rollback
 * para restaurar o estado anterior em caso de falha parcial.
 */

function transferir(array $contas, string $origem, string $destino, float $valor, bool
$interromperAposDebito): array {
    // Valida se há saldo
    if ($contas[$origem] < $valor) {
        return ['status' => 'erro', 'contas' => $contas];
    }

    // Etapa 1: Débito
    $contas[$origem] -= $valor;

    // VULNERÁVEL: pode falhar AQUI (interrupção simulada)
    // Sem rollback, o dinheiro fica perdido
    if ($interromperAposDebito) {
        return ['status' => 'interrompido', 'contas' => $contas];
    }

    // Etapa 2: Crédito (só alcançado se não houver interrupção)
    $contas[$destino] += $valor;

    return ['status' => 'sucesso', 'contas' => $contas];
}

function demo(): void {
    echo "=== VULNERÁVEL: Transação sem atomicidade/rollback ===\n";

    $contas = ['A' => 100.0, 'B' => 50.0];
    echo "Estado inicial: A=" . $contas['A'] . ", B=" . $contas['B'] . ", Total=" .
($contas['A'] + $contas['B']) . "\n";
}
```

```

// Transação que é interrompida após débito
$resultado = transferir($contas, 'A', 'B', 30.0, true);
$total = $resultado['contas']['A'] + $resultado['contas']['B'];
echo "Após interrupção: A=" . $resultado['contas']['A'] . ", B=" . $resultado
['contas']['B'] . ", Total={$total}\n";
echo "(PROBLEMA: Total era 150, agora é 120! Dinheiro desapareceu.)\n";
}

if (debug_backtrace() === []) {
    demo();
}

```

O código vulnerável faz dois updates separados sem transação. Se o segundo falha, o primeiro já foi gravado (inconsistência).

Código corrigido ([A10-tratamento-inadequado-excecoes/07-transacao-nao-atmica-sem-rollback/corrigido.php](#))

```

<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Transação com Rollback em Caso de Falha
 *
 * Se alguma etapa da transação falha, o estado anterior é restaurado.
 * Assim, ou a transferência completa ou nada muda (atomicidade).
 */

function transferir(array $contas, string $origem, string $destino, float $valor, bool
$interromperAposDebito): array {
    // Valida se há saldo
    if ($contas[$origem] < $valor) {
        return ['status' => 'erro', 'contas' => $contas];
    }

    // Salva estado anterior (para rollback)
    $saldoOrigemAntes = $contas[$origem];

    // Etapa 1: Débito
    $contas[$origem] -= $valor;

    // CORRIGIDO: se falha aqui, faz rollback do débito
    if ($interromperAposDebito) {
        $contas[$origem] = $saldoOrigemAntes; // Rollback: restaura saldo original
        return ['status' => 'erro', 'contas' => $contas, 'mensagem' => 'transacao
revertida'];
    }
}

```

```
// Etapa 2: Crédito (agora seguro, pois se chegar aqui vai completar)
$contas[$destino] += $valor;

return ['status' => 'sucesso', 'contas' => $contas];
}

function demo(): void {
    echo "=== CORRIGIDO: Transação com rollback em caso de falha ===\n";

    $contas = ['A' => 100.0, 'B' => 50.0];
    echo "Estado inicial: A=" . $contas['A'] . ", B=" . $contas['B'] . ", Total=" .
($contas['A'] + $contas['B']) . "\n";

    // Transação que é interrompida após débito
    $resultado = transferir($contas, 'A', 'B', 30.0, true);
    $total = $resultado['contas']['A'] + $resultado['contas']['B'];
    echo "Após interrupção (com rollback): A=" . $resultado['contas']['A'] . ", B=" .
$resultado['contas']['B'] . ", Total={$total}\n";
    echo "(OK: Total continua 150! Rollback restaurou o estado.)\n";
}

if (debug_backtrace() === []) {
    demo();
}
```

A correção usa **BEGIN TRANSACTION** antes, e **ROLLBACK** se qualquer passo falha. Ambos os updates são "tudo ou nada" — se qualquer um falha, os dois são desfeitos. Atomicidade garantida.

11.7.1. Como testar

Execute `php A10-tratamento-inadequado-excecoes/07-transacao-nao-atmica-sem-rollback/teste.php`. O teste simula uma transferência onde o segundo update falha propositalmente. A versão vulnerável fica com inconsistência (débito sem crédito). A versão corrigida faz rollback de ambas. Saída esperada termina em **RESULTADO: N/N verificacoes passaram**.

11.7.2. Checklist de verificação

- ☐ Operações multi-step que precisam ser "tudo ou nada" usam **BEGIN TRANSACTION**?
- ☐ Se qualquer passo falha, **ROLLBACK** é executado automaticamente?
- ☐ Try/catch ao redor de operação transacional dispara rollback em caso de exceção?

11.8. 08 — Dados Sensíveis em Debug

CWE-215: Information Exposure Through Debug Information

Modo debug (quando ativado em produção, por acidente ou negligência) pode dumpar **todas** as variáveis: senhas, tokens, números de cartão de crédito, chaves de API. Um `var_dump()` inocente numa página de erro expõe dados sensíveis pra qualquer um que vê a página. Pior ainda: modo

debug frequentemente também deixa ferramentas abertas (phpMyAdmin, API debug endpoints) que qualquer um pode acessar. A correção é: **nunca** deixar debug ativo em produção, e se ativar (temporariamente pra troubleshooting), **sempre redatar campos sensíveis** mesmo em debug.

Código vulnerável (A10-tratamento-inadequado-excecoes/08-dados-sensíveis-em-debug/vulneravel.php)

```
<?php
declare(strict_types=1);

namespace Vulneravel;

/**
 * Dados Sensíveis Expostos em Modo Debug (CWE-215) - A10:2025 Mishandling of
 * Exceptional Conditions
 *
 * Em modo debug (que pode acidentalmente continuar ativo em produção),
 * a função despeja TODOS os campos da requisição usando print_r(),
 * incluindo senhas, números de cartão de crédito e tokens de segurança.
 */

function depurarRequisicao(array $dadosRequisicao, bool $modoDebug): ?string {
    if (!$modoDebug) {
        return null;
    }

    // VULNERÁVEL: despeja tudo sem filtrar dados sensíveis
    // print_r() mostra TODOS os campos, inclusive senha, cartão de crédito, token
    return print_r($dadosRequisicao, true);
}

function demo(): void {
    echo "=== VULNERÁVEL: Exposição de dados sensíveis em debug ===\n";

    $dados = [
        'usuario' => 'joao',
        'email' => 'joao@example.com',
        'senha' => 'MinhaSenha123',
        'cartao_credito' => '4111111111111111',
        'token' => 'abc123xyz789'
    ];

    $saida = depurarRequisicao($dados, true);
    echo "Saída de debug:\n{$saida}\n";
    echo "(PROBLEMA: senha, cartão de crédito e token expostos em debug!)\n";
}

if (debug_backtrace() === []) {
    demo();
}
```

O código vulnerável despeja objeto com todos os campos (inclusive senha e cartão) quando debug=true.

Código corrigido (A10-tratamento-inadequado-excecoes/08-dados-sensíveis-em-debug/corrigido.php)

```
<?php
declare(strict_types=1);

namespace Corrigido;

/**
 * Correção: Filtro de Dados Sensíveis em Modo Debug
 *
 * Define lista de campos sensíveis que devem ser redacted (mascarados)
 * antes de exibir em modo debug. Campos não-sensíveis continuam visíveis
 * para fins de depuração, mas dados críticos estão protegidos.
 */

function depurarRequisicao(array $dadosRequisicao, bool $modoDebug): ?string {
    if (!$modoDebug) {
        return null;
    }

    // CORRIGIDO: define quais campos são sensíveis
    $camposSensiveis = ['senha', 'cartao_credito', 'token'];

    // Cria cópia dos dados para filtrar
    $dadosFiltrados = $dadosRequisicao;

    // Redacts campos sensíveis
    foreach ($camposSensiveis as $campo) {
        if (isset($dadosFiltrados[$campo])) {
            $dadosFiltrados[$campo] = '[REDACTED]';
        }
    }

    // Retorna debug info com dados sensíveis mascarados
    return print_r($dadosFiltrados, true);
}

function demo(): void {
    echo "=== CORRIGIDO: Debug com filtro de dados sensíveis ===\n";

    $dados = [
        'usuario' => 'joao',
        'email' => 'joao@example.com',
        'senha' => 'MinhaSenha123',
        'cartao_credito' => '4111111111111111',
        'token' => 'abc123xyz789'
    ];
}
```

```

    $saida = depurarRequisicao($dados, true);
    echo "Saída de debug (filtrada):\n{$saida}\n";
    echo "(OK: dados sensíveis mascarados, dados não-sensíveis visíveis para debug)\n";
};
}

if (debug_backtrace() === []) {
    demo();
}

```

A correção redige campos sensíveis com **[REDACTED]** antes de exibir, mesmo em modo debug. Nome de usuário fica visível (útil pra debug), mas senha e cartão são ocultados.

11.8.1. Como testar

Execute `php A10-tratamento-inadequado-excecoes/08-dados-sensíveis-em-debug/teste.php`. O teste ativa modo debug e inspeciona um objeto de usuário. A versão vulnerável expõe senha e cartão (PROBLEMA!). A versão corrigida redige campos sensíveis mesmo em debug. Saída esperada termina em **RESULTADO: N/N verificacoes passaram**.

11.8.2. Checklist de verificação

- ☐ Modo debug **nunca** está ativo em produção?
- ☐ Se debug é ativado (temporariamente), campos sensíveis são redatados automaticamente?
- ☐ Endpoints de debug, ferramentas administrativas, phpMyAdmin são **nunca** acessíveis publicamente?

11.9. Checklist consolidado — Tratamento Inadequado de Condições Excepcionais

Ao revisar tratamento de erros em aplicação, valide os pontos críticos:

- ☐ **Mensagens de Erro:** Cliente recebe **sempre** mensagens genéricas. Detalhes técnicos são logados apenas no servidor.
- ☐ **Exceções Tratadas:** Nenhuma exceção escapa de uma função sem ser capturada e tratada apropriadamente.
- ☐ **Validação de Retorno:** Retornos de funções críticas (I/O, BD, rede) são **sempre** verificados. Falhas silenciosas são impossíveis.
- ☐ **Default Seguro:** Switches, ternários e defaults assumem **negação/falha**, nunca **acesso/sucesso**.
- ☐ **Catch Específico:** Exceções diferentes recebem catch e tratamento específico, não genérico.
- ☐ **Fail-Safe:** Quando serviço externo falha, acesso é **negado**, não permitido.
- ☐ **Atomicidade:** Operações multi-step críticas usam transações com rollback em falha.
- ☐ **Debug Seguro:** Modo debug nunca expõe dados sensíveis, mesmo com `var_dump()`.

Chapter 12. Apêndice A: Testando SQL Injection com sqlmap na Prática

12.1. Introdução ao sqlmap

O [sqlmap](#) é uma ferramenta de código aberto que automatiza a detecção e exploração de vulnerabilidades de SQL Injection. Diferente do teste manual em código PHP (que chama funções direto), o sqlmap atua como cliente HTTP — envia requisições para uma aplicação real rodando num servidor, simula diferentes técnicas de injeção (boolean-based blind, time-based blind, error-based, UNION-based), e documenta o que consegue explorar.

Por isso, os exemplos [01-sql-injection](#) e [07-orm-injection](#) do capítulo [A05](#) têm não apenas [vulneravel.php](#) e [corrigido.php](#) (testáveis direto com [php teste.php](#)), mas também [endpoint-vulneravel.php](#) e [endpoint-corrigido.php](#) — "wrappers" HTTP finos que servem como alvo real para o sqlmap. Dessa forma, você consegue praticar uma ferramenta real de pentest num ambiente seguro e controlado antes de usá-la em qualquer coisa real.

12.2. Testando 01-sql-injection

Este exemplo demonstra SQL Injection clássica: concatenação direta de input do usuário na query de login.

Pré-requisitos:

1. Instale sqlmap: `brew install sqlmap` (macOS) ou `apt-get install sqlmap` (Linux).
2. Certifique-se de que PHP 8.1+ está disponível.

Método rápido — script automático:

Nesta pasta ([A05-injecao/01-sql-injection/](#)), há um script [testar-com-sqlmap.sh](#) que automatiza tudo:

```
./testar-com-sqlmap.sh
```

Ele: * Sobe um servidor PHP embutido em `127.0.0.1:8899`. * Roda sqlmap contra [endpoint-vulneravel.php](#) e detecta técnicas de injeção. * Roda sqlmap contra [endpoint-corrigido.php](#) e confirma que não há injeção. * Derruba o servidor automaticamente.

Método manual — passo a passo:

Passo 1: Suba o servidor

```
php -S 127.0.0.1:8899
```

Passo 2: Teste contra o endpoint vulnerável

```
sqlmap -u "http://127.0.0.1:8899/endpoint-vulneravel.php?usuario=admin&senha=x" \
-p usuario --batch --level=3 --risk=2 --technique=BE --dbms=SQLite --flush-session
```

Resultado esperado — sqlmap detecta **2 técnicas de injeção**:

```
Parameter: usuario (GET)
  Type: boolean-based blind
  Title: SQLite AND boolean-based blind - WHERE or HAVING clause (JSON)
  Payload: usuario=admin' AND CASE WHEN 1235=1235 THEN 1235 ELSE JSON(...) END--
DJln

  Type: error-based
  Title: SQLite >= 3.9 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause
  Payload: usuario=admin' AND 9708=JSON_EXTRACT(...)-- Yons
```

Passo 3: Extrair dados (prova de impacto real)

```
sqlmap -u "http://127.0.0.1:8899/endpoint-vulneravel.php?usuario=admin&senha=x" \
-p usuario --batch --level=3 --risk=2 --technique=BE --dbms=SQLite --flush-session \
--dump -T usuarios
```

Resultado — tabela **usuarios** é completamente dumpada:

```
Database: <current>
Table: usuarios
+----+-----+-----+
| id | usuario | senha_hash |
+----+-----+-----+
| 1 | admin | 8ba4b2bed568d78d3f03a56c5788c272 |
+----+-----+-----+
```

Passo 4: Teste contra o endpoint corrigido

```
sqlmap -u "http://127.0.0.1:8899/endpoint-corrigido.php?usuario=admin&senha=x" \
-p usuario --batch --level=3 --risk=2 --technique=BE --dbms=SQLite --flush-session
```

Resultado esperado — **nenhuma injeção**:

```
[WARNING] GET parameter 'usuario' does not seem to be injectable
[ERROR] all tested parameters do not appear to be injectable.
```

12.2.1. Sobre as opções sqlmap usadas

Opção	Motivo
<code>--batch</code>	Não pergunta nada interativamente, usa sempre a resposta padrão
<code>-p usuario</code>	Foca só no parâmetro <code>usuario</code> (o <code>senha</code> não é injetável aqui)
<code>--level=3 --risk=2</code>	Nível intermediário — suficiente pra detectar injeção sem demorar muito
<code>--technique=BE</code>	Testa só Boolean-based blind e Error-based (as duas que funcionam aqui)
<code>--dbms=SQLite</code>	Já sabemos o DBMS, pula fingerprint e acelera
<code>--flush-session</code>	Ignora cache anterior, garante teste limpo
<code>--dump -T usuarios</code>	Extraí os dados de verdade da tabela

12.3. Testando 07-orm-injection

Este exemplo demonstra que até uma camada ORM/Repositório é vulnerável se expuser métodos que aceitam SQL arbitrário.

Método rápido:

Nesta pasta (`A05-injecao/07-orm-injection/`), há um script `testar-com-sqlmap.sh`:

```
./testar-com-sqlmap.sh
```

Método manual:

Passo 1: Suba o servidor (porta diferente para não conflitar)

```
php -S 127.0.0.1:8900
```

Passo 2: Teste contra o endpoint vulnerável

```
sqlmap -u "http://127.0.0.1:8900/endpoint-vulneravel.php?nome=admin" \
-p nome --batch --level=3 --risk=2 --technique=BE --dbms=SQLite --flush-session
```

Resultado esperado — **2 técnicas detectadas** (injeção confirmada com apenas 5 requisições, é um contexto simples):

```
Parameter: nome (GET)
Type: boolean-based blind
Title: AND boolean-based blind - WHERE or HAVING clause
Payload: nome=admin' AND 1178=1178-- mill
```

Type: error-based

Title: SQLite >= 3.9 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause

Payload: nome=admin' AND 2084=JSON_EXTRACT(...)-- vuGk

Passo 3: Teste contra o endpoint corrigido

```
sqlmap -u "http://127.0.0.1:8900/endpoint-corrigido.php?nome=admin" \
-p nome --batch --level=3 --risk=2 --technique=BE --dbms=SQLite --flush-session
```

Resultado esperado — **nenhuma injeção**:

```
[WARNING] GET parameter 'nome' does not seem to be injectable
[ERROR] all tested parameters do not appear to be injectable.
```

12.4. Checklist Geral de sqlmap para Qualquer Projeto PHP

Este checklist adapta e expande os passos em [sqlmap-checklist-geral.md](#) em estrutura de apêndice, pronto para usar em pentest ou laboratório de segurança.

12.4.1. 0. Antes de Começar

- ☐ Você tem autorização explícita (escopo de pentest, ambiente próprio, CTF) para testar o alvo?
- ☐ Você sabe se o alvo é produção ou homologação? **Nunca** rode `--dump-all`, `--os-shell` ou opções destrutivas em produção sem combinar antes — sqlmap pode gerar muitas requisições e ficar nos logs.
- ☐ `sqlmap --version` funciona? (`brew install sqlmap` no macOS, `apt-get install sqlmap` no Linux)

12.4.2. 1. Mapear os Pontos de Entrada

Qualquer lugar que a aplicação PHP recebe input do usuário e o usa numa query é candidato:

- ☐ Parâmetros de URL (`GET ?id=1, ?usuario=x`)
- ☐ Campos de formulário (`POST`)
- ☐ Cookies (muita gente esquece — `session_id`, `preferencias`, etc.)
- ☐ Headers customizados (`X-Forwarded-For`, `User-Agent`, `Referer`)
- ☐ Corpo JSON de APIs (`Content-Type: application/json`)

12.4.3. 2. Teste Rápido de Descoberta (GET Simples)

```
sqlmap -u "http://alvo.exemplo.com/pagina.php?id=1" --batch
```

- ❑ `--batch` — não fica perguntando, usa sempre a opção padrão

Se o sqlmap disser `parameter might not be injectable`, **não** significa que está seguro — só significa que o teste padrão (level=1, risk=1) não achou. Vá para próximo passo.

12.4.4. 3. Aumentar a Profundidade do Teste

```
sqlmap -u "http://alvo.exemplo.com/pagina.php?id=1" --batch --level=5 --risk=3
```

- ❑ `--level` (1 a 5) — quantos payloads diferentes tenta, e em quais lugares (URL, headers, cookies a partir do level 2+)
- ❑ `--risk` (1 a 3) — quão "arriscados"/destrutivos são os payloads testados (risk=3 inclui `OR`-based que podem alterar dados)

`--level=5 --risk=3` é o teste mais exaustivo, mas também mais lento e "barulhento" (gera muito mais requisições).

12.4.5. 4. Focar num Parâmetro Específico (Mais Rápido)

```
sqlmap -u "http://alvo.exemplo.com/pagina.php?id=1&outro=2" -p id --batch
```

- ❑ `-p <parametro>` — testa só esse parâmetro em vez de todos (útil quando você já suspeita qual é o problema)

12.4.6. 5. Testar POST / Formulários

```
sqlmap -u "http://alvo.exemplo.com/login.php" \  
--data="usuario=admin&senha=x" --batch
```

Ou deixar o sqlmap achar formulários sozinho:

```
sqlmap -u "http://alvo.exemplo.com/login.php" --forms --batch
```

- ❑ `--data` — envia como POST com corpo especificado
- ❑ `--forms` — sqlmap busca `<form>` na página e testa os campos

12.4.7. 6. Testar Autenticado (Área Logada)

```
sqlmap -u "http://alvo.exemplo.com/painel.php?id=1" \  
--cookie="PHPSESSID=abc123xyz" --batch
```

- ❑ `--cookie="..."` — cookie de sessão válido (pegue do navegador depois de logar manualmente)

- `--auth-type=Basic --auth-cred="usuario:senha"` — se o site usa HTTP Basic Auth

12.4.8. 7. Explorar o Site Inteiro Automaticamente

```
sqlmap -u "http://alvo.exemplo.com/" --crawl=3 --forms --batch
```

- `--crawl=N` — segue links a partir da URL inicial até N níveis de profundidade, testando cada página encontrada

12.4.9. 8. Quando Encontrar uma Injeção — Extrair Informação

```
# Listar bancos de dados
sqlmap -u "..." -p id --batch --dbs

# Listar tabelas de um banco específico
sqlmap -u "..." -p id --batch -D nome_do_banco --tables

# Listar colunas de uma tabela
sqlmap -u "..." -p id --batch -D nome_do_banco -T usuarios --columns

# Dump completo de uma tabela
sqlmap -u "..." -p id --batch -D nome_do_banco -T usuarios --dump

# Dump de TUDO (cuidado - pode demorar muito)
sqlmap -u "..." -p id --batch --dump-all
```

- Sempre enumerar `--dbs` → `--tables` → `--columns` → `--dump` nessa ordem, em vez de já sair com `--dump-all` (mais controlado)

12.4.10. 9. Se o sqlmap Não Encontrar Nada de Primeira

- Tentou `--level=5 --risk=3`?
- O alvo tem WAF? Teste com `--random-agent` (varia User-Agent) e `--tamper=space2comment` (scripts de tamper em `--tamper=?`)
- O parâmetro é numérico ou string? Às vezes forçar tipo ajuda: `--param-filter` / `--test-filter`
- Confirme que a resposta muda de acordo com a query (algumas páginas sempre retornam a mesma coisa)

12.4.11. 10. Depois de Terminar

- `sqlmap --purge` limpa histórico/sessões salvas localmente (`~/.local/share/sqlmap`)
- Documente o parâmetro vulnerável, o payload usado, e request/response completo — é isso que vai no relatório de pentest
- Nunca deixe sessões/dumps de dados sensíveis de um cliente real espalhados no seu disco

12.4.12. Opções Mais Usadas — Resumo Rápido

Opção	Para quê
<code>--batch</code>	Não interativo, sempre usa resposta padrão
<code>-u "url"</code>	URL alvo
<code>-p param</code>	Foca nesse parâmetro
<code>--data="a=1&b=2"</code>	Testa via POST
<code>--cookie="..."</code>	Sessão autenticada
<code>--level=1..5</code>	Profundidade dos testes
<code>--risk=1..3</code>	Agressividade dos payloads
<code>--technique=BEUSTQ</code>	Restringe a técnicas (B=boolean, E=error, U=union, S=stacked, T=time)
<code>--dbms=mysql/postgresql/sqlite/...</code>	Já informa DBMS, pula fingerprint
<code>--crawl=N</code>	Rastreia o site
<code>--forms</code>	Testa formulários HTML
<code>--dbs / --tables / --columns / --dump</code>	Enumeração progressiva
<code>--flush-session</code>	Ignora cache anterior
<code>--random-agent</code>	User-Agent aleatório
<code>--tamper=...</code>	Scripts de ofuscação de payload

12.5. Aviso de Segurança

O sqlmap é uma ferramenta poderosa — use-a **apenas** em ambientes que você tem permissão explícita para testar:

- ☐ Seu próprio laboratório local
- ☐ Ambientes de staging/teste da sua empresa
- ☐ Pentest com escopo e contrato assinado
- ☐ Competições CTF autorizadas
- ☐ Sistemas de terceiros sem permissão explícita (é crime — Lei de Acesso Não Autorizado a Computadores)
- ☐ Produção sem aviso prévio aos donos (pode gerar muitas requisições e derrubar o serviço)

Sempre documente o que testou, quais vulnerabilidades encontrou, e mantenha confidencialidade dos dados extraídos.

Chapter 13. Apêndice B: Rodando o Projeto via Docker

13.1. Introdução

Se você não tem PHP instalado localmente (ou prefere evitar o incômodo de instalar extensões, configurar servidor web e banco de dados), pode rodar o projeto inteiro via Docker. A imagem Docker fornecida já inclui tudo: PHP 8.3, Apache web server, SQLite com PDO, e as extensões necessárias. Você só precisa ter Docker e Docker Compose instalados na sua máquina.

13.2. Subindo o Ambiente

Assumindo que você tem Docker e Docker Compose instalados, abra um terminal na raiz do projeto (`owasp-php-examples/`) e execute:

```
docker compose up -d --build
```

O flag `--build` reconstrói a imagem a partir do `Dockerfile` (necessário só na primeira execução ou se você editar o `Dockerfile`). O flag `-d` executa em background (detached mode).

O Docker vai:

1. Construir a imagem com PHP 8.3, Apache, e extensões
2. Iniciar um container rodando Apache na porta 8080 do host
3. Montar o diretório local (`owasp-php-examples/`) dentro do container em `/var/www/html`

Depois de alguns segundos, abra seu navegador em <http://localhost:8080> e você verá o índice com todas as categorias e exemplos.

Se quiser ver logs do container em tempo real:

```
docker compose logs -f
```

13.3. O Índice (`index.php`)

O arquivo `index.php` na raiz fornece uma interface web para explorar e testar o projeto. Ele oferece 3 funcionalidades principais:

1. Listar categorias e exemplos com links

A página inicial lista todas as categorias (A01 até A10), e dentro de cada categoria, um botão pra cada exemplo específico. Para cada exemplo, há links para: - Rodar teste (executa `teste.php` via HTTP, mostra resultados) - Ver código vulnerável (`highlight_file()` mostrando o PHP com destaque de sintaxe) - Ver código corrigido (mesma coisa)

2. Rodar testes

Clicando em "rodar teste" em um exemplo, o navegador faz requisição HTTP pra `teste.php` daquele exemplo. A saída é mostrada em tempo real: cada verificação, e ao final **RESULTADO: N/N verificacoes passaram.**

Um botão especial "Rodar todos os testes" executa `run-tests.sh` completo (que testa todas as 30+ subpastas) via HTTP e mostra a saída completa. Útil pra validar que tudo está funcionando.

3. Ver código-fonte com validação de caminho

Quando você clica "ver vulnerável" ou "ver corrigido", o link passa um parâmetro `ver=...` com o caminho relativo do arquivo. O `index.php` implementa **a mesma validação de path traversal ensinada no capítulo de A08** (exemplo 06-inclusao-caminho-nao-controlado).

O código valida: - O caminho real (`realpath()`) fica dentro da raiz do projeto (não escapou com `../`) - A extensão está em uma allow-list: `.php`, `.json`, `.md`, `.sh` (não permite `.exe`, `.jpg`, etc)

Se alguma validação falhar, retorna erro 400. Isso é educacional: o próprio projeto aplica as técnicas que ensina!

13.4. Parando e Editando

Para parar o container:

```
docker compose down
```

Isso para e remove o container, mas mantém a imagem (mais rápido pra `up` da próxima vez).

Importante: O diretório local é montado como volume dentro do container. Isso significa: se você edita qualquer arquivo `.php` localmente (`vulneravel.php`, `corrigido.php`, `teste.php`), a mudança é **instantaneamente refletida** dentro do container — não precisa reconstruir. Basta recarregar a página no navegador.

Se você editar o `Dockerfile` ou adicionar uma nova extensão, precisará fazer rebuild:

```
docker compose down
docker compose up -d --build
```

13.5. O Dockerfile

Abaixo está o `Dockerfile` que constrói a imagem:

Dockerfile do projeto

```
FROM php:8.3-apache

RUN apt-get update \
```

```
&& apt-get install -y --no-install-recommends libsqlite3-dev \
&& rm -rf /var/lib/apt/lists/* \
&& docker-php-ext-install pdo pdo_sqlite

COPY . /var/www/html/

RUN find /var/www/html -name "*.sh" -exec chmod +x {} \; \
&& chown -R www-data:www-data /var/www/html

EXPOSE 80
```

Este Dockerfile:

1. Parte da imagem oficial `php:8.3-apache` (já tem PHP e Apache)
2. Instala `libsqlite3-dev` (necessário pra compilar extensão PDO SQLite)
3. Habilita extensões `pdo` e `pdo_sqlite` no PHP
4. Copia o projeto inteiro (`./`) pra dentro do container em `/var/www/html`
5. Torna executáveis todos os `.sh` (scripts como `run-tests.sh`)
6. Muda permissão dos arquivos pra `www-data` (usuário sob o qual Apache roda)
7. Expõe porta 80 (mapeada em localhost:8080 pelo compose)

13.6. Docker Compose

O arquivo `docker-compose.yml`:

docker-compose.yml do projeto

```
services:
  owasp-php-curso:
    build: .
    ports:
      - "8080:80"
    volumes:
      - ./var/www/html
```

Configuração bem simples:

- `build: .` — constrói imagem a partir do Dockerfile no diretório atual
- `ports: ["8080:80"]` — mapeia porta 80 do container pra 8080 do host
- `volumes: [".:/var/www/html"]` — monta diretório local como volume (mudanças sincronizadas)

13.7. Próximos Passos

Agora que o projeto está rodando via Docker, você pode:

- Explorar cada exemplo via navegador
- Rodar os testes pra ver a diferença entre código vulnerável e corrigido
- Editar os arquivos localmente e ver mudanças refletidas instantaneamente
- Usar como ponto de partida pra suas próprias aplicações

Para mais informações sobre cada categoria, volte aos capítulos 01-10 deste livro.